解密 Frida：动态插桩框架源码之旅

前言
- 这本书是怎么来的
- 这本书适合谁
- 这本书不是什么
- 怎么读这本书
- 关于代码示例
- 致谢
- 一些建议

第1章：走进 Frida 的世界
- 1.1 什么是动态插桩
- 1.2 Frida 是什么
- 1.3 Frida 能做什么
- 1.4 Frida 与其他工具的对比
- 1.4 为什么要阅读 Frida 源码
- 1.5 Frida 的简要历史
- 1.6 本书的内容规划
- 本章小结
- 思考与讨论

第2章：源码全景地图
- 2.1 仓库顶层结构
- 2.2 子项目架构：分层设计
- 2.3 frida-gum：地基中的地基
- 2.4 frida-core：大脑和神经系统
- 2.5 绑定层：语言的桥梁
- 2.6 frida-tools：开箱即用的工具
- 2.7 构建系统
- 2.8 数据流全景图
- 2.9 该从哪里开始读
- 本章小结
- 思考与讨论

第3章：语言基础速览：C、Vala 与 JavaScript
- 3.1 C 语言：Frida 的骨骼
- 3.2 Vala 语言：Frida 的血肉
- 3.3 JavaScript：Frida 的灵魂
- 3.4 三种语言如何协作
- 3.5 阅读源码的实用技巧
- 本章小结
- 思考与讨论

第4章：程序的大门——入口与启动流程
- 4.1 为什么要关注入口？
- 4.2 frida-server 的 main() 入口
- 4.3 Application 类：真正的启动核心
- 4.4 异步启动序列
- 4.5 守护进程模式
- 4.6 frida-inject 的入口
- 4.7 DeviceManager 的初始化
- 4.8 信号处理与优雅退出
- 4.9 Environment.init() 与 Environment.configure()

本章小结
思考题
第5章：Gum：插桩引擎的心脏
5.1 Gum 是什么？
5.2 Gum 的架构全景
5.3 Interceptor：函数拦截器
5.4 Stalker：代码追踪器
5.5 Memory：内存操作
5.6 Module 和 Process：运行时信息
5.7 Cloak：隐藏自身
5.8 Gum 的初始化
5.9 Gum C API 与 GumJS 的关系
5.10 Gum 的设计哲学
本章小结
思考题
第6章：核心数据结构与接口定义
6.1 接口定义在哪里？
6.2 三大核心接口
6.3 辅助接口
6.4 ID 类型体系
6.5 AgentMessage 与消息传输
6.6 HostSessionProvider 与后端架构
6.7 会话管理的数据流
本章小结
思考题
第7章：构建系统——Meson 与交叉编译
7.1 为什么选择 Meson？
7.2 顶层 meson.build 结构
7.3 构建选项（meson.options）
7.4 语言绑定的构建
7.5 交叉编译的工作原理
7.6 平台差异的处理
7.7 子项目依赖与版本管理
7.8 实际构建命令
本章小结
思考题
第8章：Interceptor - 函数钩子的魔法
8.1 什么是函数钩子？
8.2 Interceptor 的核心 API
8.3 InvocationListener：你的”监听器”
8.4 InvocationContext：你能拿到什么？
8.5 Interceptor 的内部结构
8.6 FunctionContext：每个被 Hook 函数的”档案”
8.7 Hook 的工作原理：跳板（Trampoline）
8.8 事务机制（Transaction）
8.9 一步步跟踪：attach 到底做了什么？
8.10 线程安全与忽略机制
8.11 attach 的返回值
8.12 本章小结
讨论问题
第9章：Stalker - 代码追踪引擎
9.1 什么是动态二进制插桩？
9.2 Stalker vs Interceptor：什么时候用哪个？
9.3 Stalker 的核心 API
9.4 基本块编译模型
9.5 StalkerTransformer：你的代码转换器
9.6 EventSink：事件收集器
9.7 StalkerWriter：跨架构的代码输出
9.8 信任阈值与代码缓存
9.9 Backpatching 优化

9.10 排除区域
9.11 其他实用功能
9.12 Stalker 的完整工作流程
9.13 本章小结
讨论问题
第10章：代码生成器 - 跨架构的机器语言
10.1 为什么 Frida 需要在运行时写机器码？
10.2 Writer 的基本结构
10.3 Writer 的使用模式
10.4 常用操作对照表
10.5 带参数的函数调用
10.6 标签系统（Label System）
10.7 距离限制与处理
10.8 x86 的 Meta 寄存器
10.9 ARM64 的特殊考虑
10.10 实际例子：生成一段完整的跳板代码
10.11 辅助工具函数
10.12 本章小结
讨论问题
第11章：Relocator - 代码搬迁的艺术
11.1 为什么需要重定位？
11.2 什么指令需要修正？
11.3 Relocator 的结构
11.4 read_one / write_one 模式
11.5 EOB 和 EOI
11.6 不同架构的重定位挑战
11.7 can_relocate：预检查
11.8 Writer、Relocator 和 Interceptor 的协作
11.9 overwritten_prologue 的秘密
11.10 一个完整的重定位例子
11.11 静态重定位辅助函数
11.12 本章小结
讨论问题
第12章：进程注入——打入目标的第一步
12.1 什么是进程注入？
12.2 平台差异：每个系统有不同的”入口”
12.3 Linux 注入：Linjector 的故事
12.4 Helper 进程模式：为什么需要”中间人”？
12.5 注入流程详解
12.6 安全挑战与对抗
12.7 注入后的清理
本章小结
思考题
第13章：Agent 生命周期——注入后的世界
13.1 Agent 的入口：一切从 main 开始
13.2 Runner：Agent 的心脏
13.3 启动流程：从参数解析到事件循环
13.4 通信通道：DBus 连接
13.5 会话管理
13.6 隐身术：Cloak 机制
13.7 卸载与清理
13.8 完整生命周期图
本章小结
思考题
第14章：脚本引擎——JavaScript 遇见原生代码
14.1 两个世界的碰撞
14.2 双引擎：QuickJS 与 V8
14.3 ScriptEngine：脚本的管家
14.4 脚本创建：从源码到可执行
14.5 GumJS 绑定：桥梁的秘密

14.6 消息传递：JS 与 Host 的对话
本章小结
思考题
第15章：Gadget 模式——无需注入的插桩
15.1 什么是 Gadget?
15.2 自动初始化：constructor 的魔法
15.3 Gadget 的加载流程
15.4 配置文件：Gadget 的行为说明书
15.5 四种交互模式详解
15.6 等待与恢复机制
15.7 Gadget vs 动态注入：如何选择?
15.8 实际使用场景
15.9 Gadget 的卸载
本章小结
思考题
第16章：Linux 后端——ptrace 与 ELF 的世界
16.1 Linux 后端的整体架构
16.2 ptrace——Linux 上的万能调试接口
16.3 Linjector——Linux 注入器
16.4 /proc 文件系统——Linux 的透明窗口
16.5 Helper 进程架构
16.6 eBPF 集成——现代化的进程监控
16.7 ELF 模块处理
16.8 本章小结
讨论问题
第17章：Darwin 后端——Mach 端口与 XPC
17.1 Darwin 后端的整体架构
17.2 Mach 端口——Darwin 的进程控制基石
17.3 Spawn 控制——断点与 dyld
17.4 Fruitjector——Darwin 注入器
17.5 GumDarwinModule 与 GumDarwinMapper
17.6 代码签名与 PolicySoftener
17.7 DTrace 与 Spawn 监控
17.8 本章小结
讨论问题
第18章：Windows 后端——远程线程与调试符号
18.1 Windows 后端的整体架构
18.2 CreateRemoteThread——经典的注入手法
18.3 DbgHelp 与符号解析
18.4 多架构支持
18.5 ACL 与安全控制
18.6 Winjector 与异步监控
18.7 DEP 与 ASLR 应对
18.8 本章小结
讨论问题
第19章：移动平台——Android 与 iOS 的特殊挑战
19.1 两大移动后端概览
19.2 Android：Droidy 后端与 ADB 集成
19.3 iOS：Fruity 后端
19.4 移动平台的共同挑战
19.5 本章小结
讨论问题
第20章：DBus 协议——通信的骨架
20.1 什么是 DBus? 一个公交系统的比喻
20.2 为什么 Frida 选择 DBus?
20.3 三大核心接口
20.4 对象路径：DBus 的”门牌号”
20.5 消息的流动：一次 attach 的旅程
20.6 GDBus 的角色：幕后英雄
20.7 信号：事件的广播机制

20.8 认证：不是谁都能上车
20.9 小结
讨论问题
第21章：控制服务——网络上的 Frida
21.1 ControlService：Frida 的门面
21.2 frida-server 的启动流程
21.3 WebService：HTTP 与 WebSocket
21.4 连接的处理流程
21.5 认证与访问控制
21.6 ControlChannel：正式的对话
21.7 TLS 加密
21.8 远程调试的完整工作流
21.9 小结
讨论问题
第22章：消息系统——脚本与宿主的对话
22.1 消息系统的全景
22.2 AgentMessage：消息的结构
22.3 从脚本到宿主：send() 的旅程
22.4 批量发送：快递不是一件一件送的
22.5 流量控制：Batch ID 机制
22.6 从宿主到脚本：recv() 的旅程
22.7 错误处理：消息不能丢
22.9 AgentMessageSink：消息的终点
22.10 小结
讨论问题
第23章：Python 绑定——最受欢迎的接口
23.1 整体架构：两层包装的艺术
23.2 入口：**init**.py 的设计哲学
23.3 C 扩展：_frida 模块的秘密
23.4 核心类：Device、Session、Script
23.5 事件处理与回调机制
23.6 RPC 机制：exports 的魔法
23.7 Cancellable：优雅取消的艺术
23.8 类型提示的细致处理
23.9 常见使用模式
本章小结
讨论问题
第24章：Node.js 绑定与其他语言
24.1 Node.js 绑定：N-API 与代码生成
24.2 Python vs Node.js：绑定策略对比
24.3 Swift 绑定：原生 Apple 生态
24.4 其他语言绑定概览
24.5 绑定模式总结：通用公式
24.6 代码生成 vs 手写：工程权衡
本章小结
讨论问题
第25章：命令行工具——frida-tools 全家桶
25.1 全家桶一览
25.2 架构基石：ConsoleApplication
25.3 Reactor：异步任务调度器
25.4 frida-ps：最简单的工具
25.5 frida-kill 和 frida-ls-devices
25.7 frida：交互式 REPL
25.8 frida-trace：自动化追踪
25.9 frida-discover：函数发现
25.10 工具设计模式总结
本章小结
讨论问题
第26章：内存操作——扫描、监控与读写
26.1 为什么内存操作是核心能力

26.2 内存的基本抽象
26.3 内存读写：最基本的操作
26.4 内存扫描：大海捞针的艺术
26.5 内存访问监控（MemoryAccessMonitor）
26.6 内存分配：在目标进程中开辟空间
26.7 Frida 自己的堆管理
26.8 实际应用示例
26.9 本章小结
讨论问题
第27章：跨平台设计——一套代码，多个世界
27.1 跨平台的本质挑战
27.2 后端模式（Backend Pattern）
27.3 GumProcess：进程操作的统一抽象
27.4 GumModule：模块抽象的接口模式
27.5 Meson 构建系统中的条件编译
27.6 C 预处理器中的平台分支
27.7 TLS 抽象：一个简洁的例子
27.8 抽象泄漏：跨平台的代价
27.9 给你自己项目的启示
27.10 本章小结
讨论问题
第28章：性能优化——速度的追求
28.1 为什么性能至关重要
28.2 Stalker 的反向补丁（Backpatching）
28.3 Interceptor 的 Trampoline 设计
28.4 线程局部存储：避免锁竞争
28.5 Slab 分配器：减少系统调用
28.6 自旋锁与独立堆
28.7 最小化 Prolog/Epilog
28.8 代码块的回收利用
28.9 优化的哲学
28.10 本章小结
讨论问题
第29章：架构之美——设计模式总结
29.1 分层架构模式
29.2 后端/提供者模式（Backend/Provider）
29.3 事务模式（Transaction Pattern）
29.4 观察者模式（Observer Pattern）
29.5 工厂模式（Factory Pattern）
29.6 桥接模式（Bridge Pattern）
29.7 代理模式（Proxy Pattern）
29.8 模式总结表
29.9 为什么这些模式重要
本章小结
讨论问题
第30章：你的下一步——从读者到贡献者
30.1 搭建 Frida 开发环境
30.2 如何贡献代码
30.3 值得探索的相关项目
30.4 职业发展路径
30.5 推荐阅读
30.6 致读者
本章小结
讨论问题
附录 A：术语表
补充术语
附录 B：源码关键文件索引
frida-gum：核心引擎
frida-gum：架构后端
frida-gum：GumJS 绑定层

frida-core：服务与会话管理
frida-python 与 frida-tools
frida-core：平台注入与辅助
frida-gum：其他重要文件
构建系统与配置
frida-gum：GumJS 扩展绑定
其他子项目
按功能快速查找
附录 C：全书知识地图
C.1 Frida 全景架构图
C.2 组件依赖关系图
C.3 常用 API 速查表
C.4 章节关系图
C.5 核心数据流
关于本书
这本书是怎么写成的
开源精神
如何贡献和勘误
为什么写这本书
本书的局限性
致谢
许可协议
版本信息
联系方式

前言

> **本书由 everettjf 使用 Claude Code 分析 Frida 开源源码编写而成。** 保留出处即可自由转载。

这本书是怎么来的

Frida 是全世界最流行的动态插桩框架之一。安全研究员用它逆向分析恶意软件，移动开发者用它调试应用，游戏破解者用它修改内存——它几乎出现在每一个需要”窥探程序内部”的场景中。

当我第一次深入阅读 Frida 的源码时，我被它的工程质量所震撼。

不仅是因为它支持六个操作系统、四种 CPU 架构（这已经够复杂了），更因为它展示了一个真正的跨平台系统软件是怎么从底层一步步构建起来的。从机器码生成、到进程注入、到 JavaScript 运行时桥接，每一层都设计得精巧而实用。

我想：如果有一本书，能把这套复杂但优美的架构讲解给正在学习系统编程的开发者，让他们看到”真正底层的软件是什么样的”，那该多好。

于是有了这本书。

这本书适合谁

这本书写给**初中级开发者**。

如果你学过一门编程语言（C、Python、Java，哪个都行），知道什么是变量、函数、指针，那你就有足够的基础来读这本书。

你不需要： - 精通 C 或汇编语言 - 了解操作系统内核 - 有逆向工程经验 - 理解编译原理

这些我们都会在书中从头讲起。

这本书不是什么

这本书**不是**一本 Frida 的使用手册——它不会教你怎么用 Frida 来 hook 一个 App。

这本书**不是**一本汇编语言教程——虽然我们会介绍必要的机器码知识，但它不会覆盖指令集的方方面面。

这本书**是**一场源码探险——我们会像侦探一样，一层层剥开一个复杂系统软件的外壳，看清它内部的运作机制。在这个过程中，你会学到系统架构、跨平台设计、底层编程和工程实践——这些是比任何具体技术都更持久的知识。

怎么读这本书

如果你是编程新手： 建议按顺序阅读。前三章会帮你建立必要的背景知识。

如果你有一定经验： 可以跳过第 3 章的语言入门，直接从第 4 章开始。

如果你对某个话题特别感兴趣： 每一章都是相对独立的，你可以直接跳到感兴趣的章节。

每章末尾都有**思考题**和**动手练习**。我强烈建议你认真对待它们——被动地阅读和主动地思考，学习效果天差地别。

关于代码示例

书中的代码示例经过了大幅简化。真实的源码有复杂的错误处理、平台适配和边界情况处理，我们把这些去掉了，只保留核心逻辑。这样你可以专注于理解设计思路，而不是迷失在细节中。

如果你想看真实的代码，可以对照 Frida 的 GitHub 仓库阅读：

- **Frida 官网：** https://frida.re/
- **Frida 主仓库：** https://github.com/frida/frida
- **Frida GitHub 组织：** https://github.com/frida

致谢

感谢 Ole Andre Vadla Ravnas 和所有 Frida 贡献者写出了如此优秀的开源项目。Frida 的代码质量和架构设计值得每一位系统程序员学习。

感谢所有为开源社区贡献代码的人。没有开源文化，就不会有今天繁荣的软件世界。

感谢你选择阅读这本书。愿这段旅程能点燃你对底层编程的热情。

一些建议

准备一个笔记本。 读源码的过程中，你会产生很多疑问和想法。把它们记下来——有些问题会在后面的章节得到解答，有些会成为你进一步探索的方向。

不要急。 这不是一本需要一口气读完的小说。如果某个概念让你困惑，停下来想想，去网上搜搜相关资料，回来再继续。

动手实践。 每章末尾的思考题和动手练习不是装饰——它们是学习过程中最有价值的部分。

享受过程。 你正在做一件很酷的事——解剖一个被全世界安全研究员和开发者使用的插桩框架的内部结构。这不是每个人都有机会做的事。

让我们开始吧。

第1章：走进 Frida 的世界

> 你有没有想过，一个正在运行的程序，能不能像做手术一样，在不停机的情况下被”打开”、观察内部运转、甚至修改它的行为？

这不是科幻小说，这正是 Frida 每天在做的事情。

1.1 什么是动态插桩

在理解 Frida 之前，我们先搞清楚一个核心概念：**动态插桩**（Dynamic Instrumentation）。

想象你是一个汽车修理工。传统的调试方式相当于把发动机完全拆下来，在工作台上一个零件一个零件地检查。而动态插桩则不同，它更像是在发动机运转的时候，往里面伸入一根内窥镜，实时观察每个活塞的运动状态，甚至在不熄火的情况下调整点火时机。

用更技术化的语言来说：

- **静态分析**：程序没有运行，你阅读它的二进制代码或反编译后的代码
- **动态分析**：程序正在运行，你观察它的实际行为
- **动态插桩**：程序正在运行，你不仅能观察，还能**注入自己的代码**来改变程序的行为

```
┌──────────────────────────────────────────┐
│                 分析技术对比                 │
├──────────────┬──────────────┬──────────────┤
│   静态分析    │   动态分析    │   动态插桩    │
├──────────────┼──────────────┼──────────────┤
│ 程序未运行    │ 程序运行中    │ 程序运行中    │
│ 读代码/反编译 │ 观察行为      │ 观察+修改行为 │
│ IDA Pro 等   │ strace 等    │ Frida 等     │
```

看到的是蓝图	看到的是表演	你就是导演

1.2 Frida 是什么

Frida 是一个开源的动态插桩工具包，由 Ole Andre Vadla Ravnas 创建，目前由 NowSecure 团队维护。它的官方定义是：

> Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers.

翻译过来就是：为开发者、逆向工程师和安全研究员打造的动态插桩工具包。

如果把 Frida 比作一个工具箱，那它里面有这些”工具”：

- **frida-server**：运行在目标设备上的服务进程，是你伸入目标程序的那只”手”
- **frida-gadget**：一个可以嵌入到目标应用中的共享库，适合无法运行 server 的场景
- **frida CLI 工具**：命令行工具集，包括 frida、frida-trace、frida-ps 等
- **Python/Node.js/Swift 绑定**：让你用自己喜欢的语言编写插桩脚本

最让人兴奋的是，Frida 让你用 **JavaScript** 来编写注入到目标进程中的代码。没错，你用一段 JS 脚本，就能拦截任意函数调用、修改参数和返回值、甚至调用目标进程中的任何函数。

来看一个最简单的例子：

```
// 拦截 open() 系统调用，打印每次打开的文件路径
Interceptor.attach(Module.getExportByName(null, 'open'), {
    onEnter(args) {
        console.log('打开文件:', args[0].readUtf8String());
    }
});
```

就这么几行代码，你就能看到一个程序打开了哪些文件。这就是 Frida 的魔力。

1.3 Frida 能做什么

Frida 的应用场景非常广泛，让我用几个具体的场景来说明。

安全研究

安全研究员用 Frida 来分析 App 的通信协议、加密逻辑、认证流程。比如你想知道某个 App 是怎么加密网络请求的，用 Frida 拦截加密函数，就能看到加密前的明文和使用的密钥。

再比如，你想分析一个 App 的 SSL Pinning 机制，可以用 Frida 直接 hook 掉证书验证函数，让它始终返回”验证通过”，然后就可以用中间人代理抓取所有 HTTPS 流量了。

逆向工程

当你面对一个没有源码的程序，想理解它的内部逻辑时，Frida 就像一个 X 光机。你可以追踪函数调用链、dump 内存数据、观察对象的创建和销毁。

举一个更具体的例子：假设你在逆向一个游戏，想找到伤害计算的函数。你可以用 Frida 的 Stalker 功能追踪攻击时执行了哪些代码，再结合 Interceptor 逐步缩小范围，最终定位到那个关键函数。

应用开发与调试

开发者可以用 Frida 来动态修改应用行为而不需要重新编译。想测试某个边界条件？直接用 Frida 修改函数返回值就行了。想模拟网络超时？用 Frida 让网络请求函数延迟返回即可。

这种”热修改”的能力在调试复杂问题时尤其有用。传统方式需要修改代码、重新编译、部署、重现问题，整个循环可能要十几分钟。而用 Frida，你可以在几秒钟内修改程序行为并观察结果。

自动化测试

Frida 可以用来构建自动化测试框架，模拟各种异常情况，检验程序的健壮性。

1.4 Frida 与其他工具的对比

了解 Frida 的定位，和同类工具做个对比会很有帮助：

动态分析工具对比

工具	类型	主要平台	特点
Frida	动态插桩	全平台	JS 脚本、跨平台、灵活
GDB	调试器	Linux 为主	断点调试、底层但操作繁
LLDB	调试器	Apple 为主	类似 GDB、Xcode 集成
DynamoRIO	动态插桩	Win/Linux	学术背景、API 较底层
Intel Pin	动态插桩	Win/Linux	Intel 出品、x86 为主
Xposed	框架 Hook	Android	Android 专用、需 root

Frida 最大的优势在于**易用性和跨平台性**的平衡。你用同一套 JavaScript API 就能在 Windows、macOS、Linux、iOS、Android 上做插桩，这在其他工具中是很难实现的。

```
┌──────────────────────────────────────────┐
│              Frida 应用场景                  │
├──────────────────────────────────────────┤
│                                            │
│  安全研究      ─── 协议分析、漏洞挖掘          │
│  逆向工程      ─── 理解未知程序的逻辑          │
│  应用调试      ─── 运行时修改行为              │
│  自动化测试    ─── 模拟异常、检验健壮性        │
```

```
│  性能分析      ─── 追踪热点函数                  │
│  教育研究      ─── 学习操作系统和程序原理        │
│                                                  │
└──────────────────────────────────────────────────┘
```

1.4 为什么要阅读 Frida 源码

也许你会问：我会用 Frida 就行了，为什么要读源码呢？

这个问题很好，让我用一个比喻来回答。

会开车和懂发动机原理是两回事。大多数时候，会开车就够了。但如果你是赛车手，你必须理解发动机的每一个细节才能把车开到极限；如果你是汽车设计师，你更需要理解现有发动机的设计才能做出创新。

阅读 Frida 源码，你会收获以下这些：

第一，理解底层原理。 Frida 是怎么把代码注入到另一个进程的？Interceptor 是怎么在不修改原始二进制文件的情况下拦截函数调用的？Stalker 是怎么追踪每一条执行的指令的？这些问题的答案都藏在源码里。

第二，学习优秀的工程实践。 Frida 项目跨越多种语言（C、Vala、JavaScript、Python），支持多个平台（Windows、macOS、Linux、iOS、Android），它的架构设计、跨平台抽象、构建系统都值得学习。

第三，具备解决疑难问题的能力。 当 Frida 的行为不符合预期时，如果你了解源码，就能快速定位问题，而不是在论坛上茫然地发帖求助。

第四，为底层安全领域打下基础。 Frida 涉及进程注入、内存管理、代码生成、指令集架构等底层技术，这些知识在安全研究领域极其宝贵。

1.5 Frida 的简要历史

Frida 最早由 Ole Andre Vadla Ravnas 在 2010 年左右开始开发，最初的动机是为 NowSecure（当时叫 viaForensics）的移动安全产品提供动态分析能力。

项目的发展可以大致分为几个阶段：

- **早期（2010-2013）**：核心引擎 frida-gum 的基础架构建立，支持 x86 平台上的函数拦截和代码追踪
- **成长期（2014-2016）**：加入 JavaScript 脚本支持，增加 iOS 和 Android 平台支持，Frida 开始被安全社区广泛关注
- **成熟期（2017-2020）**：API 趋于稳定，生态系统日渐丰富，objection 等基于 Frida 的工具相继出现
- **持续演进（2021 至今）**：引入 QuickJS 作为轻量级 JS 引擎选项，持续优化性能，支持更多平台场景

如今，Frida 已经是安全研究领域最重要的工具之一，在 GitHub 上拥有大量的星标和活跃的社区。

1.6 本书的内容规划

这本书将带你从宏观到微观，逐步深入 Frida 的源码世界。我们的阅读路线大致如下：

```
┌──────────────────────────────────────────────────┐
│                    本书阅读路线                    │
├──────────────────────────────────────────────────┤
│                                                  │
│  第一部分：准备篇                                 │
│    源码全景地图 -> 语言基础速览                    │
│                                                  │
│  第二部分：核心引擎 frida-gum                     │
│    Interceptor -> Stalker -> 内存操作             │
│    代码生成 -> JavaScript 绑定                    │
│                                                  │
│  第三部分：框架层 frida-core                      │
│    进程注入 -> 会话管理 -> 设备通信               │
│    Agent 机制 -> 跨平台适配                       │
│                                                  │
│  第四部分：绑定与工具                             │
│    Python 绑定 -> Node.js 绑定                    │
│    CLI 工具 -> 构建系统                           │
│                                                  │
└──────────────────────────────────────────────────┘
```

我们的目标不是逐行翻译源码，而是帮你建立起对 Frida 架构的整体理解。当你读完这本书后，面对 Frida 的任何一个源文件，你都应该能够快速定位它在整个系统中的位置和作用。

本章小结

- **动态插桩** 是在程序运行时注入代码以观察和修改其行为的技术
- **Frida** 是目前最流行的开源动态插桩框架，支持 Windows、macOS、Linux、iOS、Android 等平台
- Frida 的核心优势在于让你用 **JavaScript** 编写注入脚本，极大降低了使用门槛
- 阅读 Frida 源码不仅能理解工具本身的原理，还能学到跨平台工程设计、底层系统编程等宝贵知识
- Frida 项目从 2010 年发展至今，已经形成了成熟的架构和活跃的社区生态

思考与讨论

1. 除了 Frida，你还知道哪些动态插桩工具？它们和 Frida 的设计理念有什么不同？（提示：可以了解一下 DynamoRIO、Pin、Xposed）
2. 在你的工作或学习中，有没有遇到过”如果能在运行时修改程序行为就好了”的场景？
3. Frida 选择 JavaScript 作为脚本语言有什么优势和劣势？如果让你选择，你会用什么语言？

第2章：源码全景地图

> 当你第一次走进一座大型图书馆时，你会先做什么？是随手抓一本书就开始读，还是先看看楼层指引图？

阅读大型开源项目的源码也是一样。在深入任何一个具体模块之前，我们需要先画出一张”全景地图”。这张地图不需要精确到每一条街道，但它应该能告诉你：哪些是主干道，哪些是小巷，以及它们之间是怎么连接的。

2.1 仓库顶层结构

让我们先看看 Frida 仓库的顶层目录：

```
frida/
├── meson.build          # 顶层构建文件，整个项目的"总指挥"
├── meson.options        # 构建选项定义
├── Makefile             # Make 入口，实际会调用 Meson
├── configure            # 配置脚本
├── releng/              # 发布工程相关脚本和工具
│   ├── frida_version.py # 版本号管理
│   ├── deps.toml        # 依赖声明
│   ├── meson/           # Meson 构建系统辅助模块
│   └── ...
├── subprojects/         # 核心子项目，这是重点
│   ├── frida-gum/       # 底层插桩引擎
│   ├── frida-core/      # 框架核心层
│   ├── frida-python/    # Python 绑定
│   ├── frida-node/      # Node.js 绑定
│   ├── frida-tools/     # CLI 命令行工具
│   ├── frida-swift/     # Swift 绑定
│   ├── frida-go/        # Go 绑定
│   ├── frida-clr/       # .NET/CLR 绑定
│   └── frida-qml/       # Qt/QML 绑定
└── tools/               # 辅助脚本
```

顶层仓库本身并不包含太多代码逻辑，它更像是一个”指挥中心”，负责把各个子项目组织在一起。真正的代码都在 `subprojects/` 目录下。

2.2 子项目架构：分层设计

Frida 采用了清晰的分层架构。如果把它比作一栋大楼，那从地基到屋顶是这样的：

```
┌──────────────────────────────────────────────────────────┐
│                          用户脚本层                          │
```

```
│                (你写的 JavaScript 脚本)                    │
├──────────────────────────────────────────────────────────┤
│                      绑定层 (Bindings)                    │
│     frida-python | frida-node | frida-swift | ...         │
├──────────────────────────────────────────────────────────┤
│                      工具层 (Tools)                       │
│         frida-tools (frida, frida-trace, ...)             │
├──────────────────────────────────────────────────────────┤
│                      核心层 (Core)                        │
│                      frida-core                           │
│          进程注入 | 会话管理 | 设备通信 | Agent          │
├──────────────────────────────────────────────────────────┤
│                     引擎层 (Engine)                       │
│                      frida-gum                            │
│      Interceptor | Stalker | Memory | CodeWriter          │
├──────────────────────────────────────────────────────────┤
│                   JS 运行时 (GumJS)                       │
│               V8 引擎  |  QuickJS 引擎                    │
├──────────────────────────────────────────────────────────┤
│                   操作系统 / 硬件                         │
│          Windows | macOS | Linux | iOS | Android          │
└──────────────────────────────────────────────────────────┘
```

我来逐层解释这个架构。

2.3 frida-gum：地基中的地基

frida-gum 是整个 Frida 的最底层引擎，用纯 C 语言编写。如果 Frida 是一辆汽车，那 frida-gum 就是发动机。

```
frida-gum/
├── gum/                      # 核心 C 代码
│   ├── gum.c/gum.h          # 初始化入口
│   ├── guminterceptor.c     # 函数拦截器 (Interceptor)
│   ├── gumstalker.c         # 代码追踪器 (Stalker)
│   ├── gummemory.c          # 内存操作
│   ├── gumprocess.c         # 进程操作
│   ├── gummodule.c          # 模块（动态库）操作
│   ├── gumcodeallocator.c # 代码内存分配
│   ├── arch-arm/            # ARM 架构特定代码
│   ├── arch-arm64/          # ARM64 架构特定代码
│   ├── arch-x86/            # x86 架构特定代码
│   ├── arch-mips/           # MIPS 架构特定代码
│   ├── backend-darwin/      # macOS/iOS 平台后端
│   ├── backend-linux/       # Linux 平台后端
│   ├── backend-windows/     # Windows 平台后端
│   └── ...
├── bindings/
│   └── gumjs/               # JavaScript 绑定层
│       ├── gumv8*.cpp       # V8 引擎绑定
│       ├── gumquick*.c      # QuickJS 引擎绑定
```

```
│        └── runtime/        # JS 运行时脚本
├── libs/                    # 内部依赖库
├── tests/                   # 测试代码
└── vapi/                    # Vala API 描述文件
```

frida-gum 中最重要的三个组件是：

Interceptor（拦截器）：能够拦截任意函数的调用。当目标函数被调用时，Frida 可以在函数入口和出口处执行你的回调代码。它的实现原理是修改函数入口处的机器指令，跳转到 Frida 准备好的一段"蹦床代码"（trampoline）。

Stalker（追踪器）：能够追踪程序执行的每一条指令。它使用动态二进制翻译技术，在程序执行前重写代码块，插入追踪逻辑。想象你在一本书的每一行旁边都插入一个书签，记录你读过这一行。

Memory（内存操作）：提供读写目标进程内存、扫描内存模式等能力。

注意 `bindings/gumjs/` 这个目录，它是连接 C 引擎和 JavaScript 世界的桥梁。这里有两套实现：基于 V8 引擎的（`gumv8*.cpp`）和基于 QuickJS 的（`gumquick*.c`）。V8 就是 Chrome 浏览器和 Node.js 使用的那个 JavaScript 引擎，性能强大但体积较大；QuickJS 是一个轻量级的 JS 引擎，适合资源受限的场景。

2.4 frida-core：大脑和神经系统

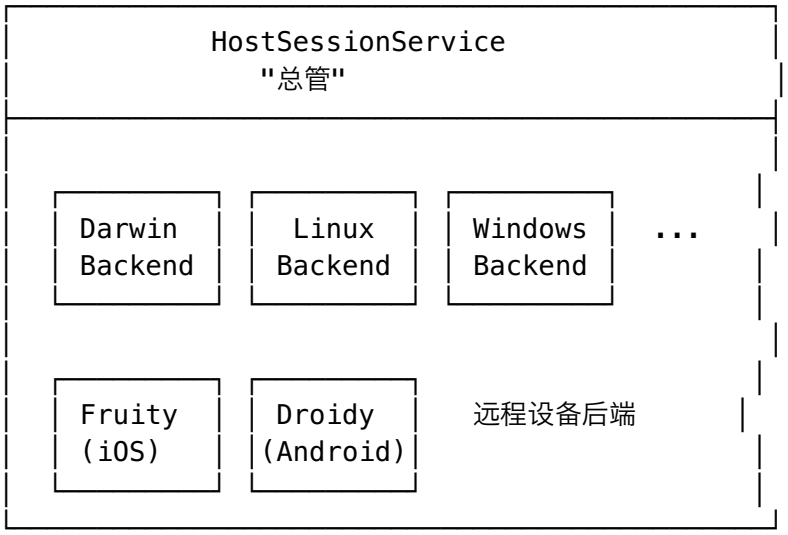
如果说 frida-gum 是肌肉，那 frida-core 就是大脑和神经系统。它负责进程管理、设备通信、代码注入等高层功能。frida-core 主要用 **Vala** 语言编写。

```
frida-core/
├── src/                         # 核心源码
│   ├── frida.vala               # 顶层 API 定义（DeviceManager 等）
│   ├── host-session-service.vala  # 会话管理服务
│   ├── control-service.vala     # 控制服务
│   ├── portal-service.vala      # Portal 服务
│   ├── agent-container.vala     # Agent 容器
│   ├── frida-glue.c             # C 语言粘合代码
│   ├── api/                     # 对外公开的 API
│   ├── compiler/                # 脚本编译器
│   ├── darwin/                  # macOS/iOS 平台实现
│   │   ├── darwin-host-session.vala
│   │   └── fruitjector.vala     # Darwin 平台注入器
│   ├── linux/                   # Linux 平台实现
│   │   ├── linux-host-session.vala
│   │   └── linjector.vala       # Linux 平台注入器
│   ├── windows/                 # Windows 平台实现
│   │   ├── windows-host-session.vala
│   │   └── winjector.vala       # Windows 平台注入器
│   ├── droidy/                  # Android 设备通信（ADB）
│   ├── fruity/                  # iOS 设备通信（USB/网络）
│   └── barebone/                # 裸机调试支持
├── server/                      # frida-server 入口
├── inject/                      # frida-inject 工具
```

```
├── portal/                     # frida-portal 服务
├── lib/                        # 库文件
├── tests/                      # 测试
└── vapi/                       # Vala API 描述
```

frida-core 中有几个关键概念需要理解：

HostSessionService：这是整个 frida-core 的”总管”。打开 `host-session-service.vala` 你会看到，它根据当前平台加载不同的后端：



Injector（注入器）：每个平台都有自己的注入器实现。Darwin 平台叫 `fruitjector`（水果注入器，因为苹果嘛），Linux 平台叫 `linjector`，Windows 平台叫 `winjector`。这些注入器负责把 Frida 的 Agent 动态库注入到目标进程中。

Agent：被注入到目标进程中的代码。它包含了 frida-gum 引擎和 JavaScript 运行时，是你的 JS 脚本实际运行的地方。

2.5 绑定层：语言的桥梁

Frida 提供了多种语言的绑定，让你可以用自己熟悉的语言来控制 Frida：

子项目	语言	用途
frida-python	Python	最常用的绑定，pip install frida
frida-node	Node.js	适合前端开发者和工具开发
frida-swift	Swift	适合 Apple 平台开发者
frida-go	Go	适合 Go 开发者
frida-clr	C#/.NET	适合 .NET 开发者
frida-qml	Qt/QML	适合 Qt 桌面应用开发

这些绑定本质上都是对 frida-core 提供的 C API 的封装。数据流是这样的：

```
┌──────────┐      ┌──────────┐      ┌──────────┐      ┌──────────┐
│ 你的     │      │ 语言     │      │ frida    │      │ frida    │
│ Python   │──────>│ 绑定     │──────>│ -core    │──────>│ -gum     │
```

```
│ 脚本        │      │ (ctypes) │      │  (Vala)  │      │    (C)    │
└────────────┘      └──────────┘      └──────────┘      └──────────┘
                                                              │
                                                              v
                                                        ┌──────────┐
                                                        │ 目标进程  │
                                                        │ 中运行的  │
                                                        │ JS 脚本   │
                                                        └──────────┘
```

2.6 frida-tools：开箱即用的工具

frida-tools 是一组用 Python 编写的命令行工具，是大多数人接触 Frida 的第一个入口：

- **frida**：交互式 REPL 环境，可以直接输入 JS 代码
- **frida-ps**：列出设备上的进程
- **frida-ls-devices**：列出可用的设备
- **frida-trace**：函数追踪工具，自动生成拦截脚本
- **frida-discover**：发现程序中有趣的函数
- **frida-kill**：终止进程

2.7 构建系统

Frida 使用 **Meson** 作为构建系统。Meson 是一个现代化的构建系统，以速度快和易用性著称。

顶层 `meson.build` 文件扮演着”总调度”的角色。打开这个文件，你会发现它的结构非常清晰：

```
# 项目定义，版本号从 releng/frida_version.py 动态获取
project('frida', 'c',
  version: run_command('releng' / 'frida_version.py').stdout().strip(),
)

# 第一步：构建 frida-gum（底层引擎）
subproject('frida-gum', default_options: gum_options)

# 第二步：构建 frida-core（核心框架）
subproject('frida-core', default_options: core_options)

# 第三步：按需构建各语言绑定
subproject('frida-python')   # 如果条件满足
subproject('frida-node')     # 如果条件满足
subproject('frida-tools')    # 如果条件满足
# ...
```

构建的顺序体现了依赖关系：先构建最底层的 gum，再构建依赖 gum 的 core，最后构建依赖 core 的各种绑定和工具。

`releng/` 目录包含了构建和发布相关的工具脚本，比如：
- `frida_version.py`：管理版本号
- `deps.toml`：声明第三方依赖
- `meson/`：Meson 构建的辅助模块
- `mkdevkit.py`：生成 devkit（开发包）

2.8 数据流全景图

当你运行一条简单的 Frida 命令时，数据在各个组件之间是这样流动的：

```
┌──────────────────────────────────────────────────────────────┐
│                          你的电脑                            │
│                                                              │
│   ┌──────────┐    ┌──────────────┐    ┌──────────────┐      │
│   │ frida    │    │ frida-python │    │ frida-core   │      │
│   │ CLI 工具 │──>│ Python 绑定  │──>│ (本地或远程  │      │
│   │          │    │              │    │  连接管理)   │      │
│   └──────────┘    └──────────────┘    └──────────────┘      │
│                                                  │           │
│                                                  │ USB/TCP   │
└──────────────────────────────────────────────────┼───────────┘
                                                   │
┌──────────────────────────────────────────────────┼───────────┐
│                          目标设备                │           │
│                                                  v           │
│   ┌──────────────┐    ┌──────────────────────────────┐      │
│   │ frida-server │    │          目标进程            │      │
│   │ (设备上的    │──>│  ┌──────────────────────┐    │      │
│   │  守护进程)   │    │  │ Agent (frida-agent)  │    │      │
│   └──────────────┘    │  │  ┌──────────┐         │    │      │
│                       │  │  │ frida-gum│         │    │      │
│                       │  │  │ +GumJS   │         │    │      │
│                       │  │  │ +你的 JS │         │    │      │
│                       │  │  └──────────┘         │    │      │
│                       │  └──────────────────────┘    │      │
│                       └──────────────────────────────┘      │
└──────────────────────────────────────────────────────────────┘
```

这张图展示了从你输入命令到代码在目标进程中执行的完整路径。理解这个数据流，你就对 Frida 的整体架构有了七八成的把握。

2.9 该从哪里开始读

面对这么多代码，我的建议是遵循”自顶向下”和”自底向上”相结合的策略：

1. **先读 frida-gum 的头文件**（`.h` 文件），理解底层引擎提供了哪些能力
2. **再读 frida-core 的 Vala 文件**，理解框架层如何组织和调度
3. **然后读 gumjs 绑定**，理解 C 引擎是怎么暴露给 JavaScript 的
4. **最后读绑定层**，理解用户 API 是怎么映射到内部实现的

不要试图一次读完所有代码。每次带着一个具体问题去读，比如”Interceptor.attach 到底是怎么实现的？“，然后沿着调用链追踪下去，效果远比漫无目的地浏览好得多。

本章小结

- Frida 仓库采用 **子项目架构**，核心代码分布在 `subprojects/` 下的各个子项目中
- **frida-gum**（C 语言）是最底层的插桩引擎，提供 Interceptor、Stalker、Memory 等核心能力
- **frida-core**（Vala 语言）是框架层，负责进程注入、会话管理、设备通信
- **绑定层**（Python/Node.js/Swift 等）将 frida-core 的 C API 封装为各语言的友好接口
- **GumJS** 是连接 C 引擎和 JavaScript 脚本世界的桥梁，支持 V8 和 QuickJS 两种引擎
- 构建系统使用 **Meson**，构建顺序遵循依赖关系：gum -> core -> bindings/tools
- 阅读源码建议带着具体问题，沿着调用链追踪

思考与讨论

1. Frida 为什么要把 gum 和 core 分成两个独立的子项目，而不是合并成一个？这种分层设计有什么好处？
2. 为什么 frida-core 选择用 Vala 而不是纯 C 来编写？你觉得这个选择有什么利弊？
3. 看看 `host-session-service.vala` 中的平台后端注册逻辑，想想如果要添加一个新平台的支持，需要改动哪些地方？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第3章：语言基础速览：C、Vala 与 JavaScript

> 如果有人告诉你，一个项目同时使用了 C、Vala、JavaScript、Python 和 C++，你的第一反应是什么？“这也太疯狂了吧！”但 Frida 就是这样一个项目，而且它把这些语言组合得相当优雅。

别担心，你不需要精通所有这些语言才能读懂 Frida 源码。这一章的目标很明确：给你配上刚好够用的”语言眼镜”，让你在后续章节中遇到代码时不至于一头雾水。

3.1 C 语言：Frida 的骨骼

frida-gum 这个最底层的引擎完全用 C 语言编写。C 语言在这里被选用的原因很简单：它是最接近硬件的高级语言，能精确控制内存布局和机器指令，而且几乎所有平台都能编译和运行 C 代码。

你需要了解的 C 基础

指针：在 Frida 源码中，指针无处不在。简单来说，指针就是一个”地址”。

```
// 一个整数
int value = 42;

// 一个指针，存储的是 value 的内存地址
int *ptr = &value;

// 通过指针读取值
printf("%d\n", *ptr);  // 输出 42

// gpointer 是 GLib 定义的通用指针类型
// 在 Frida 代码中你会经常看到它
gpointer address = (gpointer) 0x12345678;
```

把指针想象成一张写着门牌号的纸条。纸条本身不是房子，但拿着纸条你能找到房子。在 Frida 中，函数地址、内存地址、对象引用几乎都是通过指针来传递的。

结构体：C 语言用结构体来组织相关的数据。

```
// Frida 中典型的结构体定义风格
struct _GumInterceptor {
    GObject parent;          // 继承自 GObject
    GRecMutex mutex;         // 互斥锁
    GHashTable *function_contexts;  // 函数上下文哈希表
    // ...
};
```

结构体就像一个档案袋，把相关的信息装在一起。`GumInterceptor` 这个结构体包含了拦截器运作所需的全部状态。

函数指针：这是理解 Frida 回调机制的关键。

```
// 定义一个函数指针类型
typedef void (* GumInvocationCallback) (GumInvocationContext *context,
                                        gpointer user_data);

// 使用函数指针：注册一个回调
void my_callback(GumInvocationContext *context, gpointer user_data) {
    printf("函数被调用了！\n");
}

// 把 my_callback 当作参数传递
gum_interceptor_attach(interceptor, target_func, listener, NULL, 0);
```

函数指针就像是一个”任务委托”。你把一个函数交给 Frida，说”等那个函数被调用时，帮我执行这段代码”。这是整个 Interceptor 机制的基础。

GLib/GObject 类型系统

Frida 大量使用了 GLib 库，这是 GNOME 项目的基础库。你需要认识一些常见的 GLib 类型：

GLib 常见类型速查	
GLib 类型	含义
gboolean	布尔值 (TRUE/FALSE)
gint / guint	有符号/无符号整数
gsize	等同于 size_t
gpointer	通用指针 (void *)
gchar *	字符串
GArray	动态数组
GHashTable	哈希表
GBytes	不可变字节序列
GObject	所有 GObject 类的基类
GError	错误信息结构体

GObject 是 C 语言实现面向对象编程的一种约定。它不是语言特性，而是一套人为的规范。在 Frida 源码中你会看到这样的模式：

```
// 声明一个 GObject 类型
G_DECLARE_FINAL_TYPE (GumInterceptor, gum_interceptor,
                      GUM, INTERCEPTOR, GObject)

// 获取拦截器实例（单例模式）
GumInterceptor *interceptor = gum_interceptor_obtain();

// 调用方法（注意命名规范：模块_类名_方法名）
gum_interceptor_attach(interceptor, address, listener, data, flags);

// 释放引用
g_object_unref(interceptor);
```

这里有一个重要的命名约定：GLib/GObject 世界中的函数命名遵循 模块前缀_类名_方法名 的模式。所以当你看到 `gum_interceptor_attach` 时，可以解读为”gum 模块的 Interceptor 类的 attach 方法”。

3.2 Vala 语言：Frida 的血肉

Vala 可能是你第一次听说的语言，但在 Frida 中它扮演着至关重要的角色。frida-core 的大部分代码都是用 Vala 编写的。

Vala 是什么

Vala 是 GNOME 社区开发的一种编程语言，语法类似 C#/Java，但它有一个独特的特点：**Vala 编译器不生成机器码，而是先编译为 C 代码，再由 C 编译器编译为机器码。**

```
┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐
│ Vala     │     │ C 代码   │     │ 目标文件 │     │ 可执行   │
│ 源码     │──>│ (自动    │──>│ (.o)     │──>│ 文件     │
│ (.vala)  │     │  生成)   │     │          │     │          │
└──────────┘     └──────────┘     └──────────┘     └──────────┘
  valac 编译    gcc/clang        链接器
```

这意味着 Vala 代码最终会变成 C 代码，和 frida-gum 的 C 代码无缝对接。Frida 选择 Vala 的理由就很清楚了：既要高级语言的开发效率，又要和 C 底层完美融合。

Vala 语法快速上手

来看一段 frida-core 中真实存在的代码模式：

```
// 命名空间
namespace Frida {

    // 类定义，sealed 表示不可继承
    public sealed class DeviceManager : Object {

        // 信号（类似事件）
        public signal void added(Device device);
        public signal void removed(Device device);

        // 私有成员
        private Gee.ArrayList<Device> devices = new
Gee.ArrayList<Device>();

        // 构造函数
        public DeviceManager() {
            service = new
HostSessionService.with_default_backends();
        }

        // 异步方法（这是 Vala 的重要特性）
        public async void close(Cancellable? cancellable = null)
throws IOError {
            yield stop_service(cancellable);
        }

        // 同步包装
        public void close_sync(Cancellable? cancellable = null)
throws IOError {
            // ...
        }
```

```
    }
}
```

对比一下，如果你会 C#、Java 或 TypeScript，Vala 的语法应该相当亲切。但有几个 Vala 特有的概念需要注意：

async/yield：Vala 内置了异步编程支持。在 frida-core 中，几乎所有的 I/O 操作都是异步的。`async` 和 `yield` 就像餐厅点餐：你发起一个异步操作，然后 `yield` 说”做好了再叫我”，线程就可以去处理别的事了。

```
public async Device get_device(string id) throws Error {
    yield ensure_service_started();   // 等待服务启动
    foreach (var device in devices) {
        if (device.id == id)
            return device;
    }
    throw new Error.INVALID_ARGUMENT("Device not found");
}
```

可空类型与属性：类型后面加 `?` 表示可以为 null（`Cancellable? cancellable = null`）。属性用 `{ get; construct; }` 语法声明，类似 C#。所有 Vala 类默认继承 GLib.Object，自动获得引用计数和信号机制。

编译条件

frida-core 中大量使用了条件编译来处理跨平台差异：

```
private void add_local_backends() {
#if WINDOWS
    add_backend(new WindowsHostSessionBackend());
#endif
#if DARWIN
    add_backend(new DarwinHostSessionBackend());
#endif
#if LINUX
    add_backend(new LinuxHostSessionBackend());
#endif
}
```

这和 C 语言的 `#ifdef` 很像，根据编译目标平台选择不同的代码路径。

3.3 JavaScript：Frida 的灵魂

当你写 Frida 脚本时，你用的就是 JavaScript。但这个 JavaScript 运行环境有点特殊，它不是浏览器，也不是 Node.js，而是 Frida 自己搭建的一个 JS 运行时。

Frida 的 JavaScript API

Frida 在标准 JavaScript 的基础上提供了一系列全局对象和函数：

```
// Interceptor: 函数拦截
Interceptor.attach(ptr('0x12345678'), {
    onEnter(args) {
        // args[0], args[1] 是函数参数
        console.log('参数1:', args[0].toInt32());
    },
    onLeave(retval) {
        // retval 是返回值
        console.log('返回值:', retval.toInt32());
    }
});

// Module: 模块（动态库）操作
var base = Module.getBaseAddress('libc.so');
var exports = Module.enumerateExports('libc.so');

// Memory: 内存读写
var value = Memory.readU32(ptr('0x12345678'));
Memory.writeUtf8String(ptr('0x12345678'), 'hello');

// Process: 进程信息
var modules = Process.enumerateModules();
var threads = Process.enumerateThreads();
```

JS 引擎：V8 与 QuickJS

Frida 支持两种 JavaScript 引擎，这也反映在源码结构中：

```
┌──────────────────────────────────────────────────────┐
│                    GumJS 桥接层                      │
├──────────────────────────┬───────────────────────────┤
│                          │                           │
│   ┌──────────────────┐   │   ┌──────────────────┐    │
│   │   V8 后端        │   │   │ QuickJS 后端     │    │
│   │ gumv8*.cpp       │   │   │ gumquick*.c      │    │
│   │                  │   │   │                  │    │
│   │ 功能全面         │   │   │ 体积小巧         │    │
│   │ 性能强大         │   │   │ 启动快速         │    │
│   │ 体积较大         │   │   │ 资源占用少       │    │
│   └──────────────────┘   │   └──────────────────┘    │
│                          │                           │
└──────────────────────────┴───────────────────────────┘
```

在源码中，每个 JS API 都有两套实现。比如 Interceptor 的 JS 绑定： - V8 版本：`gumv8interceptor.cpp` - QuickJS 版本：`gumquickinterceptor.c`

它们提供完全相同的 JavaScript API，只是底层引擎不同。

NativePointer 与 NativeFunction

在 Frida 的 JavaScript 世界中，有两个核心类型：

```
// NativePointer：表示一个内存地址
var addr = ptr('0x7fff12345678');
var value = addr.readU32();       // 读取该地址的 32 位无符号整数

// NativeFunction：把一个地址当作函数来调用
var open = new NativeFunction(
    Module.getExportByName(null, 'open'),
    'int', ['pointer', 'int']   // 返回值类型, 参数类型列表
);
var fd = open(Memory.allocUtf8String('/etc/hosts'), 0);
```

`NativePointer` 就像一个万能遥控器，指向内存中的任何位置。`NativeFunction` 则更进一步，它不仅知道地址在哪，还知道怎么调用那个地址上的函数。

3.4 三种语言如何协作

现在让我们把三种语言串起来，看看当你执行一行 `Interceptor.attach(...)` 时，代码是如何跨越语言边界的：

```
┌──────────────────────────────────────────────────────┐
│ 第1层：JavaScript（你写的脚本）                        │
│                                                      │
│   Interceptor.attach(target, callbacks)              │
│          │                                           │
│          v                                           │
├──────────────────────────────────────────────────────┤
│ 第2层：GumJS 绑定（C/C++）                             │
│                                                      │
│   gumv8interceptor.cpp 或 gumquickinterceptor.c      │
│   解析 JS 参数，转换为 C 类型                          │
│          │                                           │
│          v                                           │
├──────────────────────────────────────────────────────┤
│ 第3层：Gum 引擎（C）                                   │
│                                                      │
│   guminterceptor.c                                   │
│   gum_interceptor_attach(self, addr, listener)       │
│   修改目标函数的机器指令，建立蹦床跳转                  │
│          │                                           │
│          v                                           │
├──────────────────────────────────────────────────────┤
│ 第4层：平台相关代码（C）                               │
│                                                      │
│   修改内存页权限、写入跳转指令                         │
│   处理缓存一致性（ARM 平台）                           │
└──────────────────────────────────────────────────────┘
```

在管理层面，Vala 代码负责更高级别的协调：你的 Python 脚本调用 frida-core（Vala）的 DeviceManager/Device/Session 来管理进程附加和脚本创建，frida-core 再把 JS 脚本送入目标进程的 Agent 中，Agent 加载 GumJS 运行时并执行你的脚本。

所以三种语言各司其职：

- **C**：干脏活累活，直接和 CPU 指令、内存打交道
- **Vala**：做管理和协调，处理网络通信、进程管理等中间层逻辑
- **JavaScript**：面向用户，提供灵活友好的脚本接口

这就像一个公司：C 是一线工人，在车间里操作机器；Vala 是中层管理者，负责调度和协调；JavaScript 是面向客户的销售代表，提供友好的服务界面。

3.5 阅读源码的实用技巧

知道了这三种语言的基本特征后，这里再给你几个阅读 Frida 源码时的实用技巧：

1. **看函数命名推断功能。** `gum_interceptor_attach` 就是 GumInterceptor 的 attach 方法，命名规范让你几乎不需要注释就能理解代码意图。
2. **从头文件入手。** 先看 `.h` 了解接口，再看 `.c` 了解实现。头文件是”菜单”，源文件是”厨房”。
3. **关注 `_init` 和 `_dispose`。** GObject 的初始化和销毁函数会告诉你一个对象管理着哪些资源。
4. **用 `vapi` 文件作为索引。** vapi 描述了 C 库的 Vala 绑定接口，是理解跨语言关系的好起点。
5. **善用搜索跨越语言边界。** 从 JS 的 `Interceptor.attach` 搜索 `gumv8interceptor`，再搜索 `gum_interceptor_attach`，就能沿着调用链从顶到底。

本章小结

- **C 语言** 用于 frida-gum 底层引擎，需要理解指针、结构体、函数指针和 GLib/GObject 类型系统
- **Vala 语言** 用于 frida-core 框架层，语法类似 C#，编译时先转换为 C 代码，天然与 GObject 集成
- **JavaScript** 是用户编写注入脚本的语言，Frida 提供了 V8 和 QuickJS 两种引擎的支持
- 三种语言各有分工：C 负责底层操作，Vala 负责管理协调，JavaScript 负责用户接口
- GumJS 绑定层（`gumv8*.cpp` 和 `gumquick*.c`）是连接 JavaScript 和 C 引擎的桥梁
- 阅读源码时，善用命名规范、头文件和 vapi 文件来快速定位代码

思考与讨论

1. Vala 编译为 C 再编译为机器码，这种”两步编译”的方式和 TypeScript 编译为 JavaScript 有什么相似之处？你觉得这种设计的核心优势是什么？
2. Frida 同时支持 V8 和 QuickJS 两种 JS 引擎，在什么场景下你会选择 QuickJS 而非 V8？

3. 如果你想追踪 `Interceptor.attach` 的完整执行路径，从 JavaScript 到最终修改目标函数的机器指令，你会怎么规划你的源码阅读路线？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第4章：程序的大门——入口与启动流程

> 你有没有想过，当你在终端敲下 `frida-server` 回车的那一刻，到它真正开始监听连接，中间到底发生了什么？这就好比你按下汽车的启动按钮，从点火到发动机平稳运转，中间经历了一整套精密的启动序列。今天我们就来拆解这个”点火”过程。

4.1 为什么要关注入口？

很多同学拿到一个大型项目的源码，第一反应是”从哪里开始看”。这个问题的答案几乎永远是：**从 main() 函数开始**。

main() 就像一栋大楼的正门。你不需要先把整栋楼的蓝图看完，只要走进正门，跟着走廊的指引，自然能到达你想去的地方。Frida 的两个核心可执行程序——frida-server 和 frida-inject——都有各自的 main()，它们的入口逻辑清晰而典型。

4.2 frida-server 的 main() 入口

frida-server 的入口代码位于 `frida-core/server/server.vala`。整个文件用 Vala 语言编写，被包裹在 `namespace Frida.Server` 中。我们来看简化后的核心流程：

```
private static int main (string[] args) {
    // 第一步：初始化运行环境
    Environment.init ();

    // 第二步：解析命令行参数
    var ctx = new OptionContext ();
    ctx.add_main_entries (option_entries, null);
    ctx.parse (ref args);

    // 第三步：配置日志
    Environment.set_verbose_logging_enabled (verbose);

    // 第四步：构建端点参数（监听地址、证书、认证等）
    var endpoint_params = new EndpointParameters (
        listen_address, 0, certificate, origin, auth_service,
asset_root
    );
```

```
    // 第五步：配置运行环境
    Environment.configure ();

    // 第六步：启动应用主循环
    return run_application (device_id, endpoint_params, options,
on_ready);
}
```

这里有几个值得注意的设计：

命令行参数的定义

frida-server 支持丰富的命令行参数，定义在一个 `OptionEntry` 数组中：

```
--listen, -l     指定监听地址
--certificate    启用 TLS 加密
--token          启用认证
--daemonize, -D 后台运行（守护进程模式）
--verbose, -v    开启详细日志
--asset-root     提供静态文件服务
```

这些参数不是摆设，它们决定了 frida-server 的运行模式。比如在远程调试场景下，你几乎一定会用到 `--listen` 和 `--certificate`。

Darwin 平台的特殊处理

在 macOS/iOS 上，main() 有一段特殊逻辑：

```
#if DARWIN
    var worker = new Thread<int> ("frida-server-main-loop", () =>
{
        var exit_code = run_application (...);
        _stop_run_loop ();
        return exit_code;
    });
    _start_run_loop ();
    return worker.join ();
#else
    return run_application (...);
#endif
```

为什么要这样做？因为 macOS/iOS 的主线程有一个系统级的 RunLoop，很多系统 API（比如 XPC 通信）要求在主线程的 RunLoop 上工作。所以 Frida 把自己的逻辑放到工作线程，把主线程让给系统 RunLoop。这就像在公司里，前台（主线程）得负责接待访客（系统事件），真正干活的人（业务逻辑）在后面的工位上。

4.3 Application 类：真正的启动核心

`run_application` 函数创建了一个 `Application` 对象，这才是核心：

```
┌──────────────────────────────────────────────────┐
│                    Application                   │
├──────────────────────────────────────────────────┤
│  device_id        : string?                      │
│  endpoint_params  : EndpointParameters           │
│  options          : ControlServiceOptions        │
│  manager          : DeviceManager?               │
│  service          : ControlService?              │
│  loop             : MainLoop                     │
├──────────────────────────────────────────────────┤
│  run()    -> 启动主循环                          │
│  start()  -> 异步初始化服务                      │
│  stop()   -> 优雅关闭                            │
└──────────────────────────────────────────────────┘
```

`run()` 方法的实现非常典型——GLib 事件循环模式：

```
public int run () {
    Idle.add (() => {
        start.begin ();  // 在主循环启动后异步开始初始化
        return false;
    });
    loop.run ();          // 阻塞在这里，直到 loop.quit()
    return exit_code;
}
```

这个模式值得记住：先用 `Idle.add` 注册一个”空闲时执行”的回调，然后启动事件循环。这样 `start()` 会在事件循环跑起来之后才执行，保证所有异步操作都有事件循环支撑。

4.4 异步启动序列

`start()` 是一个 `async` 方法，它完成了核心的初始化工作：

```
┌──────────────────────────────────────────────────┐
│              frida-server 启动序列               │
├──────────────────────────────────────────────────┤
│                                                  │
│  1. main() 被调用                                │
│     │                                            │
│  2. Environment.init()  初始化运行时             │
│     │                                            │
│  3. 解析命令行参数                               │
│     │                                            │
│  4. Environment.configure()  配置环境            │
│     │                                            │
│  5. 创建 Application 对象                        │
│     │                                            │
│  6. 注册信号处理 (SIGINT, SIGTERM)               │
│     │                                            │
│  7. Application.run() -> 启动 MainLoop           │
│     │                                            │
```

```
│  8. Application.start() 异步执行:                │
│     │                                            │
│     ├─ 若指定 device_id:                         │
│     │   ├─ 创建 DeviceManager (非本地后端)       │
│     │   ├─ 获取指定设备                          │
│     │   └─ 创建 ControlService.with_device()     │
│     │                                            │
│     └─ 若未指定 device_id:                       │
│         └─ 创建 ControlService (本地模式)        │
│     │                                            │
│  9. ControlService.start()  开始监听             │
│     │                                            │
│  10. 发送 ready 信号                             │
│                                                  │
└──────────────────────────────────────────────────┘
```

注意第 8 步的分支：frida-server 可以服务本地设备，也可以作为”代理”服务远程设备。当你传入 `--device` 参数时，它会创建一个 `DeviceManager`，通过它找到远程设备，然后为那个设备创建服务。

4.5 守护进程模式

frida-server 支持 `--daemonize` 参数来实现守护进程模式。这段代码是经典的 Unix daemon 实现：

```
if (daemonize) {
    // 1. 创建管道用于父子进程通信
    Unix.open_pipe (sync_fds, 0);

    // 2. fork 出子进程
    var pid = Posix.fork ();
    if (pid != 0) {
        // 父进程：等待子进程报告状态，然后退出
        sync_in.read (status);
        return status[0];
    }

    // 子进程继续运行
    on_ready = (success) => {
        Posix.setsid ();            // 创建新会话
        // 重定向 stdin/stdout/stderr 到 /dev/null
        Posix.dup2 (null_fd, STDIN_FILENO);
        // ... 通知父进程启动结果
    };
}
```

这个模式就像一个员工（父进程）培训了一个接班人（子进程），确认接班人能独立工作后，自己就下班了。管道（pipe）就是他们之间传递”我准备好了”这个消息的对讲机。

4.6 frida-inject 的入口

frida-inject 的入口位于 `frida-core/inject/inject.vala`，它的 main() 结构与 frida-server 类似，但目的不同——它是一个”一次性”工具，用于向目标进程注入脚本：

```
private static int main (string[] args) {
    Posix.setsid ();           // 创建独立会话
    Environment.init ();       // 初始化

    // 解析参数：目标 PID、脚本路径、运行时选择等
    ctx.parse (ref args);

    // 创建 Application 并运行
    application = new Application (
        device_id, spawn_file, target_pid, target_name,
        options, script_path, script_source, script_runtime, ...
    );
    return application.run ();
}
```

frida-inject 的 Application.start() 流程更加直接：

```
┌──────────────────────────────────────────────────────┐
│              frida-inject 启动序列                    │
├──────────────────────────────────────────────────────┤
│                                                      │
│  1. 创建 DeviceManager                               │
│     │                                                │
│  2. 获取设备（本地或指定 ID）                          │
│     │                                                │
│  3. 确定目标进程:                                     │
│     ├─ --file: spawn 新进程                           │
│     ├─ --name: 按名称查找进程                          │
│     └─ --pid:  直接使用指定 PID                        │
│     │                                                │
│  4. device.attach(pid)  附加到目标进程                 │
│     │                                                │
│  5. 创建 ScriptRunner 并加载脚本                       │
│     │                                                │
│  6. 如果是 spawn 模式，resume 恢复进程                 │
│     │                                                │
│  7. 如果 --eternalize，脚本永驻后退出                  │
│                                                      │
└──────────────────────────────────────────────────────┘
```

4.7 DeviceManager 的初始化

无论是 frida-server 还是 frida-inject，都依赖 `DeviceManager`。它是 Frida 客户端 API 的入口点，定义在 `frida-core/src/frida.vala`：

```
public sealed class DeviceManager : Object {
    private HostSessionService? service;
    private Gee.ArrayList<Device> devices;

    public DeviceManager () {
        // 默认构造：包含所有后端（本地 + 远程）
        service = new HostSessionService.with_default_backends ();
    }

    public DeviceManager.with_nonlocal_backends_only () {
        // 仅非本地后端（USB、网络等）
        service = new
HostSessionService.with_nonlocal_backends_only ();
    }
}
```

`HostSessionService` 是后端管理的核心，它根据编译条件加载不同的后端：

```
┌──────────────────────────────────────────────┐
│            HostSessionService                 │
├──────────────────────────────────────────────┤
│                                               │
│  本地后端 (add_local_backends):               │
│  ├─ WindowsHostSessionBackend  (Windows)      │
│  ├─ DarwinHostSessionBackend   (macOS/iOS)    │
│  ├─ LinuxHostSessionBackend    (Linux)        │
│  └─ FreebsdHostSessionBackend  (FreeBSD)      │
│                                               │
│  远程后端 (add_nonlocal_backends):            │
│  ├─ FruityHostSessionBackend   (iOS USB)      │
│  ├─ DroidyHostSessionBackend   (Android USB)  │
│  ├─ SocketHostSessionBackend   (TCP 网络)     │
│  └─ BareboneHostSessionBackend (裸机调试)     │
│                                               │
└──────────────────────────────────────────────┘
```

这个设计非常灵活。在 iOS 设备上运行的 frida-server 不需要 Fruity 或 Droidy 后端（因为它自己就是目标设备），所以这些后端通过条件编译 `#if !IOS && !ANDROID` 被排除了。

4.8 信号处理与优雅退出

两个程序都注册了 SIGINT 和 SIGTERM 处理器：

```
Posix.signal (Posix.Signal.INT, (sig) => {
    application.stop ();
});
```

`stop()` 方法通过 `Idle.add` 把停止操作调度到主循环中执行，确保线程安全。停止过程是有序的：先取消正在进行的 IO 操作，然后停止服务，关闭 DeviceManager，最后退出主循环。

这就像关闭一家餐厅：先停止接待新客人（取消 IO），等现有客人吃完（停止服务），打扫收拾（清理资源），最后锁门走人（退出循环）。

4.9 Environment.init() 与 Environment.configure()

你可能注意到 main() 中有两个环境相关的调用，它们被分开了：

```
Environment.init ();          // 尽早调用，做最基本的初始化
// ... 解析参数 ...
Environment.configure ();     // 在参数解析之后调用，应用配置
```

这两个函数都是 `extern`，意味着它们的实现在 C 代码中。分成两步的原因是：`init()` 需要在参数解析之前完成（因为参数解析本身可能依赖某些基础设施），而 `configure()` 需要在参数解析之后执行（因为配置可能依赖参数值）。

本章小结

- frida-server 和 frida-inject 都遵循经典的 GLib 应用模式：解析参数 -> 创建 Application -> 运行主循环
- frida-server 是一个长期运行的服务，支持守护进程模式和 TLS 加密
- frida-inject 是一个一次性工具，用于向目标进程注入脚本
- DeviceManager 通过 HostSessionService 管理多种后端，支持本地和远程设备
- Darwin 平台需要特殊的线程处理，因为系统 RunLoop 必须在主线程运行
- 优雅退出通过信号处理和有序的异步清理实现

思考题

1. 为什么 frida-server 的 `start()` 要通过 `Idle.add` 延迟到主循环启动之后才执行，而不是直接在 `run()` 中调用？如果直接调用会发生什么？

2. frida-inject 在 main() 的第一行就调用了 `Posix.setsid()`，而 frida-server 是在 fork 之后的子进程中才调用。为什么两者的时机不同？

3. 假设你要为 Frida 添加一个新的命令行参数 `--max-connections`，限制 frida-server 的最大并发连接数，你会在启动流程的哪个环节加入这个逻辑？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第5章：Gum：插桩引擎的心脏

> 如果 Frida 是一辆赛车，那 Gum 就是它的发动机。你可以换轮胎（换前端工具），可以改车身（换通信协议），但发动机不能换——因为真正让代码在目标进程里”跑起来”、让函数调用被拦截、让执行流被追踪的，全靠 Gum。今天我们打开引擎盖，看看里面到底有什么。

5.1 Gum 是什么？

Gum 是 Frida 的底层插桩引擎，项目名称叫 `frida-gum`。它是一个纯 C 语言库，提供了在运行时修改、监控、追踪程序执行的所有基础能力。

打个比方：如果你想在一条高速公路上设置检查站（拦截函数调用）、安装摄像头（追踪代码执行路径）、或者修改路标（替换函数实现），Gum 就是那个负责施工的工程队。它知道每种路面（CPU 架构）该怎么施工，知道每种交通规则（操作系统）该怎么遵守。

从 `gum.h` 头文件可以清楚地看到 Gum 的全貌：

```
// gum.h - Gum 的公开头文件，包含了所有子系统
#include <gum/guminterceptor.h>    // 函数拦截
#include <gum/gumstalker.h>        // 代码追踪
#include <gum/gummemory.h>         // 内存操作
#include <gum/gummodule.h>         // 模块管理
#include <gum/gumprocess.h>        // 进程信息
#include <gum/gumbacktracer.h>     // 调用栈回溯
#include <gum/gumexceptor.h>       // 异常处理
#include <gum/gumcloak.h>          // 隐藏自身
// ... 还有更多
```

5.2 Gum 的架构全景

Gum 的架构可以分为四个层次：

```
┌──────────────────────────────────────────────────────────┐
│                    GumJS (JavaScript 绑定)                 │
│              将 C API 暴露为 JS 可调用的接口               │
├──────────────────────────────────────────────────────────┤
│                    Gum 核心子系统                          │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐                  │
│  │Interceptor│ │ Stalker  │ │ Memory   │                  │
│  │ 函数拦截  │ │ 代码追踪 │ │ 内存操作 │                  │
│  └──────────┘ └──────────┘ └──────────┘                  │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐                  │
│  │ Module   │ │ Process  │ │ApiResolver│                  │
│  │ 模块管理 │ │ 进程信息 │ │ 符号查找 │                  │
│  └──────────┘ └──────────┘ └──────────┘                  │
├──────────────────────────────────────────────────────────┤
│                    平台抽象层                              │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐                    │
│  │Windows│ │Darwin│ │Linux │ │ QNX  │                    │
```

```
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘      │
├──────────────────────────────────────────────────────────────┤
│                      架构特定代码生成                          │
│                                                              │
│   ┌─────┐    ┌─────┐    ┌─────┐    ┌─────┐    ┌─────┐       │
│   │x86  │    │x64  │    │ARM32│    │ARM64│    │MIPS │       │
│   └─────┘    └─────┘    └─────┘    └─────┘    └─────┘       │
└──────────────────────────────────────────────────────────────┘
```

最底层是架构特定的代码生成器，每种 CPU 架构都有自己的 Writer（代码写入器），比如 `GumX86Writer`、`GumArm64Writer`。往上是平台抽象层，处理不同操作系统的内存管理、进程管理差异。再往上是核心子系统，提供高层的插桩能力。最顶层是 GumJS，把这些 C API 包装成 JavaScript 接口。

5.3 Interceptor：函数拦截器

Interceptor 是 Gum 中使用频率最高的组件。当你在 Frida 脚本里写 `Interceptor.attach(addr, {...})` 时，底层就是它在工作。

核心 API

从 `guminterceptor.h` 可以看到三组核心操作：

```
// 1. 附加监听器——在函数入口/出口插入回调
GumAttachReturn gum_interceptor_attach (
    GumInterceptor * self,
    gpointer function_address,       // 目标函数地址
    GumInvocationListener * listener, // 监听器
    gpointer listener_function_data,  // 用户数据
    GumAttachFlags flags
);

// 2. 替换函数——用你的实现替换原函数
GumReplaceReturn gum_interceptor_replace (
    GumInterceptor * self,
    gpointer function_address,       // 原函数地址
    gpointer replacement_function,   // 替换函数
    gpointer replacement_data,
    gpointer * original_function     // 返回原函数指针
);

// 3. 事务机制——批量修改，减少性能开销
void gum_interceptor_begin_transaction (GumInterceptor * self);
void gum_interceptor_end_transaction (GumInterceptor * self);
```

工作原理（简化版）

Interceptor 的原理就像在书页上贴便签纸：

```
原始函数:                          拦截后:

┌──────────────┐                   ┌──────────────┐
│ push rbp     │                   │ jmp trampoline│ <-- 开头被改写
```

```
│ mov rbp, rsp │            │ ...被覆盖... │
│ sub rsp, 0x20│            │ sub rsp, 0x20│
│ ...          │            │ ...          │
│ ret          │            │ ret          │
└──────────────┘            └──────────────┘
                                  │
                                  v
                            ┌──────────────┐
                            │ trampoline   │
                            │ 调用 onEnter │
                            │ 执行原始指令 │
                            │ 跳回原函数   │
                            │ ...          │
                            │ 调用 onLeave │
                            └──────────────┘
```

函数开头的几条指令被替换为一个跳转，跳到 Frida 生成的 trampoline 代码。trampoline 负责调用你的回调函数（onEnter），然后执行被覆盖的原始指令，再跳回原函数继续执行。函数返回时，类似的机制触发 onLeave 回调。

事务机制

注意 `begin_transaction` 和 `end_transaction` 这对 API。当你需要同时拦截多个函数时，把它们放在一个事务里可以避免每次修改都刷新 CPU 指令缓存。这就像搬家时，你不会每搬一个箱子就锁一次门，而是等所有箱子都搬完再锁。

5.4 Stalker：代码追踪器

如果说 Interceptor 是在路口设检查站，那 Stalker 就是派了一个侦探全程跟踪目标。它能追踪每一条被执行的指令。

核心概念

Stalker 使用**动态重编译**技术：它不是直接执行原始代码，而是把原始代码复制一份，在副本中插入追踪逻辑，然后执行副本。

```
// 跟踪当前线程
void gum_stalker_follow_me (
    GumStalker * self,
    GumStalkerTransformer * transformer,  // 可选的代码变换器
    GumEventSink * sink                   // 事件接收器
);

// 跟踪其他线程
void gum_stalker_follow (
    GumStalker * self,
    GumThreadId thread_id,
    GumStalkerTransformer * transformer,
```

```
    GumEventSink * sink
);
```

架构适配

从 `gumstalker.h` 中的 `GumStalkerWriter` 联合体可以看出 Stalker 需要为每种架构生成不同的代码：

```
union _GumStalkerWriter {
    gpointer instance;
    GumX86Writer * x86;
    GumArmWriter * arm;
    GumThumbWriter * thumb;
    GumArm64Writer * arm64;
    GumMipsWriter * mips;
};
```

每种 Writer 都知道如何为对应的 CPU 架构生成机器码。这是 Gum 跨平台能力的根基——同一套抽象接口，底层由架构特定的代码生成器实现。

Transformer：可编程的代码变换

Stalker 最强大的特性之一是 Transformer——你可以在代码被重编译时修改它：

```
// Transformer 接口
struct _GumStalkerTransformerInterface {
    void (* transform_block) (
        GumStalkerTransformer * self,
        GumStalkerIterator * iterator,
        GumStalkerOutput * output
    );
};
```

通过 iterator，你可以逐条遍历基本块中的指令，决定保留、修改或插入新的指令。这给了你对代码执行流的完全控制权。

5.5 Memory：内存操作

内存操作是插桩的基础。Gum 提供了跨平台的内存读写、扫描、分配、保护属性修改等功能：

```
┌──────────────────────────────────────────┐
│              Memory 子系统                 │
├──────────────────────────────────────────┤
│                                          │
│  gum_memory_read      读取内存            │
│  gum_memory_write     写入内存            │
│  gum_memory_scan      扫描内存模式        │
│  gum_memory_allocate  分配可执行内存      │
│  gum_memory_protect   修改保护属性        │
│                                          │
```

```
│  GumMemoryMap          内存映射查询        │
│  GumMemoryAccessMonitor   访问监控         │
│                                           │
└───────────────────────────────────────────┘
```

其中 `gum_memory_allocate` 特别值得关注——它分配的内存通常需要有执行权限，因为 Interceptor 和 Stalker 生成的 trampoline 代码需要放在这里。不同操作系统对可执行内存的管理策略不同（比如 iOS 的 JIT 限制），Gum 在平台抽象层统一处理了这些差异。

5.6 Module 和 Process：运行时信息

Module 子系统提供了对加载模块（动态库）的查询能力：

```
// 枚举所有已加载模块
void gum_module_enumerate (GumFoundModuleFunc func, gpointer
            user_data);

// 枚举模块导出的符号
void gum_module_enumerate_exports (const gchar * module_name,
   GumFoundExportFunc func, gpointer user_data);
```

Process 子系统提供进程级别的信息：

```
// 枚举线程
void gum_process_enumerate_threads (GumFoundThreadFunc func,
   gpointer user_data);

// 枚举内存范围
void gum_process_enumerate_ranges (GumPageProtection prot,
   GumFoundRangeFunc func, gpointer user_data);
```

这些信息对于插桩至关重要——你需要知道目标函数在哪个模块里、地址是多少、有哪些线程在运行，才能正确地进行拦截和追踪。

5.7 Cloak：隐藏自身

一个有趣的子系统是 `GumCloak`。Frida 作为插桩工具注入到目标进程后，它自身的线程、内存映射、文件描述符都会在进程中可见。如果目标程序有反调试检测，就可能发现 Frida 的存在。

Cloak 的作用就是维护一个”隐藏列表”，记录哪些资源属于 Frida 自身。当目标程序枚举线程或内存映射时，Gum 可以过滤掉这些资源。这就像间谍行动中的”清除痕迹”。

5.8 Gum 的初始化

Gum 提供了两种初始化方式：

```
// 方式一：标准初始化（用于独立使用 Gum）
void gum_init (void);
```

```
void gum_shutdown (void);
void gum_deinit (void);

// 方式二：嵌入式初始化（用于 Frida Agent 等场景）
void gum_init_embedded (void);
void gum_deinit_embedded (void);
```

嵌入式初始化是为 Frida Agent（注入到目标进程中的那部分代码）设计的。它会做一些额外的工作，比如确保 GLib 的类型系统在目标进程中正确初始化。

还有一组 fork 相关的函数值得注意：

```
void gum_prepare_to_fork (void);
void gum_recover_from_fork_in_parent (void);
void gum_recover_from_fork_in_child (void);
```

当目标进程调用 `fork()` 时，Gum 需要正确处理子进程中的状态。这些函数确保 Interceptor 和 Stalker 的内部数据结构在 fork 后保持一致。

5.9 Gum C API 与 GumJS 的关系

普通用户通过 JavaScript 使用 Frida，而 GumJS 就是连接 JavaScript 世界和 Gum C API 的桥梁：

```
┌──────────────────────────────────────────┐
│        用户 JavaScript 脚本               │
│   Interceptor.attach(addr, {              │
│     onEnter(args) { ... }                 │
│   });                                     │
├──────────────────────────────────────────┤
│              GumJS 绑定层                 │
│   将 JS 对象/回调转换为 C 结构/函数指针    │
│   管理 JS 运行时 (QuickJS 或 V8)          │
├──────────────────────────────────────────┤
│              Gum C API                    │
│   gum_interceptor_attach(...)             │
│   实际执行底层插桩操作                     │
└──────────────────────────────────────────┘
```

GumJS 支持两种 JavaScript 引擎：QuickJS（轻量级，适合嵌入式场景）和 V8（性能更好，功能更全）。用户可以通过 `--runtime qjs` 或 `--runtime v8` 来选择。

GumJS 做了大量的”翻译”工作：把 JavaScript 的回调函数包装成 C 的函数指针，把 C 的结构体转换成 JavaScript 对象，处理两个世界之间的内存管理和生命周期。

5.10 Gum 的设计哲学

回顾 Gum 的整体设计，有几个明显的特征：

1. **单例模式**：`gum_interceptor_obtain()` 返回的是进程级别的唯一实例，因为同一个进程里只能有一套拦截管理。

2. **架构与平台分离**：CPU 架构相关的代码（代码生成）和操作系统相关的代码（内存管理）被清晰地分开，使得移植到新平台只需要实现对应的那一层。

3. **最小化侵入**：Gum 尽量减少对目标进程的影响，Cloak 机制就是这一理念的体现。

4. **性能优先**：事务机制、Stalker 的信任阈值（trust threshold，控制代码缓存的复用策略）等设计都是为了在保证正确性的前提下最大化性能。

本章小结

- Gum 是 Frida 的核心插桩引擎，用纯 C 编写，提供跨平台的运行时代码修改能力
- Interceptor 通过改写函数开头的指令实现函数拦截，支持 attach（监听）和 replace（替换）两种模式
- Stalker 使用动态重编译技术追踪代码执行，Transformer 允许用户在重编译时修改指令
- Memory、Module、Process 子系统提供了插桩所需的运行时信息查询能力
- Gum 通过架构特定的 Writer（代码生成器）和平台抽象层实现跨平台支持
- GumJS 将 Gum 的 C API 暴露为 JavaScript 接口，支持 QuickJS 和 V8 两种引擎

思考题

1. Interceptor 的 attach 和 replace 分别适用于什么场景？如果你想记录一个函数的参数但不改变它的行为，应该用哪个？如果你想完全替换一个函数的返回值呢？

2. Stalker 的动态重编译会产生额外的内存和 CPU 开销。在什么场景下值得使用 Stalker，什么场景下应该优先使用 Interceptor？

3. 为什么 Gum 需要专门处理 fork() 的情况？如果不处理，子进程中可能会出现什么问题？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第6章：核心数据结构与接口定义

> 两个人要合作，首先得说好”我给你什么，你给我什么”。软件组件之间也一样——接口就是它们之间的”合同”。Frida 的各个组件之间通过一套精心设计的接口和数据结构进行通信。今天我们来读这份”合同”，看看组件之间到底是怎么约定的。

6.1 接口定义在哪里？

Frida 的核心接口定义主要集中在两个文件中：

- `frida-core/lib/base/session.vala` —— DBus 接口、ID 类型、消息结构
- `frida-core/src/host-session-service.vala` —— 后端服务接口

这些接口使用 Vala 语言定义，并通过 DBus 协议进行跨进程通信。DBus 是 Linux 世界广泛使用的进程间通信（IPC）机制，Frida 把它用作组件之间的通信协议，即使在非 Linux 平台上也是如此。

6.2 三大核心接口

Frida 的 DBus 接口体系由三个核心接口构成，它们各司其职：

```
┌──────────────────────────────────────────────────────────────┐
│                    Frida 接口体系                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│                                                              │
│   ┌──────────────────┐                                       │
│   │   HostSession    │   "前台经理"                          │
│   │  re.frida.       │   管理进程、设备级操作                │
│   │  HostSession17   │   枚举进程、spawn、attach             │
│   └──────────────────┘                                       │
│            │                                                 │
│            │ 创建                                            │
│            v                                                 │
│                                                              │
│   ┌──────────────────┐                                       │
│   │  AgentSession    │   "项目经理"                          │
│   │  re.frida.       │   管理脚本、调试器                    │
│   │  AgentSession17  │   创建/加载/卸载脚本                  │
│   └──────────────────┘                                       │
│            │                                                 │
│            │ 通信                                            │
│            v                                                 │
│                                                              │
│   ┌──────────────────┐                                       │
│   │ AgentMessageSink │   "信箱"                              │
│   │  re.frida.       │   接收 Agent 发来的消息               │
│   │  AgentMessage-   │   脚本输出、调试信息                  │
│   │  Sink17          │                                       │
│   └──────────────────┘                                       │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

HostSession：设备级操作接口

HostSession 是最高层的接口，对应一个”设备会话”。它提供的能力包括：

```
[DBus (name = "re.frida.HostSession17")]
public interface HostSession : Object {
    // 心跳检测
    async void ping (uint interval_seconds, Cancellable?
cancellable);

    // 查询系统信息
    async HashTable<string, Variant> query_system_parameters
(...);

    // 进程管理
    async HostApplicationInfo get_frontmost_application (...);
    async HostApplicationInfo[] enumerate_applications (...);
    async HostProcessInfo[] enumerate_processes (...);

    // 进程生命周期
    async uint spawn (string program, HostSpawnOptions
options, ...);
    async void resume (uint pid, ...);
    async void kill (uint pid, ...);

    // 核心操作：附加到进程
    async AgentSessionId attach (uint pid, HashTable<string,
Variant> options, ...);

    // 注入操作
    async InjectorPayloadId inject_library_file (...);
    async InjectorPayloadId inject_library_blob (...);

    // 信号：异步事件通知
    signal void spawn_added (HostSpawnInfo info);
    signal void process_crashed (CrashInfo crash);
    signal void agent_session_detached (AgentSessionId id,
SessionDetachReason reason, CrashInfo crash);
}
```

这个接口就像一个酒店的前台：你可以问它”现在有哪些房间”（枚举进程），可以”预定房间”（spawn 进程），也可以”进入已有房间”（attach 到进程）。

注意接口名称中的 17——这是接口版本号。当接口发生不兼容变更时，版本号会递增，确保新旧版本的客户端和服务端不会误连。

AgentSession：脚本级操作接口

当你 attach 到一个进程后，得到的是一个 `AgentSession`，它管理该进程内的具体操作：

```
[DBus (name = "re.frida.AgentSession17")]
public interface AgentSession : Object {
    // 会话生命周期
    async void close (...);
    async void interrupt (...);
    async void resume (uint rx_batch_id, ..., out uint
tx_batch_id);

    // 子进程管理
    async void enable_child_gating (...);
    async void disable_child_gating (...);

    // 脚本管理（核心功能）
    async AgentScriptId create_script (string source,
HashTable<string, Variant> options, ...);
    async AgentScriptId create_script_from_bytes (uint8[]
bytes, ...);
    async uint8[] compile_script (string source, ...);
    async void destroy_script (AgentScriptId script_id, ...);
    async void load_script (AgentScriptId script_id, ...);
    async void eternalize_script (AgentScriptId script_id, ...);

    // 调试器
    async void enable_debugger (AgentScriptId script_id, ...);
    async void disable_debugger (AgentScriptId script_id, ...);

    // 消息传递
    async void post_messages (AgentMessage[] messages, uint
batch_id, ...);

    // P2P 连接（性能优化）
    async void offer_peer_connection (string offer_sdp, ..., out
string answer_sdp);
    async void add_candidates (string[] candidate_sdps, ...);
}
```

这个接口就像酒店房间里的电话——你可以通过它请求各种服务（创建脚本），控制灯光空调（加载/卸载脚本），甚至打长途电话（P2P 连接）。

AgentMessageSink：消息接收接口

这是最简单但很关键的接口：

```
[DBus (name = "re.frida.AgentMessageSink17")]
public interface AgentMessageSink : Object {
    async void post_messages (AgentMessage[] messages, uint
batch_id, ...);
}
```

它只有一个方法——接收消息。这种极简设计是有意为之的：消息接收是高频操作，接口越简单越好。

6.3 辅助接口

除了三大核心接口，还有两个辅助接口值得了解：

AgentSessionProvider

```
[DBus (name = "re.frida.AgentSessionProvider17")]
public interface AgentSessionProvider : Object {
    async void open (AgentSessionId id, HashTable<string, Variant>
options, ...);
    async void migrate (AgentSessionId id, GLib.Socket
to_socket, ...);
    async void unload (...);

    signal void opened (AgentSessionId id);
    signal void closed (AgentSessionId id);
    signal void eternalized ();
}
```

这是 Agent（注入到目标进程中的那部分代码）向 Host（frida-server）提供的接口。当 Host 需要在目标进程里开启一个新的 AgentSession 时，就通过这个接口告诉 Agent。

AgentController

```
[DBus (name = "re.frida.AgentController17")]
public interface AgentController : Object {
    async HostChildId prepare_to_fork (uint parent_pid, ...);
    async HostChildId prepare_to_specialize (uint pid, string
identifier, ...);
    async void recreate_agent_thread (uint pid, uint
injectee_id, ...);
    async void wait_for_permission_to_resume (HostChildId id,
HostChildInfo info, ...);
    async void prepare_to_exec (HostChildInfo info, ...);
    async void acknowledge_spawn (HostChildInfo info,
SpawnStartState start_state, ...);
}
```

这个接口处理子进程相关的复杂场景：fork、exec、Android 的 zygote specialize 等。这些都是多进程环境下的关键操作。

6.4 ID 类型体系

Frida 定义了多种 ID 类型来标识不同的实体。这些 ID 分为两类：

基于 uint 的 ID（局部唯一）

```
public struct InjectorPayloadId {
    public uint handle;
```

```
}

public struct HostChildId {
    public uint handle;
}

public struct AgentScriptId {
    public uint handle;
}

public struct PortalMembershipId {
    public uint handle;
}
```

这类 ID 是简单的递增整数，在某个组件内部唯一即可。比如 `AgentScriptId` 只需要在一个 AgentSession 内唯一。

基于 string 的 ID（全局唯一）

```
public struct AgentSessionId {
    public string handle;

    public AgentSessionId.generate () {
        this.handle = Uuid.string_random ().replace ("-", "");
    }
}

public struct ChannelId {
    public string handle;

    public ChannelId.generate () {
        this.handle = Uuid.string_random ().replace ("-", "");
    }
}

public struct ServiceSessionId {
    public string handle;
    // 同样使用 UUID 生成
}
```

这类 ID 使用 UUID（去掉连字符）作为值，保证全局唯一。`AgentSessionId` 需要全局唯一，因为一个客户端可能同时连接多个设备上的多个进程。

两类 ID 的设计体现了一个原则：**够用就好**。不需要全局唯一的地方用简单的整数，需要全局唯一的地方才用 UUID。

```
┌──────────────────────────────────────────────────────────┐
│                        ID 类型体系                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  全局唯一 (UUID string):                                  │
│  ├─ AgentSessionId    会话标识                            │
│  ├─ ChannelId         通道标识                            │
```

```
│   └─ ServiceSessionId   服务会话标识                    │
│                                                         │
│   局部唯一 (uint):                                      │
│   ├─ AgentScriptId      脚本标识 (会话内唯一)           │
│   ├─ InjectorPayloadId 注入载荷标识                     │
│   ├─ HostChildId        子进程标识                      │
│   └─ PortalMembershipId Portal成员标识                  │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

6.5 AgentMessage 与消息传输

`AgentMessage` 是 Agent 和 Host 之间传递消息的载体：

```
public struct AgentMessage {
    public AgentMessageKind kind;   // SCRIPT 或 DEBUGGER
    public AgentScriptId script_id; // 来源脚本
    public string text;             // 文本内容 (通常是 JSON)
    public bool has_data;           // 是否有二进制数据
    public uint8[] data;            // 二进制数据
}
```

`text` 承载结构化数据（JSON），`data` 承载二进制数据，对应 Frida 脚本中 `send(message, data)` 的两个参数。消息通过批量传输优化性能——`post_messages` 接收的是 `AgentMessage[]` 数组，配合 `batch_id` 实现断线重连时的消息不丢失。

`AgentMessageTransmitter` 管理传输状态，它有一个简洁的状态机：LIVE（正常） -> INTERRUPTED（中断，等待恢复） -> CLOSED（超时关闭）。`persist_timeout` 特性让连接中断时不会立即关闭，而是等待一段时间。这就像打电话信号不好断了，你不会马上挂电话，而是等一会儿看能不能重新接通。

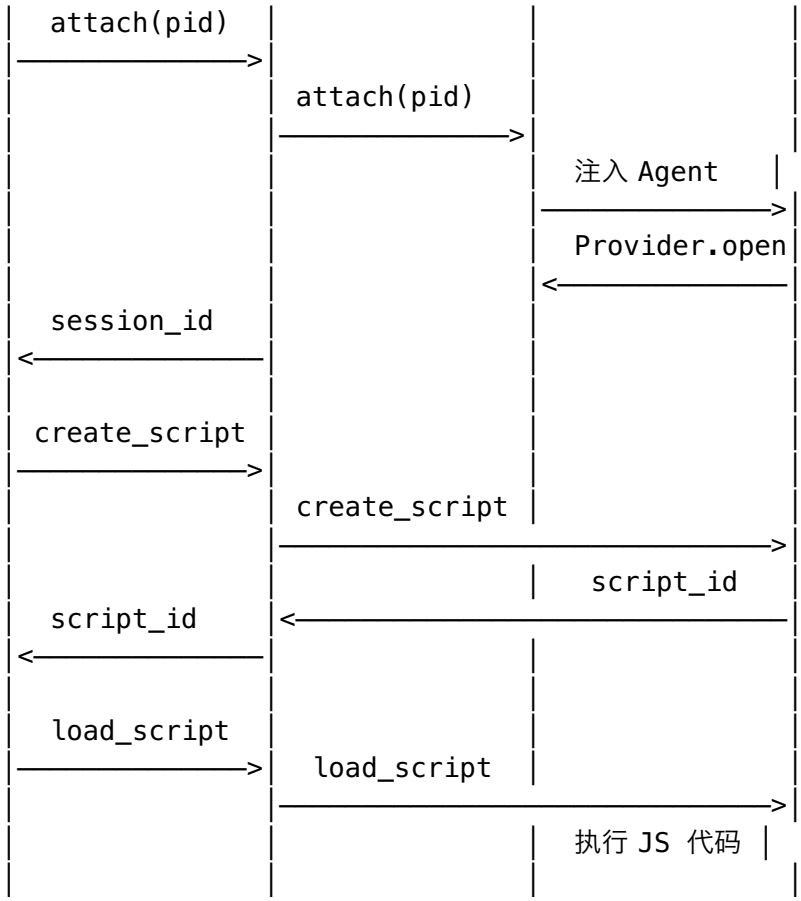
6.6 HostSessionProvider 与后端架构

`HostSessionProvider` 定义了设备后端的统一接口，每种连接方式（LOCAL/REMOTE/USB）都实现它。`HostSessionService` 管理所有 Provider——当新设备被发现（比如插入 USB），Backend 发出 `provider_available` 信号，DeviceManager 收到后就知道有新设备可用了。

6.7 会话管理的数据流

让我们把这些接口串起来，看一个完整的操作流程——从 Python 客户端 attach 到进程并创建脚本：

```
┌─────────┐     ┌─────────┐     ┌─────────┐     ┌─────────┐
│ Python  │     │ frida-  │     │ Host    │     │ Agent   │
│ Client  │     │ server  │     │ Session │     │(目标进程)│
└─────────┘     └─────────┘     └─────────┘     └─────────┘
     │               │               │               │
```



在这个流程中，HostSession 接口负责 attach 和进程管理，AgentSession 接口负责脚本的创建和加载，AgentMessageSink 负责后续的消息回传。每个接口在数据流中的角色清晰明确。

本章小结

- Frida 的核心通信基于三大 DBus 接口：HostSession（设备级）、AgentSession（进程级）、AgentMessageSink（消息传递）
- ID 类型分为全局唯一（UUID string）和局部唯一（uint）两类，按需选择
- AgentMessage 结构支持文本（JSON）和二进制数据的混合传输，采用批量发送优化性能
- AgentMessageTransmitter 通过状态机管理连接的中断和恢复
- HostSessionProvider 接口统一了不同连接方式（本地、USB、网络）的设备后端
- 接口版本号机制确保了协议的向前兼容性

思考题

1. 为什么 `AgentSessionId` 使用 UUID 字符串而 `AgentScriptId` 使用简单的 uint？如果把 `AgentScriptId` 也改成 UUID，会有什么利弊？

2. `AgentMessageTransmitter` 的 `persist_timeout` 功能在什么实际场景下特别有用？如果没有这个功能，用户体验会受到什么影响？

3. 如果你要为 Frida 添加一个新的子系统（比如”网络流量监控”），你会在哪个接口中添加新方法？还是应该定义一个全新的接口？为什么？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第7章：构建系统——Meson 与交叉编译

> 你写了一个 C 程序，在你的 Mac 上编译通过了。但你的目标是让它跑在 Android 手机、iOS 设备、Windows 电脑、甚至路由器上。怎么办？你不可能在每个平台上手动敲编译命令。Frida 面临的就是这个问题——它需要在十几种平台和架构组合上构建。今天我们来看 Frida 是怎么用 Meson 构建系统解决这个难题的。

7.1 为什么选择 Meson？

在 Frida 的历史上，构建系统经历过演变，最终选定了 Meson。为什么不用更”传统”的 CMake 或 Autotools？

Meson 的几个特点特别适合 Frida 这样的项目：

```
┌──────────────────────────────────────────────────────────────┐
│                    为什么 Meson 适合 Frida                    │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  1. 原生交叉编译支持                                          │
│     - cross file 机制简洁直观                                 │
│     - 清晰区分 host 和 build 机器                             │
│                                                              │
│  2. 子项目（subproject）管理                                  │
│     - frida-gum、frida-core 等天然适合子项目模式              │
│     - 依赖在构建时自动解析                                    │
│                                                              │
│  3. 构建速度快                                                │
│     - 后端使用 Ninja，并行编译效率高                          │
│     - 配置阶段也比 CMake 快                                   │
│                                                              │
│  4. 语法简洁可读                                              │
│     - 没有 CMake 那种"宏地狱"                                 │
│     - 不像 Autotools 需要 m4 宏和 shell 脚本                  │
│                                                              │
│  5. 多语言支持                                                │
│     - C、C++、Vala、Objective-C 都原生支持                    │
│     - Frida 恰好需要所有这些语言                              │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

特别是第 5 点——Frida 的代码库同时使用了 C（Gum 核心）、C++（V8 集成）、Vala（frida-core）、Objective-C（Darwin 平台支持），能原生支持所有这些语言的构建系统并不多。

7.2 顶层 meson.build 结构

Frida 的顶层 `meson.build` 是整个构建系统的”指挥中心”。我们来逐段解读：

```
project('frida', 'c',
  version: run_command('releng' / 'frida_version.py', check: true).stdout().strip(),
  meson_version: '>=1.1.0',
)
```

第一行就有一个有趣的设计：版本号不是硬编码的，而是通过 Python 脚本动态获取。这意味着版本号有一个”单一真相源”（single source of truth），避免了多处版本号不一致的问题。

子项目编排

接下来是核心的子项目组织：

```
# 首先构建 Gum（插桩引擎）
gum_options = [
  'frida_version=' + meson.project_version(),
  'gumjs=enabled',
]
subproject('frida-gum', default_options: gum_options)

# 然后构建 Core（核心框架）
core_options = [
  'frida_version=' + meson.project_version(),
]
# 根据条件决定是否构建各组件
foreach component : ['server', 'portal', 'inject']
  core_options += component + '=' + (get_option(component)
    .disable_auto_if(not is_cross_build)
    .disable_auto_if(is_watchos)
    .allowed() ? 'enabled' : 'disabled')
endforeach
subproject('frida-core', default_options: core_options)
```

注意构建顺序：**先 Gum，后 Core**。这是因为 Core 依赖 Gum——Core 中的 Agent 需要用 Gum 来实现插桩功能。

条件构建逻辑

最有趣的是组件的条件启用逻辑：

```
get_option('server')
    .disable_auto_if(not is_cross_build)  // 非交叉编译时禁用
```

```
    .disable_auto_if(is_watchos)          // watchOS 上禁用
    .allowed()
```

这段代码表达的意思是：frida-server 默认只在**交叉编译**时构建。为什么？因为 frida-server 是跑在目标设备上的（比如你的 Android 手机），你通常需要在开发机器上交叉编译它。在非交叉编译场景（比如在 Mac 上为 Mac 构建），你通常不需要 server。

类似地，frida-gadget 也是默认只在交叉编译时构建：

```
core_options += 'gadget=' + (get_option('gadget')
    .disable_auto_if(not is_cross_build)
    .allowed() ? 'enabled' : 'disabled')
```

7.3 构建选项（meson.options）

Frida 的构建选项定义在 `meson.options` 文件中：

```
option('server',      type: 'feature', value: 'auto',
  description: 'Build frida-server')
option('portal',      type: 'feature', value: 'disabled',
  description: 'Build frida-portal')
option('inject',      type: 'feature', value: 'auto',
  description: 'Build frida-inject')
option('gadget',      type: 'feature', value: 'auto',
  description: 'Build frida-gadget')

option('frida_python', type: 'feature', value: 'auto',
  description: 'Build Python bindings')
option('frida_node',   type: 'feature', value: 'disabled',
  description: 'Build Node.js bindings')
option('frida_swift',  type: 'feature', value: 'disabled',
  description: 'Build Swift bindings')
```

注意 `value` 的三种取值：

- `auto` —— 让构建系统根据环境自动决定（比如交叉编译时才启用 server）
- `enabled` —— 强制启用
- `disabled` —— 强制禁用

这个三值逻辑非常灵活。大多数用户不需要修改任何选项，`auto` 会做出合理的默认选择。高级用户可以通过 `-Dserver=enabled` 来强制启用或禁用特定组件。

```
┌──────────────────────────────────────────────────────────┐
│                    构建组件与默认策略                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  组件          默认值    典型场景                          │
│  ────────────  ───────  ──────────────                   │
│  server        auto     交叉编译时启用                    │
│  inject        auto     交叉编译时启用                    │
```

```
│  gadget          auto       交叉编译时启用          │
│  portal          disabled   需要时手动启用          │
│                                                     │
│  frida_python    auto       本地编译时启用          │
│  frida_node      disabled   需要时手动启用          │
│  frida_tools     auto       本地编译时启用          │
│  frida_swift     disabled   仅 macOS 可用           │
│  frida_clr       disabled   仅 Windows 可用         │
│  frida_qml       disabled   需要 Qt6                │
│                                                     │
└─────────────────────────────────────────────────────┘
```

7.4 语言绑定的构建

顶层 `meson.build` 还负责协调各语言绑定的构建：

```
# Python 绑定
if get_option('frida_python')
    .disable_auto_if(is_cross_build)
    .allowed()
  run_command(ensure_submodules, 'frida-python', check: true)
  subproject('frida-python')
endif

# Node.js 绑定
if get_option('frida_node')
    .disable_auto_if(is_cross_build)
    .allowed()
  run_command(ensure_submodules, 'frida-node', check: true)
  subproject('frida-node')
endif
```

每个绑定在构建前都会先调用 `ensure_submodules` 确保对应的 git submodule 已经被检出。这是一个贴心的设计——你不需要手动 `git submodule update`，构建系统会帮你处理。

语言绑定都有 `disable_auto_if(is_cross_build)` 这个条件。原因很直观：如果你在 Mac 上为 Android 交叉编译，你不会需要 Android 上的 Python 绑定——Python 绑定是给开发机器用的。

7.5 交叉编译的工作原理

交叉编译是 Frida 构建系统中最重要的能力之一。Meson 使用 “cross file” 来描述目标平台的信息：

```
# 示例：android-arm64 交叉编译配置
[host_machine]
system = 'android'
subsystem = 'android'
cpu_family = 'aarch64'
cpu = 'aarch64'
```

```
endian = 'little'

[binaries]
c = '/path/to/ndk/toolchains/.../aarch64-linux-android-clang'
cpp = '/path/to/ndk/toolchains/.../aarch64-linux-android-clang++'
ar = '/path/to/ndk/toolchains/.../llvm-ar'
strip = '/path/to/ndk/toolchains/.../llvm-strip'

[properties]
sys_root = '/path/to/ndk/sysroot'
```

在 frida-gum 的 `meson.build` 中，可以看到架构检测的逻辑：

```
host_os_family = host_machine.system()
if host_os_family == 'android'
  host_os_family = 'linux'   # Android 底层是 Linux
endif

if host_machine.cpu_family() == 'aarch64'
  host_arch = 'arm64'
  host_abi = 'arm64'
elif host_machine.cpu_family() == 'arm'
  host_arch = 'arm'
  host_abi = 'arm'
elif host_machine.cpu_family() == 'mips'
  host_arch = 'mips'
  if host_machine.endian() == 'little'
    host_abi = 'mipsel'
  else
    host_abi = 'mips'
  endif
```

这段代码将 Meson 提供的通用架构信息转换为 Frida 内部使用的架构/ABI 标识。注意 MIPS 的特殊处理——同一个 CPU family 根据字节序不同有不同的 ABI。

Meson 的交叉编译能力意味着在配置阶段就确定了”我要为谁编译”。构建系统会自动区分哪些代码需要用本机编译器（比如代码生成工具），哪些需要用交叉编译器（目标平台的产物）。

7.6 平台差异的处理

Frida 需要在多种平台上运行，每个平台都有自己的特殊需求。构建系统通过多种方式处理这些差异：

编译条件宏

在源码中通过预处理器宏区分平台：

```
#if DARWIN
   // macOS / iOS 特有逻辑
#elif LINUX
```

```
    // Linux / Android 特有逻辑
#elif WINDOWS
    // Windows 特有逻辑
#endif
```

Darwin 平台的语言支持

macOS/iOS 需要 Objective-C 支持，frida-gum 的构建文件中特别处理了这个：

```
languages = ['c', 'cpp']
if host_os_family == 'darwin'
  languages += ['objc', 'objcpp']
  add_languages('objc', 'objcpp', native: false)
endif
```

条件编译后端

回顾 host-session-service.vala 中的后端选择：

```
private void add_local_backends () {
#if WINDOWS
    add_backend (new WindowsHostSessionBackend ());
#endif
#if DARWIN
    add_backend (new DarwinHostSessionBackend ());
#endif
#if LINUX
    add_backend (new LinuxHostSessionBackend ());
#endif
}
```

构建系统负责定义这些条件编译宏（WINDOWS、DARWIN、LINUX 等），使得源码可以根据目标平台包含或排除特定后端。

7.7 子项目依赖与版本管理

Frida 的子项目形成清晰的依赖链：frida-gum (底层) -> frida-core (中层) -> frida-python/node/swift 等绑定 (上层)。Meson 的子项目机制自动处理构建顺序。

版本号由 Python 脚本从 git 信息动态计算，通过 `frida_version` 选项传递给每个子项目，确保所有组件版本一致。

7.8 实际构建命令

对于想要自己构建 Frida 的读者，以下是常用的构建命令：

```
# 本地构建（开发用，包含 Python 绑定和 CLI 工具）
meson setup build
ninja -C build
```

```
# 为 Android arm64 交叉编译
meson setup build-android-arm64 \
    --cross-file cross/android-arm64.txt
ninja -C build-android-arm64

# 只构建特定组件
meson setup build -Dserver=enabled -Dfrida_python=disabled

# 查看所有可用选项
meson configure build
```

每次 `meson setup` 都会创建一个独立的构建目录，你可以同时维护多个构建配置互不干扰。

本章小结

- Frida 选择 Meson 是因为它原生支持交叉编译、多语言（C/C++/Vala/ObjC）、子项目管理，且语法简洁
- 顶层 meson.build 通过 subproject() 编排 frida-gum、frida-core 和各语言绑定的构建顺序
- 构建选项使用 auto/enabled/disabled 三值逻辑，配合 disable_auto_if 实现智能的条件构建
- 交叉编译通过 cross file 描述目标平台，构建系统自动区分本机工具和目标产物
- 平台差异通过条件编译宏、语言支持检测、后端选择等多种机制处理
- 版本号通过 Python 脚本统一计算，确保所有组件版本一致

思考题

1. 为什么 frida-server 默认在交叉编译时才构建，而 frida-python 默认在本地编译时才构建？这个设计背后的使用场景假设是什么？

2. 如果你要为一个新的嵌入式平台（比如 RISC-V 架构的 Linux 设备）添加 Frida 支持，你需要在构建系统中做哪些改动？

3. Frida 把版本号从 git 信息动态生成而不是硬编码在文件中。这种做法的好处是什么？有没有潜在的问题？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第8章：Interceptor - 函数钩子的魔法

> 你有没有想过，当你在 Frida 里写下 `Interceptor.attach(addr, { onEnter, onLeave })` 的时候，底层到底发生了什么？一个正在运行的函数，怎么就突然”被你控制”了？

8.1 什么是函数钩子？

让我们用一个生活中的例子来理解。

假设你家的座机号码是 `12345678`。正常情况下，别人拨打这个号码，电话直接到你家。但有一天，你去电信公司设置了一个”呼叫转移”：所有打到 `12345678` 的电话，先转到你的手机 `98765432` 上，你接听后可以选择再转回家里的座机。

函数钩子（Hook）就是这个道理：

```
正常情况：
  调用者 ————> 目标函数

Hook 之后：
  调用者 ————> 你的代码（onEnter）————> 目标函数 ————> 你的代码
（onLeave）————> 调用者
```

Frida 的 Interceptor 就是那个”电信公司”，它负责帮你设置这个”呼叫转移”。

8.2 Interceptor 的核心 API

打开源码文件 `guminterceptor.h`，我们可以看到 Interceptor 对外暴露的几个核心函数：

```
// 获取全局唯一的 Interceptor 实例（单例模式）
GumInterceptor * gum_interceptor_obtain (void);

// 在目标函数上挂钩，注册 listener
GumAttachReturn gum_interceptor_attach (
    GumInterceptor * self,
    gpointer function_address,
    GumInvocationListener * listener,
    gpointer listener_function_data,
    GumAttachFlags flags);

// 卸载钩子
void gum_interceptor_detach (
    GumInterceptor * self,
    GumInvocationListener * listener);

// 替换整个函数（不是 hook，而是完全替换）
```

```
GumReplaceReturn gum_interceptor_replace (
    GumInterceptor * self,
    gpointer function_address,
    gpointer replacement_function,
    gpointer replacement_data,
    gpointer * original_function);

// 取消替换
void gum_interceptor_revert (
    GumInterceptor * self,
    gpointer function_address);
```

注意 `attach` 和 `replace` 是两个不同的操作：

- **attach**: 在原函数执行前后插入你的代码，原函数照常执行
- **replace**: 用你的函数完全取代原函数，但保留一个指向原函数的指针供你可选调用

8.3 InvocationListener：你的”监听器”

当你用 `Interceptor.attach()` 挂钩时，你提供的 `onEnter` 和 `onLeave` 回调函数，在 C 层对应的是 `GumInvocationListener` 接口：

```
struct _GumInvocationListenerInterface
{
  void (* on_enter) (GumInvocationListener * self,
      GumInvocationContext * context);
  void (* on_leave) (GumInvocationListener * self,
      GumInvocationContext * context);
};
```

Frida 提供了两种快捷创建方式：

```
// 创建一个 Call Listener（同时关注进入和离开）
GumInvocationListener * gum_make_call_listener (
    on_enter_callback, on_leave_callback, data, destroy);

// 创建一个 Probe Listener（只关注进入，不关心返回值）
GumInvocationListener * gum_make_probe_listener (
    on_hit_callback, data, destroy);
```

Call Listener 好比在函数门口装了两个摄像头，一个拍进门的人，一个拍出门的人。Probe Listener 只装了进门那个摄像头，更轻量。

8.4 InvocationContext：你能拿到什么？

每次你的回调被触发时，都会收到一个 `GumInvocationContext`，通过它你可以：

- 读取和修改函数参数
- 读取和修改返回值（在 onLeave 中）
- 获取 CPU 寄存器状态

- 在 onEnter 和 onLeave 之间传递自定义数据

这就像你截获了一个快递包裹，你不仅能看到包裹里的东西（参数），还能偷偷换一个（修改参数），甚至把回执单上的内容改了（修改返回值）。

8.5 Interceptor 的内部结构

让我们看看 `GumInterceptor` 对象内部长什么样：

```
struct _GumInterceptor
{
  GObject parent;
  GRecMutex mutex;                    // 递归锁，保证线程安全
  GHashTable * function_by_address;     // 地址 -> FunctionContext 映射表
  GumInterceptorBackend * backend;      // 架构相关的后端
  GumCodeAllocator allocator;           // 代码内存分配器
  volatile guint selected_thread_id;    // 线程过滤
  GumInterceptorTransaction current_transaction; // 当前事务
};
```

用一张图来理解它的结构：

```
┌──────────────────────────────────────────────────┐
│              GumInterceptor                      │
│                                                  │
│   ┌──────────────────────────────────────────┐   │
│   │   function_by_address (HashMap)          │   │
│   │                                          │   │
│   │  0x7fff1234 -> FunctionContext A         │   │
│   │  0x7fff5678 -> FunctionContext B         │   │
│   │  0x7fff9abc -> FunctionContext C         │   │
│   └──────────────────────────────────────────┘   │
│                                                  │
│   ┌──────────────┐   ┌──────────────────┐        │
│   │   Backend    │   │  CodeAllocator   │        │
│   │  (架构相关)  │   │  (内存分配)      │        │
│   └──────────────┘   └──────────────────┘        │
│                                                  │
│   ┌──────────────────────────────────────────┐   │
│   │     current_transaction                  │   │
│   │  (批量操作的事务管理)                    │   │
│   └──────────────────────────────────────────┘   │
└──────────────────────────────────────────────────┘
```

8.6 FunctionContext：每个被 Hook 函数的"档案"

每一个被 Hook 的函数，都有一个 `GumFunctionContext` 来记录它的全部信息：

```
struct _GumFunctionContext
{
  gpointer function_address;        // 原函数地址

  GumCodeSlice * trampoline_slice;   // 跳板代码的内存
  gpointer on_enter_trampoline;      // 进入跳板
  gpointer on_invoke_trampoline;     // 调用原函数的跳板
  gpointer on_leave_trampoline;      // 离开跳板

  guint8 overwritten_prologue[32];   // 被覆盖的原始字节（用于恢复）
  guint overwritten_prologue_len;    // 被覆盖了多少字节

  volatile GPtrArray * listener_entries; // 监听器列表
  gpointer replacement_function;      // 替换函数（replace 模式）
};
```

注意看 `overwritten_prologue` 这个字段，它最大 32 字节。这是因为 Interceptor 需要把目标函数的开头几条指令替换成一条跳转指令。被替换掉的原始指令需要保存起来，将来恢复时用。

8.7 Hook 的工作原理：跳板（Trampoline）

Interceptor 的核心技术就是跳板（Trampoline）。来看这张图：

```
  原函数（Hook 前）：          原函数（Hook 后）：

    ┌──────────────────┐          ┌──────────────────┐
    │ push rbp         │          │ jmp on_enter_    │  <-- 被改
写的开头
    │ mov rbp, rsp     │          │    trampoline    │
    │ sub rsp, 0x20    │          │ ...（后续不变）   │
    │ ...              │          │                  │
    │ ret              │          │ ret              │
    └──────────────────┘          └──────────────────┘
                                        |
                                        v
                                  ┌──────────────────┐
                                  │ on_enter_        │
                                  │ trampoline:      │
                                  │                  │
                                  │ 保存 CPU 上下文  │
                                  │ 调用 onEnter     │
                                  │ 恢复 CPU 上下文  │
                                  │ jmp on_invoke_   │
                                  │    trampoline    │
                                  └──────────────────┘
                                        |
                                        v
                                  ┌──────────────────┐
                                  │ on_invoke_       │
                                  │ trampoline:      │
                                  │                  │
```

```
                                        │  push rbp          │  <-- 被搬
走的原始指令
                                        │  mov rbp, rsp      │
                                        │  sub rsp, 0x20     │
                                        │  jmp 原函数+偏移   │  <-- 跳回原
函数继续执行
                                        └────────────────────┘
```

关键点在于：

1. **改写入口**：把原函数开头的几条指令替换为一条跳转指令
2. **保存原始指令**：被替换的指令搬到 `on_invoke_trampoline` 里
3. **构建跳板链**：enter trampoline -> 你的回调 -> invoke trampoline -> 原函数后半段

这就像修路：你把路口改造了一下，让所有车先经过你设的检查站，检查完了再回到原来的路上继续开。

8.8 事务机制（Transaction）

如果你要同时 Hook 很多函数，一个一个改写内存既慢又危险（在多线程环境下，你改了一半另一个线程执行到这里就崩了）。Frida 提供了事务机制：

```
gum_interceptor_begin_transaction (interceptor);

// 批量 attach 多个函数
gum_interceptor_attach (interceptor, func_a, listener_a, NULL, 0);
gum_interceptor_attach (interceptor, func_b, listener_b, NULL, 0);
gum_interceptor_attach (interceptor, func_c, listener_c, NULL, 0);

gum_interceptor_end_transaction (interceptor);
```

事务的内部结构：

```
struct _GumInterceptorTransaction
{
  gboolean is_dirty;                    // 有没有待提交的修改
  gint level;                           // 事务嵌套层级
  GQueue * pending_destroy_tasks;       // 待销毁的任务
  GHashTable * pending_update_tasks;    // 待更新的任务
  GumInterceptor * interceptor;
};
```

事务机制好比银行转账：你不会一笔一笔地操作，而是把所有转账请求收集好，然后一次性提交。`begin_transaction` 就是”开始收集”，`end_transaction` 就是”一次性提交”。在提交的那一刻，Frida 会暂停所有相关线程，批量修改内存，然后恢复运行。

8.9 一步步跟踪：attach 到底做了什么？

让我们从上到下追踪一次 `Interceptor.attach()` 调用的完整流程：

```
第1步：gum_interceptor_attach()
  │
  ├─ 加锁（GRecMutex）
  │
  ├─ 第2步：gum_interceptor_instrument()
  │     │
  │     ├─ 在 function_by_address 中查找是否已有 FunctionContext
  │     │
  │     ├─ 如果没有，创建新的 FunctionContext
  │     │    │
  │     │    └─ _gum_interceptor_backend_create_trampoline()
  │     │         │
  │     │         ├─ 分析目标函数开头的指令
  │     │         ├─ 用 Relocator 搬移原始指令
  │     │         ├─ 用 Writer 生成跳板代码
  │     │         └─ 记录被覆盖的原始字节
  │     │
  │     └─ 返回 FunctionContext
  │
  ├─ 第3步：将 Listener 添加到 FunctionContext
  │
  ├─ 第4步：安排事务更新（schedule_update）
  │     │
  │     └─ 在 end_transaction 时执行：
  │          │
  │          ├─ _gum_interceptor_backend_activate_trampoline()
  │          │    │
  │          │    └─ 把原函数开头改写为跳转指令
  │          │
  │          └─ 刷新 CPU 指令缓存
  │
  └─ 解锁
```

注意看第2步中出现了两个重要的组件：**Relocator** 和 **Writer**。它们分别负责”搬移指令”和”生成指令”，我们将在后面的章节详细介绍。

8.10 线程安全与忽略机制

Interceptor 还提供了线程控制能力：

```
// 忽略当前线程（Interceptor 的回调不会在当前线程触发）
gum_interceptor_ignore_current_thread (interceptor);
gum_interceptor_unignore_current_thread (interceptor);

// 忽略其他所有线程（只有当前线程会触发回调）
gum_interceptor_ignore_other_threads (interceptor);
gum_interceptor_unignore_other_threads (interceptor);
```

这在你的回调函数里调用了被 Hook 的函数时特别有用。如果不忽略，就会无限递归。每个线程有一个 `InterceptorThreadContext`，其中的 `ignore_level` 控制忽略层级：

```
struct _InterceptorThreadContext
{
  GumInvocationBackend listener_backend;
  GumInvocationBackend replacement_backend;
  gint ignore_level;          // > 0 时忽略这个线程
  GumInvocationStack * stack;   // 调用栈
  GArray * listener_data_slots;
};
```

8.11 attach 的返回值

attach 可能失败，返回值告诉你原因：

```
typedef enum
{
  GUM_ATTACH_OK              =  0,  // 成功
  GUM_ATTACH_WRONG_SIGNATURE  = -1,  // 函数签名不对（无法识别为有效代
            码）
  GUM_ATTACH_ALREADY_ATTACHED = -2,  // 同一个 listener 已经挂在这个函数
            上了
  GUM_ATTACH_POLICY_VIOLATION = -3,  // 权限策略不允许
  GUM_ATTACH_WRONG_TYPE       = -4,  // 类型冲突（比如已经 replace 了又要
            attach）
} GumAttachReturn;
```

另外还有一个 `GUM_ATTACH_FLAGS_FORCE` 标志，用于强制 Hook，即使 Frida 检测到目标函数可能已经被其他工具修改过。

8.12 本章小结

- Interceptor 是 Frida 最核心的 Hook 引擎，通过修改目标函数入口实现钩子
- `attach` 在函数前后插入回调，`replace` 完全替换函数
- 跳板（Trampoline）是 Hook 的核心机制：修改入口跳转 -> 执行你的代码 -> 跳回原函数
- 事务机制保证多个 Hook 操作的原子性
- 每个被 Hook 的函数都有一个 `FunctionContext` 记录其全部状态
- Interceptor 依赖 Writer（生成代码）和 Relocator（搬移代码）两个底层组件

讨论问题

1. 为什么 Interceptor 需要保存被覆盖的原始字节？如果不保存会怎样？

2. 在多线程环境下，如果线程 A 正在执行目标函数的开头几条指令，而线程 B 此时正在修改这些指令，会发生什么？Frida 如何避免这个问题？

3. `attach` 和 `replace` 各适用于什么场景？能不能对同一个函数同时使用这两种操作？

第9章：Stalker - 代码追踪引擎

> 如果 Interceptor 是在路口设了一个检查站，那 Stalker 就是给目标车辆装了一个 GPS 追踪器。它不只是在某几个点拦截你，而是追踪你走过的每一条路、每一个转弯。这到底是怎么做到的？

9.1 什么是动态二进制插桩？

我们先区分两个概念：

- **静态二进制插桩**：在程序运行之前，修改它的可执行文件，插入额外的代码。好比你在印刷之前修改了一本书的内容。
- **动态二进制插桩**（DBI）：在程序运行时，实时地转换它即将执行的代码。好比你在别人读书的时候，在他翻到下一页之前，偷偷把那一页换成了你修改过的版本。

Stalker 就是 Frida 的 DBI 引擎。它能做到：

- 追踪目标线程执行的每一条指令
- 记录所有函数调用和返回
- 在任意指令处插入自定义代码
- 实时转换正在执行的机器码

9.2 Stalker vs Interceptor：什么时候用哪个？

```
┌──────────────────┬──────────────────────┬──────────────────────────
│                  │      Interceptor     │
Stalker            │
├──────────────────┼──────────────────────┼──────────────────────────
│ 工作方式         │ 修改函数入口         │ 实时重编译代码           │
│ 粒度             │ 函数级别             │ 指令级别
│
│ 性能开销         │ 极小（只在入口处）   │ 较大（每条指令都处理）   │
│ 适用场景         │ Hook 特定函数        │ 代码覆盖率/调用追踪      │
│ 对原始代码的影响 │ 只修改函数开头几字节 │ 创建代码的完整副本       │
│ 同时追踪的范围   │ 你指定的函数         │ 整个线程的执行流         │
└──────────────────┴──────────────────────┴──────────────────────────
```

简单说：如果你只想监控几个函数的参数和返回值，用 Interceptor。如果你想知道一个线程执行了哪些代码路径，用 Stalker。

9.3 Stalker 的核心 API

让我们看看 `gumstalker.h` 中的关键接口：

```
// 创建 Stalker 实例
GumStalker * gum_stalker_new (void);

// 追踪当前线程
void gum_stalker_follow_me (GumStalker * self,
    GumStalkerTransformer * transformer,
    GumEventSink * sink);

// 停止追踪当前线程
void gum_stalker_unfollow_me (GumStalker * self);

// 追踪指定线程
void gum_stalker_follow (GumStalker * self,
    GumThreadId thread_id,
    GumStalkerTransformer * transformer,
    GumEventSink * sink);

// 停止追踪指定线程
void gum_stalker_unfollow (GumStalker * self,
    GumThreadId thread_id);
```

两个关键参数：

- **GumStalkerTransformer**: 代码转换器，决定如何修改每个基本块
- **GumEventSink**: 事件接收器，接收追踪过程中产生的事件

9.4 基本块编译模型

Stalker 的核心思想是：不修改原始代码，而是创建代码的转换副本来执行。

什么是基本块（Basic Block）？就是一段没有分支的连续指令，从第一条执行到最后一条，中间不会跳走。比如：

```
基本块 A:                    基本块 B:
  mov rax, [rdi]               cmp rax, 0
  add rax, 1                   je block_C
  mov [rdi], rax               call some_func
  jmp block_B                  jmp block_D
```

Stalker 的工作流程：

```
┌──────────────────────────────────────────────────────────────┐
│                        Stalker 执行流程                      │
│                                                              │
│  原始代码                  转换后的代码              │
│  ┌──────────┐                          ┌──────────────────┐  │
│  │ 基本块 A  │ ──编译转换──>      │ 基本块 A' (副本)  │  │
│  │           │                          │ + 插桩代码        │  │
```

```
│   └──────────┘                          └──────────────────┘   │
│         │                                      │                 │
│         v                                      v                 │
│   ┌──────────┐                          ┌──────────────────┐    │
│   │ 基本块 B  │ ──编译转换──>            │ 基本块 B' (副本)  │   │
│   │          │                          │ + 插桩代码        │   │
│   └──────────┘                          └──────────────────┘    │
│                                                                  │
│   注意：线程实际执行的是右边的转换后代码！                          │
└──────────────────────────────────────────────────────────────────┘
```

每当线程准备执行一个新的基本块时，Stalker 会：

1. 检查这个基本块是否已经被编译过
2. 如果没有，用 Transformer 编译它（读取原始指令，生成转换后的版本）
3. 让线程跳转到编译后的代码去执行
4. 当编译后的代码执行完毕（遇到跳转），重复上述过程

这就像一个实时翻译员：你说一句话，他翻译一句话给对方听。对方从来不直接听到你说的原话，只听到翻译后的版本。

9.5 StalkerTransformer：你的代码转换器

Transformer 是你控制 Stalker 行为的主要接口：

```
// 创建默认的 Transformer（原样复制所有指令）
GumStalkerTransformer * gum_stalker_transformer_make_default (void);

// 创建自定义 Transformer
GumStalkerTransformer * gum_stalker_transformer_make_from_callback (
    GumStalkerTransformerCallback callback,
    gpointer data,
    GDestroyNotify data_destroy);
```

在回调中，你通过 `GumStalkerIterator` 遍历基本块中的每条指令：

```
void my_transformer (GumStalkerIterator * iterator,
                     GumStalkerOutput * output,
                     gpointer user_data)
{
  const cs_insn * insn;

  // 遍历原始基本块中的每条指令
  while (gum_stalker_iterator_next (iterator, &insn))
  {
    // 你可以检查指令，决定怎么处理

    if (insn->id == X86_INS_CALL)
    {
      // 在 call 指令前插入一个回调
      gum_stalker_iterator_put_callout (iterator,
          my_callout, NULL, NULL);
```

```
    }

    // 保留原始指令（复制到输出）
    gum_stalker_iterator_keep (iterator);
  }
}
```

keep 是关键操作：它把当前指令复制到转换后的代码中。如果你不调用 keep，这条指令就会被”吃掉”。你也可以用 put_callout 在任意位置插入 C 函数回调。

9.6 EventSink：事件收集器

EventSink 负责接收 Stalker 产生的各种事件。先看事件类型：

```
enum _GumEventType
{
  GUM_NOTHING  = 0,
  GUM_CALL     = 1 << 0,   // 函数调用
  GUM_RET      = 1 << 1,   // 函数返回
  GUM_EXEC     = 1 << 2,   // 每条指令执行
  GUM_BLOCK    = 1 << 3,   // 基本块执行
  GUM_COMPILE  = 1 << 4,   // 基本块编译
};
```

不同事件携带不同信息：

```
struct _GumCallEvent {
  GumEventType type;
  gpointer location;    // call 指令的地址
  gpointer target;      // 被调用函数的地址
  gint depth;           // 调用深度
};

struct _GumExecEvent {
  GumEventType type;
  gpointer location;    // 当前执行的指令地址
};

struct _GumBlockEvent {
  GumEventType type;
  gpointer start;       // 基本块起始地址
  gpointer end;         // 基本块结束地址
};
```

EventSink 接口的设计很优雅：

```
struct _GumEventSinkInterface
{
  GumEventType (* query_mask) (GumEventSink * self);  // 你关心哪些事件?
  void (* start) (GumEventSink * self);               // 追踪开始
  void (* process) (GumEventSink * self,              // 处理每个事件
```

```
    const GumEvent * event, GumCpuContext * cpu_context);
  void (* flush) (GumEventSink * self);              // 刷新缓冲区
  void (* stop) (GumEventSink * self);               // 追踪结束
};
```

`query_mask` 非常重要：如果你只关心 `GUM_CALL`，Stalker 就不会为每条指令都生成事件上报代码，大幅减少性能开销。

9.7 StalkerWriter：跨架构的代码输出

Stalker 需要在不同架构上生成代码，因此使用了一个联合体来统一接口：

```
union _GumStalkerWriter
{
  gpointer instance;
  GumX86Writer * x86;
  GumArmWriter * arm;
  GumThumbWriter * thumb;
  GumArm64Writer * arm64;
  GumMipsWriter * mips;
};

struct _GumStalkerOutput
{
  GumStalkerWriter writer;
  GumInstructionEncoding encoding;
};
```

这意味着 Stalker 在 x86 平台上用 `GumX86Writer` 生成代码，在 ARM64 平台上用 `GumArm64Writer`，但对上层提供统一的抽象。

9.8 信任阈值与代码缓存

Stalker 编译过的基本块会被缓存起来复用，但问题是：怎么知道缓存的代码还是有效的？

```
// 获取/设置信任阈值
gint gum_stalker_get_trust_threshold (GumStalker * self);
void gum_stalker_set_trust_threshold (GumStalker * self,
    gint trust_threshold);
```

信任阈值（trust threshold）控制缓存策略：

- **-1**: 永远不信任缓存，每次都重新编译（最安全但最慢）
- **0**: 信任缓存，但不计数（默认值，适合大多数场景）
- **N > 0**: 基本块被执行 N 次后，才完全信任它（折中方案）

为什么需要这个？因为有些程序会动态修改自己的代码（自修改代码）。如果 Stalker 总是用缓存的旧版本，就会执行错误的代码。

9.9 Backpatching 优化

Stalker 的一个重要优化是 backpatching（回填修补）。

考虑这个场景：基本块 A 的末尾跳转到基本块 B。第一次执行时，Stalker 必须从 A' 返回到 Stalker 引擎，查找或编译 B，再跳到 B'。但如果 B 已经编译好了，后续执行时可以直接从 A' 跳到 B'，跳过 Stalker 引擎这个中间环节。

```
首次执行:
  A'末尾 ──> Stalker引擎 ──> 查找/编译B ──> B'

Backpatch 后:
  A'末尾 ──────────────────────────────────────> B'
```

这就像你第一次去一个地方需要查地图，但走过一次后就记住路了，下次直接走。

Stalker 通过 Observer 接口通知外部关于 backpatch 的操作：

```
struct _GumStalkerObserverInterface
{
  GumStalkerNotifyBackpatchFunc notify_backpatch;
  // ... 各种计数器 ...
  GumStalkerIncrementFunc increment_call_imm;
  GumStalkerIncrementFunc increment_call_reg;
  GumStalkerIncrementFunc increment_jmp_imm;
  GumStalkerIncrementFunc increment_ret;
  // ...
};
```

9.10 排除区域

有些代码你不希望 Stalker 去转换（比如 Frida 自己的代码），可以用 exclude 排除：

```
void gum_stalker_exclude (GumStalker * self,
    const GumMemoryRange * range);
```

当执行流进入排除区域时，Stalker 会让它直接原生执行，离开后再恢复追踪。

9.11 其他实用功能

```
// 添加调用探针（当特定地址被 call 时触发）
GumProbeId gum_stalker_add_call_probe (GumStalker * self,
    gpointer target_address, GumCallProbeCallback callback,
    gpointer data, GDestroyNotify notify);

// 在指定线程上执行函数
gboolean gum_stalker_run_on_thread (GumStalker * self,
    GumThreadId thread_id, GumStalkerRunOnThreadFunc func,
```

```
    gpointer data, GDestroyNotify data_destroy);

// 使指定地址的缓存失效（代码被修改后需要调用）
void gum_stalker_invalidate (GumStalker * self,
    gconstpointer address);

// 预编译指定地址的代码
void gum_stalker_prefetch (GumStalker * self,
    gconstpointer address, gint recycle_count);
```

`run_on_thread` 特别有趣：它允许你在目标线程的上下文中执行一段代码。这对于需要在特定线程上操作的场景非常有用。

9.12 Stalker 的完整工作流程

把所有的概念串起来：

```
┌──────────────────────────────────────────────────────────┐
│                    Stalker 完整流程                      │
│                                                          │
│  1. follow_me() / follow(thread_id)                      │
│     │                                                    │
│  2. 获取目标线程当前的 PC（程序计数器）                  │
│     │                                                    │
│  3. 查找该 PC 对应的基本块是否已编译                     │
│     │                                                    │
│     ├─ 是 ──> 直接执行编译后的代码                       │
│     │                                                    │
│     └─ 否 ──> 4. 编译基本块                              │
│                  │                                       │
│                  ├─ 用 Capstone 反汇编原始指令           │
│                  ├─ 调用 Transformer 处理每条指令        │
│                  ├─ 用 Writer 生成转换后的代码           │
│                  ├─ 插入事件上报代码（根据 EventSink     │
│                  │  的 query_mask）                      │
│                  └─ 缓存编译结果                         │
│                                                          │
│  5. 执行编译后的代码                                     │
│     │                                                    │
│  6. 遇到跳转/调用/返回时                                 │
│     │                                                    │
│     ├─ 产生对应事件发送给 EventSink                      │
│     └─ 回到第3步处理下一个基本块                         │
│                                                          │
│  7. unfollow_me() / unfollow(thread_id) 停止追踪         │
└──────────────────────────────────────────────────────────┘
```

9.13 本章小结

- Stalker 是 Frida 的动态二进制插桩引擎，能追踪线程执行的每一条指令

- 核心原理是实时编译：将原始代码的基本块转换成插桩后的副本来执行
- Transformer 让你控制如何转换每条指令，EventSink 让你收集执行事件
- Backpatching 优化减少了反复查找编译代码的开销
- 信任阈值控制代码缓存的策略，平衡安全性和性能
- 与 Interceptor 相比，Stalker 粒度更细但开销更大

讨论问题

1. Stalker 为什么选择”复制并转换代码”的方案，而不是像 Interceptor 那样在原地修改代码？

2. 如果目标程序存在自修改代码（比如解密 shellcode 后执行），Stalker 如何处理？trust_threshold 在这种场景下应该怎么设置？

3. 在 JavaScript 层面使用 `Stalker.follow()` 时，开启 `GUM_EXEC` 事件（追踪每条指令）和只开启 `GUM_CALL` 事件，性能差异为什么会很大？从源码层面解释原因。

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第10章：代码生成器 - 跨架构的机器语言

> 你有没有想过一个问题：Frida 在运行时需要往内存中写入新的机器码（比如跳转指令、跳板代码），这些机器码是怎么生成的？总不能手工一个字节一个字节地拼吧？

10.1 为什么 Frida 需要在运行时写机器码？

回忆一下前两章的内容：

- Interceptor 需要在目标函数开头写一条跳转指令
- Interceptor 需要生成跳板代码（保存寄存器、调用回调、恢复寄存器）
- Stalker 需要把原始基本块编译成插桩后的副本

这些操作都需要在运行时动态生成机器码。而且 Frida 是跨平台的，需要支持 x86、x86_64、ARM、ARM64（AArch64）、MIPS 等架构。每种架构的指令编码都不同，手动拼字节简直是噩梦。

所以 Frida 设计了一套 **Writer** 抽象层：你告诉它”我要生成一条 push 指令”，它自动帮你把这条指令编码成对应架构的机器码。

这就像一个多语言翻译器：你用”通用描述”说出你要做的事，它帮你翻译成法语、德语或日语。

10.2 Writer 的基本结构

以 `GumX86Writer` 为例，它的结构体：

```
struct _GumX86Writer
{
  volatile gint ref_count;
  gboolean flush_on_destroy;

  GumCpuType target_cpu;        // 目标 CPU 类型（x86 还是 x64）
  GumAbiType target_abi;        // 目标 ABI

  guint8 * base;                // 代码缓冲区起始地址
  guint8 * code;                // 当前写入位置（游标）
  GumAddress pc;                // 当前 PC 值

  GumMetalHashTable * label_defs;  // 标签定义表
  GumMetalArray label_refs;        // 标签引用列表
};
```

再看 `GumArm64Writer`：

```
struct _GumArm64Writer
{
  volatile gint ref_count;
  gboolean flush_on_destroy;

  GumArm64DataEndian data_endian;  // 数据端序
  GumOS target_os;                 // 目标操作系统
  GumPtrauthSupport ptrauth_support; // 指针认证支持

  guint32 * base;               // 代码缓冲区（注意是 guint32*）
  guint32 * code;               // 当前写入位置
  GumAddress pc;                // 当前 PC 值

  GumMetalHashTable * label_defs;
  GumMetalArray label_refs;
  GumMetalArray literal_refs;        // 字面量引用
  const guint32 * earliest_literal_insn;
};
```

注意一个重要区别：x86 的 `code` 指针是 `guint8 *`（字节指针），而 ARM64 的是 `guint32 *`（4字节指针）。这是因为 x86 指令变长（1到15字节不等），而 ARM64 指令固定4字节。

```
x86 指令（变长）：

┌──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┐
│55│48│89│e5│48│83│ec│20│48│8b│05│xx│xx│ ...
│ 1字节 │     3字节      │     3字节      │
│ push  │  mov rbp,rsp  │  sub rsp,32   │
└──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┘
```

ARM64 指令（固定4字节）：

```
┌──────────┬──────────┬──────────┬──────────┐
│ fd7bbfa9 │ fd030091 │ ff830051 │ e00f40f9 │
│  stp     │  mov     │  sub     │  ldr     │
│ (4字节)  │ (4字节)  │ (4字节)  │ (4字节)  │
└──────────┴──────────┴──────────┴──────────┘
```

10.3 Writer 的使用模式

所有 Writer 都遵循相同的使用模式：

```
// 1. 创建或初始化
GumX86Writer writer;
gum_x86_writer_init (&writer, code_buffer);

// 2. 生成指令
gum_x86_writer_put_push_reg (&writer, GUM_X86_RBP);
gum_x86_writer_put_mov_reg_reg (&writer, GUM_X86_RBP, GUM_X86_RSP);
gum_x86_writer_put_sub_reg_imm (&writer, GUM_X86_RSP, 0x20);

// 3. 刷新（处理延迟的标签引用等）
gum_x86_writer_flush (&writer);

// 4. 清理
gum_x86_writer_clear (&writer);
```

Writer 就像一支笔，`init` 是告诉它从哪里开始写，每个 `put_*` 调用写入一条指令，写完后 `flush` 确保所有内容都已最终确定。

10.4 常用操作对照表

下面这张表列出了常见操作在不同架构下的 Writer API：

```
┌──────────────────┬──────────────────────────┬──────────────────────
│      操作        │       X86 Writer         │       ARM64
Writer           │
├──────────────────┼──────────────────────────┼──────────────────────
│ 函数调用         │ put_call_address(addr)   │
put_bl_imm(addr)            │
│                  │ put_call_reg(reg)        │
put_blr_reg(reg)            │
├──────────────────┼──────────────────────────┼──────────────────────
│ 无条件跳转       │ put_jmp_address(addr)    │
put_b_imm(addr)             │
│                  │ put_jmp_reg(reg)         │
put_br_reg(reg)             │
├──────────────────┼──────────────────────────┼──────────────────────
│ 函数返回         │ put_ret()                │
put_ret()                   │
├──────────────────┼──────────────────────────┼──────────────────────
```

```
│ 压栈              │ put_push_reg(reg)           │
put_push_reg_reg(a, b)       │
│                   │                             │ (ARM64 必须成对压
栈)          │
├──────────────────┼──────────────────────────┼──────────────────────
│ 出栈              │ put_pop_reg(reg)            │ put_pop_reg_reg(a,
b)        │
├──────────────────┼──────────────────────────┼──────────────────────
│ 寄存器赋值        │ put_mov_reg_u64(reg, val)   │
put_ldr_reg_u64(reg, val)    │
│                   │ put_mov_reg_reg(dst, src)   │
put_mov_reg_reg(dst, src)    │
├──────────────────┼──────────────────────────┼──────────────────────
│ 加法              │ put_add_reg_imm(reg, val)   │
put_add_reg_reg_imm(d,s,v)   │
├──────────────────┼──────────────────────────┼──────────────────────
│ NOP               │ put_nop()                   │
put_nop()                    │
├──────────────────┼──────────────────────────┼──────────────────────
│ 断点              │ put_breakpoint()            │
put_brk_imm(0)               │
├──────────────────┼──────────────────────────┼──────────────────────
│ 保存全部寄存器    │ put_pushax()                │
put_push_all_x_registers()   │
│ 恢复全部寄存器    │ put_popax()                 │
put_pop_all_x_registers()    │
├──────────────────┼──────────────────────────┼──────────────────────
│ 标签定义          │ put_label(id)               │
put_label(id)                │
│ 跳转到标签        │ put_jmp_near_label(id)      │
put_b_label(id)              │
└──────────────────┴──────────────────────────┴──────────────────────
```

注意 ARM64 的压栈必须成对操作（`push_reg_reg`），这是因为 ARM64 要求栈指针 16 字节对齐，两个 8 字节寄存器正好 16 字节。而 x86 可以单个寄存器压栈。

10.5 带参数的函数调用

Writer 最强大的功能之一是生成带参数的函数调用。这在不同架构下差异巨大：

- x86_64 Linux/macOS: 前6个参数通过 rdi, rsi, rdx, rcx, r8, r9 传递
- x86_64 Windows: 前4个参数通过 rcx, rdx, r8, r9 传递
- ARM64: 前8个参数通过 x0-x7 传递

Writer 帮你屏蔽了这些差异：

```
// X86: 生成一个带参数的调用
gum_x86_writer_put_call_address_with_arguments (&writer,
    GUM_CALL_CAPI,        // 调用约定
    target_address,       // 函数地址
```

```
    2,                  // 参数个数
    GUM_ARG_ADDRESS, some_ptr,  // 参数1：地址
    GUM_ARG_REGISTER, GUM_X86_RBX); // 参数2：寄存器值

// ARM64: 同样的操作
gum_arm64_writer_put_call_address_with_arguments (&writer,
    target_address,
    2,
    GUM_ARG_ADDRESS, some_ptr,
    GUM_ARG_REGISTER, ARM64_REG_X19);
```

Writer 内部会自动生成”把参数放到正确位置”的代码，你不需要关心具体的调用约定。

10.6 标签系统（Label System）

在生成代码时，经常需要向前跳转到一个还没生成的位置。比如：

```
  cmp rax, 0
  je skip_block       <-- 这里需要跳到 skip_block，但它还没生成
  ... 一些代码 ...
skip_block:            <-- 到这里才定义
  ... 继续 ...
```

Writer 用标签系统来解决这个”先有鸡还是先有蛋”的问题：

```
// 定义一个标签 ID
const gchar * skip_label = "skip_block";

// 生成跳转到标签的指令（此时目标地址未知）
gum_x86_writer_put_jcc_near_label (&writer,
    X86_INS_JE, skip_label, GUM_NO_HINT);

// 生成其他代码 ...
gum_x86_writer_put_nop (&writer);

// 定义标签（记录当前位置）
gum_x86_writer_put_label (&writer, skip_label);

// flush 时，Writer 回填之前跳转指令中的偏移量
gum_x86_writer_flush (&writer);
```

内部实现的原理：

```
┌──────────────────────────────────────────────────────────┐
│  Writer 标签解析过程                                      │
│                                                          │
│  第1步：遇到 jcc_near_label("skip")                       │
│    - 写入 jcc 操作码                                      │
│    - 偏移量暂时填 0                                       │
│    - 记录到 label_refs: {id="skip", 位置=0x10}            │
│                                                          │
```

```
│  第2步：遇到 put_label("skip")                          │
│    - 记录到 label_defs: {id="skip", 地址=0x20}          │
│                                                          │
│  第3步：flush()                                          │
│    - 遍历 label_refs                                     │
│    - 找到 "skip" 的定义地址 0x20                         │
│    - 计算偏移量: 0x20 - 0x10 - 指令长度                  │
│    - 回填到第1步写入的位置                               │
└──────────────────────────────────────────────────────────┘
```

ARM64 Writer 还有一个额外的 `literal_refs` 机制，用于处理字面量池。因为 ARM64 没有大立即数指令，加载一个 64 位常量需要从附近内存读取，Writer 会在代码末尾放置一个字面量池。

10.7 距离限制与处理

不同架构的跳转指令有不同的距离限制：

```
┌──────────────┬──────────────────────────────────────┐
│   架构       │  直接跳转最大距离                    │
├──────────────┼──────────────────────────────────────┤
│ x86 short jmp│  +/- 127 字节                        │
│ x86 near jmp │  +/- 2GB                             │
│ ARM64 B      │  +/- 128MB (GUM_ARM64_B_MAX_DISTANCE)│
│ ARM64 ADRP   │  +/- 4GB (GUM_ARM64_ADRP_MAX_DISTANCE)│
└──────────────┴──────────────────────────────────────┘
```

Writer 提供了辅助函数来检查距离：

```
// x86: 判断能否直接跳转
gboolean gum_x86_writer_can_branch_directly_between (
    GumAddress from, GumAddress to);

// ARM64: 判断能否直接跳转
gboolean gum_arm64_writer_can_branch_directly_between (
    GumArm64Writer * self, GumAddress from, GumAddress to);
```

如果超出直接跳转范围，Frida 需要使用间接跳转：先把目标地址加载到寄存器，再通过寄存器跳转。ARM64 Writer 的 `put_branch_address` 函数会自动处理这种情况：

```
// 这个函数会自动选择最优策略:
// 距离近 -> 直接 B 指令
// 距离远 -> LDR + BR 组合
void gum_arm64_writer_put_branch_address (
    GumArm64Writer * self, GumAddress address);
```

10.8 x86 的 Meta 寄存器

`GumX86Writer` 定义了一组有趣的”Meta 寄存器”：

```
enum _GumX86Reg
{
  GUM_X86_EAX,    // 32位
  GUM_X86_RAX,    // 64位
  GUM_X86_XAX,    // Meta: 32位模式下是 EAX，64位模式下是 RAX
  GUM_X86_XCX,    // Meta: ECX 或 RCX
  GUM_X86_XSP,    // Meta: ESP 或 RSP
  GUM_X86_XIP,    // Meta: EIP 或 RIP
  // ...
};
```

Meta 寄存器以 x 开头（如 `XAX`, `XSP`），它们会根据 Writer 的目标 CPU 类型自动选择 32 位或 64 位版本。这让同一段代码生成逻辑可以同时支持 x86 和 x64。

10.9 ARM64 的特殊考虑

ARM64 Writer 还需要处理一些 ARM64 特有的问题：

指针认证（Pointer Authentication）：Apple Silicon 支持 PAC，指针在使用前需要签名验证。Writer 提供了：

```
// 去除指针签名
gboolean gum_arm64_writer_put_xpaci_reg (
    GumArm64Writer * self, arm64_reg reg);

// 对地址进行签名
GumAddress gum_arm64_writer_sign (
    GumArm64Writer * self, GumAddress value);
```

端序（Endianness）：虽然 ARM64 指令总是小端序，但数据可以是大端序。Writer 的 `data_endian` 字段控制写入数据时的端序。

10.10 实际例子：生成一段完整的跳板代码

让我们看看 Interceptor 可能生成的 on_enter 跳板代码（简化版）：

```
// ARM64 版本的 on_enter trampoline 伪代码
void generate_enter_trampoline (GumArm64Writer * cw,
                                GumFunctionContext * ctx)
{
  // 保存所有通用寄存器
  gum_arm64_writer_put_push_all_x_registers (cw);

  // 保存所有浮点寄存器
  gum_arm64_writer_put_push_all_q_registers (cw);

  // 把 FunctionContext 指针作为第一个参数
  gum_arm64_writer_put_ldr_reg_address (cw, ARM64_REG_X0,
      GUM_ADDRESS (ctx));
```

```
  // 把 CPU 上下文指针作为第二个参数
  gum_arm64_writer_put_mov_reg_reg (cw,
      ARM64_REG_X1, ARM64_REG_SP);

  // 调用 C 函数处理回调
  gum_arm64_writer_put_call_address_with_arguments (cw,
      GUM_ADDRESS (_gum_function_context_begin_invocation),
      2,
      GUM_ARG_REGISTER, ARM64_REG_X0,
      GUM_ARG_REGISTER, ARM64_REG_X1);

  // 恢复浮点寄存器
  gum_arm64_writer_put_pop_all_q_registers (cw);

  // 恢复通用寄存器
  gum_arm64_writer_put_pop_all_x_registers (cw);

  // 跳到 on_invoke_trampoline 执行原函数
  gum_arm64_writer_put_b_imm (cw,
      GUM_ADDRESS (ctx->on_invoke_trampoline));
}
```

这段代码展示了 Writer 的典型使用方式：一条一条地”说出”你想要的指令，Writer 把它们编码成真正的机器码。

10.11 辅助工具函数

Writer 还提供了一些方便的辅助函数：

```
// 获取当前写入位置
gpointer gum_x86_writer_cur (GumX86Writer * self);

// 获取已写入的偏移量
guint gum_x86_writer_offset (GumX86Writer * self);

// 直接写入原始字节
void gum_x86_writer_put_bytes (GumX86Writer * self,
    const guint8 * data, guint n);

// 写入填充（padding）
void gum_x86_writer_put_nop_padding (GumX86Writer * self, guint n);

// 获取第 N 个参数对应的寄存器
GumX86Reg gum_x86_writer_get_cpu_register_for_nth_argument (
    GumX86Writer * self, guint n);
```

最后一个函数很巧妙：它根据当前的 ABI（调用约定）告诉你第 N 个参数应该放在哪个寄存器里。

10.12 本章小结

- Writer 是 Frida 的代码生成抽象层，将”生成指令”的操作与具体的指令编码解耦
- 每种架构有对应的 Writer：`GumX86Writer`、`GumArm64Writer` 等
- x86 指令变长，ARM64 指令固定 4 字节，这导致两者的 Writer 实现细节不同
- 标签系统解决了前向引用的问题，通过 flush 时回填偏移量实现
- Writer 自动处理调用约定、距离限制、指针认证等平台差异
- 每个 `put_*` 调用直接在缓冲区中写入编码后的机器码字节

讨论问题

1. 为什么 ARM64 Writer 需要一个 `literal_refs` 机制而 x86 Writer 不需要？这和两种架构的立即数处理方式有什么关系？

2. 在使用标签系统时，如果一个标签被引用了但从未被定义，`flush` 时会发生什么？Frida 如何处理这种错误？

3. 假设你需要在 ARM64 上生成一段代码，跳转到一个 200MB 之外的地址。`put_b_imm` 会失败（超出 128MB 限制），你会怎么用 Writer API 来实现这个跳转？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第11章：Relocator - 代码搬迁的艺术

> 假设你家门口有一块路牌，上面写着”前方 500 米右转到公园”。现在城市改造，要把这块路牌搬到 2 公里外的新路口。你能原封不动地搬过去吗？当然不行——因为从新位置出发，500 米后右转到达的根本不是公园了。你必须重新计算距离，改成”前方 2500 米右转到公园”。这就是代码重定位的核心问题。

11.1 为什么需要重定位？

回忆第 8 章 Interceptor 的工作原理：Hook 一个函数时，需要把函数开头的几条指令”搬走”，放到 invoke trampoline 里执行，然后在原位置写入跳转指令。

```
原函数:
 地址 0x1000:  push rbp              ┐
 地址 0x1001:  mov rbp, rsp          │ 被搬走
 地址 0x1004:  sub rsp, 0x20         │
 地址 0x1008:  call 0x2000           ┘ <-- 这条有问题!
 地址 0x100D:  ...
```

```
搬到 trampoline（地址 0x5000）：
 地址 0x5000:  push rbp              -- OK，这条不受地址影响
 地址 0x5001:  mov rbp, rsp          -- OK
 地址 0x5004:  sub rsp, 0x20         -- OK
 地址 0x5008:  call 0x2000           -- 还是 OK 吗？
```

等一下，最后那条 `call 0x2000` 需要仔细看。在 x86 上，`call` 指令实际存储的不是绝对地址 `0x2000`，而是相对偏移量。在原位置 `0x1008`，偏移量是 `0x2000 - 0x100D = 0x0FF3`。但搬到 `0x5008` 后，同样的偏移量会让你跳到 `0x500D + 0x0FF3 = 0x6000`，完全错误。

这就是 Relocator 要解决的问题：**把指令从一个地址搬到另一个地址时，修正所有与地址相关的编码。**

11.2 什么指令需要修正？

不是所有指令都需要修正。需要修正的主要是使用 **PC 相对寻址** 的指令：

```
┌──────────────────────────────────────────────────────────────────────┐
│  需要重定位的指令类型                                                  │
│                                                                      │
│  x86/x64:                                                            │
│  ├─ call rel32         (相对调用)                                     │
│  ├─ jmp rel8/rel32     (相对跳转)                                     │
│  ├─ jcc rel8/rel32     (条件跳转)                                     │
│  ├─ loop/loopz/loopnz (循环指令)                                      │
│  └─ mov/lea [rip+off] (RIP 相对寻址，x64 特有)                        │
│                                                                      │
│  ARM64:                                                              │
│  ├─ b/bl imm           (分支/调用)                                    │
│  ├─ b.cond imm         (条件分支)                                     │
│  ├─ cbz/cbnz imm       (比较分支)                                     │
│  ├─ tbz/tbnz imm       (测试位分支)                                   │
│  ├─ adr/adrp           (PC 相对取地址)                                │
│  └─ ldr literal        (PC 相对加载字面量)                            │
│                                                                      │
│  不需要重定位的指令:                                                   │
│  ├─ mov reg, reg       (寄存器间操作)                                 │
│  ├─ add reg, imm       (立即数运算)                                   │
│  ├─ push/pop           (栈操作)                                       │
│  └─ ret                (返回)                                         │
└──────────────────────────────────────────────────────────────────────┘
```

11.3 Relocator 的结构

先看 x86 和 ARM64 两种 Relocator 的结构体，它们几乎一致：

```
struct _GumX86Relocator
{
  volatile gint ref_count;
```

```
  csh capstone;            // Capstone 反汇编引擎

  const guint8 * input_start;  // 原始代码起始地址
  const guint8 * input_cur;    // 当前读取位置
  GumAddress input_pc;         // 当前 PC 值
  cs_insn ** input_insns;      // 已读取的指令数组
  GumX86Writer * output;       // 输出 Writer

  guint inpos;             // 输入游标位置
  guint outpos;            // 输出游标位置

  gboolean eob;            // End Of Block（遇到分支）
  gboolean eoi;            // End Of Input（不可继续）
};

struct _GumArm64Relocator
{
  volatile gint ref_count;

  csh capstone;

  const guint8 * input_start;
  const guint8 * input_cur;
  GumAddress input_pc;
  cs_insn ** input_insns;
  GumArm64Writer * output;    // 注意：输出到 ARM64 Writer

  guint inpos;
  guint outpos;

  gboolean eob;
  gboolean eoi;
};
```

结构完全对称。Relocator 的设计模式是：**用 Capstone 读取（反汇编）原始指令，用 Writer 写入（重新编码）修正后的指令。**

```
┌──────────────┐      ┌──────────────────┐      ┌──────────────┐
│  原始代码    │      │    Relocator     │      │  新位置代码  │
│  (地址 A)    │──────>│                  │──────>│  (地址 B)    │
│              │      │  Capstone 读取   │      │              │
│  push rbp    │      │  分析指令类型    │      │  push rbp    │
│  mov rbp,rsp │      │  修正地址引用    │      │  mov rbp,rsp │
│  call 0x2000 │      │  Writer 写入     │      │  call 0x2000 │
│              │      │                  │      │  (偏移已修正)│
└──────────────┘      └──────────────────┘      └──────────────┘
```

11.4 read_one / write_one 模式

Relocator 的核心 API 是两个简洁的操作：`read_one` 和 `write_one`。

```
// 从输入读取一条指令（用 Capstone 反汇编）
guint gum_x86_relocator_read_one (GumX86Relocator * self,
    const cs_insn ** instruction);

// 把读取的指令重定位后写入输出
gboolean gum_x86_relocator_write_one (GumX86Relocator * self);

// 跳过一条指令（读了但不写）
void gum_x86_relocator_skip_one (GumX86Relocator * self);

// 一次性读取并写入所有指令
void gum_x86_relocator_write_all (GumX86Relocator * self);
```

典型的使用方式：

```
GumX86Relocator relocator;
GumX86Writer writer;
const cs_insn * insn;

// 初始化
gum_x86_writer_init (&writer, trampoline_buffer);
gum_x86_relocator_init (&relocator, original_function, &writer);

// 循环读取指令，直到搬够了需要的字节数
guint total_bytes = 0;
while (total_bytes < required_bytes)
{
  total_bytes = gum_x86_relocator_read_one (&relocator, &insn);

  // 你可以在这里检查指令
  // 比如打印: printf("  %s %s\n", insn->mnemonic, insn->op_str);
}

// 把所有读取的指令写入输出（自动修正地址）
gum_x86_relocator_write_all (&relocator);

// 在末尾加一条跳转，回到原函数的剩余部分
gum_x86_writer_put_jmp_address (&writer,
    GUM_ADDRESS (original_function + total_bytes));

gum_x86_writer_flush (&writer);
gum_x86_relocator_clear (&relocator);
gum_x86_writer_clear (&writer);
```

这段代码就是 Interceptor 构建 invoke trampoline 的核心逻辑（简化版）。

11.5 EOB 和 EOI

Relocator 有两个重要的状态标志：

```
gboolean eob;   // End Of Block
gboolean eoi;   // End Of Input
```

- **EOB (End Of Block)**: 遇到了分支指令（jmp、call、ret 等），表示当前基本块结束了。后续指令可能不会被执行到。
- **EOI (End Of Input)**: 遇到了无法继续读取的情况。比如一条无条件跳转 `jmp` 之后，原始代码流就断了，不应该再继续读取。

```
  mov rax, 1
  cmp rax, 0
  je somewhere       <-- EOB = true (条件跳转，可能跳走)
  add rax, 1         <-- 还可以继续读取
  jmp elsewhere      <-- EOB = true, EOI = true (无条件跳转，后面不再可达)
  ???                <-- 不应该继续读取了
```

Interceptor 在搬移指令时会检查这些标志：如果还没搬够需要的字节数就遇到了 EOI，说明目标函数的开头包含一条无条件跳转，这种情况需要特殊处理。

11.6 不同架构的重定位挑战

x86/x64 的挑战

x86 的主要难点是变长指令。你不知道一条指令有多长，直到你完全解码它。而且 x86 有很多种寻址模式，重定位逻辑相当复杂。

另外，x64 引入了 RIP 相对寻址，很多全局变量访问都使用这种方式：

```
; 原始位置 0x1000:
mov rax, [rip + 0x1234]    ; 实际访问 0x1000 + 7 + 0x1234 = 0x223B

; 搬到 0x5000 后，必须修改偏移量:
mov rax, [rip + 0xD234]    ; 保证还是访问 0x223B
```

ARM64 的挑战

ARM64 的指令虽然固定 4 字节，但它的 PC 相对寻址范围比较有限：

指令	PC 相对寻址范围
B (分支)	+/- 128 MB
B.cond	+/- 1 MB
CBZ/CBNZ	+/- 1 MB
TBZ/TBNZ	+/- 32 KB
ADR	+/- 1 MB
ADRP	+/- 4 GB
LDR (literal)	+/- 1 MB

如果指令搬移后，原始目标超出了新位置的寻址范围怎么办？Relocator 需要将一条简单的指令”展开”成多条指令：

```
原始（在 0x1000）：
  b.eq 0x2000           ; 条件跳转，目标在 1MB 内

搬到 0x80000000（很远的地方）后：
  b.ne skip             ; 反转条件，跳过下面的跳转
  ldr x17, [pc, #8]     ; 从附近的字面量池加载目标地址
  br x17                ; 间接跳转
skip:
  ...
  .quad 0x2000           ; 字面量池中存放原始目标地址
```

一条指令变成了好几条，这就是为什么 ARM64 Relocator 的 `can_relocate` 函数需要一个 `available_scratch_reg` 参数：

```
gboolean gum_arm64_relocator_can_relocate (
    gpointer address,
    guint min_bytes,
    GumRelocationScenario scenario,
    guint * maximum,
    arm64_reg * available_scratch_reg);  // 需要一个临时寄存器
```

而 x86 版本不需要这个参数，因为 x86 的间接跳转可以直接用内存地址，不需要额外的寄存器。

11.7 can_relocate：预检查

在实际搬移代码之前，Frida 会先检查能否成功重定位：

```
// x86 版本：简洁
gboolean gum_x86_relocator_can_relocate (
    gpointer address,
    guint min_bytes,     // 至少需要搬移多少字节
    guint * maximum);    // 输出：最多能搬多少字节

// ARM64 版本：更复杂
gboolean gum_arm64_relocator_can_relocate (
    gpointer address,
    guint min_bytes,
    GumRelocationScenario scenario,
    guint * maximum,
    arm64_reg * available_scratch_reg);
```

为什么需要预检查？因为有些指令可能无法安全重定位：

- 指令引用了相邻指令的地址（循环指令等）
- 代码中间有数据混杂（ARM Thumb 模式常见）
- 搬移后超出了可能的修正范围

11.8 Writer、Relocator 和 Interceptor 的协作

让我们看看这三者如何在 Hook 过程中协同工作：

```
┌──────────────────────────────────────────────────────────────┐
│  Interceptor Hook 流程中的角色分工                              │
│                                                                │
│  1. Interceptor 决定要 Hook 函数 foo()                          │
│     │                                                          │
│  2. Relocator 检查: can_relocate(foo, 跳转指令长度)             │
│     │                                                          │
│  3. Writer 创建 on_enter trampoline:                            │
│     │   ├─ put_push_all_registers()                            │
│     │   ├─ put_call(begin_invocation)                          │
│     │   ├─ put_pop_all_registers()                             │
│     │   └─ put_jmp(on_invoke_trampoline)                       │
│     │                                                          │
│  4. Relocator + Writer 创建 on_invoke trampoline:               │
│     │   ├─ Relocator: read_one() -- 读原始指令                  │
│     │   ├─ Relocator: write_one() -- 修正后写入                 │
│     │   ├─ (重复直到搬够字节)                                   │
│     │   └─ Writer: put_jmp(foo + 偏移) -- 跳回原函数            │
│     │                                                          │
│  5. Writer 创建 on_leave trampoline:                            │
│     │   ├─ put_push_all_registers()                            │
│     │   ├─ put_call(end_invocation)                            │
│     │   ├─ put_pop_all_registers()                             │
│     │   └─ put_ret() 或 put_jmp(返回地址)                       │
│     │                                                          │
│  6. Writer 改写原函数入口:                                      │
│     │   └─ put_jmp(on_enter_trampoline)                        │
│     │                                                          │
│  7. 刷新指令缓存，Hook 生效                                     │
└──────────────────────────────────────────────────────────────┘
```

三者的关系简单来说：

- **Writer** 是”笔”，负责写入机器码
- **Relocator** 是”搬运工”，负责把代码从一个地方搬到另一个地方并修正地址
- **Interceptor** 是”总指挥”，决定搬什么、往哪搬、怎么连接

11.9 overwritten_prologue 的秘密

回顾 `GumFunctionContext` 中的字段：

```
guint8 overwritten_prologue[32];
guint overwritten_prologue_len;
```

为什么是 32 字节？

在 x86_64 上，一条 `jmp` 跳转指令通常需要 5 字节（1 字节操作码 + 4 字节偏移）。但如果目标太远（超过 2GB），可能需要 14 字节（6 字节操作码 + 8 字节绝对地址）。所以 Frida 需要搬走至少这么多字节的原始代码。

在 ARM64 上，最简单的跳转是 4 字节的 `B` 指令，但如果距离超过 128MB，可能需要更长的指令序列。

32 字节的缓冲区足以覆盖所有架构的最坏情况。

11.10 一个完整的重定位例子

假设我们要 Hook 这个函数：

```
; 原始函数在 0x401000
0x401000: push rbp
0x401001: mov rbp, rsp
0x401004: call 0x402000      ; 相对偏移 = 0x402000 - 0x401009 = 0x0FF7
0x401009: pop rbp
0x40100A: ret
```

Interceptor 需要用 5 字节的 `jmp` 替换开头。我们需要搬走前 9 字节（到 `call` 指令结束）：

```
; trampoline 在 0x700000
0x700000: push rbp          ; 原样复制，无需修正
0x700001: mov rbp, rsp      ; 原样复制，无需修正
0x700004: call 0x402000     ; 偏移需要修正！
                            ; 新偏移 = 0x402000 - 0x700009 = 0xFFF01FF7
                            ; (在 32 位偏移范围内，可以修正)
0x700009: jmp 0x401009      ; Writer 生成：跳回原函数剩余部分

; 原函数被改写为:
0x401000: jmp on_enter_trampoline  ; 5 字节
0x401005: nop                      ; 填充
0x401006: nop
0x401007: nop
0x401008: nop
0x401009: pop rbp                  ; 不变
0x40100A: ret                      ; 不变
```

Relocator 在这个过程中自动完成了 `call` 指令偏移量的修正。

11.11 静态重定位辅助函数

Relocator 还提供了一个简便的静态函数，不需要手动管理 Reader/Writer 的生命周期：

```
// 一步到位：从 from 复制至少 min_bytes 到 to，返回实际复制的字节数
guint gum_x86_relocator_relocate (
    gpointer from,
    guint min_bytes,
    gpointer to);
```

```
guint gum_arm64_relocator_relocate (
    gpointer from,
    guint min_bytes,
    gpointer to);
```

这个函数内部创建临时的 Writer 和 Relocator，完成重定位后自动清理。适合简单场景。

11.12 本章小结

- Relocator 解决的核心问题是：将指令从一个地址搬到另一个地址时，修正所有 PC 相对寻址
- 内部使用 Capstone 反汇编引擎解析指令，使用 Writer 生成修正后的指令
- `read_one` / `write_one` 是核心操作模式，逐条读取、逐条写入
- `eob` 和 `eoi` 标志帮助判断是否到达基本块或代码流的边界
- ARM64 的重定位比 x86 更复杂，因为 PC 相对寻址范围有限，可能需要将一条指令展开为多条
- Writer、Relocator、Interceptor 三者协作完成整个 Hook 流程

讨论问题

1. 如果目标函数的前几条指令中包含一条 `jmp` 到自身的指令（比如一个 spin loop），Relocator 搬移后这条指令应该跳到哪里？跳到 trampoline 中的新位置还是原来的位置？

2. 为什么 ARM64 Relocator 的 `can_relocate` 需要一个 `available_scratch_reg` 参数而 x86 版本不需要？结合两种架构的间接跳转机制来分析。

3. 在现代操作系统中，代码段通常是不可写的。Interceptor 在改写函数入口时需要先修改内存保护属性。这个操作在 Frida 的源码中是由谁负责的？（提示：回顾第8章的 `GumCodeAllocator` 和事务机制）

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第12章：进程注入——打入目标的第一步

> 你有没有想过，当你在终端敲下 `frida -p 1234` 的时候，Frida 是怎么把自己的代码”塞”进一个正在运行的进程里的？这就像一个特工要潜入一栋戒备森严的大楼——他不能从正门走，他需要找到一条隐秘的通道，悄悄进入，然后在里面建立自己的据点。

12.1 什么是进程注入？

在操作系统的世界里，每个进程都有自己独立的地址空间，就好比每户人家都有自己的房间，门是锁着的。进程注入，简单来说，就是把一段代码（通常是一个动态库）加载到目标进程的地址空间里，让它在那里执行。

用一个生活中的比喻来理解：

```
你的进程（Frida 客户端）        目标进程（被分析的程序）

┌──────────────────────┐        ┌──────────────────────┐
│                      │        │                      │
│  "我要派特工进去"     │ ──── │  "正常运行中..."      │
│                      │  注入  │                      │
│                      │        │  ┌──────────────┐    │
│                      │        │  │ Frida Agent  │    │
│                      │        │  │ (特工已就位)  │    │
│                      │        │  └──────────────┘    │
└──────────────────────┘        └──────────────────────┘
```

特工要进入大楼，需要几个关键步骤：

1. **找到入口** - 利用操作系统提供的调试机制（比如 ptrace）
2. **打开门锁** - 暂停目标进程，获取控制权
3. **携带装备** - 把 Agent 动态库放到目标能访问的地方
4. **激活特工** - 让目标进程加载并执行这个动态库

12.2 平台差异：每个系统有不同的”入口”

不同操作系统提供了不同的注入手段，就像不同大楼有不同的安保系统：

```
┌──────────────────────────────────────────────────────┐
│                    Frida 注入机制总览                 │
├──────────────┬──────────────────────────────────────┤
│ 平台         │ 核心机制                              │
├──────────────┼──────────────────────────────────────┤
│ Linux        │ ptrace + dlopen                      │
│ macOS        │ task_for_pid + mach_vm_* + thread_create │
│ Windows      │ CreateRemoteThread + LoadLibrary     │
│ Android      │ ptrace + dlopen (与 Linux 类似)      │
│ iOS          │ 通常用 Gadget 模式（越狱环境用 substrate） │
├──────────────┼──────────────────────────────────────┤
│ 通用方案     │ Gadget 模式（不需要注入，第15章详述）  │
└──────────────┴──────────────────────────────────────┘
```

虽然每个平台细节不同，但核心思路是一致的：利用操作系统的调试接口，在目标进程中执行一个”加载动态库”的操作。今天我们重点分析 Linux 平台的实现。

12.3 Linux 注入：Linjector 的故事

在 Frida 的源码中，Linux 注入的核心类是 `Linjector`，位于 `frida-core/src/linux/linjector.vala`。我们来看它的结构：

```
// 简化后的 Linjector 结构
public sealed class Linjector : Object, Injector {
    public LinuxHelper helper;      // 实际干活的助手进程
    public TemporaryDirectory tempdir;  // 临时文件目录

    private HashMap<uint, uint> pid_by_id;     // 注入 ID -> 目标
PID
    private HashMap<uint, TemporaryFile> blob_file_by_id;  // blob
临时文件

    // 核心注入方法
    public async uint inject_library_file (uint pid, string path,
        string entrypoint, string data);
    public async uint inject_library_blob (uint pid, Bytes blob,
        string entrypoint, string data);
    public async uint inject_library_fd (uint pid, UnixInputStream
library_so,
        string entrypoint, string data);
}
```

注意到了吗？Linjector 本身并不直接操作 ptrace，它把脏活累活都委托给了一个叫 `LinuxHelper` 的对象。这是一个非常聪明的设计。

12.4 Helper 进程模式：为什么需要”中间人”？

这里有一个很多人忽略的细节：Frida 并不是直接从主进程去 ptrace 目标的，而是启动了一个独立的 Helper 进程来完成注入。为什么要多此一举呢？

想象一下这个场景：你是一个快递员（Frida 主进程），要往一个小区（目标进程）里送包裹。但小区保安（操作系统权限检查）只认住户和物业人员。怎么办？你找了一个物业的朋友（Helper 进程）帮你送进去。

```
┌──────────────────────────────────────────────────────┐
│                    注入架构图                         │
│                                                      │
│                                                      │
│  ┌──────────────┐    ┌──────────────┐               │
│  │ Frida 主进程 │    │ Helper 进程  │               │
│  │              │    │ (特权助手)   │               │
│  │  Linjector   │──>│              │               │
│  │              │    │  ptrace()    │               │
│  └──────────────┘    │  dlopen()    │               │
│                      │              │               │
│                      └──────────────┘               │
│                            │                        │
│                            v                        │
│                      ┌──────────────┐               │
│                      │  目标进程    │               │
```

```
│                          │                  │                  │
│                          │                  │                  │
│                          │  ┌──────────┐    │                  │
│                          │  │  Agent   │    │                  │
│                          │  │  .so     │    │                  │
│                          │  └──────────┘    │                  │
│                          │                  │                  │
└──────────────────────────┴──────────────────┴──────────────────┘
```

Helper 进程模式有几个好处:

1. **权限隔离** - Helper 可以以 root 权限运行，而 Frida 主进程不需要
2. **架构适配** - 32 位和 64 位目标可以用不同架构的 Helper（factory32/factory64）
3. **稳定性** - 如果注入过程出了问题，崩溃的是 Helper 而不是 Frida 主进程

从 `LinuxHelperProcess` 的源码可以看到这种双架构设计:

```
// 简化示意
public sealed class LinuxHelperProcess : Object, LinuxHelper {
    private HelperFactory? factory32;  // 32位 Helper
    private HelperFactory? factory64;  // 64位 Helper

    // 根据目标进程的架构选择合适的 Helper
    private async LinuxHelper obtain_for_pid (uint pid) {
        var cpu_type = cpu_type_from_pid (pid);
        return yield obtain_for_cpu_type (cpu_type);
    }
}
```

12.5 注入流程详解

让我们把整个注入过程串起来。当你调用 `inject_library_file` 时，实际发生了以下步骤:

```
时间线 ──────────────────────────────────────────────────────>

[1] Linjector.inject_library_file()
    │
    ├── 打开 .so 文件，获取文件描述符
    │
    v
[2] Linjector.inject_library_fd()
    │
    ├── 分配注入 ID
    ├── 记录 pid_by_id 映射
    │
    v
[3] helper.inject_library()    // 委托给 Helper 进程
    │
    ├── [3a] ptrace(ATTACH, pid)        暂停目标进程
    ├── [3b] 在目标进程中分配内存     mmap 远程调用
```

```
    ├── [3c] 写入引导代码和参数      写入 .so 路径等
    ├── [3d] 创建远程线程执行 dlopen     加载 Agent
    ├── [3e] dlopen 调用 entrypoint     通常是 frida_agent_main
    ├── [3f] ptrace(DETACH, pid)        恢复目标进程
    │
    v
[4] Agent 开始运行（下一章详述）
```

这里有个有趣的细节：Frida 支持多种传递动态库的方式。我们从源码中可以看到三种：

- **inject_library_file** - 传文件路径，目标进程通过路径加载
- **inject_library_blob** - 传二进制数据，先写到临时文件或 memfd 再加载
- **inject_library_fd** - 直接传文件描述符，最灵活的方式

特别值得注意的是 `MemoryFileDescriptor`（memfd）的使用：

```
// 简化示意
if (MemoryFileDescriptor.is_supported ()) {
    // 现代 Linux 内核支持 memfd_create
    // 不需要在磁盘上创建临时文件，更隐蔽
    FileDescriptor fd = MemoryFileDescriptor.from_bytes (name,
blob);
    return yield inject_library_fd (pid, fd, entrypoint, data);
} else {
    // 回退方案：写入临时文件
    var file = new TemporaryFile (name, blob, tempdir);
    return yield inject_library_file (pid, file.path, entrypoint,
data);
}
```

memfd 是 Linux 3.17+ 引入的特性，允许创建一个只存在于内存中的匿名文件。使用 memfd 有两个好处：一是更快（不需要磁盘 IO），二是更隐蔽（不会在文件系统留下痕迹）。

12.6 安全挑战与对抗

进程注入本质上是一种”越权”操作，操作系统自然会设置各种障碍：

```
┌──────────────────────────────────────────────────┐
│                 安全机制与应对策略                 │
├──────────────────┬───────────────────────────────┤
│ 安全机制          │ Frida 的应对                  │
├──────────────────┼───────────────────────────────┤
│ ptrace 限制       │ Helper 进程以 root 运行       │
│ (YAMA ptrace      │ 或调整 ptrace_scope 设置      │
│  scope)           │                               │
├──────────────────┼───────────────────────────────┤
│ SELinux           │ 在某些模式下需要宽松策略      │
│                   │ Gadget 模式可绕过             │
├──────────────────┼───────────────────────────────┤
│ 文件系统权限      │ 调整临时文件/目录权限         │
```

```
│                    │  使用 memfd 避免文件系统        │
├────────────────────┼──────────────────────────────┤
│  代码签名          │  iOS/macOS 上通常用 Gadget    │
│                    │  而非运行时注入                 │
├────────────────────┼──────────────────────────────┤
│  seccomp           │  Helper 在 seccomp 之前注入    │
│                    │  或使用允许的系统调用           │
└────────────────────┴──────────────────────────────┘
```

从 Linjector 的源码中我们还能看到一些细节处理：

```
// 确保临时目录权限正确
private void ensure_tempdir_prepared () {
    if (did_prep_tempdir) return;
    if (tempdir.is_ours)
        adjust_directory_permissions (tempdir.path);
    did_prep_tempdir = true;
}
```

这些看似简单的权限调整，是实际工程中不可或缺的细节。很多注入失败的原因，往往就是文件权限不对。

12.7 注入后的清理

注入不是一锤子买卖。当 Agent 完成任务或者用户断开连接时，需要清理现场：

```
// 当注入完成或 Agent 卸载时的回调
private void on_uninjected (uint id) {
    pid_by_id.unset (id);           // 移除 PID 映射
    blob_file_by_id.unset (id);     // 删除临时文件引用
    uninjected (id);                 // 发送信号通知上层
}
```

还有一个 `demonitor` 方法，允许在不卸载 Agent 的情况下停止监控。这在某些场景下很有用——比如你希望 Agent 继续运行，但不再需要 Frida 端保持连接。

本章小结

- **进程注入**是 Frida 动态插桩的第一步，核心思路是利用操作系统调试接口在目标进程中加载动态库
- Linux 上的注入通过 **Linjector** 类实现，它委托 **LinuxHelper** 进程完成实际的 ptrace + dlopen 操作
- **Helper 进程模式**提供了权限隔离、架构适配和稳定性保障
- 支持多种库传递方式：文件路径、二进制 blob、文件描述符，其中 **memfd** 是最现代且隐蔽的方式
- 不同平台有不同的安全挑战，Frida 针对每种都有对应策略

思考题

1. 为什么 Frida 在 Linux 上选择了 Helper 进程的架构，而不是直接在主进程中调用 ptrace？如果去掉 Helper 层会有什么问题？
2. memfd_create 相比临时文件有哪些优势？在什么场景下可能无法使用 memfd？
3. 如果目标进程开启了 seccomp 限制，禁止了 dlopen 相关的系统调用，Frida 还能注入吗？有什么替代方案？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第13章：Agent 生命周期——注入后的世界

> 上一章我们看到了 Frida 如何把 Agent 这个”特工”送入目标进程。但故事才刚刚开始——特工进了大楼之后，他要做什么？怎么和总部联系？任务完成后又怎么安全撤退？这就是 Agent 生命周期的故事。

13.1 Agent 的入口：一切从 main 开始

当 Helper 进程通过 dlopen 加载 Agent 动态库后，会调用指定的入口函数。在 Frida 中，这个入口函数就是 `frida_agent_main`。让我们看看它做了什么：

```
// frida-core/lib/agent/agent.vala（简化）
namespace Frida.Agent {
    public void main (string agent_parameters,
                      ref UnloadPolicy unload_policy,
                      void * injector_state) {
        if (Runner.shared_instance == null)
            Runner.create_and_run (agent_parameters,
                ref unload_policy, injector_state);
        else
            Runner.resume_after_transition (
                ref unload_policy, injector_state);
    }
}
```

这段代码虽然短，但信息量很大。它做了一个判断：如果是第一次被调用（shared_instance 为 null），就创建一个新的 Runner 并开始运行；如果不是第一次（比如 fork 之后子进程恢复），就执行恢复流程。

你可以把 `Runner` 想象成特工的”行动指挥中心”——它负责协调 Agent 的整个生命周期。

13.2 Runner：Agent 的心脏

Runner 是 Agent 中最核心的类，它实现了多个接口，身兼数职：

```
┌──────────────────────────────────────────────────────┐
│                    Runner 的角色                      │
├──────────────────────────────────────────────────────┤
│                                                      │
│  ProcessInvader        -- 作为入侵者管理自身          │
│  AgentSessionProvider  -- 提供会话管理能力            │
│  ExitHandler           -- 处理进程退出事件            │
│  ForkHandler           -- 处理 fork 事件              │
│  SpawnHandler          -- 处理进程创建事件            │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Runner 内部维护了大量状态，我们来看它最重要的几个成员：

```
// 简化后的 Runner 关键成员
private sealed class Runner : Object {
    public static Runner shared_instance = null;  // 全局单例

    private MainContext main_context;     // GLib 主事件循环上下文
    private MainLoop main_loop;          // 事件循环
    private DBusConnection connection;   // 与 Host 的通信通道
    private AgentController? controller; // 远端控制器的代理

    // 脚本相关
    private ScriptBackend? qjs_backend;  // QuickJS 后端
    private ScriptBackend? v8_backend;   // V8 后端

    // 安全与监控
    private ExitMonitor? exit_monitor;   // 监控进程退出
    private Interceptor interceptor;     // 函数拦截器
    private Exceptor? exceptor;          // 异常处理器

    // 进程生命周期
    private ForkMonitor? fork_monitor;   // 监控 fork
    private SpawnMonitor? spawn_monitor; // 监控进程创建
}
```

13.3 启动流程：从参数解析到事件循环

Agent 的启动是一个精心编排的过程。让我们跟着 `create_and_run` 走一遍：

```
┌──────────────────────────────────────────────────────┐
│                Agent 启动时间线                       │
│                                                      │
│  [1] Environment._init()                             │
│       └── 初始化 GLib、Gum 等基础设施                 │
```

```
│                                                          │
│   [2] 检测自身路径和内存范围                               │
│        └── detect_own_range_and_path()                   │
│        └── Gum.Cloak.add_range()  // 隐藏自身             │
│                                                          │
│   [3] 创建 Runner 实例                                    │
│        └── new Runner(agent_parameters, ...)             │
│                                                          │
│   [4] runner.run()                                       │
│        ├── push_thread_default()  // 设置线程上下文        │
│        ├── start.begin()          // 开始异步启动          │
│        └── main_loop.run()        // 进入事件循环          │
│                                                          │
│   [5] start() 异步流程                                    │
│        ├── 解析 agent_parameters                          │
│        ├── 初始化 Interceptor, ExitMonitor 等             │
│        ├── 建立与 Host 的 DBus 连接                       │
│        └── 预热 JS 引擎线程                               │
│                                                          │
│   [6] 事件循环运行中... (等待指令)                         │
└──────────────────────────────────────────────────────────┘
```

其中参数解析这一步值得细看。agent_parameters 是一个用竖线分隔的字符串：

```
// 参数格式: "传输地址|选项1|选项2|..."
string[] tokens = agent_parameters.split ("|");
unowned string transport_uri = tokens[0];  // 第一个是传输地址

foreach (unowned string option in tokens[1:]) {
    if (option == "eternal")
        ensure_eternalized ();         // 永不卸载模式
    else if (option == "sticky")
        stop_thread_on_unload = false; // 卸载时不停线程
    else if (option == "exit-monitor:off")
        enable_exit_monitor = false;   // 关闭退出监控
    // ... 更多选项
}
```

这种简单的文本协议设计得很巧妙——通过字符串传参避免了复杂的结构体跨进程传递问题。

13.4 通信通道：DBus 连接

Agent 与 Host（Frida 客户端）之间的通信是通过 DBus 协议建立的。但请注意，这里并不是系统的 DBus 总线，而是一个点对点的 DBus 连接，跑在 Unix socket（Linux）或命名管道（Windows）上。

```
┌──────────────────────────────────────────────────────────┐
│            Agent <-> Host 通信架构                         │
│                                                          │
│   Host 端            Agent 端            │
```

```
│   ┌──────────────┐                ┌──────────────┐      │
│   │ Frida        │                │ Runner       │      │
│   │ Client       │                │              │      │
│   │              │   DBus over    │ Agent        │      │
│   │ Agent        │<──────────────>│ Session      │      │
│   │ Controller   │  Unix Socket   │ Provider     │      │
│   │              │                │              │      │
│   └──────────────┘                └──────────────┘      │
│                                                         │
│  控制命令 ────────────────>  创建脚本/Hook/...          │
│  <────────────────  消息/事件/数据                       │
└─────────────────────────────────────────────────────────┘
```

在 Linux 上，通信通道的建立有一个特别有趣的细节。从 AgentContainer 的源码我们可以看到：

```
// 简化示意
// Linux 使用 socketpair 创建连接
int agent_ctrlfds[2];
Posix.socketpair (AF_UNIX, SOCK_STREAM, 0, agent_ctrlfds);

// fd[0] 给 Host 端
// fd[1] 传给 Agent（通过 injector_state）
```

而在 Agent 端的 `create_and_run` 方法中：

```
// Agent 侧接收 socket fd
var linjector_state = (LinuxInjectorState *)
opaque_injector_state;
int agent_ctrlfd = linjector_state->agent_ctrlfd;

// 构建传输地址: "socket:FD号"
agent_parameters = "socket:%d%s".printf (agent_ctrlfd,
agent_parameters);
```

这样 Agent 就通过继承的文件描述符直接与 Host 通信，无需经过任何文件系统或网络。这个方案既高效又安全。

13.5 会话管理

建立连接后，Host 可以通过 AgentSessionProvider 接口创建多个会话（Session）。每个会话可以独立加载脚本、进行 Hook 操作：

```
┌──────────────────────────────────────────────────────┐
│          Agent 内部会话结构                          │
│                                                      │
│  Runner (AgentSessionProvider)                       │
│  │                                                   │
│  ├── Session A                                       │
│  │   ├── ScriptEngine                                │
│  │   │   ├── Script 1 (hook malloc)                  │
│  │   │   └── Script 2 (trace calls)                  │
```

```
│   │   └── DBus 接口注册                          │
│   │                                              │
│   ├── Session B                                  │
│   │   ├── ScriptEngine                           │
│   │   │   └── Script 3 (custom logic)            │
│   │   └── DBus 接口注册                          │
│   │                                              │
│   └── Direct Connections (直连)                  │
│       └── 不经过主连接的独立通道                  │
└──────────────────────────────────────────────────┘
```

13.6 隐身术：Cloak 机制

一个优秀的特工，最重要的技能之一就是不被发现。Frida Agent 也是如此。在启动过程中，你会看到这样的代码：

```
// 隐藏 Agent 自身的内存范围
cached_agent_range = detect_own_range_and_path (mapped_range, out
cached_agent_path);
Gum.Cloak.add_range (cached_agent_range);

// 隐藏文件描述符
Gum.Cloak.add_file_descriptor (injector_state.fifo_fd);
```

`Gum.Cloak` 是 Frida 的隐身系统。当目标进程试图枚举自己的内存映射（比如读 `/proc/self/maps`）或文件描述符时，被 Cloak 标记的范围会被自动隐藏。同时，Agent 的线程也会被隐藏：

```
var ignore_scope = new ThreadIgnoreScope (FRIDA_THREAD);
// 在这个 scope 内，Frida 的线程对 Stalker 等机制不可见
```

13.7 卸载与清理

Agent 的退出有几种不同的方式，对应不同的 UnloadPolicy：

```
┌──────────────────────────────────────────────────┐
│              卸载策略 (UnloadPolicy)              │
├──────────────┬──────────────────────────────────┤
│  IMMEDIATE   │ 立即卸载，清理所有资源            │
│              │ 这是默认行为                      │
├──────────────┼──────────────────────────────────┤
│  RESIDENT    │ Agent 永驻内存，不卸载            │
│              │ 用于 eternalize 场景              │
├──────────────┼──────────────────────────────────┤
│  DEFERRED    │ 延迟卸载                          │
│              │ 用于 fork/exec 等进程转换场景     │
└──────────────┴──────────────────────────────────┘
```

正常卸载流程：

```
// 简化的关闭流程
// 1. 停止事件循环
main_loop.quit ();

// 2. 根据 stop_reason 决定后续
if (stop_reason == PROCESS_TRANSITION) {
    unload_policy = DEFERRED;    // fork 后保留
} else if (is_eternal) {
    unload_policy = RESIDENT;    // 永驻模式
    keep_running_eternalized (); // 在新线程继续运行
} else {
    release_shared_instance ();  // 释放单例
    Environment._deinit ();      // 清理环境
}
```

“Eternalize” 是一个特别有趣的概念。当脚本调用了某些需要永久存在的 Hook 时，Agent 可以进入永驻模式——即使 Frida 客户端断开连接，Agent 依然在目标进程中运行，Hook 继续生效：

```
private void keep_running_eternalized () {
    // 在一个新线程中继续运行事件循环
    agent_gthread = new Thread<bool> ("frida-eternal-agent", () =>
{
        main_context.push_thread_default ();
        main_loop.run ();  // 永远不会返回
        main_context.pop_thread_default ();
        return true;
    });
}
```

13.8 完整生命周期图

把所有阶段串起来：

```
┌──────────────────────────────────────────────────────────────┐
│                Agent 完整生命周期                              │
│                                                               │
│  dlopen() ──> frida_agent_main()                              │
│                     │                                         │
│                     v                                         │
│               Runner.create_and_run()                         │
│                     │                                         │
│                     │                                         │
│            ┌────────────────┐                                 │
│            v                v                                 │
│    Environment._init()   检测自身范围                          │
│            │             并 Cloak 隐藏                         │
│            v                                                  │
│    创建 Runner 单例                                            │
│            │                                                  │
│            v                                                  │
│    runner.run()                                               │
│            │                                                  │
│    ┌──────────────┐                                           │
```

```
│    v              v                                          │
│  start()    main_loop.run()                                  │
│    │              ^                                          │
│    v              │ (事件驱动)                                 │
│  解析参数          │                                          │
│  初始化拦截器      │                                          │
│  建立 DBus 连接    │                                          │
│  注册服务 ────────┘                                          │
│                                                              │
│  ═══════ 运行阶段 ═══════                                    │
│  接收指令 -> 创建会话 -> 加载脚本 -> 执行 Hook                 │
│                                                              │
│  ═══════ 退出阶段 ═══════                                    │
│  main_loop.quit()                                            │
│        │                                                     │
│        ├── IMMEDIATE: 清理并卸载                              │
│        ├── RESIDENT:  新线程继续运行                           │
│        └── DEFERRED:  等待进程转换后恢复                       │
└──────────────────────────────────────────────────────────────┘
```

本章小结

- Agent 的入口是 `frida_agent_main`，通过 **Runner** 单例管理整个生命周期
- 启动过程包括：环境初始化、自身隐藏（Cloak）、参数解析、DBus 连接建立
- Agent 与 Host 通过 **点对点 DBus 协议**通信，Linux 上使用 socketpair 传递文件描述符
- 支持多种卸载策略：**IMMEDIATE**（立即）、**RESIDENT**（永驻）、**DEFERRED**（延迟）
- **Cloak 机制**让 Agent 对目标进程”隐身”，隐藏内存范围、文件描述符和线程

思考题

1. Agent 为什么选择 DBus 协议而不是更简单的 JSON-RPC？DBus 在这个场景下有什么优势？
2. “Eternalize” 模式下，如果 Frida 客户端已经断开，Agent 的 Hook 还能继续工作。这是怎么实现的？会不会有内存泄漏的风险？
3. 在 fork 场景下，Agent 需要处理哪些特殊情况？父进程和子进程的 Agent 如何区分和独立运行？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第14章：脚本引擎——JavaScript 遇见原生代码

> 思考一个问题：一段 JavaScript 代码，是怎么在一个 C/C++ 编写的原生进程里运行的？更神奇的是，这段 JS 代码还能读写进程内存、拦截函数调用、修改参数和返回值。这背后到底发生了什么？

14.1 两个世界的碰撞

我们先来感受一下这种”神奇”。当你写一个 Frida 脚本：

```
Interceptor.attach(Module.findExportByName(null, "open"), {
    onEnter: function(args) {
        console.log("Opening: " + args[0].readUtf8String());
    }
});
```

这段 JavaScript 代码做了以下事情： 1. 在原生进程中找到 `open` 函数的地址 2. 在那个地址上设置了一个钩子 3. 每次 `open` 被调用时，读取第一个参数（一个 C 字符串指针） 4. 把内容发送到控制台

JavaScript 本身没有操作内存指针的能力，C 语言本身也不认识 JavaScript。是谁在中间搭了这座桥？答案就是 **GumJS** - Frida 的 JavaScript 绑定层。

```
┌──────────────────────────────────────────────────────────┐
│                Frida 脚本引擎架构                         │
│                                                          │
│   ┌──────────────────────────────────────────────────┐   │
│   │            JavaScript 世界                        │   │
│   │                                                  │   │
│   │  Interceptor.attach(...)                         │   │
│   │  Memory.readUtf8String(...)                      │   │
│   │  Process.enumerateModules(...)                   │   │
│   └──────────────────────────────────────────────────┘   │
│                        │ GumJS 绑定层                     │
│   ┌────────────────────v─────────────────────────────┐   │
│   │            C/C++ 桥接层                           │   │
│   │                                                  │   │
│   │  gumquickinterceptor.c / gumv8interceptor.c      │   │
│   │  gumquickmemory.c      / gumv8memory.c           │   │
│   │  gumquickprocess.c     / gumv8process.c          │   │
│   └──────────────────────────────────────────────────┘   │
│                        │                                 │
│   ┌────────────────────v─────────────────────────────┐   │
│   │            Gum 核心库                             │   │
│   │                                                  │   │
│   │  GumInterceptor  (函数拦截)                       │   │
│   │  GumStalker      (代码追踪)                       │   │
│   │  GumMemory        (内存操作)                      │   │
```

```
│   └──────────────────────────────────────────┘   │
└──────────────────────────────────────────────────┘
```

14.2 双引擎：QuickJS 与 V8

Frida 支持两个 JavaScript 引擎，就像一辆跑车配了两个发动机，你可以根据需要切换：

```
┌──────────────────────────────────────────────────┐
│          JavaScript 引擎对比                      │
├──────────────┬──────────────┬──────────────────┤
│              │   QuickJS    │       V8         │
├──────────────┼──────────────┼──────────────────┤
│  体积        │ 很小 (~300KB) │ 较大 (~5MB)      │
│  启动速度    │ 极快         │ 较慢             │
│  执行性能    │ 一般         │ 极好 (JIT)       │
│  内存占用    │ 低           │ 较高             │
│  适用场景    │ 嵌入式/资源受限 │ 性能敏感场景     │
│  文件前缀    │ gumquick*    │ gumv8*           │
└──────────────┴──────────────┴──────────────────┘
```

从 GumJS 的源码目录结构就能清楚看到这种双引擎设计：

```
gumjs/
├── gumquickinterceptor.c    <-- QuickJS 版本
├── gumv8interceptor.cpp     <-- V8 版本
├── gumquickmemory.c
├── gumv8memory.cpp
├── gumquickprocess.c
├── gumv8process.cpp
├── gumquickcore.c
├── gumv8core.cpp
├── gumquickstalker.c
├── gumv8stalker.cpp
└── ... (每个 API 模块都有两个版本)
```

每个 JS API（Interceptor、Memory、Process 等）都有两套实现：一套用 QuickJS 的 C API，一套用 V8 的 C++ API。虽然两套代码在 JS 引擎接口调用上不同，但底层最终都调用同样的 Gum 核心库。

在 ScriptEngine 中，引擎的选择是这样的：

```
// 简化示意
private Gum.ScriptBackend pick_backend (ScriptRuntime runtime) {
    if (runtime == DEFAULT)
        runtime = preferred_runtime;  // 默认选择
    return invader.get_script_backend (runtime);
}
```

用户可以在创建脚本时指定引擎，也可以使用默认值。在 Agent 的 Runner 中，两个后端按需初始化：

```
// Runner 中的后端管理
private Gum.ScriptBackend? qjs_backend;  // QuickJS，按需创建
private Gum.ScriptBackend? v8_backend;   // V8，按需创建
```

14.3 ScriptEngine：脚本的管家

ScriptEngine 是管理脚本生命周期的核心类。它的设计很清晰：

```
// 简化后的 ScriptEngine
public sealed class ScriptEngine : Object {
    // 信号：脚本发出的消息和调试信息
    public signal void message_from_script (AgentScriptId id,
string json, Bytes? data);
    public signal void message_from_debugger (AgentScriptId id,
string message);

    // 脚本实例映射表
    private HashMap<AgentScriptId?, ScriptInstance> instances;
    private uint next_script_id = 1;

    // 核心操作
    public async ScriptInstance create_script (string? source,
Bytes? bytes, ScriptOptions options);
    public async void load_script (AgentScriptId script_id);
    public async void destroy_script (AgentScriptId script_id);
    public void post_to_script (AgentScriptId script_id, string
json, Bytes? data);
}
```

每个脚本的生命都要经历这样的旅程：

```
┌──────────────────────────────────────────────────────────┐
│                    脚本生命周期状态机                     │
│                                                          │
│   create_script()          load_script()                 │
│        │                       │                         │
│        v                       v                         │
│   ┌─────────┐              ┌─────────┐                  │
│   │ CREATED │ ──────────>  │ LOADING │                  │
│   └─────────┘              └─────────┘                  │
│                                 │                        │
│                                 v                        │
│                            ┌─────────┐                   │
│                            │ LOADED  │ <── 正常运行      │
│                            └─────────┘                   │
│                                 │                        │
│                  ┌──────────────┼──────────────┐         │
│                  v              v              v         │
│          ┌──────────────┐  ┌──────────┐  ┌──────────┐   │
│          │ ETERNALIZED  │  │ DISPOSED │  │ (close)  │   │
│          │ (永驻)       │  └──────────┘  └──────────┘   │
│          └──────────────┘                               │
```

```
│        └──────────────┘         │                          │
│                                 v                          │
│                          ┌──────────┐                      │
│                          │ UNLOADED │                      │
│                          └──────────┘                      │
│                                 │                          │
│                                 v                          │
│                          ┌──────────┐                      │
│                          │ DESTROYED │                     │
│                          └──────────┘                      │
└────────────────────────────────────────────────────────────┘
```

14.4 脚本创建：从源码到可执行

当你发送一段 JavaScript 源码给 Frida，`create_script` 方法会这样处理：

```
// 简化示意
public async ScriptInstance create_script (string? source, Bytes?
bytes,
    ScriptOptions options) throws Error {

    // 1. 分配唯一 ID
    var script_id = AgentScriptId (next_script_id++);

    // 2. 选择 JS 引擎后端
    Gum.ScriptBackend backend = pick_backend (options.runtime);

    // 3. 创建脚本（编译）
    Gum.Script script;
    if (source != null)
        script = yield backend.create (name, source,
options.snapshot);
    else
        script = yield backend.create_from_bytes (bytes,
options.snapshot);

    // 4. 排除自身范围（避免 Stalker 追踪 Agent 自己）
    script.get_stalker ().exclude (invader.get_memory_range ());

    // 5. 包装成 ScriptInstance
    var instance = new ScriptInstance (script_id, script);
    instances[script_id] = instance;

    return instance;
}
```

这里有一个关键概念：**snapshot**（快照）。Frida 支持预编译脚本并保存快照，下次加载时可以跳过编译步骤，大幅提升启动速度。这就像把一本书翻译好放在书架上，下次要用时直接取，不用再翻译。

14.5 GumJS 绑定：桥梁的秘密

GumJS 绑定层是整个系统中最精妙的部分。让我们以 `Interceptor.attach` 为例，看看一次 JS 调用如何穿越到原生世界。

以 QuickJS 版本为例，`gumquickinterceptor.c` 中大致是这样的结构：

```
// 伪代码，展示绑定层的工作原理
static JSValue gumjs_interceptor_attach (JSContext * ctx,
    JSValue this_val, int argc, JSValue * argv)
{
    // 1. 从 JS 参数中提取目标地址
    gpointer target;
    _gum_quick_native_pointer_get (ctx, argv[0], &target);

    // 2. 从 JS 参数中提取回调对象
    JSValue on_enter = JS_GetPropertyStr (ctx, argv[1], "onEnter");
    JSValue on_leave = JS_GetPropertyStr (ctx, argv[1], "onLeave");

    // 3. 调用 Gum 核心的 C API
    GumAttachReturn result = gum_interceptor_attach (
        self->interceptor,
        target,
        GUM_INVOCATION_LISTENER (listener),
        NULL
    );

    // 4. 返回 JS 对象
    return wrap_as_js_listener (listener);
}
```

这就是桥梁的工作方式：

```
┌──────────────────────────────────────────────────────────┐
│          一次 Interceptor.attach 的旅程                  │
│                                                          │
│   JS 世界:                                               │
│   Interceptor.attach(ptr("0x12345"), {                   │
│       onEnter(args) { ... }                              │
│   })                                                     │
│        │                                                 │
│        v                                                 │
│   GumJS 绑定层:                                          │
│   gumjs_interceptor_attach()                             │
│        │   解析 JS 参数                                  │
│        │   转换类型 (JSValue -> C pointer)               │
│        │   创建 InvocationListener                       │
│        v                                                 │
│   Gum 核心:                                              │
│   gum_interceptor_attach()                               │
│        │   修改目标函数的机器码                          │
│        │   插入跳转指令到 trampoline                     │
```

```
│        v                                                  │
│   原生世界:                                                 │
│   目标函数被调用时 -> trampoline -> 回调 JS 函数            │
└──────────────────────────────────────────────────────────┘
```

GumJS 目录下的模块覆盖了 Frida JS API 的所有功能:

```
┌──────────────────────────────────────────────────────────┐
│              GumJS 模块映射                                 │
├──────────────────────┬──────────────────────────────────┤
│  JS API              │  对应 C/C++ 模块                    │
├──────────────────────┼──────────────────────────────────┤
│  Interceptor         │  gumquickinterceptor.c            │
│  Stalker             │  gumquickstalker.c                │
│  Memory              │  gumquickmemory.c                 │
│  Process             │  gumquickprocess.c                │
│  Module              │  gumquickmodule.c                 │
│  Thread              │  gumquickthread.c                 │
│  Socket              │  gumquicksocket.c                 │
│  File                │  gumquickfile.c                   │
│  Database (SQLite)│  gumquickdatabase.c               │
│  ApiResolver         │  gumquickapiresolver.c            │
│  CModule             │  gumquickcmodule.c                │
│  Symbol              │  gumquicksymbol.c                 │
│  Instruction         │  gumquickinstruction.c            │
│  CodeWriter          │  gumquickcodewriter.c             │
│  CodeRelocator       │  gumquickcoderelocator.c          │
└──────────────────────┴──────────────────────────────────┘
```

14.6 消息传递：JS 与 Host 的对话

脚本运行过程中，JS 代码和 Frida 客户端之间需要互相发消息。这个通信管道经历了多层传递:

```
┌──────────────────────────────────────────────────────────┐
│              消息传递路径                                   │
│                                                            │
│   JS 脚本端:                                               │
│   send({type: "info", payload: "hello"})                   │
│        │                                                   │
│        v                                                   │
│   Gum.Script 消息处理器                                     │
│   on_message(json, data)                                   │
│        │                                                   │
│        ├── RPC 消息?  -> rpc_client.try_handle_message     │
│        │                                                   │
│        └── 普通消息 -> ScriptInstance.message 信号          │
│                 │                                          │
│                 v                                          │
│              ScriptEngine.message_from_script 信号         │
│                 │                                          │
```

```
│                  v                                          │
│            通过 DBus 连接发送到 Host                        │
│                  │                                          │
│                  v                                          │
│  Python/Node.js 端:                                         │
│  script.on('message', function(message, data) { })          │
└──────────────────────────────────────────────────────────────┘
```

消息到达 ScriptInstance 后，会先检查是否为 RPC 响应（Frida 内置了 RPC 机制，支持 Host 调用脚本的 `rpc.exports` 方法），非 RPC 消息则作为普通消息转发。反方向，Host 通过 `post_to_script` 向脚本发消息，但只有处于 LOADING/LOADED/DISPOSED 状态的脚本才能接收。

ScriptEngine 还支持 JavaScript 调试器接口，通过 `enable_debugger` 启用后，调试消息通过 DBus 传回 Host，最终转发给 Chrome DevTools。

本章小结

- Frida 的脚本引擎通过 **GumJS 绑定层**把 JavaScript API 映射到 Gum 核心库的 C API
- 支持 **QuickJS**（轻量快速）和 **V8**（高性能 JIT）两个 JS 引擎，每个 API 模块都有两套实现
- **ScriptEngine** 管理脚本的完整生命周期：创建、编译、加载、执行、卸载、销毁
- 消息传递经历多层：JS -> Gum.Script -> ScriptInstance -> ScriptEngine -> DBus -> Host
- 内置 **RPC 机制**支持双向方法调用，**调试器接口**支持 Chrome DevTools 连接

思考题

1. 为什么 Frida 要同时支持 QuickJS 和 V8 两个引擎？在实际使用中你会怎么选择？
2. GumJS 的每个模块都有 Quick 和 V8 两个版本，这是否会导致代码维护困难？你能想到什么方法来减少重复代码？
3. 当一个 Interceptor Hook 被触发时，从原生函数调用到 JS onEnter 回调执行，大概会经历哪些步骤？这个过程的性能开销在哪里？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第15章：Gadget 模式——无需注入的插桩

> 前面几章我们看到了 Frida 通过 ptrace 注入 Agent 的精妙设计。但如果目标环境根本不允许 ptrace 呢？比如 iOS 上未越狱的设备，或者一个嵌入式系统里没有调试器支持。这时候该怎么办？Frida 说：注入不了？那我就直接住进去。

15.1 什么是 Gadget?

如果说传统注入是”派特工潜入大楼”，那 Gadget 模式就是”装修的时候就把窃听器砌进墙里了”。Gadget 是一个特殊的 Frida 动态库，你在打包应用或编译程序时直接把它链接进去。当程序启动时，Gadget 会自动初始化，完全不需要运行时注入。

```
┌──────────────────────────────────────────────────────────────┐
│          传统注入 vs Gadget 模式                              │
│                                                              │
│  传统注入:                                                    │
│                                                              │
│  ┌──────────┐   ptrace    ┌──────────┐                      │
│  │  Frida   │  ──────>    │ 目标进程  │                      │
│  │  Client  │   运行时    │          │                      │
│  └──────────┘   注入      │ +Agent   │                      │
│                           └──────────┘                      │
│                                                              │
│  Gadget 模式:                                                 │
│  ┌──────────────────────────────┐                            │
│  │  目标程序 (编译/打包时)        │                            │
│  │  ┌──────────────────────┐    │                            │
│  │  │ frida-gadget.so/dylib │    │                            │
│  │  │ (已经在里面了)         │    │                            │
│  │  └──────────────────────┘    │                            │
│  └──────────────────────────────┘                            │
│          │                                                   │
│          v   程序启动时自动初始化                              │
│  ┌──────────┐                                                │
│  │  Frida   │   连接到 Gadget 监听的端口                       │
│  │  Client  │                                                │
│  └──────────┘                                                │
└──────────────────────────────────────────────────────────────┘
```

15.2 自动初始化：constructor 的魔法

Gadget 最巧妙的地方在于它的初始化完全是自动的。当动态库被加载时，操作系统的动态链接器会自动调用标记了 `constructor` 属性的函数。Frida 在 `gadget-glue.c` 中利用了这一机制：

```
// gadget-glue.c（简化）

#if defined (HAVE_WINDOWS)
// Windows: DLL 入口点
BOOL WINAPI DllMain (HINSTANCE instance, DWORD reason, LPVOID
            reserved) {
    switch (reason) {
        case DLL_PROCESS_ATTACH:
            frida_gadget_load (NULL, NULL, NULL);
            break;
        case DLL_PROCESS_DETACH:
            if (reserved == NULL)  // 动态卸载时才清理
```

```
            frida_gadget_unload ();
        break;
    }
    return TRUE;
}

#elif defined (HAVE_DARWIN)
// macOS/iOS: constructor 属性
__attribute__ ((constructor)) static void
frida_on_load (int argc, const char * argv[],
               const char * envp[], const char * apple[],
               int * result)
{
    // Darwin 平台可以从 apple[] 参数获取额外配置
    frida_parse_apple_parameters (apple, &found_range, &range, &config_data);
    frida_gadget_load (found_range ? &range : NULL, config_data, result);
}

#else
// Linux/其他: constructor 属性
__attribute__ ((constructor)) static void frida_on_load (void) {
    frida_gadget_load (NULL, NULL, NULL);
}

__attribute__ ((destructor)) static void frida_on_unload (void) {
    frida_gadget_unload ();
}
#endif
```

看到了吗？每个平台都有自己的”自动初始化”机制，但核心都是调用 `frida_gadget_load()`。这就像不同品牌的汽车有不同的启动方式，但最终都是点燃发动机。

15.3 Gadget 的加载流程

`frida_gadget_load()` 最终会调用 Vala 层的 `Frida.Gadget.load()` 函数。让我们跟着源码走一遍：

```
┌──────────────────────────────────────────────────────────┐
│                Gadget 加载流程                            │
│                                                          │
│   [1] frida_gadget_load()                                │
│         │                                                │
│         v                                                │
│   [2] Gadget.load()                                      │
│         ├── 检查是否已加载 (防重入)                        │
│         ├── Environment.init()                           │
│         ├── detect_location()  // 检测自身位置            │
│         │                                                │
│         v                                                │
│   [3] 加载配置文件                                         │
```

```
│        ├── 尝试读取 frida-gadget.config.json          │
│        └── 没有配置文件则使用默认值                    │
│        │                                              │
│        v                                              │
│   [4] 根据配置创建 Controller                          │
│        ├── ScriptInteraction    -> ScriptRunner       │
│        ├── ScriptDirectory      -> ScriptDirRunner    │
│        ├── ListenInteraction    -> ControlServer      │
│        └── ConnectInteraction   -> ClusterClient      │
│        │                                              │
│        v                                              │
│   [5] 决定是否阻塞等待                                 │
│        ├── listen + on_load:wait  -> 阻塞直到连接      │
│        └── listen + on_load:resume -> 不阻塞           │
│        │                                              │
│        v                                              │
│   [6] 启动 Controller，进入工作状态                    │
└──────────────────────────────────────────────────────┘
```

从源码中可以看到这个清晰的分支逻辑:

```
// 简化示意
public void load (Gum.MemoryRange? mapped_range,
                  string? config_data, int * result) {
    if (loaded) return;
    loaded = true;

    Environment.init ();
    location = detect_location (mapped_range);

    // 加载配置
    config = (config_data != null)
        ? parse_config (config_data)
        : load_config (location);

    // 根据交互模式创建对应的 Controller
    var interaction = config.interaction;
    if (interaction is ScriptInteraction) {
        controller = new ScriptRunner (config, location);
    } else if (interaction is ScriptDirectoryInteraction) {
        controller = new ScriptDirectoryRunner (config, location);
    } else if (interaction is ListenInteraction) {
        controller = new ControlServer (config, location);
    } else if (interaction is ConnectInteraction) {
        controller = new ClusterClient (config, location);
    }
}
```

15.4 配置文件：Gadget 的行为说明书

Gadget 的行为由配置文件 `frida-gadget.config.json` 控制。这个文件需要放在与 Gadget 动态库相同的目录下。让我们看看 Frida 支持的配置选项：

```
{
  "interaction": {
    "type": "listen",
    "address": "0.0.0.0",
    "port": 27042,
    "on_port_conflict": "fail",
    "on_load": "wait"
  },
  "teardown": "minimal",
  "runtime": "default",
  "code_signing": "optional"
}
```

从源码中的 Config 类定义，我们可以看到所有配置项：

```
// 简化的 Config 结构
private sealed class Config : Object {
    public Object interaction;    // 交互模式（核心配置）
    public TeardownRequirement teardown;  // 清理策略
    public ScriptRuntime runtime;         // JS 引擎选择
    public CodeSigningPolicy code_signing; // 代码签名策略
}
```

15.5 四种交互模式详解

这是 Gadget 最重要的部分。四种交互模式决定了 Gadget 的工作方式：

```
┌──────────────────────────────────────────────────────────┐
│                Gadget 交互模式                            │
│                                                          │
│   ┌──────────────────────────────────────────────────┐   │
│   │  1. listen (监听模式) -- 默认                      │   │
│   │     Gadget 启动一个 TCP/Unix 服务器                 │   │
│   │     等待 Frida 客户端连接                           │   │
│   │     适合: 开发调试，需要交互式分析                   │   │
│   └──────────────────────────────────────────────────┘   │
│                                                          │
│   ┌──────────────────────────────────────────────────┐   │
│   │  2. connect (连接模式)                             │   │
│   │     Gadget 主动连接到指定的 Frida Portal            │   │
│   │     适合: 设备在 NAT 后面，无法被直连               │   │
│   └──────────────────────────────────────────────────┘   │
│                                                          │
│   ┌──────────────────────────────────────────────────┐   │
│   │  3. script (脚本模式)                              │   │
│   │     Gadget 直接加载指定的 JS 脚本文件               │   │
│   │                                                  │   │
```

```
│   │      不需要 Frida 客户端连接                          │    │
│   │      适合: 自动化测试, 生产环境监控                  │    │
│   └──────────────────────────────────────────────────┘    │
│                                                            │
│   ┌──────────────────────────────────────────────────┐    │
│   │  4. script-directory (脚本目录模式)                  │    │
│   │      Gadget 加载指定目录下的所有脚本                 │    │
│   │      支持文件变更时自动重扫描                        │    │
│   │      适合: 多脚本管理, 热更新场景                    │    │
│   └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

每种模式对应的配置文件示例:

Listen 模式（默认，最常用）：

```
{
  "interaction": {
    "type": "listen",
    "address": "127.0.0.1",
    "port": 27042,
    "on_load": "wait"
  }
}
```

当 `on_load` 设为 `"wait"` 时，Gadget 会在加载时阻塞应用程序的主线程，直到 Frida 客户端连接上来。这在调试启动阶段的代码时特别有用。设为 `"resume"` 则允许应用正常启动，客户端随时可以连接。

Script 模式（独立运行）：

```
{
  "interaction": {
    "type": "script",
    "path": "/path/to/agent.js",
    "parameters": {
      "target_function": "secret_check"
    },
    "on_change": "reload"
  }
}
```

从源码中的 ScriptInteraction 类可以看到，脚本模式支持传入自定义参数和变更行为:

```
private sealed class ScriptInteraction : Object {
    public string path;            // 脚本文件路径
    public Json.Node parameters;   // 传给脚本的参数
    public ChangeBehavior on_change;  // 文件变更时的行为
    // on_change: IGNORE (忽略) 或 RELOAD (重新加载)
}
```

Connect 模式和 Script-Directory 模式的配置类似，分别指定远程地址/token 和脚本目录路径。Script-Directory 模式还支持进程过滤（通过 `ProcessFilter` 按可执行文件名、Bundle ID 或 ObjC 类名匹配），每个脚本可以有独立配置，只在符合条件的进程中运行。

15.6 等待与恢复机制

Gadget 的一个关键设计是 “wait for resume” 机制。在 listen 模式下，应用可能需要暂停在 constructor 中，等 Frida 客户端连接并完成 Hook 设置后再继续运行：

```
// 简化示意
wait_for_resume_needed = true;

var listen_interaction = config.interaction as ListenInteraction;
if (listen_interaction != null
    && listen_interaction.on_load == LoadBehavior.RESUME) {
    wait_for_resume_needed = false;
}

if (wait_for_resume_needed && Environment.can_block_at_load_time
()) {
    // 在当前线程阻塞，直到客户端连接并调用 resume
    var loop = new MainLoop (wait_for_resume_context, true);
    wait_for_resume_loop = loop;
    loop.run ();  // 阻塞在这里
}
```

当 Frida 客户端调用 `device.resume(pid)` 时，Gadget 端的 `resume()` 方法会修改状态为 STARTED，通过条件变量唤醒阻塞的线程，并退出等待循环。这个机制保证了你可以在应用的第一行业务代码执行之前就完成所有 Hook 设置。

15.7 Gadget vs 动态注入：如何选择？

Gadget vs 动态注入 对比

	动态注入	Gadget
需要修改应用?	不需要	需要
需要 root/越狱?	通常需要	不需要
启动时 Hook?	有时间窗口	可以完美覆盖
适用环境	开发/测试设备	任何环境
对 App Store	无影响	不能上架
调试便利性	即连即用	需要重新打包
自动化测试	需要额外步骤	天然支持
CI/CD 集成	较复杂	简单

15.8 实际使用场景

场景一：iOS 应用分析（非越狱）

在 iOS 上，没有越狱就无法使用 ptrace 注入。常见做法是： 1. 解包 IPA 文件 2. 将 `FridaGadget.dylib` 放入 Frameworks 目录 3. 修改二进制的 Load Commands，添加对 Gadget 的依赖 4. 重签名后安装

应用启动时，iOS 的 dyld 会自动加载 Gadget，触发 constructor。

场景二：嵌入式设备 - 编译时链接 `frida-gadget.so`，配置 script 模式指向监控脚本，设备启动后自动插桩。

场景三：自动化测试 - CI/CD 中构建内嵌 Gadget 的测试版应用，配合 script 模式自动收集覆盖率、检测内存泄漏，无需额外 Frida 服务端。

15.9 Gadget 的卸载

Gadget 的卸载通过 `destructor`（Linux/macOS）或 `DLL_PROCESS_DETACH`（Windows）触发。卸载时，Gadget 会在工作线程中停止 Controller，等待所有异步操作完成。配置中的 `teardown` 项控制清理程度：`"full"` 完全清理所有资源（适合测试），`"minimal"`（默认）只做最少清理以加快退出速度。

本章小结

- **Gadget** 是 Frida 的嵌入式插桩方案，通过编译/打包时嵌入，利用 `constructor/DllMain` 自动初始化
- 支持四种交互模式：**listen**（等待连接）、**connect**（主动连接）、**script**（独立脚本）、**script-directory**（多脚本目录）
- 通过 `frida-gadget.config.json` 配置文件控制行为，配置文件放在 Gadget 动态库的同级目录
- “wait for resume” 机制可以在应用启动前完成所有 Hook 设置
- Gadget 模式特别适合 **iOS 非越狱分析**、**嵌入式设备**和 **CI/CD 自动化测试**场景

思考题

1. 在 iOS 非越狱环境下使用 Gadget，重签名后的应用无法上架 App Store。如果你需要在生产环境中做类似的监控，有什么替代方案？
2. Gadget 的 listen 模式中，`on_load: "wait"` 会阻塞主线程。如果应用有启动超时机制（比如 iOS 的 watchdog），可能会被系统杀掉。你会怎么处理这个问题？
3. 比较 Gadget 的 script 模式和 connect 模式，在一个需要远程管理的物联网设备上，你会选择哪种？为什么？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第16章：Linux 后端——ptrace 与 ELF 的世界

> 你有没有想过，当你在 Linux 上运行 `frida -p 1234` 的时候，Frida 到底是怎么把代码”塞”进另一个进程的？在 Linux 这个开放的世界里，内核给了我们一把万能钥匙——ptrace。让我们打开这扇门，看看 Frida 在 Linux 上的全部秘密。

16.1 Linux 后端的整体架构

```
┌──────────────────────────────────────────────┐
│              LinuxHostSession                 │
│  (管理本地 Linux 主机上的所有调试会话)          │
├──────────────┬──────────────┬────────────────┤
│ Linjector    │ SpawnGater   │ ProcessEnumerator │
│ (注入器)     │ (eBPF拦截)   │ (进程枚举)        │
├──────────────┴──────────────┴────────────────┤
│           LinuxHelperBackend                  │
│  (fork/ptrace/注入的核心实现)                  │
├──────────────────────────────────────────────┤
│   Linux Kernel: ptrace, /proc, eBPF           │
└──────────────────────────────────────────────┘
```

打个比方，`LinuxHostSession` 就像一个指挥中心，`Linjector` 是执行注入任务的特种兵，而 `LinuxHelperBackend` 是特种兵手里的装备库——里面有 ptrace、fork 这些底层武器。

16.2 ptrace——Linux 上的万能调试接口

16.2.1 spawn 的实现：fork + ptrace + execve

当你调用 `frida.spawn("/path/to/app")` 时，Frida 内部发生了这些事情：

```
// 简化自 frida-helper-backend.vala 中的 spawn 实现
pid = fork();
if (pid == 0) {
    setsid();                  // 创建新会话
    ptrace(PTRACE_TRACEME);      // 告诉内核："我愿意被父进程跟踪"
    raise(SIGSTOP);              // 停下来等待父进程
    execve(path, argv, envp);    // 替换为目标程序
}
// 父进程
wait_for_signal(pid, SIGSTOP);   // 等待子进程停在 SIGSTOP
ptrace(PTRACE_CONT, pid);        // 让子进程继续执行
wait_for_signal(pid, SIGTRAP);   // 等待 execve 触发的 SIGTRAP
// 现在目标程序已加载但还没开始运行，可以注入了！
```

这个流程就像导演喊”预备——开始——暂停！“：子进程 fork 出来后先用 PTRACE_TRACEME 声明自己愿意被跟踪，然后 raise(SIGSTOP) 停下来。父进程收到信号后让子进程继续，子进程执行 execve 时会自动触发 SIGTRAP，此时目标程序已加载到内存但还没开始运行——正好是注入的最佳时机。

16.2.2 syscall 拦截：等待安全状态

注入代码不能在任意时刻进行。Frida 的做法是等待目标进入”安全”的系统调用：

```
// 简化自 frida-helper-backend-glue.c
gboolean _frida_syscall_satisfies(gint syscall_id, FridaSyscallMask mask) {
  switch (syscall_id) {
    case __NR_read: case __NR_readv:
      return (mask & SYSCALL_MASK_READ) != 0;
    case __NR_poll: case __NR_epoll_wait:
      return (mask & SYSCALL_MASK_POLL_LIKE) != 0;
    case __NR_futex:
      return (mask & SYSCALL_MASK_FUTEX) != 0;
    case __NR_accept:
      return (mask & SYSCALL_MASK_ACCEPT) != 0;
  }
  return FALSE;
}
```

read、poll、accept 这类调用意味着进程正在”等待”——就像一个人在咖啡厅里发呆，这时候你去找他搭话，远比他在高速公路上飙车时安全得多。

16.2.3 exec transition——跟踪程序替换

Frida 还能处理 exec 转换。当进程调用 execve 替换自身时，Frida 需要检测并重新建立控制：

```
// 简化自 frida-helper-backend.vala
public async void prepare_exec_transition(uint pid) {
    var session = yield ExecTransitionSession.open(pid);
    exec_transitions[pid] = session;
    update_process_status(pid, EXEC_PENDING);
}

public async void await_exec_transition(uint pid) {
    yield exec_transitions[pid].wait_for_exec(cancellable);
    // exec 完成后重新建立监控
}
```

这在监控通过 shell 脚本启动的程序时特别有用——shell 先 fork，子进程再 exec 成目标程序。

16.3 Linjector——Linux 注入器

`Linjector`（“L”代表 Linux）的注入分五步：

```
┌──────────────────────────────────────────────┐
│              Linjector 注入五步法              │
├──────────────────────────────────────────────┤
│  1. ptrace(ATTACH) 附加到目标进程              │
│  2. 在目标地址空间分配内存                      │
│  3. 将 agent .so 内容写入目标进程              │
│  4. 修改寄存器，让目标执行 dlopen 加载 agent    │
│  5. agent 加载后调用指定的入口函数              │
└──────────────────────────────────────────────┘
```

Frida 支持使用 memfd_create 来避免文件系统操作：

```
// 简化自 linjector.vala
if (MemoryFileDescriptor.is_supported()) {
    FileDescriptor fd = MemoryFileDescriptor.from_bytes(name,
blob);
    return yield inject_library_fd(pid, fd_stream, ...);
}
// 回退到临时文件方式
var file = new TemporaryFile.from_stream(name, blob_stream,
tempdir);
return yield inject_library_file(pid, file.path, ...);
```

memfd 就像一个”虚拟U盘”——数据只存在于内存中，不会留下文件痕迹。

16.4 /proc 文件系统——Linux 的透明窗口

Frida 大量使用 /proc 文件系统来获取进程信息。`ProcMapsSnapshot` 解析 `/proc/<pid>/maps` 获取内存映射：

```
// 简化自 proc-maps.vala
public static ProcMapsSnapshot from_pid(uint32 pid) {
    var snap = new ProcMapsSnapshot();
    var it = ProcMapsIter.for_pid(pid);  // 读取 /proc/<pid>/maps
    while (it.next()) {
        var m = new Mapping();
        m.start = it.start_address;
        m.end = it.end_address;
        m.readable = it.flags[0] == 'r';
        m.executable = it.flags[2] == 'x';
        m.path = it.path;
        maps.add(m);
    }
    return snap;
}
```

/proc/PID/maps 就像一份详细的”地产登记簿”，记录了进程的每一块内存是谁的、能做什么。遍历 /proc 下的数字目录就能枚举所有进程。

16.5 Helper 进程架构

Frida 使用独立的 Helper 进程来完成 ptrace 等敏感操作：

```
┌──────────────────┐          D-Bus IPC          ┌──────────────────────┐
│   Frida Client   │<──────────────────────>│   frida-helper       │
│  (你的脚本进程)   │                        │  (特权辅助进程)        │
└──────────────────┘                        └──────────────────────┘
                                                        │
                                                        │ ptrace
                                                        v
                                              ┌──────────────────────┐
                                              │    Target Process    │
                                              └──────────────────────┘
```

两个核心原因：**权限隔离**（Helper 可以以 root 权限运行）和**稳定性**（Helper 崩溃不影响客户端）。

16.6 eBPF 集成——现代化的进程监控

16.6.1 SpawnGater——进程创建拦截

SpawnGater 使用 eBPF 程序拦截新进程创建，通过 Ringbuf 将事件从内核传递到用户空间：

```
// 简化自 spawn-gater.vala
public void start() throws Error {
    var obj = BpfObject.open("spawn-gater.elf",
        Data.HelperBackend.get_spawn_gater_elf_blob().data);
    events_reader = new
BpfRingbufReader(obj.maps.get_by_name("events"));
    obj.load();
    foreach (var program in obj.programs)
        links.add(program.attach());  // 附加到内核钩子
}
```

16.6.2 ActivitySampler——性能采样

ActivitySampler 为每个 CPU 核心创建 perf event，通过 eBPF 采样目标进程的调用栈，就像”交通摄像头”定时拍快照。

16.7 ELF 模块处理

在 Linux 上，可执行文件和共享库使用 ELF（Executable and Linkable Format）格式。Frida 的 `GumElfModule`（位于 `gum/backend-linux/gummodule-linux.c`）负责解析 ELF 文件：

```
┌──────────────────────────────────────┐
│              ELF 文件结构              │
├──────────────────────────────────────┤
│  ELF Header (魔数、架构、入口点)        │
├──────────────────────────────────────┤
│  Program Headers (LOAD/DYNAMIC等)     │
├──────────────────────────────────────┤
│  .text     (代码段)                   │
│  .dynsym   (动态符号表)               │
│  .dynstr   (动态字符串表)             │
│  .plt/.got (链接/偏移表)              │
├──────────────────────────────────────┤
│  Section Headers                     │
└──────────────────────────────────────┘
```

Frida 需要解析 ELF 来完成三件事：找到 dlopen/dlsym 等函数的地址、枚举模块中的导出符号、确定模块的加载基址和内存布局。这些信息对于注入和 hook 都至关重要。

16.8 本章小结

- Linux 后端的核心武器是 **ptrace** 系统调用，提供进程跟踪、内存读写、寄存器操作
- **spawn** 通过 fork + ptrace(TRACEME) + execve 实现，在 SIGTRAP 处暂停目标
- **注入** 通过 Linjector 完成，本质是在目标进程中执行 dlopen 加载 agent .so
- **/proc** 文件系统用于枚举进程和解析内存映射
- **Helper 进程** 提供权限隔离和稳定性保障
- **eBPF** 带来现代化的内核级监控，SpawnGater 拦截进程创建，ActivitySampler 采样活动
- **memfd** 优化让注入完全在内存中完成，不留磁盘痕迹

讨论问题

1. ptrace 的 PTRACE_TRACEME 和 PTRACE_ATTACH 有什么区别？在什么场景下 Frida 会使用 PTRACE_ATTACH？

2. eBPF 相比传统的 ptrace 监控有什么优势和局限？为什么 Frida 同时使用两种技术？

3. memfd_create 在较老的 Linux 内核上不可用，Frida 的回退方案（临时文件）有什么潜在的安全风险？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第17章：Darwin 后端——Mach 端口与 XPC

> 如果说 Linux 是一间门窗大开的房子，那 Apple 的平台就是一座层层设防的堡垒。在 macOS 和 iOS 上，Frida 不能像 Linux 那样直接用 ptrace 为所欲为——它必须学会使用 Mach 端口、理解代码签名、应对策略限制。让我们来看看 Frida 是如何在 Apple 的地盘上施展身手的。

17.1 Darwin 后端的整体架构

```
┌──────────────────────────────────────────────────────────┐
│                    DarwinHostSession                      │
│  (macOS/iOS 主会话，管理设备上的调试会话)                  │
├──────────────┬──────────────┬──────────────────────────┤
│Fruitjector│ FruitController│ PolicySoftener            │
│(注入器)    │ (iOS应用管理)  │ (策略软化)                 │
├──────────────┴──────────────┴──────────────────────────┤
│                 DarwinHelperBackend                      │
│  (Mach端口操作、进程spawn、代码注入)                      │
├──────────────────────────────────────────────────────────┤
│  Mach Kernel: task_port, thread, exception_port          │
└──────────────────────────────────────────────────────────┘
```

17.2 Mach 端口——Darwin 的进程控制基石

每个 Darwin 进程都有一个 task port，拿到这个端口就相当于拿到了进程的”遥控器”。通过它你可以：

```
┌──────────────────────────────────────────────────────┐
│                task_port 能做的事情                   │
├──────────────────────────────────────────────────────┤
│  mach_vm_allocate()     - 在目标进程分配内存          │
│  mach_vm_write()        - 向目标进程写入数据          │
│  mach_vm_read()         - 读取目标进程内存            │
│  mach_vm_protect()      - 修改内存保护属性            │
│  thread_create()        - 在目标进程创建线程          │
│  thread_set_state()     - 修改线程寄存器              │
│  task_threads()         - 枚举所有线程                │
└──────────────────────────────────────────────────────┘
```

这就像拿到了一个人的银行账号加密码——存钱、取钱、转账、查余额，几乎什么都能做。

17.2.1 注入的 Mach 端口操作

```
// 简化自 frida-helper-backend-glue.m
struct FridaInjectInstance {
    guint pid;
```

```
    mach_port_t task;                // 目标进程的 task port
    mach_vm_address_t payload_address;   // 注入代码地址
    mach_vm_size_t payload_size;
    FridaAgentContext *agent_context;    // Agent 上下文
    mach_port_t thread;              // 注入线程
};
```

注入的关键步骤如下：

```
┌──────────────┐
│ 1. 获取      │  task_for_pid(pid) -> task_port
│ task port    │
├──────────────┤
│ 2. 分配      │  mach_vm_allocate(task, &addr, size)
│ 远程内存     │
├──────────────┤
│ 3. 写入      │  mach_vm_write(task, addr, payload, size)
│ 注入代码     │
├──────────────┤
│ 4. 设置      │  mach_vm_protect(task, addr, size, RX)
│ 内存权限     │
├──────────────┤
│ 5. 创建      │  thread_create_running(task, ...)
│ 远程线程     │
├──────────────┤
│ 6. 监控      │  dispatch_source 监控线程完成
│ 线程执行     │
└──────────────┘
```

17.2.2 AgentContext——两阶段执行

注入代码分两个阶段：先在 Mach 线程中创建 POSIX 线程，再由 POSIX 线程执行 dlopen。为什么这么绕？因为 Mach 线程是内核级线程，缺少 TLS 等用户态基础设施：

```
// 简化自 frida-helper-backend-glue.m
struct FridaAgentContext {
    // Mach 线程阶段
    GumAddress mach_task_self_impl;
    GumAddress pthread_create_from_mach_thread_impl;
    // POSIX 线程阶段
    GumAddress dlopen_impl;        // 加载 agent dylib
    GumAddress dlsym_impl;         // 查找入口函数
    GumAddress dlclose_impl;
    // 数据存储
    gchar dylib_path_storage[256];
    gchar entrypoint_name_storage[256];
    gchar entrypoint_data_storage[4096];
};
```

17.3 Spawn 控制——断点与 dyld

在 Darwin 上 spawn 使用 `posix_spawn` 配合异常端口。Frida 需要在 dyld（动态链接器）的执行过程中设置硬件断点，精确控制加载流程：

```
┌──────────────────────────────────────────────┐
│          Spawn 断点阶段（简化版）              │
├──────────────────────────────────────────────┤
│  DETECT_FLAVOR  -> 判断 dyld 版本             │
│  [V4+] LIBSYSTEM_INITIALIZED                 │
│  [V3-] SET_HELPERS -> DLOPEN_LIBC            │
│  [共同] CF_INITIALIZE -> CLEANUP -> DONE     │
└──────────────────────────────────────────────┘
```

源码中 `FridaSpawnInstance` 维护了最多 4 个硬件断点，每到一个检查站（断点阶段）就暂停进程，Frida 确认没问题后才放行——就像跑步比赛中每隔一段距离设置检查站。

Frida 必须区分 dyld V4+（现代 macOS/iOS）和 V3-（旧版），因为两者的初始化路径完全不同。

17.4 Fruitjector——Darwin 注入器

`Fruitjector`（“水果注入器”）是 Darwin 平台的注入器，它优先尝试 mmap 方式，回退到文件方式：

```
// 简化自 fruitjector.vala
public async uint inject_library_resource(uint pid, AgentResource
resource, ...) {
    var blob = yield helper.try_mmap(resource.blob);
    if (blob == null)
        return yield inject_library_file(pid,
resource.get_file().path, ...);
    return yield helper.inject_library_blob(pid, resource.name,
blob, ...);
}
```

17.5 GumDarwinModule 与 GumDarwinMapper

`GumDarwinModule` 解析 Mach-O 格式（Darwin 的可执行文件格式），`GumDarwinMapper` 更进一步，能在目标进程中完整重建 Mach-O 模块：

```
// 简化自 gumdarwinmapper.c
struct GumDarwinMapper {
  GumDarwinModule *module;
  GumDarwinModuleResolver *resolver;
  gsize vm_size;
  GumAddress process_chained_fixups;  // 现代 dyld 的链式修复
```

```
    GumAddress tlv_get_addr_addr;       // TLV 支持
};
```

它就像一个”模块搬运工”，把 dylib 从磁盘搬到目标进程内存中，处理好所有重定位和符号绑定。

Mach-O 与 ELF 的一个重要区别是 **Chained Fixups**（链式修复）。现代 macOS/iOS 的 dyld 使用这种紧凑格式来描述需要修复的指针。GumDarwinMapper 必须正确处理这些修复，否则加载的模块无法正常运行。

另一个关键点是 **Objective-C 元数据**。Darwin 上大量使用 ObjC，Mach-O 文件中包含 `__objc_classlist`、`__objc_methnames` 等 section。Frida 的 `GumObjcApiResolver`（位于 `gum/backend-darwin/gumobjcapiresolver.c`）专门用于解析和搜索 ObjC 的类和方法。

17.6 代码签名与 PolicySoftener

Apple 平台最大的障碍之一是**代码签名**。在 iOS 上，所有可执行代码必须经过签名验证。Frida 通过 `PolicySoftener` 接口应对不同越狱环境：

```
// 简化自 policy-softener.vala
// 自动选择合适的实现
if (InternalIOSTVOSPolicySoftener.is_available())
    softener = new InternalIOSTVOSPolicySoftener();
else if (ElectraPolicySoftener.is_available())
    softener = new ElectraPolicySoftener();
else if (Unc0verPolicySoftener.is_available())
    softener = new Unc0verPolicySoftener();
else
    softener = new IOSTVOSPolicySoftener();
```

PolicySoftener 还会临时放宽 iOS 的内存限制，防止注入 agent 后因内存占用过大而被系统杀掉（jetsam）。它保存原始限制，在 20 秒后自动恢复：

```
// 简化自 policy-softener.vala
protected virtual ProcessEntry perform_softening(uint pid) {
    MemlimitProperties? saved_limits = null;
    if (!DarwinHelperBackend.is_application_process(pid)) {
        saved_limits = try_commit_memlimit_properties(
            pid, MemlimitProperties.without_limits());
    }
    var entry = new ProcessEntry(pid, saved_limits);
    // 20秒后自动恢复原始限制
    var expiry = new TimeoutSource.seconds(20);
    expiry.set_callback(() => { forget(pid); return false; });
    return entry;
}
```

注意 PolicySoftener 有多个实现——对应不同的越狱方案（Electra、unc0ver 等），因为每种越狱对系统策略的修改方式不同。在非越狱设备上只能使用 NullPolicySoftener（什么都不做）。

17.7 DTrace 与 Spawn 监控

在 macOS 上，Frida 使用 DTrace（内置的动态跟踪框架）实现进程 spawn 监控，类似 Linux 上的 eBPF。`DTraceAgent` 在 Helper 后端构造时初始化，当启用 spawn gating 时，它监视系统中新进程的创建。

17.8 本章小结

- Darwin 后端的核心是 **Mach 端口**，通过 task port 可以完全控制另一个进程
- **注入** 分两阶段：Mach 线程创建 POSIX 线程，POSIX 线程执行 dlopen
- **Spawn 控制** 通过异常端口和硬件断点实现，需要精确跟踪 dyld 初始化流程
- **GumDarwinMapper** 能在目标进程中完整重建 Mach-O 模块
- **代码签名** 是 Apple 平台特有的挑战，PolicySoftener 适配不同越狱环境
- **DTrace** 在 macOS 上提供进程监控能力

讨论问题

1. 为什么注入需要两个阶段（Mach 线程 -> POSIX 线程），而不能直接在 Mach 线程上执行 dlopen?

2. dyld V4+ 和 V3- 的区别对 Frida 意味着什么？为什么需要在运行时检测版本?

3. 在没有越狱的 iOS 设备上，Frida 面临哪些根本性的限制?

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第18章：Windows 后端——远程线程与调试符号

> 在 Windows 的世界里，注入 DLL 几乎是一项”民间传统艺术”。从游戏外挂到安全工具，无数程序都在使用 CreateRemoteThread + LoadLibrary 这个经典组合。Frida 的 Windows 后端也不例外，但它把这个”传统技艺”做到了工程化的极致。让我们揭开 Windows 后端的面纱。

18.1 Windows 后端的整体架构

```
┌──────────────────────────────────────────────────┐
│              WindowsHostSession                  │
│   (管理 Windows 本地调试会话)                     │
├──────────┬──────────────────────────────────────┤
│ Winjector │  ApplicationEnumerator               │
```

```
│ (注入器)  │  ProcessEnumerator                    │
├──────────┴──────────────────────────────────────┤
│        WindowsHelperBackend                      │
│  (CreateRemoteThread, DLL注入)                   │
├──────────────────────────────────────────────────┤
│  Windows Kernel: Process/Thread/Memory APIs      │
└──────────────────────────────────────────────────┘
```

18.2 CreateRemoteThread——经典的注入手法

Windows 提供了一个堪称”为注入而生”的 API：`CreateRemoteThread`。核心注入流程：

```
// 简化自 frida-helper-backend-glue.c
void inject_library_file(uint32 pid, const char *path,
        const char *entrypoint, const char *data,
        void **inject_instance, void **thread_handle) {
    frida_enable_debug_privilege();  // 启用调试特权

    HANDLE process = OpenProcess(
        PROCESS_VM_OPERATION | PROCESS_VM_READ | PROCESS_VM_WRITE
        | PROCESS_CREATE_THREAD | PROCESS_QUERY_INFORMATION,
        FALSE, pid);

    FridaRemoteWorkerContext rwc;
    frida_remote_worker_context_init(&rwc, &details);

    // 创建远程线程执行注入代码
    HANDLE thread = CreateRemoteThread(process, NULL, 0,
        rwc.entrypoint, rwc.argument, 0, NULL);

    // 如果失败，尝试更底层的 API
    if (thread == NULL) {
        RtlCreateUserThread(process, NULL, FALSE, 0, NULL, NULL,
            rwc.entrypoint, rwc.argument, &thread, &client_id);
    }
}
```

18.2.1 RemoteWorkerContext——远程执行上下文

Frida 在目标进程中构建完整的”工作代码”：

```
struct FridaRemoteWorkerContext {
    gboolean stay_resident;
    gpointer load_library_impl;      // LoadLibraryW
    gpointer get_proc_address_impl;  // GetProcAddress
    gpointer free_library_impl;      // FreeLibrary
    gpointer virtual_free_impl;      // VirtualFree
    WCHAR dll_path[MAX_PATH + 1];
```

```
    gchar entrypoint_name[256];
    gchar entrypoint_data[MAX_PATH];
    gpointer entrypoint;            // 远程代码入口
    gpointer argument;              // 远程数据地址
};
```

远程线程在目标进程中执行的伪代码：

```
// 这段代码在目标进程中执行（简化版）
DWORD WINAPI remote_worker(FridaRemoteWorkerContext *ctx) {
    HMODULE mod = ctx->load_library_impl(ctx->dll_path);
    FARPROC entry = ctx->get_proc_address_impl(mod, ctx->entrypoint_name);
    entry(ctx->entrypoint_data);          // 调用 agent 入口
    if (!ctx->stay_resident)
        ctx->free_library_impl(mod);      // 如果不常驻则卸载
    ctx->virtual_free_impl(ctx, 0, MEM_RELEASE);  // 释放自身
    return 0;
}
```

注意这里的函数指针（load_library_impl 等）是在 Frida 端通过枚举 kernel32.dll 的导出表解析出来的，不是硬编码的地址。

18.2.2 RtlCreateUserThread——备用方案

有些受保护的进程会拒绝 CreateRemoteThread。Frida 回退到 ntdll.dll 中未文档化的 `RtlCreateUserThread`——正门进不去就走侧门，这个更底层的 API 有时能绕过限制。

18.3 DbgHelp 与符号解析

Windows 后端的一大特色是内置完整的调试符号支持。Frida 为每种架构都捆绑了 `dbghelp.dll` 和 `symsrv.dll`：

```
// 简化自 windows-host-session.vala
agent = new AgentDescriptor(
    PathTemplate("<arch>\\frida-agent.dll"),
    new AgentResource[] {
        new AgentResource("arm64\\dbghelp.dll", ...),
        new AgentResource("arm64\\symsrv.dll", ...),
        new AgentResource("x86_64\\dbghelp.dll", ...),
        new AgentResource("x86_64\\symsrv.dll", ...),
        new AgentResource("x86\\dbghelp.dll", ...),
        new AgentResource("x86\\symsrv.dll", ...),
    }
);
```

dbghelp.dll 提供符号查找（SymFromName/SymFromAddr）、符号枚举和栈回溯（StackWalk64）。symsrv.dll 能从 Microsoft 符号服务器自动下载 PDB 文件。Frida 自带这两个 DLL 而不用系统的，是为了确保一致的行为——就像修理工自带全套工具上门，不依赖客户家里有没有工具箱。

18.4 多架构支持

Windows 上一个 64 位系统可以同时运行 32 位和 64 位进程（WoW64），加上 ARM64 Windows，Frida 必须为三种架构准备 agent：

```
public unowned string arch_name_from_pid(uint pid) {
    switch (cpu_type_from_pid(pid)) {
        case CpuType.IA32:   return "x86";
        case CpuType.AMD64:  return "x86_64";
        case CpuType.ARM64:  return "arm64";
    }
}
```

这意味着 Frida 的 Windows 发布包中实际包含六个二进制文件（三种架构的 agent DLL 各一个，加上对应的 dbghelp.dll）。Helper 进程在注入前通过 `Gum.Windows.cpu_type_from_pid()` 判断目标架构，选择正确的 DLL 版本。

18.5 ACL 与安全控制

Frida 在使用临时文件时需要正确设置 ACL（访问控制列表），否则目标进程可能没权限读取注入的 DLL：

```
// 简化自 winjector-glue.c
void frida_winjector_set_acls_as_needed(const char *path) {
   LPCWSTR sddl = frida_access_get_sddl_string_for_temp_directory();
   if (sddl != NULL) {
      SECURITY_DESCRIPTOR *sd;
      ConvertStringSecurityDescriptorToSecurityDescriptor(sddl, ...);
      PACL dacl;
      GetSecurityDescriptorDacl(sd, &present, &dacl, &defaulted);
      SetNamedSecurityInfo(path, SE_FILE_OBJECT,
            DACL_SECURITY_INFORMATION,
         NULL, NULL, dacl, NULL);
   }
}
```

SeDebugPrivilege 是另一个关键——这张”万能通行证”让 Frida 能打开几乎任何进程的句柄，通常只有管理员才能获得。

18.6 Winjector 与异步监控

`Winjector` 封装了注入逻辑，`WaitHandleSource` 将 Windows 句柄等待机制集成到 GLib 事件循环中：

```
// 简化自 frida-helper-backend.vala
private void monitor_remote_thread(uint id, void *instance,
        void *waitable_thread_handle) {
    var source = WaitHandleSource.create(waitable_thread_handle,
true);
    source.set_callback(() => {
```

```
        _free_inject_instance(instance, out is_resident);
        uninjected(id);
        return false;
    });
    source.attach(main_context);
}
```

在提升权限模式下，Frida 将 DLL 复制到安全目录（AssetDirectory），确保目标进程有权限访问。

18.7 DEP 与 ASLR 应对

DEP 防止数据区域执行代码——Frida 的注入代码分配为 PAGE_EXECUTE_READWRITE 内存。**ASLR** 让每次加载地址不同——Frida 通过枚举 kernel32.dll 的导出表来动态解析函数地址，不依赖硬编码：

```
static gboolean collect_kernel32_export(const GumExportDetails *details,
        gpointer user_data) {
    FridaRemoteWorkerContext *rwc = user_data;
    if (strcmp(details->name, "LoadLibraryW") == 0)
        rwc->load_library_impl = GSIZE_TO_POINTER(details->address);
    else if (strcmp(details->name, "GetProcAddress") == 0)
        rwc->get_proc_address_impl = GSIZE_TO_POINTER(details->address);
    // ...
    return !has_resolved_all();
}
```

18.8 本章小结

- Windows 注入的核心是 **CreateRemoteThread + LoadLibrary**，回退方案是 **RtlCreateUserThread**
- **RemoteWorkerContext** 包含远程执行所需的所有函数指针和数据
- Frida 自带 **dbghelp.dll** 和 **symsrv.dll** 提供完整符号解析
- **ACL** 设置确保目标进程能访问注入的 DLL
- **SeDebugPrivilege** 是操作其他进程的前提
- **WaitHandleSource** 将 Windows 句柄等待集成到 GLib 事件循环
- 通过正确的内存分配和动态地址解析应对 **DEP** 和 **ASLR**

讨论问题

1. CreateRemoteThread 和 RtlCreateUserThread 各在什么情况下会失败？Windows 上还有哪些代码注入技术？

2. Frida 为什么要自带 dbghelp.dll？这样做有什么潜在的兼容性问题？

3. Windows 的 Protected Process Light (PPL) 机制对 Frida 的注入有什么影响？

第19章：移动平台——Android 与 iOS 的特殊挑战

> 当 Frida 从桌面走向口袋里的手机，一切都变得更加困难。Android 有 SELinux 和沙箱，iOS 有代码签名和越狱检测。在移动平台上，Frida 不仅要完成注入，还要学会”伪装”和”潜行”。本章我们来看看 Frida 是如何在这两大移动系统上求生存的。

19.1 两大移动后端概览

```
┌──────────────────┬──────────────────┐
│     Android      │       iOS        │
│   Droidy 后端    │   Fruity 后端    │
├──────────────────┼──────────────────┤
│ droidy-client    │ fruity-host-sess │
│ droidy-host-sess │ usbmux / lockdown│
│ injector / jdwp  │ xpc / dtx        │
├──────────────────┼──────────────────┤
│ ADB Protocol     │ usbmux Protocol  │
│ (TCP :5037)      │ (Unix socket)    │
├──────────────────┼──────────────────┤
│  adb server      │  usbmuxd         │
│  USB / TCP       │  USB / WiFi      │
└──────────────────┴──────────────────┘
```

两个后端都运行在**主机端**（你的电脑上），通过各自的协议与设备通信。

19.2 Android：Droidy 后端与 ADB 集成

19.2.1 设备发现

Droidy 通过 ADB 协议发现和连接 Android 设备：

```
// 简化自 droidy-client.vala
public sealed class DeviceTracker : Object {
    public async void open(Cancellable? cancellable) {
        client = yield Client.open(cancellable);
        var devices = yield client.request_data("host:track-
devices-l");
        yield update_devices(devices);
    }

    private async void update_devices(string encoded) {
```

```
        foreach (var line in encoded.split("\n")) {
            // 解析: "<serial>\t<state>\t<model:xxx ...>"
            if (type == "device")
                device_attached(new DeviceDetails(serial, name));
        }
    }

    private async string detect_name(string serial) {
        return yield ShellCommand.run("getprop ro.product.model",
serial);
    }
}
```

19.2.2 ShellSession——ADB Shell v2

Frida 使用 Shell v2 协议与设备交互，支持结构化的输入输出和退出码获取：

```
// 简化自 droidy-client.vala
public async void open(string device_serial) {
    var client = yield Client.open();
    yield client.request("host:transport:" + device_serial);
    yield client.request_protocol_change("shell,v2,raw:");
}
```

19.2.3 Gadget 注入——JDWP 的妙用

除了传统的 ptrace 注入（需要设备端 frida-server），Frida 还支持通过 Gadget + JDWP 方式注入 Java 应用：

```
// 简化自 droidy/injector.vala
private async GadgetDetails inject_gadget() {
    string so_path = "/data/local/tmp/frida-gadget-" + instance_id
+ ".so";
    string unix_socket = "frida:" + package;

    // 1. 推送 gadget .so 到设备
    yield FileSync.send(gadget, so_meta, so_path, device_serial);

    // 2. 复制到应用数据目录
    yield shell.check_call("cp " + so_path + " /data/data/" +
package + "/gadget.so");

    // 3. 通过 JDWP 断点触发 System.loadLibrary 加载 gadget
    // 4. gadget 监听 Unix socket 等待 Frida 连接
}
```

Gadget 模式的优势是不需要 root 权限，只要应用是 debuggable 的就行。

19.2.4 SELinux 的挑战

SELinux 给 Frida 带来多重障碍：

- **ptrace 限制**：非特权进程不能使用 ptrace，解决方案是 frida-server 以 root 运行
- **文件访问限制**：不同 SELinux 上下文的进程不能随意读文件，解决方案是 memfd_create
- **域转换限制**：注入的代码继承目标进程的上下文，某些操作可能被拒绝
- **执行保护**：SELinux 可能禁止执行非标签化代码

19.2.5 RoboLauncher——Android 应用管理

设备端的 frida-server 使用 `RoboLauncher` 管理应用启动和注入：

```
// 简化自 linux-host-session.vala (Android 部分)
#if ANDROID
    robo_launcher = new RoboLauncher(this, io_cancellable);
    robo_launcher.spawn_added.connect(on_spawn_added);
    crash_monitor = new CrashMonitor();
#endif
```

RoboLauncher 通过 Activity Manager（am start）启动应用，在 Zygote fork 出新进程后、Java 代码执行前完成注入。

19.3 iOS：Fruity 后端

19.3.1 usbmuxd——TCP 隧道

iOS 设备通过 usbmuxd 提供 TCP 隧道功能：

```
// 简化自 usbmux.vala
private async bool init_async(int io_priority, Cancellable?
cancellable) {
    #if WINDOWS
        connectable = new InetSocketAddress(
            new InetAddress.loopback(IPV4), 27015);
    #else
        connectable = new UnixSocketAddress("/var/run/usbmuxd");
    #endif
    connection = yield new
SocketClient().connect_async(connectable);
}
```

通过 usbmuxd，Frida 可以连接到设备上任意 TCP 端口，比如 frida-server 监听的 27042 端口。

19.3.2 Lockdown——设备配对认证

连接 iOS 设备前需要通过 Lockdown 协议完成配对：

```
// 简化自 lockdown.vala
public static async LockdownClient open(UsbmuxDevice device) {
    var usbmux = yield UsbmuxClient.open();
    Plist pair_record = yield
usbmux.read_pair_record(device.udid);

    // 从配对记录提取 TLS 证书
    var cert = pair_record.get_bytes_as_string("HostCertificate");
    var key = pair_record.get_bytes_as_string("HostPrivateKey");
    tls_certificate = new TlsCertificate.from_pem(cert + "\n" +
key);

    yield usbmux.connect_to_port(device.id, 62078);  // Lockdown
端口
    // 发送 StartSession，升级到 TLS 加密通信
}
```

19.3.3 XPC 与 DiscoveryService

较新的 iOS 版本上，Frida 使用 XPC 协议发现设备服务：

```
// 简化自 xpc.vala
public sealed class DiscoveryService : Object {
    public string query_udid() {
        return new VariantReader(handshake_body)
            .read_member("Properties").read_member("UniqueDeviceID'
            .get_string_value();
    }
    public ServiceInfo get_service(string identifier) {
        // 从握手响应中查找服务的端口号
    }
}
```

19.3.4 越狱 vs 非越狱

越狱设备	非越狱设备
frida-server root 运行	只能使用 Gadget 模式
可以 attach 任意进程	只能注入自己签名的应用
支持 spawn gating	不支持 spawn gating
PolicySoftener 处理限制	受代码签名严格限制
完整的系统级控制	功能受限

越狱设备上，frida-server 通过 task_for_pid 获取 task port，使用 Mach 端口注入（见第17章）。非越狱设备只能将 Gadget dylib 嵌入应用重新签名后安装。

19.4 移动平台的共同挑战

19.4.1 资源约束

移动设备 CPU 和内存有限，iOS 的 jetsam 会杀掉内存占用过大的进程。Frida 使用事件驱动模型避免忙等待，消息序列化也要高效。

19.4.2 反调试对抗

常见 Frida 检测手段	Frida 应对
扫描 27042 端口	自定义端口
查找 frida-server 进程	进程重命名
搜索 "LIBFRIDA" 字符串	代码修改
ptrace(TRACEME) 自保护	绕过检查
检查 /proc/self/maps	memfd 注入
dlopen 行为监控	Gadget 伪装

19.5 本章小结

- **Droidy 后端** 通过 ADB 协议与 Android 设备通信，支持设备发现和 Shell 交互
- **Gadget 注入** 利用 JDWP 将 agent 注入到 debuggable 的 Java 应用中，不需要 root
- **SELinux** 是 Android 上的主要障碍，Frida 通过 root 权限和 memfd 应对
- **Fruity 后端** 通过 usbmuxd 建立 TCP 隧道，Lockdown 完成设备认证
- **XPC** 用于发现 iOS 设备上的服务
- 越狱 iOS 拥有完整能力，非越狱只能用 Gadget 模式
- 移动应用的**反调试检测**是实际使用中的重要挑战
- 两大平台共同特点是资源受限和安全策略严格

讨论问题

1. Android 上 JDWP 注入 Gadget 与 frida-server 的 ptrace 注入相比，各有什么优缺点？

2. iOS 非越狱设备使用 Frida Gadget 需要重签名应用，这在实际工作中会遇到哪些问题？

3. 如果你在开发移动安全应用想检测 Frida，你会采用哪些手段？反过来如何规避？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第20章：DBus 协议——通信的骨架

> 你有没有想过，当你在 Python 脚本里调用 `session.attach(pid)` 的时候，这个请求是如何穿越进程边界，传递到 frida-server 里面去的？两个完全独立的进程，是怎么”说上话”的？

20.1 什么是 DBus？一个公交系统的比喻

在理解 Frida 的通信机制之前，我们先聊聊 DBus 这个概念。

想象一座城市的公交系统。乘客（消息发送者）站在站台上，告诉司机”我要去 XX 路”（调用某个方法）。公交车沿着固定的线路（总线）把乘客送到目的地。到了站之后，目的地的服务窗口（服务对象）接待乘客，完成业务。

DBus 就是 Linux 世界里的这套”公交系统”。它是一个进程间通信（IPC）协议，允许不同进程通过一条”总线”来交换消息。DBus 定义了三个核心概念：

```
┌──────────────────────────────────────────────────────┐
│                    DBus 核心概念                     │
├──────────────────────────────────────────────────────┤
│                                                      │
│  1. 接口 (Interface)                                 │
│     - 定义了一组方法和信号                           │
│     - 类似于 Java/C# 的 interface                    │
│     - 例如: re.frida.HostSession17                   │
│                                                      │
│  2. 对象路径 (Object Path)                           │
│     - 服务端暴露的"地址"                             │
│     - 例如: /re/frida/HostSession                    │
│                                                      │
│  3. 方法调用与信号 (Method Call & Signal)            │
│     - 方法调用: 请求-响应模式                        │
│     - 信号: 单向广播通知                             │
│                                                      │
└──────────────────────────────────────────────────────┘
```

需要注意，Frida 并没有使用系统级的 DBus 守护进程（比如 dbus-daemon）。它用的是 DBus 的**协议格式**，通过 GDBus 库在自己的 TCP/Unix Socket 连接上跑 DBus 消息。这就好比，你不需要公交公司的调度中心，你自己买了辆小巴，按照公交车的规矩在自己修的路上跑。

20.2 为什么 Frida 选择 DBus？

这是一个很好的问题。Frida 完全可以自己设计一套二进制协议，为什么要借用 DBus？原因有几个：

第一，GLib 生态的天然选择。 Frida 的核心代码使用 Vala 语言编写，Vala 编译到 C 后依赖 GLib/GObject 生态。GLib 自带 GDBus 库，提供了现成的、经过大量实战检验的 DBus 实现。用它就像搭积木一样方便。

第二，接口定义的优雅。 Vala 的 `[DBus]` 注解可以直接把一个接口声明为 DBus 接口，编译器自动生成序列化和反序列化代码。开发者只需要写接口定义，不需要手工处理协议细节。

第三，内置的异步支持。 GDBus 天然支持异步方法调用，完美契合 Frida 的 async/await 编程模型。

来看一段源码中的接口定义：

```
// 来自 session.vala（简化版）
[DBus (name = "re.frida.HostSession17")]
public interface HostSession : Object {
    public abstract async void ping (uint interval_seconds,
        Cancellable? cancellable) throws GLib.Error;

    public abstract async uint spawn (string program,
        HostSpawnOptions options,
        Cancellable? cancellable) throws GLib.Error;

    public abstract async AgentSessionId attach (uint pid,
        HashTable<string, Variant> options,
        Cancellable? cancellable) throws GLib.Error;

    // 信号：进程崩溃时广播通知
    public signal void process_crashed (CrashInfo crash);
    // 信号：Agent 会话断开时通知
    public signal void agent_session_detached (
        AgentSessionId id, SessionDetachReason reason, CrashInfo
crash);
}
```

看到 `[DBus (name = "re.frida.HostSession17")]` 这行了吗？Vala 编译器会根据这个注解，自动为 HostSession 接口生成 DBus 的代理（Proxy）和骨架（Skeleton）代码。名字里的数字 17 是协议版本号，每当接口发生不兼容变更时会递增。

20.3 三大核心接口

Frida 的 DBus 通信围绕三个核心接口展开。如果把 Frida 比作一家医院，那么：

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  HostSession（前台挂号）                                  │
│  ├── enumerate_processes()  列出所有"病人"（进程）        │
│  ├── spawn()                创建新进程                    │
│  ├── attach()               挂号，建立会话                │
│  └── kill()                 强制结束进程                  │
│                                                          │
│  AgentSessionProvider（手术室调度）                       │
│  ├── open()          打开一个 Agent 会话                  │
│  ├── migrate()       将会话迁移到新连接                   │
│  └── unload()        卸载 Agent                           │
```

```
│                                                                  │
│  AgentSession (医生坐诊)                                          │
│  ├── create_script()    创建脚本                                  │
│  ├── load_script()      加载脚本                                  │
│  ├── post_messages()    发送消息                                  │
│  └── close()            结束会话                                  │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

HostSession 是整个 Frida 服务的入口。客户端连接到 frida-server 后，第一件事就是获取 HostSession 的代理对象。通过它可以枚举进程、启动新进程、附加到目标进程。

AgentSessionProvider 负责管理注入到目标进程中的 Agent。当你调用 `attach()` 后，Frida 会把一个动态库注入到目标进程，这个库里运行的就是 Agent。AgentSessionProvider 负责打开和管理这些 Agent 的会话。

AgentSession 是你和目标进程中 Agent 之间的直接对话通道。创建脚本、加载脚本、收发消息，都走这个接口。

20.4 对象路径：DBus 的”门牌号”

每个 DBus 服务对象都需要注册到一个特定的路径上，就像每家店铺都有自己的门牌号。Frida 的对象路径定义在 `session.vala` 的 `ObjectPath` 命名空间中：

```
namespace ObjectPath {
    public const string HOST_SESSION =
        "/re/frida/HostSession";
    public const string AGENT_SESSION_PROVIDER =
        "/re/frida/AgentSessionProvider";
    public const string AGENT_SESSION =
        "/re/frida/AgentSession";
    public const string AGENT_MESSAGE_SINK =
        "/re/frida/AgentMessageSink";
    public const string AUTHENTICATION_SERVICE =
        "/re/frida/AuthenticationService";

    // 带 ID 的路径：每个 Agent 会话有独立地址
    public static string for_agent_session (AgentSessionId id) {
        return AGENT_SESSION + "/" + id.handle;
    }
}
```

注意 `for_agent_session()` 这个方法——当你同时调试多个进程时，每个 Agent 会话都有自己独一无二的路径，比如 `/re/frida/AgentSession/abc123`。这样同一条 DBus 连接上就可以承载多个会话，互不干扰。

20.5 消息的流动：一次 attach 的旅程

让我们追踪一次完整的 `attach()` 调用，看看 DBus 消息是如何流动的：

```
┌──────────┐                                  ┌──────────────┐
│  Python  │                                  │ frida-server │
│  Client  │                                  │              │
└──────────┘                                  └──────────────┘
     │                                                │
     │  1. DBus Method Call                           │
     │  path: /re/frida/HostSession                   │
     │  method: attach(pid=1234, options={})          │
     │  ──────────────────────────────────────────>   │
     │                                                │
     │              2. Server 收到请求                 │
     │              注入 Agent 到 pid 1234             │
     │              创建 AgentSession                  │
     │              返回 session_id                    │
     │                                                │
     │  3. DBus Method Return                         │
     │  session_id = "a1b2c3"                         │
     │  <──────────────────────────────────────────   │
     │                                                │
     │  4. DBus Method Call                           │
     │  path: /re/frida/AgentSession/a1b2c3           │
     │  method: create_script(source="...")           │
     │  ──────────────────────────────────────────>   │
     │                                                │
```

整个过程对客户端来说是透明的。Python 绑定层把你的 `session.attach(1234)` 翻译成 DBus 方法调用，GDBus 库负责序列化、发送、等待响应、反序列化。你甚至感受不到 DBus 的存在。

20.6 GDBus 的角色：幕后英雄

GDBus 是 GLib 库提供的 DBus 实现。在 Frida 中，它扮演了以下角色：

序列化/反序列化。 GDBus 使用 GVariant 格式对方法参数和返回值进行编码。GVariant 是一种紧凑的二进制格式，支持基本类型、数组、字典等复杂结构。

连接管理。 `GDBusConnection` 封装了一条 DBus 连接。注意看 `dbus.vala` 中的一段巧妙代码：

```
// 获取 GDBus 内部使用的 MainContext
// 这个 hack 是为了让 libnice (ICE 库) 知道正确的上下文
public async MainContext get_dbus_context () {
    var input = new DummyInputStream ();
    var output = new MemoryOutputStream (null);
    var connection = yield new DBusConnection (
        new SimpleIOStream (input, output),
        null, 0, null, null);

    // 通过 filter 捕获 GDBus 内部线程的 MainContext
    uint filter_id = connection.add_filter ((conn, msg, incoming)
=> {
```

```
        MainContext ctx = MainContext.ref_thread_default ();
        // 拿到了！这就是 GDBus 的私有上下文
        get_context_request.resolve (ctx);
        return msg;
    });

    // 触发一次 proxy 获取来激活 filter
    do_get_proxy.begin (connection, io_cancellable);
    // ...
}
```

这段代码看起来有点绕，但它解决了一个实际问题：GDBus 内部有自己的线程和 MainContext，而 Frida 的 P2P 连接（基于 libnice/ICE）需要知道这个上下文。所以 Frida 创建了一个”假连接”，通过消息过滤器”偷”出 GDBus 的内部上下文。这是典型的工程智慧——不修改库的源码，而是通过巧妙的方式获取所需信息。

对象注册与代理。 服务端通过 `connection.register_object()` 把实现类注册到某个路径上；客户端通过 `connection.get_proxy()` 获取远程对象的本地代理。

```
// 服务端注册
registration_id = connection.register_object (
    ObjectPath.AGENT_SESSION, agent_session);

// 客户端获取代理
var proxy = yield connection.get_proxy (
    null, ObjectPath.HOST_SESSION,
    DBusProxyFlags.NONE, cancellable);
```

20.7 信号：事件的广播机制

除了方法调用，DBus 还支持信号（Signal），这是一种单向的事件广播机制。Frida 大量使用信号来通知异步事件：

```
// HostSession 中的信号定义
public signal void spawn_added (HostSpawnInfo info);
public signal void process_crashed (CrashInfo crash);
public signal void agent_session_detached (
    AgentSessionId id,
    SessionDetachReason reason,
    CrashInfo crash);
```

当目标进程崩溃时，frida-server 会发射 `process_crashed` 信号。GDBus 会把这个信号广播给所有连接的客户端。客户端的 GDBus 代理收到信号后，触发本地的事件回调。整个过程不需要客户端轮询。

这就好比医院的广播系统——护士不需要挨个通知家属，通过广播就能让所有等候的人听到。

20.8 认证：不是谁都能上车

DBus 层面本身有一套简单的认证机制。但 Frida 在更高层实现了自己的认证。源码中定义了 `AuthenticationService` 接口：

```
[DBus (name = "re.frida.AuthenticationService17")]
public interface AuthenticationService : Object {
    public abstract async string authenticate (
        string token, Cancellable? cancellable) throws GLib.Error;
}
```

`StaticAuthenticationService` 是最常用的实现，使用 SHA256 哈希比较 token：

```
public async string authenticate (string token, ...) {
    string input_hash = Checksum.compute_for_string (SHA256,
token);
    uint accumulator = 0;
    for (uint i = 0; i != input_hash.length; i++) {
        accumulator |= input_hash[i] ^ token_hash[i];
    }
    if (accumulator != 0)
        throw new Error.INVALID_ARGUMENT ("Incorrect token");
    return "{}";
}
```

注意这里使用了常量时间比较（constant-time comparison），通过位或运算累积差异，避免了时序攻击。这是安全编程的最佳实践。

20.9 小结

本章我们深入了解了 Frida 的 DBus 通信骨架。核心要点：

- **Frida 使用 DBus 协议格式（不是系统 DBus 服务）**，通过 GDBus 库在自建连接上传输消息
- **三大核心接口**：HostSession（服务入口）、AgentSessionProvider（Agent 管理）、AgentSession（脚本交互）
- **对象路径**是 DBus 对象的”门牌号”，Frida 为每个 Agent 会话分配唯一路径
- **GDBus 负责序列化、连接管理、对象注册**，Vala 的 `[DBus]` 注解让接口定义极其简洁
- **信号机制**让事件通知不需要轮询，天然支持异步编程模型
- **认证使用常量时间比较**，防止时序攻击

讨论问题

1. Frida 为什么不使用 gRPC 或 Protocol Buffers 来替代 DBus？从历史演进和生态绑定的角度分析。
2. 对象路径中的版本号 `17`（如 `re.frida.HostSession17`）意味着什么？如果客户端和服务端版本不匹配会发生什么？
3. `get_dbus_context()` 中通过”假连接”获取 GDBus 私有上下文的做法，你能想到其他获取库内部状态的类似技巧吗？

第21章：控制服务——网络上的 Frida

> 当你在电脑上运行 `frida -H 192.168.1.100 -p 1234` 连接到手机上的 frida-server 时，你有没有好奇过：frida-server 到底在监听什么？它是如何把网络上飞来的字节流变成”附加到进程”这样的具体操作的？

21.1 ControlService：Frida 的门面

如果把 Frida 比作一家餐厅，那么 ControlService 就是餐厅的大门和前台。它负责：

- 监听网络端口，等待客户端连接
- 验证来客身份（认证）
- 把客户端的请求转发给后厨（HostSession）
- 管理所有正在进行的”用餐”（Agent 会话）

在源码中，`ControlService` 定义在 `control-service.vala` 中。我们先看它的整体架构：

```
┌──────────────────────────────────────────────────────────────┐
│                        ControlService                        │
│                                                              │
│  ┌──────────────┐      ┌──────────────────────────────┐     │
│  │  WebService  │      │      ConnectionHandler       │     │
│  │  (HTTP/WS)   │──────>│                              │     │
│  │  监听端口    │      │  ┌──────────────────────┐    │     │
│  └──────────────┘      │  │ AuthenticationChannel│    │     │
│                        │  │  (验证身份)          │    │     │
│                        │  └──────────────────────┘    │     │
│                        │              │               │     │
│                        │              │ 验证通过      │     │
│                        │              v               │     │
│                        │  ┌──────────────────────┐    │     │
│                        │  │  ControlChannel      │    │     │
│                        │  │  (正式对话)          │    │     │
│                        │  └──────────────────────┘    │     │
│                        └──────────────────────────────┘     │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │                HostSession (后端服务)                 │    │
│  │  DarwinHostSession / LinuxHostSession / ...          │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

21.2 frida-server 的启动流程

让我们从 `server.vala` 的 `main()` 函数开始，看看 frida-server 是如何启动的。

```
// server.vala (简化版)
private static int main (string[] args) {
    // 解析命令行参数
    // --listen, --certificate, --token, --origin 等
    var ctx = new OptionContext ();
    ctx.add_main_entries (option_entries, null);
    ctx.parse (ref args);

    // 构建端点参数
    var endpoint_params = new EndpointParameters (
        listen_address,       // 如 "0.0.0.0:27042"
        0,
        parse_certificate (certpath),  // TLS 证书
        origin,               // 允许的 Origin
        auth_service,         // 认证服务
        asset_root            // 静态文件目录
    );

    // 创建并运行 ControlService
    service = new ControlService (endpoint_params, options);
    yield service.start (cancellable);
}
```

默认端口是 `27042`——这是 Frida 的标志性端口号。整个启动过程可以概括为：

```
解析命令行参数
       │
       v
构建 EndpointParameters (地址、证书、Token)
       │
       v
创建 ControlService
       │  ├── 创建平台对应的 HostSession
       │  │   (Darwin/Linux/Windows/FreeBSD)
       │  └── 创建 WebService (HTTP 服务器)
       │
       v
service.start()
       │  ├── 启动 WebService 监听端口
       │  └── 可选: 预加载 (preload)
       │
       v
进入主循环，等待连接
```

21.3 WebService：HTTP 与 WebSocket

Frida 的网络传输层基于 HTTP 和 WebSocket。`WebService` 类封装了这一层，定义在 `socket.vala` 中：

```
// socket.vala (简化版)
public sealed class WebService : Object {
    public signal void incoming (
        IOStream connection,
        SocketAddress remote_address,
        DynamicInterface? dynamic_iface);

    public WebServiceFlavor flavor;     // CONTROL 或 CLUSTER
    public EndpointParameters endpoint_params;

    public async void start (Cancellable? cancellable);
    public void stop ();
}
```

当一个客户端通过 WebSocket 连接到 frida-server 时，发生了什么？

```
┌──────────┐          ┌──────────────────┐
│  Client  │          │    frida-server  │
│          │          │                  │
└──────────┘          └──────────────────┘
     │                          │
     │  1. TCP 连接              │
     │  ────────────────────────> │
     │                          │
     │  2. HTTP Upgrade 请求     │
     │  GET / HTTP/1.1           │
     │  Upgrade: websocket       │
     │  ────────────────────────> │
     │                          │
     │  3. 101 Switching         │
     │  <──────────────────────── │
     │                          │
     │  4. WebSocket 帧          │
     │  (承载 DBus 消息)         │
     │  <──────────────────────> │
     │                          │
```

WebSocket 在这里充当了一个”管道”。一旦 WebSocket 连接建立，Frida 就在上面跑 DBus 协议。WebSocket 的帧机制天然支持消息边界，非常适合承载 DBus 的消息格式。

`WebServiceFlavor` 有两个值：`CONTROL`（用于 frida-server 的控制通道）和 `CLUSTER`（用于 Portal 集群模式）。

21.4 连接的处理流程

当 WebService 收到一个新连接后，会触发 `incoming` 信号。ControlService 在 `on_server_connection` 中处理这个信号：

```
// control-service.vala (简化版)
private void on_server_connection (IOStream connection,
        SocketAddress remote_address,
        DynamicInterface? dynamic_iface) {
    handler.handle_server_connection.begin (connection);
}
```

`ConnectionHandler.handle_server_connection` 是关键方法：

```
public async void handle_server_connection (IOStream
raw_connection) {
    // 在 WebSocket 流之上创建 DBus 连接
    var connection = yield new DBusConnection (
        raw_connection,
        null,
        DELAY_MESSAGE_PROCESSING,  // 先别处理消息
        null,
        io_cancellable);

    // 根据是否需要认证，创建不同的 Peer
    AuthenticationService? auth_service =
        parent.endpoint_params.auth_service;

    if (auth_service != null)
        peers[connection] = new AuthenticationChannel (
            this, connection, auth_service);
    else
        peers[connection] = new ControlChannel (
            this, connection);

    // 开始处理消息
    connection.start_message_processing ();
}
```

注意 `DELAY_MESSAGE_PROCESSING` 这个标志。创建 DBusConnection 时先不处理消息，等到 Peer 对象（AuthenticationChannel 或 ControlChannel）完全初始化后，再调用 `start_message_processing()`。这避免了初始化过程中的竞态条件。

21.5 认证与访问控制

如果 frida-server 启动时指定了 `--token` 参数，连接的处理会多一个认证步骤：

```
┌──────────┐                          ┌──────────────┐
│  Client  │                          │ frida-server │
└────┬─────┘                          └──────┬───────┘
```

```
│                                          │
│  连接建立                                 │
│  ──────────────────────────────────>     │
│                                          │
│  此时服务端创建 AuthenticationChannel      │
│  注册 UnauthorizedHostSession            │
│  (所有方法都返回"未授权")                  │
│                                          │
│  调用 authenticate(token)                │
│  path: /re/frida/AuthenticationService
│  ──────────────────────────────────>     │
│                                          │
│  [验证 token 的 SHA256 哈希]              │
│                                          │
│  认证成功: 返回 "{}"                      │
│  <──────────────────────────────────     │
│                                          │
│  服务端"升级"连接:                         │
│  AuthenticationChannel                   │
│      --> ControlChannel                  │
│  注册真正的 HostSession                   │
│                                          │
│  现在可以正常调用了                        │
│  attach(pid=1234)                        │
│  ──────────────────────────────────>     │
```

在认证通过之前，服务端注册的是一个 `UnauthorizedHostSession`——这个类实现了 HostSession 的所有方法，但每个方法都简单地抛出”未授权”错误。这种设计很巧妙：客户端看到的接口是一样的，只是在认证前所有操作都被拒绝。

认证通过后，`promote_authentication_channel` 方法把 AuthenticationChannel 替换为 ControlChannel：

```
public async void promote_authentication_channel (
        AuthenticationChannel channel) {
    DBusConnection connection = channel.connection;

    peers.unset (connection);
    yield channel.close (io_cancellable);

    // 创建新的 ControlChannel，注册真正的 HostSession
    peers[connection] = new ControlChannel (this, connection);
}
```

21.6 ControlChannel：正式的对话

ControlChannel 是认证后的核心通信通道。它在 DBus 连接上注册各种服务对象，让客户端可以正常操作：

```
ControlChannel 注册的 DBus 对象
┌──────────────────────────────────────────────────────────┐
```

```
│                                                   │
│  /re/frida/HostSession                            │
│    -> 进程枚举、attach、spawn 等                    │
│                                                   │
│  /re/frida/TransportBroker                        │
│    -> TCP 传输协商 (用于 P2P 优化)                  │
│                                                   │
│  /re/frida/BusSession                             │
│    -> Bus 消息通道                                  │
│                                                   │
│  /re/frida/AgentMessageSink/{session_id}          │
│    -> 每个 Agent 会话的消息接收端                    │
│                                                   │
└───────────────────────────────────────────────────┘
```

当客户端调用 `attach()` 附加到某个进程后，ControlService 会：

1. 调用底层 HostSession 的 `attach()` 方法，注入 Agent
2. 获取 Agent 的 `AgentSession` 对象
3. 在客户端的 DBus 连接上注册该 AgentSession
4. 获取客户端侧的 `AgentMessageSink` 代理

这样，后续的脚本操作和消息传递就可以直接在 AgentSession 上进行了。

21.7 TLS 加密

frida-server 支持 TLS 加密，只需启动时指定证书：

```
frida-server --certificate /path/to/cert.pem
```

在源码中，证书通过 `EndpointParameters` 传递：

```
var endpoint_params = new EndpointParameters (
    listen_address,
    0,
    parse_certificate (certpath),  // TLS 证书
    origin,
    auth_service,
    asset_root
);
```

有了 TLS，WebSocket 连接就变成了 WSS（WebSocket Secure），所有通信内容都被加密。这在远程调试时尤其重要——你可不希望你的调试流量在网络上被嗅探。

21.8 远程调试的完整工作流

让我们把所有知识串起来，看看远程调试的完整工作流：

```
手机端                          电脑端

┌──────────────────┐                    ┌──────────────────┐
│  frida-server    │                    │  frida CLI       │
```

```
│                    │                    │                    │
│  1. 启动           │                    │                    │
│  监听 27042        │                    │                    │
│                    │  2. TCP 连接       │                    │
│                    │ <──────────────────│  frida -H phone    │
│                    │                    │                    │
│                    │  3. WebSocket      │                    │
│                    │  升级 + TLS        │                    │
│                    │ <──────────────────>│                   │
│                    │                    │                    │
│  4. DBus 连接      │  5. authenticate   │                    │
│  建立              │ <──────────────────│  --token=secret    │
│                    │                    │                    │
│  6. 认证通过       │                    │                    │
│  注册真实          │  7. attach(pid)    │                    │
│  HostSession       │ <──────────────────│  -p 1234           │
│                    │                    │                    │
│  8. 注入 Agent     │  9. session_id     │                    │
│  到目标进程        │ ──────────────────>│                    │
│                    │                    │                    │
│                    │  10. 创建/加载     │                    │
│                    │  脚本、收发消息    │                    │
│                    │ <──────────────────>│                   │
└────────────────────┘                    └────────────────────┘
```

21.9 小结

本章我们深入分析了 Frida 的控制服务架构。核心要点：

- **ControlService 是 frida-server 的核心**，负责接受连接、认证客户端、管理会话
- **WebService 提供 HTTP/WebSocket 传输**，DBus 消息在 WebSocket 之上传输
- **认证采用两阶段设计**：先用 UnauthorizedHostSession 拒绝一切，认证后升级为真正的 ControlChannel
- **默认端口 27042**，支持 TLS 加密和 Token 认证
- **ConnectionHandler 管理所有 Peer（连接对）**，每个 DBus 连接对应一个 Peer
- **frida-server 支持守护进程模式**（`--daemonize`），使用经典的 Unix fork + pipe 模式

讨论问题

1. 为什么 Frida 选择 WebSocket 而不是直接的 TCP 作为传输层？WebSocket 带来了哪些额外的好处？
2. 认证失败时，为什么不直接断开连接，而是要注册一个 UnauthorizedHostSession？从协议设计的角度思考。
3. 在真实的渗透测试场景中，如果你发现目标机器上运行着没有开启认证的 frida-server，意味着什么？如何利用，又该如何防范？

第22章：消息系统——脚本与宿主的对话

> 当你在 Frida 脚本里调用 `send({type: "hook", address: "0x12345"})` 的时候，这条消息是怎么从目标进程里面”飞”到你的 Python 脚本的？中间经历了怎样的旅程？如果消息发得太快，会不会”堵车”？

22.1 消息系统的全景

Frida 的消息系统是连接脚本世界（JavaScript）和宿主世界（Python/Node.js/C）的桥梁。它要解决的核心问题是：两个完全不同的运行时环境，如何可靠地交换数据？

想象一下快递系统：你在网上下了一个订单（`send()`），快递公司收到后把包裹装车（序列化），经过分拣中心（消息队列），最终送到你家门口（`on('message', callback)`）。Frida 的消息系统就是这样一套”快递网络”。

```
┌──────────────────────────────────────────────────────────────┐
│                    消息系统全景                                │
│                                                              │
│  ┌──────────┐                    ┌──────────────┐           │
│  │ JS 脚本  │                    │  Python 宿主 │           │
│  │          │                    │              │           │
│  │ send()   │                    │ on('message')│           │
│  │ recv()   │                    │ post()       │           │
│  └──────────┘                    └──────────────┘           │
│       │                                  ^                   │
│       v                                  |                   │
│  ┌──────────────────────────────────────────────────┐       │
│  │            AgentMessageTransmitter               │       │
│  │   ┌──────────────────────────────────────────┐   │       │
│  │   │          PendingMessage 队列             │   │       │
│  │   │  [msg1] -> [msg2] -> [msg3] -> ...       │   │       │
│  │   └──────────────────────────────────────────┘   │       │
│  │                                                  │       │
│  │  批量发送 (Batch)     流量控制 (Batch ID)        │       │
│  └──────────────────────────────────────────────────┘       │
│       │                                  ^                   │
│       v                                  |                   │
│  ┌──────────────────────────────────────────────────┐       │
│  │           AgentMessageSink (DBus)                │       │
│  └──────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────┘
```

22.2 AgentMessage：消息的结构

每条消息在 Frida 内部用 `AgentMessage` 结构体表示，定义在 `session.vala` 中：

```
public struct AgentMessage {
    public AgentMessageKind kind;     // 消息类型
    public AgentScriptId script_id;   // 来源脚本 ID
    public string text;               // JSON 文本内容
    public bool has_data;             // 是否有二进制附件
    public uint8[] data;              // 二进制数据
}

public enum AgentMessageKind {
    SCRIPT = 1,    // 来自脚本的消息
    DEBUGGER       // 来自调试器的消息
}
```

各字段一目了然：`kind` 区分脚本消息和调试器消息，`script_id` 在多脚本场景下标识来源，`text` 承载 JSON 格式的结构化数据，`has_data` + `data` 携带可选的二进制附件。

为什么把 text 和 data 分开？ 这是一个巧妙的设计决策。JSON 很适合传递结构化的元数据（hook 地址、函数名、参数值等），但传递大量二进制数据时效率很低——需要 Base64 编码，体积膨胀 33%。所以 Frida 允许消息同时携带 JSON 文本和原始二进制数据，各取所长。

在 JavaScript 端，这对应着：

```
// 只发送 JSON
send({type: "log", message: "函数被调用了"});

// 发送 JSON + 二进制数据
var buf = Memory.readByteArray(ptr, 256);
send({type: "dump", address: ptr.toString()}, buf);
```

22.3 从脚本到宿主：send() 的旅程

当脚本调用 `send()` 时，消息经历了一段复杂的旅程。让我们逐步追踪：

```
JS 脚本: send({type: "hook"}, binary_data)
        |
        v
ScriptEngine.message_from_script 信号
        |
        v
BaseAgentSession.on_message_from_script()
        |
        v
AgentMessageTransmitter.post_message_from_script()
        |   创建 PendingMessage，加入队列
        |
```

```
        v
AgentMessageTransmitter.maybe_deliver_pending_messages()
        │  从队列中取出消息，组装成批次
        │
        v
AgentMessageSink.post_messages(batch, batch_id)
        │  通过 DBus 发送到宿主端
        │
        v
宿主端收到消息，触发 on('message') 回调
```

在 BaseAgentSession 中，消息从脚本引擎流向发送器：

```
// base-agent-session.vala (简化版)
private void on_message_from_script (
        AgentScriptId script_id, string json, Bytes? data) {
    // 直接转交给 transmitter
    transmitter.post_message_from_script (script_id, json, data);
}
```

AgentMessageTransmitter 是消息的”分拣中心”。它不会立刻发送每条消息，而是先放入队列：

```
public void post_message_from_script (
        AgentScriptId script_id, string json, Bytes? data) {
    pending_messages.offer (new PendingMessage (
        next_serial++,           // 递增的序列号
        AgentMessageKind.SCRIPT,
        script_id,
        json,
        data
    ));
    maybe_deliver_pending_messages ();
}
```

22.4 批量发送：快递不是一件一件送的

这里有一个关键的性能优化：**批量发送**。Frida 不是每收到一条消息就立刻通过 DBus 发送，而是把多条消息打包成一个批次（batch）一起发。

就像快递公司不会每收到一个包裹就发一趟车，而是凑够一车再发。这样可以大幅减少 DBus 调用的开销。

```
private void maybe_deliver_pending_messages () {
    if (state != LIVE)
        return;

    AgentMessageSink? sink = (nice_message_sink != null)
        ? nice_message_sink : message_sink;
    if (sink == null)
        return;
```

```
    // 组装批次，限制总大小不超过 4MB
    var batch = new Gee.ArrayList<PendingMessage> ();
    size_t total_size = 0;
    size_t max_size = 4 * 1024 * 1024;  // 4MB 上限

    PendingMessage? m;
    while ((m = pending_messages.peek ()) != null) {
        size_t message_size = m.estimate_size_in_bytes ();
        // 如果加上这条消息会超过限额，并且批次不为空
        if (total_size + message_size > max_size && !
batch.is_empty)
            break;
        pending_messages.poll ();
        batch.add (m);
        total_size += message_size;
    }

    // 发送批次
    sink.post_messages (items_arr, batch_id, cancellable);
}
```

注意几个细节：

1. **4MB 的批次上限**——防止单次 DBus 调用传输过多数据导致阻塞
2. **至少发一条消息**——即使单条消息超过 4MB，也会被发送（`!batch.is_empty` 检查）
3. **优先使用 P2P 连接**——如果有 nice_message_sink（ICE/DTLS 直连），优先走这条路

22.5 流量控制：Batch ID 机制

消息系统还需要处理一个经典问题：**流量控制**。如果脚本 hook 了一个高频调用的函数，每秒可能产生数千条消息。如果宿主端处理不过来怎么办？

Frida 使用了一套基于 Batch ID 的流量控制机制：

`batch_id` 是批次中最后一条消息的序列号。宿主端通过 `post_messages` 的参数回传 `rx_batch_id`，告诉 Agent 端”我已经处理到了第几批”。在会话中断和恢复时，这个机制尤为重要：

```
public void resume (uint rx_batch_id, out uint tx_batch_id) {
    if (rx_batch_id != 0) {
        // 丢弃已确认的消息
        PendingMessage? m;
        while ((m = pending_messages.peek ()) != null
               && m.delivery_attempts > 0
               && m.serial <= rx_batch_id) {
            pending_messages.poll ();
        }
    }

    // 重新开始投递
    delivery_cancellable = new Cancellable ();
```

```
    state = LIVE;
    maybe_deliver_pending_messages ();

    // 告诉对方我们收到了哪些
    tx_batch_id = last_rx_batch_id;
}
```

当连接断开后重新恢复时，Agent 端告诉宿主”我上次收到你确认的是 batch X”，然后只重发 X 之后的消息。宿主端也告诉 Agent”我上次收到你的消息到 batch Y”。这样双方都不会丢失消息，也不会重复处理。

22.6 从宿主到脚本：recv() 的旅程

消息不仅可以从脚本流向宿主，也可以反向流动。在 JavaScript 端：

```
// 接收来自宿主的消息
recv('poke', function(message) {
  console.log('收到宿主消息:', JSON.stringify(message));
});
```

在 Python 端：

```
script.post({'type': 'poke', 'data': 'hello'})
```

这条反向消息的路径是：

```
Python: script.post(message)
        |
        v
AgentSession.post_messages([message], batch_id)
        |  通过 DBus 发送到 Agent 端
        |
        v
BaseAgentSession.post_messages()
        |
        v
ScriptEngine.post_to_script(script_id, text, data)
        |
        v
JS 脚本的 recv() 回调被触发
```

在 BaseAgentSession 中处理接收到的消息：

```
public async void post_messages (AgentMessage[] messages,
        uint batch_id, Cancellable? cancellable) {
    // 检查当前状态是否允许接收
    transmitter.check_okay_to_receive ();

    foreach (var m in messages) {
        switch (m.kind) {
            case SCRIPT:
                script_engine.post_to_script (
                    m.script_id,
```

```
                    m.text,
                    m.has_data ? new Bytes (m.data) : null);
                break;
            case DEBUGGER:
                script_engine.post_to_debugger (
                    m.script_id, m.text);
                break;
        }
    }

    // 确认接收批次
    transmitter.notify_rx_batch_id (batch_id);
}
```

注意这里的 `check_okay_to_receive()`——如果会话处于中断状态（INTERRUPTED），就拒绝接收消息并抛出异常。这是流量控制的一部分。

22.7 错误处理：消息不能丢

每条待发送的消息被包装为 `PendingMessage`，记录了序列号（`serial`）、投递尝试次数（`delivery_attempts`）等元数据。消息管道中的错误处理设计得很谨慎：

```
private async void deliver_batch (AgentMessageSink sink,
        Gee.ArrayList<PendingMessage> messages, void * items) {
    bool success = false;
    pending_deliveries++;
    try {
        foreach (var message in messages)
            message.delivery_attempts++;

        yield sink.post_messages (items_arr, batch_id,
            delivery_cancellable);
        success = true;
    } catch (GLib.Error e) {
        // 发送失败！把消息放回队列
        pending_messages.add_all (messages);
        // 按序列号重新排序，保证顺序
        pending_messages.sort ((a, b) => a.serial - b.serial);
    } finally {
        pending_deliveries--;
        // 所有投递完成且成功，重置序列号
        if (pending_deliveries == 0 && success)
            next_serial = 1;
    }
}
```

这里有三个关键设计：

1. **失败后放回队列**——消息不会因为一次网络故障就丢失

2. **排序保证顺序**——放回的消息按序列号排序，确保消费者收到的消息顺序正确
3. **序列号重置**——当所有消息都成功投递后，序列号重置为 1，避免无限增长

22.9 AgentMessageSink：消息的终点

`AgentMessageSink` 是消息链的最后一环，定义为一个 DBus 接口：

```
[DBus (name = "re.frida.AgentMessageSink17")]
public interface AgentMessageSink : Object {
    public abstract async void post_messages (
        AgentMessage[] messages,
        uint batch_id,
        Cancellable? cancellable) throws GLib.Error;
}
```

这个接口有两个实现位置：

1. **宿主端**——在 ControlService 中，当客户端调用 `attach()` 后，服务端获取客户端 DBus 连接上的 AgentMessageSink 代理。Agent 发出的消息通过这个代理传递到客户端。

2. **P2P 端**——如果建立了 WebRTC（ICE/DTLS）直连，消息会走 `nice_message_sink`，绕过 frida-server 直接从 Agent 传到客户端。Transmitter 中通过 `nice_message_sink != null` 判断优先走 P2P 路径。

22.10 小结

本章我们深入探索了 Frida 的消息系统。核心要点：

- **AgentMessage 结构体**包含消息类型、脚本 ID、JSON 文本和可选的二进制数据，text + data 分离设计兼顾了灵活性和效率
- **AgentMessageTransmitter 是消息的分拣中心**，维护 PendingMessage 队列，负责批量发送和重试
- **批量发送**将多条消息打包成一个 DBus 调用，单批上限 4MB，大幅减少通信开销
- **Batch ID 机制**实现流量控制和可靠传输，支持断线后的消息恢复
- **失败后消息放回队列**并按序列号排序，保证消息不丢失、不乱序
- **中断/恢复机制**（`interrupt/resume`）允许短暂断线后继续会话，`persist_timeout` 超时后才彻底关闭

讨论问题

1. 为什么 Frida 选择 JSON 作为消息的文本格式而不是 Protocol Buffers 或 MessagePack？从 JavaScript 运行时的角度思考。
2. 4MB 的批次大小限制是如何确定的？如果你需要 dump 一段 100MB 的内存，消息系统会如何处理？
3. 在 `deliver_batch` 中，失败后把消息放回队列并重新排序。如果排序本身的开销很大（比如队列中有上万条消息），你会如何优化？

第23章：Python 绑定——最受欢迎的接口

> 你有没有想过，当你在 Python 里写 `import frida` 的时候，这短短一行代码背后到底发生了什么？Python 是怎么和 Frida 的 C 核心对话的？

Python 是 Frida 最受欢迎的语言绑定，也是大多数人接触 Frida 的第一站。无论你是写自动化测试脚本、安全分析工具，还是 CTF 比赛中的快速原型，Python 绑定都是首选。本章我们就来拆解这个绑定层的内部结构，看看它是怎么把 C 语言的底层能力，包装成我们熟悉的 Pythonic API 的。

23.1 整体架构：两层包装的艺术

Frida 的 Python 绑定并不是一层简单的封装，而是采用了两层结构：

```
┌──────────────────────────────────────────────────┐
│              你的 Python 脚本                      │
│          import frida                             │
├──────────────────────────────────────────────────┤
│         frida/__init__.py                         │
│    便捷函数: attach(), spawn(), kill()            │
├──────────────────────────────────────────────────┤
│         frida/core.py                             │
│    Python 包装层: Device, Session, Script         │
│    事件处理, RPC 机制, 类型提示                   │
├──────────────────────────────────────────────────┤
│         frida/_frida/                             │
│    C 扩展模块 (extension.c)                       │
│    直接调用 frida-core 的 C API                   │
├──────────────────────────────────────────────────┤
│         frida-core (C 库)                         │
│    Frida 的核心实现                               │
└──────────────────────────────────────────────────┘
```

这就像去餐厅吃饭。`__init__.py` 是菜单上的推荐套餐，让你一句话就能点好菜；`core.py` 是服务员，帮你协调厨房的各种操作；`_frida/extension.c` 是厨房和前台之间的传菜口；而 `frida-core` 就是真正的厨房。

23.2 入口：init.py 的设计哲学

打开 `frida/__init__.py`，你会发现它的设计理念非常清晰——提供最简单的顶层 API：

```python
# frida/__init__.py 的核心结构

from . import _frida    # 加载 C 扩展
from . import core      # 加载 Python 包装层

__version__ = _frida.__version__

# 便捷函数，全部委托给 core 层
get_device_manager = core.get_device_manager

def attach(target, realm=None, persist_timeout=None):
    """附加到进程"""
    return get_local_device().attach(target, realm=realm,
                                     persist_timeout=persist_timeout)

def spawn(program, argv=None, envp=None, env=None, cwd=None,
          stdio=None):
    """启动进程并暂停"""
    return get_local_device().spawn(program=program, argv=argv, ...)

def get_local_device():
    return get_device_manager().get_local_device()

def get_usb_device(timeout=0):
    return get_device_manager().get_usb_device(timeout)
```

注意这里的模式：所有顶层函数都是通过 `get_local_device()` 来完成的。这意味着 `frida.attach(pid)` 等价于 `frida.get_local_device().attach(pid)`。这种设计让 80% 的场景（本地调试）变得极其简洁。

同时，十多种异常类型（`ServerNotRunningError`、`ProcessNotFoundError`、`TimedOutError` 等）在 C 扩展中定义，通过 `__init__.py` 统一导出给用户。

23.3 C 扩展：_frida 模块的秘密

`_frida` 是整个绑定的心脏，位于 `frida/_frida/extension.c`，直接 `#include <frida-core.h>` 调用 C API。同时提供类型存根文件 `__init__.pyi`，让 IDE 能理解 C 扩展的接口：

```python
# _frida/__init__.pyi（类型存根，简化）
class Device(Object):
    @property
    def id(self) -> Optional[str]: ...
    @property
    def type(self) -> Optional[str]: ...  # "local", "remote", "usb"
    def attach(self, pid: int, ...) -> "Session": ...
    def spawn(self, program: str, ...) -> int: ...

class Session(Object):
    def create_script(self, source: str, ...) -> Script: ...
```

```
class Script(Object):
    def load(self) -> None: ...
    def post(self, message: str, ...) -> None: ...
```

所有 C 扩展对象继承自统一的 `Object` 基类，提供 `on()` / `off()` 方法注册信号处理器，与 GObject 信号系统一脉相承。

23.4 核心类：Device、Session、Script

`core.py` 中定义的三大核心类构成了 Frida 操作的基本流水线：

```
┌──────────┐  attach()   ┌──────────┐  create_script()
┌──────────┐
│  Device  │ ──────────> │  Session  │ ──────────────> │
Script   │
│          │             │           │
│          │
│ 代表一台 │             │ 代表与目标 │                    │ 代表注入
│
│ 设备     │             │ 进程的会话 │                    │ 的JS脚本
│
└──────────┘             └──────────┘
└──────────┘
```

每个 Python 包装类内部都持有一个 `_impl` 成员，指向对应的 C 扩展对象：

```
class Session:
    def __init__(self, impl):
        self._impl = impl   # _frida.Session 的实例

    @cancellable
    def create_script(self, source, name=None, runtime=None):
        kwargs = {"name": name, "runtime": runtime}
        _filter_missing_kwargs(kwargs)   # 移除值为 None 的参数
        return Script(self._impl.create_script(source, **kwargs))

    @cancellable
    def detach(self):
        self._impl.detach()
```

这种 “Python 包装类 + C 实现类” 的模式，让 Python 层可以自由添加功能（比如 RPC 机制、消息路由），而不需要修改 C 代码。

注意 `_filter_missing_kwargs` 这个辅助函数——它把值为 `None` 的参数从字典中移除，这样 C 扩展就能正确使用默认值。这是一个很实用的小技巧。

23.5 事件处理与回调机制

Frida 是事件驱动的，Python 绑定中最重要的事件机制就是 `on()` / `off()` 模式：

```python
# Script 类的事件处理
class Script:
    def __init__(self, impl):
        self._impl = impl
        self._on_message_callbacks = []

        # 注册内部消息处理器
        impl.on("destroyed", self._on_destroyed)
        impl.on("message", self._on_message)

    def on(self, signal, callback):
        if signal == "message":
            # message 信号由 Python 层管理
            self._on_message_callbacks.append(callback)
        else:
            # 其他信号直接委托给 C 扩展
            self._impl.on(signal, callback)

    def _on_message(self, raw_message, data):
        message = json.loads(raw_message)
        mtype = message["type"]

        if mtype == "log":
            # 日志消息：调用日志处理器
            self._log_handler(message["level"], message["payload"])
        elif mtype == "send" and is_rpc_message(message):
            # RPC 响应：路由到 RPC 处理器
            self._on_rpc_message(...)
        else:
            # 用户消息：分发给所有注册的回调
            for callback in self._on_message_callbacks[:]:
                callback(message, data)
```

这里有一个关键的设计：`message` 信号没有直接委托给 C 扩展，而是在 Python 层拦截并做了路由。这是因为 Frida 的消息通道承载了三种不同类型的消息：

```
┌──────────────────────────────────────────────────────┐
│                Script 消息通道                        │
├──────────────────┬──────────────────┬──────────────────┤
│  type="log"      │ type="send"      │ type="send"      │
│  日志消息        │ RPC 响应         │ 用户消息         │
│                  │ payload 以       │                  │
│  -> 日志处理器   │ "frida:rpc"      │ -> 用户回调      │
│                  │ 开头             │                  │
│                  │ -> RPC 处理器    │                  │
└──────────────────┴──────────────────┴──────────────────┘
```

23.6 RPC 机制：exports 的魔法

RPC 是 Frida Python 绑定中最优雅的功能之一。你可以在 JS 脚本中导出函数，然后直接在 Python 中调用：

```
// JS 端
rpc.exports = {
    add: function(a, b) { return a + b; },
    readMemory: function(addr, size) { return
            Memory.readByteArray(ptr(addr), size); }
};
```

```
# Python 端
script.load()
api = script.exports_sync
result = api.add(1, 2)        # 直接调用！
data = api.read_memory(addr, 64)  # camelCase 自动转 snake_case
```

这是怎么实现的？关键在 `ScriptExportsSync` 类：

```
class ScriptExportsSync:
    def __init__(self, script):
        self._script = script

    def __getattr__(self, name):
        script = self._script
        js_name = _to_camel_case(name)  # read_memory -> readMemory

        def method(*args, **kwargs):
            request, data = make_rpc_call_request(js_name, args)
            return script._rpc_request(request, data, **kwargs)

        return method
```

它利用 Python 的 `__getattr__` 魔术方法，把任意属性访问转化为 RPC 调用。`_to_camel_case` 函数负责把 Python 风格的 `snake_case` 转成 JavaScript 风格的 `camelCase`。

每个 RPC 调用都有唯一的 `request_id`，Python 通过 `post()` 发送请求 `["frida:rpc", id, "call", "add", [1, 2]]`，JS 端执行后通过 `send()` 返回结果，Python 端的 `_on_rpc_message` 根据 `request_id` 匹配并唤醒等待的条件变量。异步版本 `ScriptExportsAsync` 则使用 `asyncio.Future` 替代条件变量。

23.7 Cancellable：优雅取消的艺术

Frida 的几乎所有 I/O 操作都支持取消，这通过 `@cancellable` 装饰器和 `Cancellable` 类实现：

```
def cancellable(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
```

```
        cancellable = kwargs.pop("cancellable", None)
        if cancellable is not None:
            with cancellable:   # push_current / pop_current
                return f(*args, **kwargs)
        return f(*args, **kwargs)
    return wrapper
```

使用方式很简单：

```
cancellable = frida.Cancellable()

# 在另一个线程中：
cancellable.cancel()

# 在主线程中，这个调用会被取消：
try:
    device.attach(pid, cancellable=cancellable)
except frida.OperationCancelledError:
    print("操作已取消")
```

这个模式来自 GLib 的 `GCancellable`，被 Python 绑定完整保留了下来。

23.8 类型提示的细致处理

Frida 的 Python 绑定充分利用 `typing` 高级特性。消息类型用 `TypedDict` + `Literal` 精确定义，`on()` 方法用 `@overload` 让 IDE 能根据信号名推断回调类型：

```
class ScriptErrorMessage(TypedDict):
    type: Literal["error"]
    description: str
    stack: NotRequired[str]

class Script:
    @overload
    def on(self, signal: Literal["destroyed"],
           callback: ScriptDestroyedCallback) -> None: ...
    @overload
    def on(self, signal: Literal["message"],
           callback: ScriptMessageCallback) -> None: ...
```

为兼容不同 Python 版本，代码做了条件导入（`NotRequired` 在 3.11 之前从 `typing_extensions` 导入），确保从 Python 3.7 到最新版本都有良好的类型检查体验。

23.9 常见使用模式

把上面的知识串起来，最经典的 Frida Python 使用模式是这样的：

```
import frida
import sys
```

```python
# 1. 附加到目标进程
session = frida.attach("目标进程")

# 2. 创建并加载脚本
script = session.create_script("""
    Interceptor.attach(Module.findExportByName(null, 'open'), {
        onEnter: function(args) {
            send({type: 'open', path: args[0].readUtf8String()});
        }
    });
""")

# 3. 注册消息回调
def on_message(message, data):
    if message['type'] == 'send':
        print(f"[*] {message['payload']}")

script.on('message', on_message)

# 4. 加载脚本（此时 hook 生效）
script.load()

# 5. 保持运行
sys.stdin.read()
```

这四个步骤——attach、create_script、on('message')、load——构成了 Frida 最基本的工作流。整个流程背后，Python 绑定帮你处理了 C API 调用、线程安全、消息路由、JSON 序列化等所有复杂细节。

本章小结

- Frida Python 绑定采用两层架构：C 扩展 `_frida` 负责与 frida-core 通信，`core.py` 提供 Pythonic 的包装
- `__init__.py` 通过便捷函数让 80% 的使用场景只需一行代码
- 三大核心类 Device -> Session -> Script 构成操作流水线
- 消息通道承载三种消息（日志、RPC、用户消息），由 Python 层路由分发
- RPC 机制通过 `__getattr__` 魔术方法和请求/响应模式实现透明调用
- `@cancellable` 装饰器为所有 I/O 操作提供取消支持
- 类型提示使用了 TypedDict、Literal、overload 等高级特性

讨论问题

1. 为什么 Frida 选择在 Python 层而不是 C 扩展层实现 RPC 机制？这种设计有什么优缺点？
2. `ScriptExportsSync` 使用 `__getattr__` 实现动态方法调用，这种模式在什么场景下可能带来问题（比如自动补全、类型检查）？Frida 是如何缓解这些问题的？
3. 如果你要为 Frida 的 Python 绑定添加一个新功能（比如支持异步 `async for` 遍历进程列表），你会在哪一层实现？为什么？

第24章：Node.js 绑定与其他语言

> 如果你会做红烧肉，那换一个锅、换一种火候，做出来的菜味道会不同，但核心步骤是一样的。Frida 的各语言绑定就是这个道理——底层都是同一个 frida-core，但每种语言都有自己的”锅”和”火候”。那这些绑定到底是怎么造出来的？

上一章我们详细分析了 Python 绑定。但 Frida 的野心不止于此——它为 Node.js、Swift、.NET、Go 等多种语言都提供了绑定。本章我们来看看这些绑定的实现方式，找出其中的共通模式。

24.1 Node.js 绑定：N-API 与代码生成

Node.js 是 Frida 生态中第二重要的语言绑定。与 Python 绑定手写 C 扩展不同，Node.js 绑定大量使用了代码生成技术。

整体架构

```
┌──────────────────────────────────────────────────┐
│              你的 Node.js 脚本                    │
│        import frida from "frida"                 │
├──────────────────────────────────────────────────┤
│         src/frida.ts                             │
│     TypeScript 包装层                            │
│     类型定义 + 信号处理 + 便捷方法               │
├──────────────────────────────────────────────────┤
│         frida_binding (N-API)                    │
│     C 原生模块，自动生成                         │
│     基于 GIR 描述文件生成绑定代码                │
├──────────────────────────────────────────────────┤
│         frida-core (C 库)                        │
└──────────────────────────────────────────────────┘
```

Node.js 绑定使用 N-API (Node-API) 版本 8，这是 Node.js 提供的稳定 ABI 接口。使用 N-API 意味着编译一次就能在多个 Node.js 版本上运行，不需要为每个版本重新编译。

代码生成器：frida_bindgen

这是 Node.js 绑定最有趣的部分。在 `src/frida_bindgen/` 目录下有一套完整的代码生成系统：

```
src/frida_bindgen/
├── model.py        # 从 GIR XML 解析 API 模型
├── codegen.py      # 根据模型生成代码
├── customization.py  # 自定义调整
```

```
├── loader.py       # 加载 GIR 文件
├── assets/         # 代码模板和辅助代码
│   ├── codegen_helpers.ts
│   ├── codegen_helpers.c
│   ├── codegen_types.h
│   └── codegen_prototypes.h
└── cli.py          # 命令行入口
```

GIR (GObject Introspection Repository) 是 GLib 世界中描述 API 的标准 XML 格式。Frida 的核心库基于 GObject，天然支持生成 GIR 文件。代码生成器读取这个 GIR 文件，自动生成三种代码：

```
# codegen.py 的核心
def generate_all(model):
    return {
        "ts":  generate_ts(model),          # TypeScript 包装层
        "dts": generate_napi_dts(model),     # 类型声明文件
        "c":   generate_napi_bindings(model), # C 语言 N-API 绑定
    }
```

这意味着当 frida-core 的 API 发生变化时，只需要重新运行代码生成器，Node.js 绑定就能自动更新。这比手写维护要高效得多。

从 GIR 到 TypeScript

生成器把 GIR XML 解析成结构化的模型（`ObjectType`、`Method`、`Signal` 等数据类），然后生成 TypeScript 包装层。生成的代码通过 `bindings` 包加载 N-API 原生模块，并用 `Promise` 处理异步操作（符合 Node.js 的事件循环模型）：

```
// 生成的 frida.ts（简化示意）
import bindings from "bindings";
const binding = bindings({ bindings: "frida_binding" });

export class Device {
    private impl: NativeDevice;
    get id(): string { return this.impl.id; }

    async attach(pid: number): Promise<Session> {
        const impl = await this.impl.attach(pid);
        return new Session(impl);
    }
}
```

24.2 Python vs Node.js：绑定策略对比

两种绑定采用了截然不同的策略：

```
┌──────────────┬──────────────────┬──────────────────┐
│              │ Python 绑定      │ Node.js 绑定     │
├──────────────┼──────────────────┼──────────────────┤
│ C 绑定实现   │ 手写 extension.c │ 自动生成 (GIR)   │
```

```
│ 包装层语言      │ Python              │ TypeScript          │
│ 异步模型        │ threading + cond    │ Promise + async     │
│ 类型信息        │ .pyi 存根文件       │ .d.ts 声明文件      │
│ 分发方式        │ PyPI (pip)          │ npm + prebuild      │
│ 原生模块 ABI    │ Python C API        │ N-API (稳定ABI)     │
│ API 更新方式    │ 手动维护            │ 代码生成            │
│ RPC 支持        │ Python 层实现       │ TypeScript 层实现   │
└─────────────────┴─────────────────────┴─────────────────────┘
```

Python 绑定更”手工”，可以做更精细的控制；Node.js 绑定更”自动化”，维护成本更低。两者各有优劣。

24.3 Swift 绑定：原生 Apple 生态

Swift 绑定是 Frida 对 Apple 生态的正式支持，位于 `frida-swift` 项目中。它不是简单的 C API 封装，而是用地道的 Swift 风格重新设计了 API。

项目结构

```
frida-swift/
├── Package.swift       # Swift Package Manager 配置
├── Frida/              # 主要源码
│   ├── Device.swift
│   ├── Session.swift
│   ├── Script.swift
│   ├── DeviceManager.swift
│   ├── Bus.swift
│   ├── Compiler.swift
│   └── ...
├── FridaCore/          # C 桥接层
└── Tests/              # 测试
```

Swift 风格的 API

Swift 绑定充分利用了 Swift 的语言特性：

```swift
// Device.swift（简化示意）
public final class Device: Sendable, Identifiable {
    // 使用 AsyncStream 处理事件，非常 Swift 化
    public var events: Events {
        eventSource.makeStream()
    }

    public typealias Events = AsyncStream<Event>

    // 枚举表示设备类型
    public enum Kind: UInt, Codable {
        case local
        case remote
        case usb
    }
```

```swift
    // 枚举表示各种事件
    public enum Event {
        case spawnAdded(SpawnDetails)
        case childAdded(ChildDetails)
        case processCrashed(CrashDetails)
        case output(data: [UInt8], fd: Int, pid: UInt)
        case lost
    }

    // 内部持有 C 的 opaque pointer
    internal let handle: OpaquePointer

    init(handle: OpaquePointer) {
        self.handle = handle
        // 通过 GObject 信号连接 Swift 事件
        connectSignal(instance: self, handle: handle,
                      signal: "spawn-added", handler: onSpawnAdded)
        connectSignal(instance: self, handle: handle,
                      signal: "lost", handler: onLost)
    }
}
```

关键设计点：

1. 使用 Swift 的 `AsyncStream` 而非回调来处理事件，这与 Swift 的结构化并发模型完美契合
2. 使用 `@frozen enum` 来定义类型安全的事件和设备种类
3. 通过 `OpaquePointer` 桥接 C 的句柄，而不是 `Unsafe` 指针
4. 遵循 `Sendable` 协议，保证线程安全

C 桥接

Swift 不能直接调用 GObject 的 API，需要一个 C 桥接层 `FridaCore`：

```
┌──────────────┐
│ Swift 代码   │
│ Device.swift │
├──────────────┤
│ FridaCore    │  <-- C 桥接层
│ C headers    │      暴露 frida-core 的 C API
├──────────────┤
│ frida-core   │
│ (C 库)       │
└──────────────┘
```

Swift 通过 `import FridaCore` 直接调用 `frida_device_attach_sync()` 等 C 函数，然后在 Swift 层做类型转换和错误处理。

24.4 其他语言绑定概览

QML 绑定 (frida-qml)

QML 是 Qt 的声明式 UI 语言。Frida 的 QML 绑定让你可以用 Qt Quick 构建 Frida 的图形化工具：

```
// 概念示意
import Frida 1.0

DeviceListModel {
    id: deviceModel
    onDeviceAdded: {
        console.log("新设备:", device.name)
    }
}

ListView {
    model: deviceModel
    delegate: Text { text: model.name }
}
```

这个绑定主要面向需要构建 GUI 工具的场景，比如 Frida 的官方 GUI 工具就基于此。

.NET/CLR 绑定 (frida-clr)

.NET 绑定让 C# 和 F# 开发者能使用 Frida。它通过 P/Invoke 调用 frida-core 的 C API：

```
// 概念示意
using Frida;

var device = DeviceManager.GetLocalDevice();
var session = device.Attach(pid);
var script = session.CreateScript("console.log('hello');");
script.Message += (sender, e) => {
    Console.WriteLine(e.Message);
};
script.Load();
```

Go 绑定 (frida-go)

Go 绑定通过 cgo 调用 frida-core：

```
// 概念示意
import "github.com/AdiEcho/frida-go"

device := frida.GetLocalDevice()
session, _ := device.Attach(pid)
script, _ := session.CreateScript("console.log('hello');")
script.On("message", func(msg string, data []byte) {
```

```
    fmt.Println(msg)
})
script.Load()
```

24.5 绑定模式总结：通用公式

观察这些绑定，可以提炼出一个通用的绑定创建公式：

```
┌──────────────────────────────────────────────────────────┐
│              创建 Frida 语言绑定的通用模式                  │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  1. 原生桥接层                                            │
│     ├── 调用 frida-core 的 C API                          │
│     ├── 处理内存管理 (GObject ref/unref)                   │
│     └── 类型转换 (C 类型 <-> 目标语言类型)                 │
│                                                          │
│  2. 包装层                                                │
│     ├── 用目标语言的习惯重新设计 API                       │
│     ├── 实现事件/信号系统                                  │
│     │   (回调、Promise、AsyncStream 等)                    │
│     ├── 实现 RPC 机制                                      │
│     └── 错误处理 (C 的 GError -> 语言异常)                 │
│                                                          │
│  3. 分发层                                                │
│     ├── 包管理器集成 (pip/npm/SPM/NuGet)                   │
│     ├── 预编译二进制分发                                   │
│     └── 类型定义文件 (.pyi/.d.ts 等)                       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

如果你要为新语言创建绑定

假设你想为 Rust 创建 Frida 绑定，步骤大致是：

1. **创建 C FFI 层**：用 Rust 的 `extern "C"` 和 `bindgen` 生成 frida-core 的 Rust 声明
2. **处理 GObject 生命周期**：实现 `Drop` trait 来自动调用 `g_object_unref`
3. **包装核心类**：`Device`、`Session`、`Script`，用 Rust 的 enum 和 Result 处理错误
4. **实现事件系统**：用 channels 或 async streams 传递信号
5. **实现 RPC**：基于 serde_json 做序列化
6. **发布到 crates.io**：提供预编译的 frida-core 库

关键挑战在于 GObject 的引用计数和 Rust 的所有权系统之间的协调，以及异步操作的集成方式。

24.6 代码生成 vs 手写：工程权衡

Node.js 绑定的代码生成方式值得深入思考：

```
手写绑定的优点:
├── 完全控制 API 的每个细节
├── 可以做语言特有的优化
└── 更容易处理特殊情况

代码生成的优点:
├── API 变更时自动同步
├── 一致性好，不容易遗漏
├── 减少样板代码
└── 可以同时生成多种语言的绑定
```

Frida 的做法是混合模式：Node.js 用代码生成处理大部分机械性的绑定代码，然后通过 `customization.py` 和手写的 TypeScript 包装层来处理需要特殊逻辑的部分。这种 “80% 自动化 + 20% 手工” 的策略，在大型项目中是非常实用的工程决策。

GIR 在这里扮演了关键角色——它是 frida-core API 的”单一事实来源”(Single Source of Truth)。任何语言绑定都可以从同一个 GIR 文件出发，保证了各语言 API 的一致性。

本章小结

- Node.js 绑定使用 N-API（稳定 ABI），通过 GIR 代码生成器自动生成 C 绑定和 TypeScript 类型
- Swift 绑定用地道的 Swift 风格重新设计了 API，使用 AsyncStream 处理事件、enum 表示类型
- 各语言绑定遵循统一模式：原生桥接层 + 包装层 + 分发层
- GIR (GObject Introspection) 是连接 frida-core 和各语言绑定的桥梁
- 代码生成 vs 手写是工程权衡，Frida 采用混合策略
- 创建新语言绑定的核心挑战是：GObject 生命周期管理、异步模型适配、事件系统映射

讨论问题

1. N-API 的稳定 ABI 特性对 Frida 的 Node.js 绑定分发有什么具体好处？如果没有 N-API，需要怎么做？
2. Swift 绑定使用 `AsyncStream` 而 Python 绑定使用回调函数来处理事件。如果要为 Python 绑定也加上 `async for` 支持，你会怎么设计？
3. 假设你要为一种新语言（比如 Kotlin Native 或 Zig）创建 Frida 绑定，你会选择代码生成还是手写？为什么？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第25章：命令行工具——frida-tools 全家桶

> 你安装 Frida 后第一个敲的命令是什么？大概是 `frida-ps` 看看有哪些进程，或者 `frida -U 目标App` 直接开干。这些命令行工具用起来顺手到让人忘记它们的存在。但你有没有好奇过，这些工具内部是怎么组织的？为什么它们的行为如此一致？

frida-tools 是 Frida 官方提供的命令行工具集合，它构建在 Python 绑定之上。本章我们深入这个”全家桶”，看看它的架构设计和每个工具的实现原理。

25.1 全家桶一览

先来看看 frida-tools 提供了哪些工具。在 `setup.py` 的 `entry_points` 中，注册了所有命令行工具：

通过 `setup.py` 的 `entry_points` 注册了 17 个命令，按功能分类：

```
┌──────────────────────────────────────────────────────────┐
│                    frida-tools 工具集                    │
├──────────────┬──────────────────────────────────────────┤
│ 交互调试     │ frida           交互式 REPL              │
├──────────────┼──────────────────────────────────────────┤
│ 追踪分析     │ frida-trace     函数追踪                 │
│              │ frida-strace    系统调用追踪             │
│              │ frida-itrace    指令级追踪               │
│              │ frida-discover  函数发现                 │
├──────────────┼──────────────────────────────────────────┤
│ 设备与进程   │ frida-ls-devices  列出设备               │
│              │ frida-ps          列出进程               │
│              │ frida-kill        终止进程               │
├──────────────┼──────────────────────────────────────────┤
│ 文件操作     │ frida-ls     列出远程文件                │
│              │ frida-pull   拉取远程文件                │
│              │ frida-push   推送文件                    │
│              │ frida-rm     删除远程文件                │
├──────────────┼──────────────────────────────────────────┤
│ 开发工具     │ frida-create   创建项目模板              │
│              │ frida-compile  编译 TypeScript agent     │
│              │ frida-pm       包管理                    │
│              │ frida-apk      APK 操作                  │
└──────────────┴──────────────────────────────────────────┘
```

25.2 架构基石：ConsoleApplication

所有工具都继承自同一个基类 `ConsoleApplication`，这是整个工具集一致性的秘密：

```python
# application.py（简化示意）
class ConsoleApplication:
    def __init__(self, run_until_return, on_stop=None):
        self._reactor = Reactor(run_until_return, on_stop)
        self._device = None
        self._session = None
        # 解析命令行参数 ...

    def run(self):
        # 1. 解析参数
        parser = argparse.ArgumentParser()
        self._add_options(parser)       # 子类添加自定义参数
        options = parser.parse_args()
        self._initialize(parser, options, args)

        # 2. 选择设备
        if self._needs_device():
            self._device = self._pick_device()

        # 3. 附加到目标（如果需要）
        if self._needs_target():
            self._attach(target)

        # 4. 启动 Reactor
        self._start()
        self._reactor.run()
```

这个基类处理了所有工具共有的逻辑：

```
┌──────────────────────────────────────────┐
│          ConsoleApplication 基类          │
├──────────────────────────────────────────┤
│ 命令行参数解析      -D 设备ID / -U USB     │
│                      -n 进程名 / -p PID    │
│ 设备选择逻辑     本地/USB/远程            │
│ 进程附加逻辑     by name / by pid         │
│ 信号处理       Ctrl+C 优雅退出            │
│ 输出格式化      终端检测/颜色支持          │
│ Reactor 生命周期  启动/停止/清理            │
└──────────────────────────────────────────┘
          │
     ┌────┴────┬─────────┬──────────┐
     │         │         │          │
  frida   frida-ps  frida-kill  ...
```

每个具体工具只需要覆写几个方法：

- `_add_options()`: 添加自己特有的命令行参数
- `_initialize()`: 解析参数后的初始化
- `_start()`: 真正的业务逻辑
- `_needs_device()` / `_needs_target()`: 声明是否需要设备/目标

25.3 Reactor：异步任务调度器

`Reactor` 是异步任务调度核心——主线程运行用户交互，后台线程处理异步任务：

```
class Reactor:
    def __init__(self, run_until_return, on_stop=None):
        self._pending = collections.deque([])  # 任务队列
        self.io_cancellable = frida.Cancellable()

    def run(self):
        worker = threading.Thread(target=self._run)  # 后台线程
        worker.start()
        self._run_until_return(self)  # 主线程：用户交互
        self.stop()
        worker.join()

    def schedule(self, f, delay=None):
        """将任务加入队列，可选延迟执行"""
        when = time.time() + (delay or 0)
        self._pending.append((f, when))
```

就像餐厅的前台接待客人（主线程），后厨按订单做菜（工作线程），`schedule()` 就是下单。

25.4 frida-ps：最简单的工具

`frida-ps` 是理解工具架构的最佳入门：

```
class PSApplication(ConsoleApplication):
    def _add_options(self, parser):
        parser.add_argument("-a", "--applications",
                            action="store_true",
                            help="list only applications")
        parser.add_argument("-j", "--json",
                            dest="output_format", const="json",
                            help="output results as JSON")

    def _start(self):
        if self._list_only_applications:
            self._list_applications()
        else:
            self._list_processes()

    def _list_processes(self):
        # 就这么简单：调用 Python 绑定的 API
        processes = self._device.enumerate_processes()

        for process in sorted(processes, key=cmp):
            print(f"{process.pid:>6}  {process.name}")
```

```
        self._exit(0)
```

整个工具不到 100 行核心代码。它所做的事情： 1. 继承 `ConsoleApplication`（自动获得 `-D`、`-U` 等设备选择参数） 2. 添加自己的 `-a`（仅应用）和 `-j`（JSON 输出）参数 3. 在 `_start()` 中调用 `self._device.enumerate_processes()` 4. 格式化输出

25.5 frida-kill 和 frida-ls-devices

`frida-kill` 同样精简，只有 30 行代码——解析目标参数，调用 `self._device.kill()`，处理 `ProcessNotFoundError`。`infer_target()` 辅助函数自动判断输入是 PID 还是进程名。

`frida-ls-devices` 则稍有不同，它声明 `_needs_device() = False`（不需要特定设备），使用 `prompt_toolkit` 构建 TUI 界面，异步查询每个设备的系统参数，查询过程中还有旋转动画提示加载状态。

25.7 frida：交互式 REPL

`frida` 命令本身是整个工具集中最复杂的，它是一个完整的交互式 JavaScript REPL：

```
class REPLApplication(ConsoleApplication):
    def __init__(self):
        self._script = None
        self._ready = threading.Event()
        self._completer = FridaCompleter(self)
        self._compilers = {}

        super().__init__(self._process_input, self._on_stop)

        # 构建交互式终端
        self._cli = PromptSession(
            lexer=PygmentsLexer(JavascriptLexer),  # JS 语法高亮
            history=FileHistory(history_file),      # 命令历史
            completer=self._completer,              # 自动补全
            complete_in_thread=True,
            enable_open_in_editor=True,             # 支持用编辑器打开
            tempfile_suffix=".js",
        )
```

用户输入的 JS 代码通过 `Script.post()` 发送到目标进程的 JS 引擎执行，结果通过消息通道返回并格式化显示。REPL 还支持自动补全（`FridaCompleter` 查询目标进程的 API）、TypeScript 实时编译、文件监控（`-l script.js` 修改后自动重载）和 autoperform（自动在 Java/ObjC 运行时上下文中执行）。

25.8 frida-trace：自动化追踪

`frida-trace` 是日常使用频率最高的工具之一。它能自动 hook 指定的函数并打印调用日志：

```
# 追踪所有 open* 函数
frida-trace -i "open*" 目标进程

# 追踪特定模块的导入
frida-trace -t libssl.so 目标进程

# 追踪 Objective-C 方法
frida-trace -m "-[NSURLSession *]" 目标App
```

`TracerApplication` 继承 `ConsoleApplication` 和 `UI`，通过 `TracerProfileBuilder` 解析 `-i`、`-x`、`-m`、`-a` 等参数构建追踪配置。工作原理：

1. 根据用户指定的模式（`-i`, `-m`, `-a` 等），构建一个追踪配置 `TracerProfile`
2. 在目标进程中枚举匹配的函数
3. 为每个函数生成一个 handler 脚本（自动生成在 `__handlers__/` 目录）
4. 用 `Interceptor.attach()` 挂钩这些函数
5. 函数被调用时，通过消息通道把参数、返回值等信息发回
6. Python 端格式化并彩色输出

用户可以编辑自动生成的 handler 脚本来自定义行为：

```
// __handlers__/libc.so/open.js（自动生成）
{
    onEnter(log, args, state) {
        log(`open("${args[0].readUtf8String()}")`);
    },
    onLeave(log, retval, state) {
        log(`=> ${retval}`);
    }
}
```

frida-trace 还内置了一个 WebSocket 服务器，提供了 Web UI 来可视化追踪结果。

25.9 frida-discover：函数发现

`frida-discover` 通过采样来发现目标进程中活跃的函数：

```
class Discoverer:
    def start(self, session, runtime, ui):
        script = session.create_script(
            name="discoverer",
            source=self._create_discover_script(),
            runtime=runtime
        )
        script.on("message", on_message)
```

```
        script.load()

        # 通过 RPC 启动采样
        params = script.exports_sync.start()
        ui.on_sample_start(params["total"])

    def stop(self):
        # 停止采样，获取结果
        result = self._script.exports_sync.stop()
        # 解析结果：模块函数 + 动态函数
```

它的原理是利用 Stalker（Frida 的代码追踪引擎）对代码执行进行采样，统计哪些函数被调用了，按调用频率排序输出。这对于逆向一个不熟悉的程序特别有用——你可以快速知道”这个程序主要在跑哪些函数”。

25.10 工具设计模式总结

三个核心模式贯穿整个 frida-tools：(1) **继承 + 模板方法**：`ConsoleApplication` 定义工作流骨架，子类只实现差异部分；(2) **Reactor 异步调度**：主线程处理交互，工作线程执行回调；(3) **Script + RPC**：Python 端控制流程，JavaScript agent 端在目标进程中执行。

这三个模式的组合让添加新工具变得非常简单：

```
from frida_tools.application import ConsoleApplication

class MyToolApplication(ConsoleApplication):
    def _add_options(self, parser):
        parser.add_argument("--my-option", help="我的选项")

    def _start(self):
        # 你的逻辑
        script = self._session.create_script("""
            // 你的 JavaScript agent
        """)
        script.on("message", self._on_message)
        script.load()

    def _on_message(self, message, data):
        print(message)

def main():
    app = MyToolApplication()
    app.run()
```

几十行代码，你就拥有了一个完整的命令行工具，自带设备选择、进程附加、Ctrl+C 处理等所有功能。

本章小结

- frida-tools 提供了 17 个命令行工具，覆盖调试、追踪、设备管理、文件操作、开发等场景

- 所有工具继承自 `ConsoleApplication` 基类，共享参数解析、设备选择、生命周期管理等公共逻辑
- `Reactor` 是异步任务调度的核心，协调主线程交互和后台任务执行
- frida（REPL）使用 prompt_toolkit 提供语法高亮、自动补全、命令历史等交互功能
- frida-trace 通过 TracerProfile 构建追踪配置，自动生成可编辑的 handler 脚本
- 复杂工具都遵循 “Script + RPC” 模式：Python 端控制，JavaScript agent 端执行
- 创建自定义工具只需继承 ConsoleApplication 并实现几个方法

讨论问题

1. frida-tools 的所有工具都是 Python 实现的，这在性能上会有什么限制？有没有可能用其他语言重写某些工具？
2. Reactor 模式和 Python 的 asyncio 有什么异同？为什么 frida-tools 没有直接使用 asyncio？
3. 如果你要设计一个新的 Frida 命令行工具（比如 `frida-heap` 用于堆内存分析），你会如何组织 Python 端和 JavaScript agent 端的职责划分？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第26章：内存操作——扫描、监控与读写

> 如果你能随意翻看别人书架上的每一本书，找到某一页的某一句话，甚至在上面涂涂改改，你会做什么？在动态插桩的世界里，内存就是那个书架，而 Frida 给了你一把万能钥匙。

26.1 为什么内存操作是核心能力

在逆向工程和安全研究中，几乎所有的工作最终都归结为”读内存”和”写内存”。Hook 函数是在代码段写入跳转指令，读取返回值是在栈上读内存，修改游戏数值是在堆上写内存。可以说，没有内存操作能力的插桩框架就像一把没有刀刃的瑞士军刀——好看但没用。

Frida 的内存操作能力主要集中在 `frida-gum` 的 `gummemory.h` 中，它提供了一套完整的内存读写、扫描、保护、分配的 API。让我们逐一拆解。

26.2 内存的基本抽象

在深入具体操作之前，我们先看看 Frida 如何描述内存。这就好比你要去图书馆找书，首先得知道图书馆的编目系统。

内存范围（GumMemoryRange）

```
struct _GumMemoryRange
{
  GumAddress base_address;  // 起始地址
  gsize size;               // 大小
};
```

简单得不能再简单——一个起始地址加一个大小，就描述了内存中的一块连续区域。Frida 还贴心地提供了一个宏来判断某个地址是否在范围内：

```
#define GUM_MEMORY_RANGE_INCLUDES(r, a) \
    ((a) >= (r)->base_address && \
     (a) < ((r)->base_address + (r)->size))
```

页面保护（GumPageProtection）

```
enum _GumPageProtection
{
  GUM_PAGE_NO_ACCESS = 0,
  GUM_PAGE_READ      = (1 << 0),  // 可读
  GUM_PAGE_WRITE     = (1 << 1),  // 可写
  GUM_PAGE_EXECUTE   = (1 << 2),  // 可执行
};
```

通过位运算组合出 `GUM_PAGE_RW`、`GUM_PAGE_RX`、`GUM_PAGE_RWX` 等常用保护属性。值得注意的是，现代操作系统越来越限制 RWX 页面，Frida 通过 `gum_query_rwx_support()` 查询支持程度。

26.3 内存读写：最基本的操作

读取内存

```
guint8 * gum_memory_read (gconstpointer address,
                          gsize len,
                          gsize * n_bytes_read);
```

看起来很简单对吧？但实现可没那么简单。以 Linux 平台为例，Frida 的读取策略分两步：

```
┌──────────────────────────────────────────────┐
│              gum_memory_read 流程              │
├──────────────────────────────────────────────┤
│                                              │
│  1. 优先使用 process_vm_readv 系统调用        │
│     (内核 >= 3.2 才支持)                      │
│      ┌──────────┐    ┌──────────┐            │
│      │ local    │ <-- │ remote   │            │
│      │ iov_base │    │ iov_base │            │
│      │ iov_len  │    │ iov_len  │            │
│      └──────────┘    └──────────┘            │
```

```
│                                                   │
│  2. 如果系统调用不可用(ENOSYS),                    │
│     回退到检查权限后直接 memcpy                    │
│                                                   │
└───────────────────────────────────────────────────┘
```

为什么要用 `process_vm_readv` 而不是直接 `memcpy`？因为直接 `memcpy` 如果遇到不可读的内存会导致段错误（SIGSEGV）。`process_vm_readv` 是一个安全的系统调用，如果读取失败会返回错误码而不是崩溃。这就像是用一根绝缘的钓鱼竿去触碰电线——即使电线带电，你也不会触电。

写入内存

```
gboolean gum_memory_write (gpointer address,
                           const guint8 * bytes,
                           gsize len);
```

写入内存同样使用 `process_vm_writev` 作为首选方案，回退方案是检查可写性后直接 `memcpy`。

修补代码段

代码段通常是只读可执行的（RX），你不能直接写入。Frida 提供了 `gum_memory_patch_code` 来安全地修改代码：

```
gboolean gum_memory_patch_code (gpointer address, gsize size,
    GumMemoryPatchApplyFunc apply, gpointer apply_data);
```

它的工作原理大致是：先把目标页面改为可写，执行你的修改回调，然后恢复原来的保护属性，最后刷新指令缓存。这个”刷新指令缓存”的步骤（`gum_clear_cache`）在 ARM 架构上尤其重要，因为 ARM 的指令缓存和数据缓存是分离的。

26.4 内存扫描：大海捞针的艺术

内存扫描是逆向工程中的常用操作。比如你想找到某个加密密钥在内存中的位置，或者定位某个字符串。Frida 的内存扫描 API 是：

```
void gum_memory_scan (const GumMemoryRange * range,
                      const GumMatchPattern * pattern,
                      GumMemoryScanMatchFunc func,
                      gpointer user_data);
```

模式匹配的两种方式

Frida 支持两种扫描模式：

十六进制模式（带通配符）：

```
// JavaScript 层的用法
Memory.scan(address, size, "48 8b ?? 10 ff");
```

?? 表示任意字节。在 C 层，这种模式被解析为一系列 Token，每个 Token 要么是精确匹配，要么是带掩码的匹配。

正则表达式模式：

```
// 内部支持正则扫描
gum_memory_scan_regex(range, regex, func, user_data);
```

扫描算法的优化

看源码中的 `gum_memory_scan_raw` 实现，可以发现一个聪明的优化：

```
┌──────────────────────────────────────────────────────┐
│              内存扫描优化策略                          │
├──────────────────────────────────────────────────────┤
│                                                      │
│   1. 从所有 Token 中找到最长的精确匹配 Token          │
│   2. 以此 Token 作为"锚点"进行扫描                    │
│   3. 找到锚点匹配后，再验证完整模式                   │
│                                                      │
│   类比：在一本书中找"红色的大象在跳舞"                 │
│   - 先快速翻页找"大象"（最长的确定性词）              │
│   - 找到后再看前后文是否匹配完整句子                  │
│                                                      │
└──────────────────────────────────────────────────────┘
```

这种”最长锚点优先”的策略大大减少了不必要的完整模式匹配，提升了扫描速度。

26.5 内存访问监控（MemoryAccessMonitor）

如果说内存扫描是”主动查找”，那内存访问监控就是”被动等待”。它能告诉你：谁在什么时候读了或写了某块内存。

```
struct _GumMemoryAccessDetails
{
  GumThreadId thread_id;        // 哪个线程
  GumMemoryOperation operation; // 读、写还是执行
  gpointer from;                // 从哪条指令发起的
  gpointer address;             // 访问的目标地址
  guint range_index;            // 命中了第几个监控范围
  guint page_index;             // 范围内第几页
  guint pages_completed;        // 已完成监控的页数
  guint pages_total;            // 总监控页数
  GumCpuContext * context;      // CPU 上下文快照
};
```

它的工作原理非常巧妙：

```
┌──────────────────────────────────────────────────────┐
│        MemoryAccessMonitor 工作原理                   │
```

```
┌──────────────────────────────────────────────────────┐
│                                                      │
│  1. 将目标内存页的保护设为 NO_ACCESS                  │
│                                                      │
│  2. 当有代码尝试访问时:                               │
│     -> 触发异常/信号 (SIGSEGV/异常)                   │
│     -> Frida 捕获异常                                 │
│     -> 记录访问详情                                   │
│     -> 恢复页面保护                                   │
│     -> 调用用户回调                                   │
│     -> 继续执行                                       │
│                                                      │
│  类似于: 在书页上放一根头发                           │
│  如果有人翻过这页, 头发就会掉落                       │
│                                                      │
└──────────────────────────────────────────────────────┘
```

`auto_reset` 参数控制是否在每次触发后自动重新设置陷阱。如果设为 `TRUE`，每次访问都会触发回调；如果设为 `FALSE`，则每页只触发一次。

在不同平台上，异常的捕获机制不同： - **Windows**: 使用 Vectored Exception Handler - **POSIX (Linux/macOS)**: 使用信号处理器捕获 SIGSEGV

26.6 内存分配：在目标进程中开辟空间

Frida 经常需要在目标进程中分配内存。比如写入 Trampoline 代码、存储 Hook 的上下文数据等。

基本分配

```
gpointer gum_alloc_n_pages (guint n_pages, GumPageProtection prot);
```

近距离分配

这是一个非常重要的能力。在 x86-64 和 ARM64 上，很多跳转指令只能跳到相对距离有限的地址。所以 Frida 需要在目标地址附近分配内存：

```
gpointer gum_alloc_n_pages_near (guint n_pages,
    GumPageProtection prot,
    const GumAddressSpec * spec);
```

`GumAddressSpec` 描述了”靠近哪里”和”最远多远”：

```
struct _GumAddressSpec
{
  gpointer near_address;  // 靠近这个地址
  gsize max_distance;     // 最大距离
};
```

这就像是你要在学校旁边租房，`near_address` 是学校地址，`max_distance` 是你愿意走的最远距离。

内存生命周期管理

Frida 还提供了更细粒度的内存管理 API：

```
┌──────────────────────────────────────────┐
│          内存生命周期 API                  │
├──────────────────────────────────────────┤
│ gum_memory_allocate   -- 分配              │
│ gum_memory_recommit   -- 重新提交物理页     │
│ gum_memory_decommit   -- 释放物理页         │
│ gum_memory_discard    -- 标记可丢弃        │
│ gum_memory_release    -- 释放虚拟地址空间   │
│ gum_memory_free       -- 完全释放          │
└──────────────────────────────────────────┘
```

这套 API 对应了操作系统虚拟内存管理的各个阶段，让 Frida 能够精确控制内存的使用。

26.7 Frida 自己的堆管理

一个有趣的细节：Frida 没有直接使用系统的 `malloc`，而是嵌入了 `dlmalloc`（Doug Lea 的 malloc 实现），创建了独立的内存空间（mspace）：

```
gum_mspace_main = create_mspace(0, TRUE);
gum_mspace_internal = create_mspace(0, TRUE);
```

为什么要这样做？因为 Frida 运行在目标进程中，如果和目标进程共用堆分配器，可能会导致：1. 插桩代码影响目标进程的内存布局 2. 目标进程的堆损坏波及到 Frida 自身 3. 在 Hook `malloc/free` 时产生无限递归

使用独立的 mspace 就像是在别人家的厨房里自带了一套锅碗瓢盆——你用你的，他用他的，互不干扰。

26.8 实际应用示例

让我们用 Frida 的 JavaScript API 做几个实际的内存操作：

```
// 1. 读取内存
var buf = Memory.readByteArray(ptr("0x12345678"), 16);

// 2. 写入内存
Memory.writeByteArray(ptr("0x12345678"),
    [0x90, 0x90, 0x90, 0x90]);  // NOP sled

// 3. 扫描内存中的特定模式
Memory.scan(module.base, module.size, "48 89 5c 24 ?? 57",
{
    onMatch: function(address, size) {
```

```
        console.log("Found at: " + address);
    },
    onComplete: function() {
        console.log("Scan complete");
    }
});

// 4. 监控内存访问
MemoryAccessMonitor.enable([
    { base: ptr("0x12345678"), size: 4096 }
], {
    onAccess: function(details) {
        console.log(details.operation +
            " at " + details.address +
            " from " + details.from);
    }
});

// 5. 在目标进程中分配内存
var page = Memory.alloc(4096);
Memory.protect(page, 4096, 'rwx');
```

26.9 本章小结

- Frida 用 `GumMemoryRange` 和 `GumPageProtection` 构建了统一的内存抽象
- 内存读写优先使用安全的系统调用（`process_vm_readv/writev`），避免因访问无效内存而崩溃
- 内存扫描采用”最长精确 Token 锚点”策略优化性能
- `MemoryAccessMonitor` 利用页面保护机制实现访问监控，巧妙地把操作系统的异常处理变成了监控工具
- 近距离内存分配（`alloc_near`）解决了跳转距离限制的问题
- Frida 使用独立的 `dlmalloc` mspace 避免与目标进程的堆管理冲突

讨论问题

1. `MemoryAccessMonitor` 通过修改页面保护来监控访问，这种方式对性能有什么影响？在什么场景下这种开销是可接受的？

2. 为什么 Frida 要区分 `gum_memory_decommit` 和 `gum_memory_free`？在插桩场景中，精细的内存生命周期管理有什么好处？

3. 如果目标进程使用了自定义的内存分配器（比如 jemalloc 或 tcmalloc），Frida 使用独立的 dlmalloc mspace 是否足够安全？可能还存在哪些潜在的冲突？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第27章：跨平台设计——一套代码，多个世界

> 你有没有想过，一个软件怎么做到在 Windows、macOS、Linux、iOS、Android 上都能运行？这不是简单地把代码复制五份然后分别改改就行的。Frida 的跨平台设计是一个值得深入学习的工程案例。

27.1 跨平台的本质挑战

想象一下，你要写一封信，收件人分别在中国、美国、日本。内容是一样的，但你需要用中文、英文、日文各写一遍。更复杂的是，每个国家的信封格式、邮编规则、地址写法都不一样。

操作系统之间的差异也是如此：

同一功能，不同平台的实现方式		
功能	Linux	macOS/iOS
读进程内存	process_vm_readv	mach_vm_read
修改内存保护	mprotect	mach_vm_protect
枚举线程	读/proc/self/task	task_threads
枚举模块	dl_iterate_phdr	_dyld_image_count + _dyld_get_image_*
TLS操作	pthread_key_create	pthread_key_create（或自定义实现）
页大小查询	sysconf	getpagesize

Frida 的策略是：**定义统一的接口，每个平台各自实现**。这听起来像是教科书上的话，但 Frida 的做法有很多值得学习的细节。

27.2 后端模式（Backend Pattern）

打开 `frida-gum/gum/` 目录，你会看到这样的目录结构：

```
gum/
├── gummemory.h              <-- 统一接口定义
├── gummemory.c              <-- 跨平台通用逻辑
├── gumprocess.h             <-- 统一接口定义
├── gummodule.h              <-- 统一接口定义
│
```

```
├── backend-darwin/
│   ├── gummemory-darwin.c   <-- macOS/iOS 实现
│   ├── gumprocess-darwin.c
│   └── gummodule-darwin.c
│
├── backend-linux/
│   ├── gummemory-linux.c    <-- Linux/Android 实现
│   ├── gumprocess-linux.c
│   └── gummodule-linux.c
│
├── backend-windows/
│   ├── gummemory-windows.c  <-- Windows 实现
│   ├── gumprocess-windows.c
│   └── gummodule-windows.c
│
├── backend-freebsd/         <-- FreeBSD 实现
├── backend-qnx/             <-- QNX 实现
│
├── backend-arm64/           <-- ARM64 架构特定
├── backend-arm/             <-- ARM 架构特定
├── backend-x86/             <-- x86 架构特定
│
└── backend-posix/           <-- POSIX 通用实现
    ├── gumtls-posix.c
    └── gummemoryaccessmonitor-posix.c
```

注意这里有两个维度的后端：**操作系统后端**和**CPU架构后端**。一个运行在 Linux 上的 ARM64 设备，会同时使用 `backend-linux/` 和 `backend-arm64/` 中的代码。

```
┌──────────────────────────────────────────┐
│            后端的两个维度                  │
│                                          │
│  ┌──────┐   ┌──────────────────┐         │
│  │ 上层 │   │ gumprocess.h     │         │
│  │ 接口 │   │ gummemory.h      │         │
│  │      │   │ gummodule.h      │         │
│  └──────┘   └──────────────────┘         │
│      │                │                  │
│      v                v                  │
│  ┌──────┐   ┌──────────────────┐         │
│  │ OS 层 │   │ darwin / linux / win │     │
│  │ 后端 │   │ 内存、进程、模块管理 │       │
│  └──────┘   └──────────────────┘         │
│      │                                   │
│      v                                   │
│  ┌──────┐   ┌──────────────────┐         │
│  │ 架构层 │   │ arm64 / arm / x86 │       │
│  │ 后端 │   │ 寄存器、指令、Stalker │      │
│  └──────┘   └──────────────────┘         │
```

27.3 GumProcess：进程操作的统一抽象

`gumprocess.h` 定义了跨平台的进程操作接口。我们来看几个关键函数：

```
// 获取进程 ID —— 每个平台都有，但调用方式不同
GumProcessId gum_process_get_id (void);

// 枚举所有线程 —— Linux 读 /proc，macOS 用 Mach API
void gum_process_enumerate_threads (GumFoundThreadFunc func,
    gpointer user_data, GumThreadFlags flags);

// 枚举所有模块 —— Linux 用 dl_iterate_phdr，macOS 用 dyld API
void gum_process_enumerate_modules (GumFoundModuleFunc func,
    gpointer user_data);

// 枚举内存范围
void gum_process_enumerate_ranges (GumPageProtection prot,
    GumFoundRangeFunc func, gpointer user_data);
```

每个平台的 `gumprocess-xxx.c` 文件实现相同的函数签名，但内部实现完全不同。以 `gum_process_enumerate_ranges` 为例：

- **Linux**: 解析 `/proc/self/maps` 文件
- **macOS**: 使用 `mach_vm_region` 迭代虚拟内存区域
- **Windows**: 使用 `VirtualQuery` 遍历地址空间

这些实现细节被完全封装在后端文件中，上层代码只需要调用统一的接口。

27.4 GumModule：模块抽象的接口模式

`GumModule` 使用了 GObject 的接口（Interface）机制，这是一个更优雅的跨平台设计：

```
struct _GumModuleInterface
{
  GTypeInterface parent;

  const gchar * (* get_name) (GumModule * self);
  const gchar * (* get_path) (GumModule * self);
  const GumMemoryRange * (* get_range) (GumModule * self);
  void (* enumerate_imports) (GumModule * self, ...);
  void (* enumerate_exports) (GumModule * self, ...);
  void (* enumerate_symbols) (GumModule * self, ...);
  GumAddress (* find_export_by_name) (GumModule * self, ...);
  // ...
};
```

这是一个虚函数表（vtable）。不同平台创建不同的 GumModule 实现：

```
┌──────────────────────────────────────────────────┐
│          GumModule 的平台实现                     │
│                                                  │
│            GumModule (接口)                       │
│              ┌──────────────┐                    │
│              │ get_name     │                    │
│              │ get_path     │                    │
│              │ enum_imports │                    │
│              │ enum_exports │                    │
│              │ find_export  │                    │
│              └──────────────┘                    │
│                     │                            │
│     ┌───────────────┼──────────────┐             │
│     v               v              v             │
│  Darwin          Linux          Windows          │
│  (Mach-O)       (ELF)           (PE)             │
│   ┌─────────┐   ┌─────────┐    ┌─────────┐       │
│   │解析     │   │解析     │    │解析     │       │
│   │load     │   │.dynsym  │    │export   │       │
│   │commands │   │.symtab  │    │table    │       │
│   └─────────┘   └─────────┘    └─────────┘       │
│                                                  │
└──────────────────────────────────────────────────┘
```

这样设计的好处是：当你在 JavaScript 中调用 `Module.enumerateExports()` 时，底层会自动分发到正确的平台实现，而你完全不需要关心目标是 ELF、Mach-O 还是 PE 格式。

27.5 Meson 构建系统中的条件编译

Frida 使用 Meson 构建系统来管理跨平台编译。看看 `gum/meson.build` 中的关键片段：

```
# 根据目标操作系统选择后端源文件
if host_os_family == 'windows'
  gum_sources += [
    'backend-windows' / 'gummemory-windows.c',
    'backend-windows' / 'gumprocess-windows.c',
    'backend-windows' / 'gummodule-windows.c',
    'backend-windows' / 'gumtls-windows.c',
    # ...
  ]
elif host_os_family == 'darwin'
  gum_sources += [
    'backend-darwin' / 'gummemory-darwin.c',
    'backend-darwin' / 'gumprocess-darwin.c',
    'backend-darwin' / 'gummodule-darwin.c',
    # ...
  ]
elif host_os_family == 'linux'
```

```
  gum_sources += [
    'backend-linux' / 'gummemory-linux.c',
    'backend-linux' / 'gumprocess-linux.c',
    'backend-linux' / 'gummodule-linux.c',
    'backend-posix' / 'gumtls-posix.c',
    # ...
  ]
endif
```

注意 `backend-posix/` 目录的存在。POSIX 是 Unix 系操作系统的公共标准，Linux 和 macOS 都遵循。所以有些功能（比如 TLS、异常处理）可以共享 POSIX 实现，不需要每个平台都写一遍。

```
┌──────────────────────────────────────────────────┐
│            代码复用的层次                          │
│                                                   │
│   最上层：gummemory.c（完全通用）                  │
│            所有平台共享的逻辑                      │
│            如：内存扫描、模式匹配                  │
│                                                   │
│   中间层：backend-posix/（Unix 通用）              │
│            Linux + macOS + FreeBSD 共享            │
│            如：TLS、异常处理器                     │
│                                                   │
│   底层：backend-linux/ 或 backend-darwin/          │
│          平台专属实现                              │
│          如：内存读写、进程枚举                    │
│                                                   │
└──────────────────────────────────────────────────┘
```

27.6 C 预处理器中的平台分支

除了构建系统层面的文件选择，Frida 在代码内部也大量使用预处理器来处理平台差异：

```
// gummemory.c 中的平台分支示例
gpointer
gum_sign_code_pointer (gpointer value)
{
#ifdef HAVE_PTRAUTH
  return ptrauth_sign_unauthenticated (value,
      ptrauth_key_asia, 0);
#else
  return value;
#endif
}
```

ARM64 上的指针认证（PAC）只在支持的硬件上存在，所以用 `#ifdef` 来区分。

再看 Linux 内存后端中，不同 CPU 架构的系统调用号差异：

```
// gummemory-linux.c
#if defined (HAVE_I386) && GLIB_SIZEOF_VOID_P == 4
# define GUM_SYS_PROCESS_VM_READV   347
#elif defined (HAVE_I386) && GLIB_SIZEOF_VOID_P == 8
# define GUM_SYS_PROCESS_VM_READV   310
#elif defined (HAVE_ARM)
# define GUM_SYS_PROCESS_VM_READV   (__NR_SYSCALL_BASE + 376)
#elif defined (HAVE_ARM64)
# define GUM_SYS_PROCESS_VM_READV   270
#endif
```

同一个操作系统上，不同的 CPU 架构连系统调用号都不一样。这就是跨平台开发的残酷现实。

27.7 TLS 抽象：一个简洁的例子

线程局部存储（TLS）是跨平台抽象的理想教学案例。`gumtls.h` 定义了四个函数：`gum_tls_key_new`、`gum_tls_key_free`、`gum_tls_key_get_value`、`gum_tls_key_set_value`。POSIX 实现（`gumtls-posix.c`）用 `pthread_key_create/pthread_getspecific`，Windows 实现用 `TlsAlloc/TlsGetValue`，macOS 还有独立的 `gumtls-darwin.c` 处理 Darwin 特有细节。四个函数，三个平台，每个平台几十行代码——接口简单，实现清晰。

27.8 抽象泄漏：跨平台的代价

但现实没有那么美好。在某些地方，平台差异大到无法完全抽象掉。这就是所谓的”抽象泄漏”（Leaky Abstraction）。

例子1：macOS 的 Mach 端口

macOS 的进程间通信基于 Mach 端口。很多 Darwin 特有的功能（如 `gumdarwinmapper.c`、`gumdarwinsymbolicator.c`）没有跨平台的对应物，只能作为平台专属模块存在。

例子2：Android 的特殊性

虽然 Android 是 Linux 内核，但它的用户空间差异很大。所以 Frida 专门有 `gumandroid.c` 来处理 Android 特有的问题，比如 linker namespace、SELinux 策略等。

例子3：符号类型的差异

`GumSymbolType` 枚举把 Mach-O 特有的类型（`GUM_SYMBOL_UNDEFINED`、`GUM_SYMBOL_INDIRECT` 等）和 ELF 特有的类型（`GUM_SYMBOL_OBJECT`、`GUM_SYMBOL_FUNCTION` 等）放在同一个枚举里。接口统一了，但数据模型必须包含所有平台的并集——这是务实的妥协。

同样，Stalker 的每种 CPU 架构都有数千行的独立实现（`gumstalker-x86.c`、`gumstalker-arm.c`、`gumstalker-arm64.c`），因为指令编码差异太大，无法用 `#ifdef` 解决。

27.9 给你自己项目的启示

从 Frida 的跨平台设计中，我们可以提炼出几个实用原则：

```
┌──────────────────────────────────────────────────────┐
│          跨平台设计的实用原则                          │
├──────────────────────────────────────────────────────┤
│                                                      │
│  1. 接口先行：先定义 .h 文件，再写各平台 .c            │
│                                                      │
│  2. 分层复用：通用 > POSIX > 平台专属                  │
│     能共享的尽量共享，不能共享的才分开                  │
│                                                      │
│  3. 构建系统做大切分（选文件）                         │
│     预处理器做小切分（选代码段）                       │
│                                                      │
│  4. 接受抽象泄漏：有些平台特性无法完美抽象             │
│     用注释标明，而不是硬往接口里塞                     │
│                                                      │
│  5. 目录结构即架构：一眼就能看出哪些是通用的           │
│     哪些是平台专属的                                   │
│                                                      │
└──────────────────────────────────────────────────────┘
```

27.10 本章小结

- Frida 使用”统一接口 + 多后端实现”的模式处理跨平台差异
- 后端分为两个维度：操作系统后端（darwin/linux/windows）和 CPU 架构后端（arm64/arm/x86）
- `GumModule` 使用 GObject 接口机制实现多态，`GumProcess` 使用同名函数的不同实现文件
- Meson 构建系统负责根据目标平台选择正确的源文件
- `backend-posix/` 提供了 Unix 系平台的共享实现，减少重复代码
- 抽象泄漏不可避免，务实的做法是在保持接口统一的前提下，用注释标明平台特有的部分
- 好的目录结构本身就是最好的架构文档

讨论问题

1. Frida 的后端模式和经典的”策略模式”有什么异同？在 C 语言中没有虚函数的情况下，GumModule 是如何通过 GObject 接口实现多态的？

2. 如果你要给 Frida 添加一个新的操作系统后端（比如 Fuchsia），你需要实现哪些文件？从 meson.build 的结构中能得到什么指引？

3. Frida 选择在构建系统层面（而非运行时）切换平台实现，这种静态分发策略相比运行时策略（如动态加载插件）有什么优缺点？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第28章：性能优化——速度的追求

> 给一辆行驶中的汽车换轮胎，你不仅要换得上，还要换得快——不然车就翻了。Frida 的性能优化就是在解决这个问题：如何在不拖慢目标程序的前提下，完成复杂的插桩任务？

28.1 为什么性能至关重要

动态插桩工具面临一个根本矛盾：你插入的每一行监控代码都会增加目标程序的执行时间。如果一个函数原本执行 100 纳秒，你的 Hook 引入了 10 微秒的开销，那就是 100 倍的减速。在实时系统、游戏、高频交易等场景中，这种减速是不可接受的。

所以 Frida 在每一个关键路径上都做了精心的优化。让我们逐一拆解这些优化策略。

28.2 Stalker 的反向补丁（Backpatching）

Stalker 是 Frida 的代码跟踪引擎，它通过动态翻译（JIT）来跟踪程序的每一条指令。在上一章我们了解了 Stalker 的基本原理，这里我们聚焦它的性能优化。

问题：间接跳转的开销

当 Stalker 遇到一个间接跳转（比如 `blr x8` 或 `call [rax]`），它不知道目标地址是什么，必须查找翻译后的代码块：

```
┌──────────────────────────────────────────────────────┐
│     没有优化的间接跳转处理                              │
│                                                       │
│  原始代码: blr x8                                      │
│                                                       │
│  翻译后:                                               │
│    1. 保存 CPU 上下文                                  │
│    2. 调用 C 函数查找目标块                            │
│       -> 查哈希表                                      │
│       -> 如果没翻译过，翻译目标代码                     │
│    3. 恢复 CPU 上下文                                  │
│    4. 跳转到翻译后的代码                               │
│                                                       │
│  开销：保存/恢复上下文 + 函数调用 + 哈希查找            │
│  每次间接跳转都要走这条慢路径                           │
└──────────────────────────────────────────────────────┘
```

解决方案：内联缓存 + 反向补丁

Stalker 使用了两层优化：

第一层：内联缓存（Inline Cache, IC）

```
┌──────────────────────────────────────────────────────┐
│      内联缓存工作原理                                  │
│                                                      │
│   翻译后的代码附带一个小缓存表(IC Entry):              │
│                                                      │
│   ┌──────────────────────────────┐                   │
│   │ slot[0]: target=0xA -> 0xA' │  命中!            │
│   │ slot[1]: target=0xB -> 0xB' │  直接跳           │
│   │ slot[2]: (empty)            │  未命中           │
│   └──────────────────────────────┘                   │
│                                                      │
│   执行流程:                                           │
│     cmp x8, [slot0.target]                           │
│     b.eq slot0.translated     // 快速路径             │
│     cmp x8, [slot1.target]                           │
│     b.eq slot1.translated     // 快速路径             │
│     b slow_path               // 慢速路径             │
│                                                      │
│   快速路径: 2-3 条比较指令, 无函数调用                 │
│   慢速路径: 完整的查找流程                             │
└──────────────────────────────────────────────────────┘
```

第二层：反向补丁（Backpatching）

当 Stalker 第一次翻译完一个代码块后，它可以”回去”把之前的间接跳转直接改写为直接跳转：

```
┌──────────────────────────────────────────────────────┐
│      反向补丁示例                                      │
│                                                      │
│   初始状态（块 A 翻译时，块 B 还未翻译）:              │
│     块A: ... -> [查找B的入口] -> 慢路径                │
│                                                      │
│   翻译块 B 之后，回去修改块 A:                         │
│     块A: ... -> [直接跳转到B'] -> 快速!                │
│                                                      │
│   这就是"反向"的含义:                                  │
│   翻译新块时，回头修补指向它的旧跳转                   │
└──────────────────────────────────────────────────────┘
```

源码中 `gumstalker.h` 定义了 `gum_stalker_prefetch_backpatch`、`gum_stalker_backpatch_get_from/to` 等 API，`GumStalkerObserver` 接口允许外部观察者接收反向补丁通知，支持构建更高级的优化。

28.3 Interceptor 的 Trampoline 设计

Interceptor 是 Frida 最常用的 Hook 机制。它的性能关键在于 Trampoline（跳板）的设计。

Trampoline 的核心思想是**快慢分离**：先检查有没有活跃的 listener，如果没有就直接跳到原始代码（零开销快速路径）；只有在需要时才走完整的保存上下文、调用回调、恢复上下文的慢速路径。

另一个关键优化是**近距离分配**。Trampoline 代码通过 `gum_alloc_n_pages_near` 分配在目标函数附近。在 ARM64 上，近距离跳转只需一条 `B` 指令（4字节），而远距离跳转需要 `ADRP+ADD+BR` 三条指令（12字节以上）。在热点函数中，这种差距会被放大数百万倍。

28.4 线程局部存储：避免锁竞争

在多线程环境中，锁是性能杀手。Frida 大量使用线程局部存储（TLS）来避免加锁：

```
// gumtls.h
typedef gsize GumTlsKey;

GumTlsKey gum_tls_key_new (void);
gpointer gum_tls_key_get_value (GumTlsKey key);
void gum_tls_key_set_value (GumTlsKey key, gpointer value);
```

Stalker 的 `GumExecCtx`（执行上下文）就是按线程存储的：

```
┌──────────────────────────────────────────────┐
│     线程局部存储避免锁竞争                      │
│                                               │
│  传统方式（需要锁）:                            │
│    Thread A ──┐                               │
│    Thread B ──┼── lock ── shared_ctx ── unlock│
│    Thread C ──┘      （串行化，性能瓶颈）      │
│                                               │
│  TLS 方式（无锁）:                              │
│    Thread A ─── TLS[key] ─── ctx_A            │
│    Thread B ─── TLS[key] ─── ctx_B            │
│    Thread C ─── TLS[key] ─── ctx_C            │
│    （完全并行，零竞争）                         │
│                                               │
└──────────────────────────────────────────────┘
```

每个被跟踪的线程都有自己独立的执行上下文（`GumExecCtx`），包含独立的代码生成器、代码缓存等。这样不同线程的 Stalker 实例之间完全不需要同步。

唯一需要同步的场景是跨线程操作，比如从主线程停止另一个线程的跟踪。这时 Frida 使用原子操作而非互斥锁：

```
struct _GumExecCtx
{
  volatile gint state;  // 用原子操作而非锁来管理状态
  // ...
};
```

28.5 Slab 分配器：减少系统调用

频繁的 mmap/munmap 系统调用开销很大。Stalker 使用 Slab 分配策略来批量管理内存：

```
┌──────────────────────────────────────────────────┐
│          Slab 分配策略                            │
│                                                  │
│   传统方式：每个代码块单独 mmap                    │
│     mmap(4KB) -> 块A                             │
│     mmap(4KB) -> 块B                             │
│     mmap(4KB) -> 块C                             │
│     3次系统调用                                   │
│                                                  │
│   Slab 方式：预分配大块，内部切分                  │
│     mmap(64KB) -> Slab                           │
│     ┌──────────────────────────┐                 │
│     │ 块A │ 块B │ 块C │ ... │ 空闲 │             │
│     └──────────────────────────┘                 │
│     1次系统调用                                   │
│                                                  │
└──────────────────────────────────────────────────┘
```

从源码中可以看到 Slab 的数据结构：

```
struct _GumSlab
{
  guint8 * data;          // 数据区起始
  guint offset;           // 当前分配偏移
  guint size;             // 可用大小
  guint memory_size;      // 实际映射大小
  GumSlab * next;         // 链接到下一个 Slab
};

struct _GumCodeSlab
{
  GumSlab slab;
  gpointer invalidator;  // 代码失效处理器
};

struct _GumDataSlab
{
  GumSlab slab;
};
```

Stalker 的执行上下文中维护着多种 Slab：

```
struct _GumExecCtx
{
  // ...
  GumCodeSlab * code_slab;     // 翻译后的代码
  GumSlowSlab * slow_slab;     // 慢速路径代码
  GumDataSlab * data_slab;     // 数据（如内联缓存）
```

```
  GumCodeSlab * scratch_slab;  // 临时工作区
  // ...
};
```

分配新的代码块时，只需移动 `offset` 指针即可，就像在一个大笔记本上往后翻页写字——不需要每次都去文具店买新笔记本。

当一个 Slab 写满了，就分配新的 Slab 并通过 `next` 指针链接。这种链式 Slab 结构在空间和时间上都取得了很好的平衡。

28.6 自旋锁与独立堆

Frida 嵌入了 dlmalloc 并维护两个独立的 mspace（`gum_mspace_main` 和 `gum_mspace_internal`），既避免与目标进程冲突，又通过分别控制锁粒度来优化性能。

在代码生成的关键路径上，Stalker 的 `GumExecCtx` 使用 `GumSpinlock`（自旋锁）而非互斥锁来保护代码 Slab。自旋锁在获取失败时循环重试而非休眠，避免了上下文切换的微秒级开销，对于只需几十条指令的极短临界区来说是更好的选择。

28.7 最小化 Prolog/Epilog

插入回调时必须保存和恢复 CPU 上下文。Frida 提供三个级别：`GUM_PROLOG_NONE`（0条指令，纯跳转）、`GUM_PROLOG_MINIMAL`（约10条指令，只保存必要寄存器）、`GUM_PROLOG_FULL`（约40条指令，保存全部寄存器和 SIMD）。Stalker 还缓存了各级别 Prolog/Epilog 的代码地址（`last_prolog_minimal` 等），多个代码块通过 `BL` 共享同一份代码，既节省空间又利用指令缓存。

28.8 代码块的回收利用

`GumExecBlock` 中的 `recycle_count` 记录代码块被重新翻译的次数。对于频繁自失效的代码块（如自修改代码），Stalker 据此调整策略，避免无限增长。

28.9 优化的哲学

回顾 Frida 的所有性能优化，可以归纳出几个核心原则：

```
┌──────────────────────────────────────────────┐
│        Frida 性能优化的核心原则                │
├──────────────────────────────────────────────┤
│                                              │
│  1. 快慢分离                                  │
│     热路径极致优化，冷路径可以慢               │
│                                              │
│  2. 空间换时间                                │
│     内联缓存、Slab 预分配、Prolog 缓存         │
│                                              │
│  3. 避免同步                                  │
```

```
│       TLS 替代锁、自旋锁替代互斥锁                          │
│       原子操作替代临界区                                     │
│                                                             │
│   4. 批量处理                                               │
│       Slab 批量分配、反向补丁批量应用                        │
│                                                             │
│   5. 渐进优化                                               │
│       第一次走慢路径，逐步优化为快路径                        │
│       内联缓存从空到满的过程                                 │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

这些原则不仅适用于 Frida，也适用于任何需要极致性能的系统软件。关键洞察是：**一次性的高开销操作（如翻译代码块）可以被分摊到后续无数次的快速执行中**——这就是 JIT 编译的核心思想。

28.10 本章小结

- Stalker 的反向补丁和内联缓存将间接跳转的开销从”函数调用+哈希查找”降低到”几条比较指令”
- Interceptor 的 Trampoline 设计在 Hook 被禁用时几乎零开销
- 线程局部存储（TLS）让每个线程拥有独立的执行上下文，消除了锁竞争
- Slab 分配器通过批量分配内存，将多次系统调用减少为一次
- 自旋锁用于保护极短的临界区，避免了上下文切换的开销
- 分级的 Prolog/Epilog 让不需要完整上下文的操作只付最小代价
- Frida 嵌入 dlmalloc 并使用独立 mspace，既避免冲突又优化分配性能
- 性能优化的核心是”快慢分离”——让常见情况极快，罕见情况可以慢

讨论问题

1. Stalker 的内联缓存大小是有限的（通常只有几个槽位）。如果一个间接跳转的目标非常多变（比如虚函数表的分发），内联缓存会频繁失效。你能想到什么应对策略？

2. Frida 使用自旋锁保护代码生成的临界区。如果目标进程是一个只有单核 CPU 的嵌入式设备，自旋锁还是好的选择吗？为什么？

3. “渐进优化”策略（第一次慢，之后快）和”预编译”策略（启动时全部翻译）各有什么优缺点？在什么场景下你会选择哪种？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第29章：架构之美——设计模式总结

> 当你读完一整本关于 Frida 源码的书，回过头来看，你会发现一个有趣的现象：那些让你觉得”设计得真巧妙”的地方，其实都遵循着一些经典的设计模式。这些模式并非刻意为之，而是工程师在解决真实问题时，自然而然地收敛到了最优解。

Frida 的代码库跨越了 C、Vala、JavaScript、Python 多种语言，支持 Windows、macOS、Linux、iOS、Android 等多个平台。要管理如此庞大的复杂度，仅靠个人英雄主义是不够的——你需要好的架构和设计模式。

29.1 分层架构模式

如果让我用一句话概括 Frida 的整体架构，那就是：**严格分层，逐层抽象**。

想象一栋大楼。地基是不可见的，但它承载着整栋建筑。每一层楼都只需要知道自己下面那层提供了什么，而不需要关心地基是怎么打的。Frida 的架构也是如此：

```
┌──────────────────────────────────────────┐
│          用户脚本层 (JavaScript)          │
│   Interceptor.attach / Stalker.follow / ...  │
├──────────────────────────────────────────┤
│          绑定层 (GumJS Bindings)          │
│   gumquickinterceptor.c / gumv8stalker.c  │
├──────────────────────────────────────────┤
│          核心引擎层 (Gum)                 │
│   guminterceptor.c / gumstalker.c / ...   │
├──────────────────────────────────────────┤
│          平台抽象层 (Backend)             │
│   backend-arm64/ backend-x86/ backend-darwin/│
├──────────────────────────────────────────┤
│          操作系统 / 硬件                  │
│   ptrace / mach_vm / /proc/maps / ...     │
└──────────────────────────────────────────┘
```

这种分层的好处是什么？

- **关注点分离**：写 JavaScript 绑定的人不需要懂 ARM64 指令编码
- **可替换性**：V8 引擎可以替换为 QuickJS，只需改绑定层
- **可测试性**：每一层都可以独立测试

在 Frida 的源码中，你能清晰地看到这些层次对应的目录结构：

- `frida-gum/gum/` —— 核心引擎层
- `frida-gum/gum/backend-*` —— 平台抽象层
- `frida-gum/bindings/gumjs/` —— 绑定层
- `frida-core/src/` —— 服务与会话管理
- `frida-python/` / `frida-node/` —— 用户 API 层

29.2 后端/提供者模式（Backend/Provider）

Frida 需要在多种 CPU 架构和操作系统上运行。它是怎么做到的？答案是**后端模式**。

这就像一个万能充电器。你的手机需要充电，充电器提供一个统一的 USB 接口（抽象接口），但内部根据不同国家的电压标准（不同平台）使用不同的变压模块（不同后端）。

以 `GumStalker` 为例，它的头文件 `gumstalker.h` 定义了统一接口：

```
// 统一接口——所有平台都用这组 API
GumStalker * gum_stalker_new (void);
void gum_stalker_follow_me (GumStalker * self,
    GumStalkerTransformer * transformer,
    GumEventSink * sink);
void gum_stalker_unfollow_me (GumStalker * self);
```

但具体实现分散在不同的后端目录中：

```
frida-gum/gum/
├── backend-arm64/gumstalker-arm64.c     // ARM64 实现
├── backend-arm/gumstalker-arm.c         // ARM32 实现
├── backend-x86/gumstalker-x86.c         // x86/x64 实现
└── gumstalker.h                         // 统一头文件
```

编译系统（Meson）根据目标平台选择编译哪个后端文件。这种模式在 `GumInterceptor`、`GumRelocator`、`GumWriter` 等组件中都能看到。

同样，`frida-core` 中的 `HostSession` 也采用了提供者模式：

```
frida-core/src/
├── darwin/darwin-host-session.vala      // macOS/iOS
├── linux/linux-host-session.vala        // Linux/Android
├── host-session-service.vala            // 统一接口
```

29.3 事务模式（Transaction Pattern）

`GumInterceptor` 的 `begin_transaction` / `end_transaction` 是一个教科书级别的事务模式实现。

想象你在银行转账。你不希望钱从 A 账户扣了，但还没到 B 账户的时候系统崩溃。银行用事务来保证：要么全部完成，要么全部回滚。

Frida 的 Interceptor 面临类似问题：当你同时 hook 多个函数时，你不希望 hook 到一半时目标进程恰好执行到了那段被修改了一半的代码。

```
// 事务模式使用方式
gum_interceptor_begin_transaction (interceptor);

// 批量操作——此时修改还没有"生效"
gum_interceptor_attach (interceptor, target_a, listener, NULL);
gum_interceptor_attach (interceptor, target_b, listener, NULL);
gum_interceptor_attach (interceptor, target_c, listener, NULL);
```

```
// 提交——所有修改一次性生效
gum_interceptor_end_transaction (interceptor);
```

在 `end_transaction` 内部，Frida 会：1. 暂停目标进程的所有其他线程 2. 一次性刷新所有待写入的 hook 代码 3. 刷新 CPU 指令缓存 4. 恢复所有线程

这样就保证了原子性——外部观察者要么看到 hook 前的状态，要么看到 hook 后的状态，不会看到中间状态。

29.4 观察者模式（Observer Pattern）

观察者模式在 Frida 中无处不在。核心思想是：**当某件事发生时，通知所有关心这件事的人。**

最典型的例子是 `GumEventSink`。当 Stalker 追踪代码执行时，它会产生大量事件（执行了哪条指令、调用了哪个函数、跳转到了哪里）。这些事件通过 `GumEventSink` 接口传递给订阅者：

```
// GumEventSink 接口定义
struct _GumEventSinkInterface {
  GTypeInterface parent;

  GumEventType (* query_mask) (GumEventSink * self);
  void (* start)   (GumEventSink * self);
  void (* process)  (GumEventSink * self,
                     const GumEvent * event,
                     GumCpuContext * cpu_context);
  void (* flush)   (GumEventSink * self);
  void (* stop)    (GumEventSink * self);
};
```

`query_mask` 方法让观察者声明自己关心哪些类型的事件，这样 Stalker 就不会产生没人需要的事件，避免了性能浪费。

在 `frida-core` 层面，`SessionService` 的信号机制也是观察者模式：

```
┌──────────────┐                    ┌──────────────┐
│              │   信号: "detached"  │              │
│              │ ──────────────────> │              │
│   Session    │   信号: "message"   │   Listener   │
│              │ ──────────────────> │              │
└──────────────┘                    └──────────────┘
```

GObject 的信号系统天然支持这种模式，Frida 大量使用 `g_signal_connect` 和 `g_signal_emit` 来实现组件间的松耦合通信。

29.5 工厂模式（Factory Pattern）

Frida 中的 Session 创建就是工厂模式的典型应用。你不需要自己 `new` 一个 Session 对象，而是通过 Device 来”生产”它：

```
# Python 端——用户看到的简洁 API
device = frida.get_usb_device()
session = device.attach(pid)       # Device 是 Session 的工厂
script = session.create_script(src) # Session 是 Script 的工厂
```

在内部，`device.attach()` 触发了一连串的创建过程：

```
Device.attach(pid)
  └─> HostSession.attach_to(pid)
        └─> AgentSession 创建
              └─> Agent 注入到目标进程
                    └─> ScriptBackend 初始化
```

每一层都是下一层的工厂。这种设计隐藏了复杂的创建逻辑，用户只需要一行 `device.attach(pid)` 就完成了注入、通信建立、脚本引擎初始化等一系列操作。

`GumScriptBackend` 也是工厂模式的体现——它根据配置创建不同类型的 Script 对象（V8 或 QuickJS）：

```
// 根据运行时选择创建不同的脚本后端
GumScriptBackend * backend;
backend = gum_script_backend_obtain_qjs ();  // 或 obtain_v8()
```

29.6 桥接模式（Bridge Pattern）

GumJS 层是整个 Frida 中最精彩的桥接模式实现。它在 C 语言的高性能世界和 JavaScript 的高表达力世界之间架起了一座桥。

```
┌──────────────────────┐          ┌──────────────────────┐
│   JavaScript 世界    │          │       C 世界         │
│                      │          │                      │
│ Interceptor.attach   │ ──桥──>  │ gum_interceptor_     │
│                      │          │   attach             │
│ ptr("0x1234")        │ ──桥──>  │ GumAddress           │
│                      │          │                      │
│ callback(args)       │ <──桥──  │ GumInvocationContext │
└──────────────────────┘          └──────────────────────┘
```

桥接的核心挑战在于**类型转换**和**生命周期管理**：

- JavaScript 的 `NativePointer` 需要桥接为 C 的 `gpointer`
- C 的回调需要安全地调用 JavaScript 函数
- JavaScript 的垃圾回收器不能回收还在被 C 引用的对象

为了支持两个不同的 JS 引擎，Frida 对每个需要暴露给 JS 的模块都实现了两套绑定：

```
bindings/gumjs/
├── gumquickinterceptor.c     // QuickJS 版桥接
├── gumv8interceptor.cpp      // V8 版桥接
├── gumquickstalker.c         // QuickJS 版桥接
```

```
├── gumv8stalker.cpp          // V8 版桥接
└── ...
```

这正是桥接模式的精髓：**抽象和实现可以独立变化**。无论底层用 QuickJS 还是 V8，上层的 `Interceptor.attach()` API 都是一样的。

29.7 代理模式（Proxy Pattern）

Frida 的 RPC 机制是代理模式的完美体现。当你在 Python 中调用 `script.exports.my_function()` 时，你调用的并不是真正的函数——你调用的是一个代理，它把调用序列化为 JSON 消息，通过管道发送到目标进程，在那里反序列化并执行真正的函数，然后把结果原路返回。

```
┌──────────┐   JSON/RPC   ┌──────────┐   实际调用   ┌──────────┐
│ Python   │ ──────────> │ Frida    │ ──────────> │ 目标进程 │
│ 代理     │ <──────────  │ 管道     │ <──────────  │ 真实函数 │
└──────────┘   返回结果   └──────────┘   执行结果   └──────────┘
```

对调用者来说，代理对象和本地对象没有区别——这就是代理模式的力量。

29.8 模式总结表

设计模式	Frida 中的应用	关键文件
分层架构	Gum/GumJS/Core/ Python 四层	整体目录结构
后端/提供者	Stalker/Interceptor 多架构支持	backend-*/ *-host-session.vala
事务	Interceptor 的 begin/end	guminterceptor.c
观察者	EventSink/GObject 信号	gumstalker.h GObject signal
工厂	Device->Session-> Script 创建链	frida.vala host-session-*.vala
桥接	GumJS 双引擎绑定	gumquick*.c gumv8*.cpp
代理	RPC exports 机制	script.py / rpc.js

29.9 为什么这些模式重要

你可能会想：“我知道这些模式有什么用？我又不写 Frida。”

但事实是，这些模式解决的是**通用问题**：

- 你的项目需要支持多个平台？学习后端模式
- 你的操作需要原子性？学习事务模式
- 你需要在不同语言间传递数据？学习桥接模式
- 你需要解耦组件之间的通信？学习观察者模式

Frida 源码是一本活生生的设计模式教科书，而且每个模式都经过了真实项目的验证。

本章小结

- Frida 采用**严格分层架构**，从硬件到用户脚本共五层
- **后端模式**让 Frida 能用统一接口支持多种 CPU 架构和操作系统
- **事务模式**保证了多个 hook 的原子性提交
- **观察者模式**通过 EventSink 和 GObject 信号实现了松耦合通信
- **工厂模式**隐藏了 Session 和 Script 的复杂创建逻辑
- **桥接模式**让 JavaScript 和 C 两个世界无缝协作
- **代理模式**让跨进程调用像本地调用一样简单
- 这些模式不是学术概念，而是解决真实工程问题的经验结晶

讨论问题

1. 如果你要为 Frida 添加一个新的 CPU 架构支持（比如 RISC-V），后端模式需要你修改哪些文件？你能从现有的 ARM64 后端中借鉴什么？

2. 事务模式中”暂停所有线程再刷新代码”的做法有什么潜在问题？在高并发场景下你会如何优化？

3. 假设你要设计一个类似 Frida 的工具，但目标是在浏览器中运行（比如 WebAssembly 插桩）。你会保留哪些设计模式，又会放弃哪些？为什么？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

第30章：你的下一步——从读者到贡献者

> 读完一本书之后，最危险的事情是什么？是把书合上，然后什么都不做。知识如果不转化为行动，就像下载了一个文件却从来不打开——它占据着你的硬盘，却从未真正属于你。

恭喜你走到了这里。你已经从 Frida 的用户界面一路深入到了汇编指令和内存布局的层面。但这只是一个开始。这一章，我们聊聊接下来你可以做什么。

30.1 搭建 Frida 开发环境

要从”读源码”进化到”改源码”，你首先需要一个能编译 Frida 的开发环境。

基本要求

```
┌──────────────────────────────────────────────┐
│                 开发环境基本要求                 │
├──────────────────────────────────────────────┤
│ 操作系统：macOS / Linux（推荐 Ubuntu 22.04）  │
│ Python：3.7+                                  │
│ Node.js：18+                                  │
│ Git：最新版                                   │
│ 磁盘空间：至少 30GB（完整编译非常占空间）       │
│ 内存：建议 16GB+                              │
└──────────────────────────────────────────────┘
```

克隆与编译

Frida 使用 Meson 构建系统，并通过 `releng` 目录下的脚本管理依赖：

```
# 克隆 Frida 主仓库
git clone --recurse-submodules https://github.com/nicedayzhu/frida.git
cd frida

# 查看可用的构建目标
make

# 编译 frida-gum（核心引擎）
make gum-linux-x86_64     # Linux x64
make gum-macos-arm64      # macOS Apple Silicon

# 编译 frida-core（完整功能）
make core-linux-x86_64

# 编译 Python 绑定
make python-linux-x86_64
```

第一次编译会很慢，因为需要下载和编译所有依赖（V8 引擎尤其耗时）。喝杯咖啡，耐心等待。

开发工作流

修改代码后的典型工作流程：

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│ 修改源码 │ ──> │ 重新编译 │ ──> │ 运行测试 │ ──> │ 手动验证 │
└──────────┘    └──────────┘    └──────────┘    └──────────┘
```

^ |
└──────────── 发现问题 ────────────┘

如果你只修改了 `frida-gum` 中的某个文件，不需要重新编译整个项目。Meson 的增量编译会只重新编译受影响的部分。

```
# 进入构建目录直接用 ninja 增量编译
cd build/gum-linux-x86_64
ninja

# 运行 Gum 的单元测试
./tests/gum-tests
./tests/gum-tests /Interceptor    # 只跑 Interceptor 相关测试
```

30.2 如何贡献代码

Frida 是一个活跃的开源项目，贡献者来自世界各地。你的贡献可以是任何形式的。

从小处开始

不要一上来就试图重写 Stalker 引擎。好的第一步包括：

- **修复文档中的错误**：最低门槛，但非常有价值
- **改进错误信息**：让用户看到更友好的报错
- **添加测试用例**：发现一个边界条件没有被测试到？写个 test
- **修复小 bug**：GitHub Issues 中标记为 “good first issue” 的问题

贡献流程

```
# 1. Fork 仓库到你的 GitHub 账号

# 2. 创建功能分支
git checkout -b fix/interceptor-null-check

# 3. 修改代码并确保测试通过
# ... 编辑文件 ...
make gum-linux-x86_64
cd build/gum-linux-x86_64 && ./tests/gum-tests

# 4. 提交并推送
git add -A
git commit -m "Fix null pointer check in interceptor attach"
git push origin fix/interceptor-null-check

# 5. 在 GitHub 上创建 Pull Request
```

代码风格

Frida 的代码风格需要注意：

- **C 代码**：GLib 风格，函数名用 `gum_module_function_name` 格式
- **Vala 代码**：类似 C#，PascalCase 类名，snake_case 方法名
- **缩进**：C 用 2 空格（或 tab，取决于子项目），Vala 用 tab
- **注释**：简洁明了，不写废话

30.3 值得探索的相关项目

Frida 不是孤岛。它处于一个丰富的工具生态系统中。学完 Frida 之后，这些项目值得你深入了解：

动态二进制插桩工具

DBI 工具对比

工具	开发语言	典型场景	学习价值
Frida	C/Vala/JS	逆向/安全	你已经学了！
DynamoRIO	C	程序分析	底层 DBI 原理
Intel Pin	C++	性能分析	JIT 编译技术
QBDI	C++	模糊测试	QuarkslaB 出品
Valgrind	C	内存检测	影子内存技术

逆向工程工具

- **Radare2 (r2)**：开源的逆向工程框架，和 Frida 有良好的集成（r2frida 项目）。学习 r2 的架构设计，你会发现它和 Frida 有异曲同工之妙。
- **Ghidra**：NSA 开源的反编译工具。它的反编译算法值得深入研究。
- **IDA Pro**：商业逆向工具的标杆。了解它能帮助你理解”反编译器需要解决什么问题”。

虚拟化与模拟

- **QEMU**：全系统模拟器。Frida 的 Stalker 和 QEMU 的 TCG（Tiny Code Generator）在原理上有很多共通之处——都是将一种指令翻译为另一种。
- **Unicorn Engine**：基于 QEMU 的 CPU 模拟框架。很多安全研究者把它和 Frida 结合使用。

编译器与运行时

- **V8**：Frida 的默认 JS 引擎。理解 V8 的 JIT 编译能帮助你理解 GumJS 的性能特性。
- **QuickJS**：Frida 的轻量级 JS 引擎选项。Fabrice Bellard 的杰作，代码量小，适合学习。

- **LLVM**：虽然 Frida 不直接使用 LLVM，但 LLVM 的中间表示和优化理念对理解代码变换很有帮助。

30.4 职业发展路径

学完 Frida 源码之后，你已经掌握了一组非常有价值的技能。这些技能可以引向多个职业方向。

安全研究员

你掌握的技能	适用方向
动态插桩原理	漏洞挖掘
内存布局理解	Exploit 开发
跨平台系统编程	移动安全
代码注入技术	恶意软件分析

安全研究是一个需求旺盛的领域。无论是甲方安全团队（大厂的安全部门）还是乙方安全公司（安全厂商），都需要能深入理解系统底层的人。

工具开发者

你现在理解了一个世界级工具框架的内部设计。这意味着你有能力：

- 为 Frida 开发插件和扩展
- 构建基于 Frida 的自动化安全测试平台
- 设计和实现自己的动态分析工具

系统/基础架构工程师

Frida 源码中涉及的技术——进程管理、内存操作、JIT 编译、跨进程通信、多平台抽象——这些都是系统编程的核心技能。

编译器/运行时工程师

如果你对 Stalker 的代码翻译技术特别感兴趣，编译器和语言运行时是一个深入方向。从 JIT 编译到垃圾回收，从指令选择到寄存器分配，这些领域有无数迷人的问题等着你。

30.5 推荐阅读

书籍

- 《深入理解计算机系统》(CS:APP) —— 如果你还没读过，现在就开始
- 《程序员的自我修养——链接、装载与库》—— 理解可执行文件格式
- 《编译器设计》(Engineering a Compiler) —— JIT 编译的理论基础
- 《The Art of Software Security Assessment》—— 安全审计经典
- 《Android Internals》—— 如果你做移动安全

在线资源

- **Frida 官方文档** (frida.re) —— 永远的起点
- **Frida CodeShare** —— 社区分享的脚本，学习最佳实践
- **OleBegemann 的博客** —— 深入的 Frida 技术文章
- **r2con / OffensiveCon 演讲视频** —— 了解安全社区的前沿
- **GitHub 上的 awesome-frida** —— 资源合集

源码阅读清单

读完 Frida 源码后，推荐继续阅读这些项目的源码：

1. **QuickJS** —— 10 万行代码的完整 JS 引擎，适合一个人读完
2. **SQLite** —— 教科书级别的 C 项目，测试覆盖率极高
3. **Redis** —— 优雅的 C 语言数据结构实现
4. **Lua** —— 最小的可嵌入脚本引擎之一

30.6 致读者

写这本书的过程中，我（以及辅助我的 Claude Code）一起深入了 Frida 的每一个角落。从 `Interceptor.attach` 的第一次调用，到 Stalker 引擎中复杂的代码翻译逻辑，再到 GumJS 如何在 C 和 JavaScript 之间架起桥梁——每一次深入都让人感叹：好的系统软件就像一座精心设计的大厦，远看宏伟，近看精致。

但这座大厦不是一天建成的。Frida 从 2010 年的第一行代码到今天，经历了无数次重构、优化和扩展。它的作者 Ole Andre Vadla Ravnas 和整个社区用十多年的时间，打磨出了我们今天看到的这个工具。

你也是这个社区的一部分。当你用 Frida 做研究时，当你发现一个 bug 并提交 issue 时，当你写了一篇博客分享你的经验时——你都在为这个生态做贡献。

不要害怕源码。你已经证明了自己能读懂它。

不要害怕贡献。每一个伟大的贡献者都是从第一个 typo fix 开始的。

不要害怕失败。在安全研究和系统编程领域，失败是最好的老师。

下一步，打开终端，克隆代码，开始你自己的探索。

我们在 Issue 和 Pull Request 中见。

本章小结

- **搭建开发环境**：Frida 使用 Meson 构建系统，首次编译需要较长时间和充足的磁盘空间
- **从小处贡献**：修复文档、改进错误信息、添加测试用例都是好的开始
- **探索相关项目**：DynamoRIO、QEMU、r2 等工具与 Frida 互补
- **多条职业路径**：安全研究、工具开发、系统编程、编译器工程
- **持续学习**：推荐的书籍和源码项目可以帮助你不断深入

讨论问题

1. 如果你要基于 Frida 构建一个产品（比如自动化安全测试平台），你会选择哪些 Frida 的能力作为核心功能？你会如何设计架构？
2. Frida 的 Stalker 和 QEMU 的 TCG 都是动态代码翻译引擎。它们的设计目标有什么不同？这种不同如何影响了它们的架构选择？
3. 回顾你读这本书的过程，哪一章对你影响最大？如果你要向一个朋友推荐从哪一章开始读，你会选哪一章？为什么？

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

附录 A：术语表

> 本术语表按英文字母顺序排列，收录了本书中出现的核心技术术语。每个术语提供中文解释和英文原名，方便读者在阅读英文资料时对照使用。

ABI（Application Binary Interface，应用二进制接口） 定义了函数调用时参数如何传递、返回值如何返回、寄存器如何分配等底层约定。不同平台（如 ARM64 的 AAPCS、x86-64 的 System V ABI）有不同的 ABI 规范。Frida 的 Interceptor 在 hook 函数时必须严格遵守目标平台的 ABI。

Agent（代理） 在 Frida 中，Agent 是被注入到目标进程中的动态链接库（.so / .dylib / .dll）。它包含了 JavaScript 运行时和 Gum 引擎，是用户脚本实际执行的载体。对应源码中 `frida-core/src/agent-container.vala`。

ARM / ARM64（ARM 架构） 移动设备主流的 CPU 架构。ARM 指 32 位版本（ARMv7），ARM64 指 64 位版本（AArch64/ARMv8）。Frida 的 Gum 引擎对两种架构都有完整支持。

Backpatching（回补丁） Stalker 引擎中的一种优化技术。当代码块第一次被翻译时，某些跳转目标可能尚未确定。Backpatching 是在目标确定后回过头来修补之前的跳转指令，避免重复查找。

Binary（二进制文件） 编译后的可执行程序或库文件。不同平台有不同的二进制格式：Linux 用 ELF，macOS/iOS 用 Mach-O，Windows 用 PE。

Bootstrapper（引导器） 注入过程中用于初始化 Agent 的一小段代码。它负责加载动态库、解析符号、调用入口函数等基本引导工作。

Code Signing（代码签名） iOS 和 macOS 上的安全机制，要求所有执行的代码都必须经过数字签名验证。Frida 需要处理代码签名问题才能在这些平台上注入代码和修改内存中的指令。

CModule Frida 提供的一种机制，允许用户直接在脚本中编写 C 代码，由 TinyCC 编译后在目标进程中执行。适合性能敏感的场景。

D-Bus（Desktop Bus） 一种进程间通信协议。Frida 内部使用了基于 D-Bus 协议的通信机制来实现 frida-server 与客户端之间的消息传递。

DBI（Dynamic Binary Instrumentation，动态二进制插桩） 在程序运行时修改其二进制代码以插入分析逻辑的技术。Frida 是 DBI 工具的代表之一。

Detour / Trampoline（跳板） Hook 技术的核心。在目标函数开头写入一条跳转指令（detour），跳转到一块预先分配的代码区域（trampoline），在那里执行用户的回调逻辑，然后再跳回原函数。

DWARF 一种调试信息格式，存储在可执行文件中，包含变量名、类型、行号等信息。Frida 的 Gum 引擎可以利用 DWARF 信息进行符号解析。

ELF（Executable and Linkable Format） Linux 和 Android 上的标准可执行文件格式。Frida 的 `backend-elf` 目录包含了解析 ELF 文件的代码。

Entrypoint（入口点） 程序或动态库开始执行的地址。Frida 在注入 Agent 时需要找到并调用其入口点。

ExecMemory（可执行内存） 具有执行权限的内存区域。Frida 动态生成的代码（如 trampoline、Stalker 翻译后的代码块）都需要写入可执行内存。

Frida 本书的主角。一个跨平台的动态插桩框架，支持 Windows、macOS、Linux、iOS、Android 等。项目主页：frida.re。

Gadget Frida 提供的一个特殊的共享库，可以嵌入到目标应用中（而不是远程注入）。适用于无法使用 frida-server 的场景，例如未越狱的 iOS 设备上的重打包应用。

GLib / GObject GNOME 项目的基础库。GLib 提供了数据结构、事件循环、线程等基础设施。GObject 提供了 C 语言的面向对象系统（类、继承、接口、信号）。Frida 的 C 代码大量使用了 GLib/GObject。

Gum Frida 的核心引擎库（`frida-gum`）。提供了 Interceptor、Stalker、Memory、Module 等底层能力。名字没有特殊含义，就是一个简短的项目名。

GumJS Gum 的 JavaScript 绑定层（`frida-gum/bindings/gumjs/`）。负责将 Gum 的 C API 暴露给 JavaScript 世界，支持 V8 和 QuickJS 两种引擎。

Hook（钩子） 拦截函数调用的技术。通过修改目标函数的代码，在函数执行前后插入自定义逻辑。Frida 的 `Interceptor.attach()` 就是 hook 操作。

Inject / Injection（注入） 将代码加载到另一个进程的地址空间中的操作。Frida 在不同平台使用不同的注入方式：Linux 用 ptrace，macOS 用 mach_vm + task_for_pid，Android 可能用 /proc/pid/mem 等。

Interceptor（拦截器） Frida 最常用的功能组件。通过 inline hook 技术拦截函数调用，允许用户在函数执行前（onEnter）和执行后（onLeave）运行自定义代码。源码在 `guminterceptor.c`。

JIT（Just-In-Time Compilation，即时编译） 在程序运行时将代码编译为机器码的技术。V8 引擎使用 JIT 编译 JavaScript。Stalker 也使用类似 JIT 的技术动态翻译和插桩代码块。

Mach-O（Mach Object） macOS 和 iOS 上的可执行文件格式。Frida 需要解析 Mach-O 格式来查找函数、段、符号等信息。

Meson Frida 使用的构建系统。比 CMake 更现代、更简洁，配置文件使用声明式语法。对应每个子项目中的 `meson.build` 文件。

Module（模块） 一个已加载的可执行文件或动态链接库。在 Frida 中，`Module.findExportByName()` 用于在已加载的模块中查找导出符号。

N-API（Node-API） Node.js 的稳定 C/C++ 插件接口。`frida-node` 使用 N-API 将 Frida 的 C 核心包装为 Node.js 模块。

NativePointer（原生指针） Frida JavaScript API 中表示内存地址的对象。支持算术运算、读写操作等。底层对应 C 中的 `gpointer`。

PE（Portable Executable） Windows 上的可执行文件格式（.exe / .dll）。

Process（进程） 操作系统中程序运行的实例。Frida 的 `Process` API 提供了查询进程信息（模块列表、线程列表、内存范围等）的能力。

ptrace Linux 上的进程追踪系统调用。Frida 在 Linux/Android 上使用 ptrace 来附加到目标进程并注入代码。

QuickJS Fabrice Bellard 开发的轻量级 JavaScript 引擎。Frida 将其作为 V8 的替代选项，适合资源受限的环境。

Relocator（重定位器） Frida 中负责”搬迁”指令的组件。当 Interceptor hook 一个函数时，被覆盖的原始指令需要被复制到 trampoline 中。但某些指令（如 PC 相对跳转）不能简单复制，需要 Relocator 进行调整。

RPC（Remote Procedure Call，远程过程调用） Frida 提供的机制，允许 Python/Node.js 端直接调用目标进程中 JavaScript 脚本暴露的函数。通过 `script.exports` 和 `rpc.exports` 实现。

Script（脚本） 在 Frida 中，Script 是用户编写的 JavaScript 代码的运行时容器。一个 Script 对应一个 JS 引擎实例和一组绑定。

Session（会话） Frida 客户端与目标进程之间的连接。通过 `device.attach(pid)` 创建。一个 Session 可以创建多个 Script。

Slab（内存块） Stalker 引擎中用于分配翻译后代码的内存区域。预先分配大块内存，然后从中按需划分小块，避免频繁的系统调用。

Stalker（追踪器） Frida 的动态代码追踪引擎。能够追踪目标线程执行的每一条指令、每一次函数调用和每一次跳转。通过动态代码翻译（类似 JIT）实现。源码在 `gumstalker.c` 和 `backend-*/gumstalker-*.c`。

Symbol（符号） 函数名或变量名在二进制文件中的表示。调试信息、导出表、符号表中都包含符号。Frida 的 `DebugSymbol` API 用于解析地址到符号名。

Task Port（任务端口） macOS/iOS 上的 Mach IPC 概念。`task_for_pid()` 获取目标进程的任务端口后，就可以读写其内存、创建线程等。Frida 在 Darwin 平台的注入依赖此机制。

Thumb（Thumb 指令集） ARM 架构的一种压缩指令编码模式。Thumb 指令为 16 位宽（而标准 ARM 指令为 32 位），Thumb-2 混合了 16 位和 32 位指令。Frida 的 ARM 后端需要正确处理 ARM/Thumb 模式切换。

Trampoline（跳板） 见 “Detour” 条目。Trampoline 是 hook 过程中创建的代码片段，用于跳转到用户回调，然后跳回原始函数继续执行。

V8 Google 开发的高性能 JavaScript 引擎（Chrome 中使用的引擎）。Frida 默认使用 V8 作为 GumJS 的脚本后端。

Vala 一种编译为 C 代码的编程语言，语法类似 C#。Frida-core 大量使用 Vala 编写，它提供了 GObject 之上的现代语法糖，包括异步编程、类型推断等。

Writer（代码写入器） Frida 中用于动态生成机器码的组件。例如 `GumArm64Writer` 提供了一组 API 来生成 ARM64 指令，而不需要手动编码二进制。

XPC（XPC Services） macOS/iOS 上的进程间通信机制。Frida 在 Darwin 平台使用 XPC 与系统服务交互，也通过 XPC 与 helper 进程通信。对应 `frida-core/src/darwin/` 中的相关代码。

补充术语

以下术语在阅读源码时也会经常遇到：

Allocation（内存分配） 在 Frida 中，内存分配不仅指普通的堆内存分配，还包括可执行内存的分配。`gumcodeallocator.c` 专门处理可执行代码内存的分配和管理。

Attach（附加） 将 Frida 连接到目标进程的操作。附加后才能注入 Agent 和运行脚本。在 `frida-core` 层对应 `HostSession.attach_to()` 方法。

Capstone 一个开源的反汇编引擎，支持多种 CPU 架构。Frida 在某些场景下使用 Capstone 来反汇编目标代码，辅助分析指令结构。

Context（上下文） 在 Frida 中，Context 通常指 CPU 上下文（`GumCpuContext`），包含了所有寄存器的值。在 `onEnter` 回调中可以通过 `this.context` 访问和修改寄存器。

DeviceManager（设备管理器） Frida 客户端的入口类，负责发现和管理可用的设备（本地设备、USB 设备、远程设备）。

Dispose / Finalize（销毁/终结） GObject 对象的两个析构阶段。Dispose 释放对其他对象的引用（可被多次调用），Finalize 释放自身资源（只调用一次）。理解这两个阶段对于阅读 Frida 的 GObject 代码很重要。

Enumerate（枚举） 遍历某类资源的操作。如 `Process.enumerateModules()` 枚举已加载模块，`Module.enumerateExports()` 枚举导出符号。Frida 中大量使用回调式枚举模式。

FFI（Foreign Function Interface，外部函数接口） 一种允许某种语言调用另一种语言编写的函数的机制。Frida 的 `NativeFunction` 和 `NativeCallback` 底层使用 libffi 来实现跨语言函数调用。

Fruitjector Frida 在 macOS/iOS 平台上的代码注入器。名字来源于 Darwin（达尔文）-> Fruit（水果）的联想。对应 `frida-core/src/darwin/fruitjector.vala`。

GMainLoop / GMainContext（GLib 主循环） GLib 提供的事件循环机制。Frida 的异步操作（如等待消息、处理信号）都基于 GMainLoop。理解 GLib 事件循环对于理解 Frida 的异步架构至关重要。

GumInvocationContext（调用上下文） Interceptor hook 函数时，传递给回调的上下文对象。包含了函数参数、CPU 上下文、线程 ID 等信息。

GumInvocationListener（调用监听器） Interceptor 的回调接口。实现 `on_enter` 和 `on_leave` 方法来处理函数调用事件。

Helper（辅助进程） Frida 在某些平台上使用特权辅助进程来执行需要高权限的操作（如在 macOS 上使用 task_for_pid）。对应 `frida-core/src/darwin/frida-helper-*.vala` 等文件。

Inline Hook（内联钩子） 直接修改目标函数开头的指令来实现 hook 的技术。与 GOT/PLT hook 不同，inline hook 可以 hook 任意地址的函数，不限于导入函数。Frida 的 Interceptor 使用的就是 inline hook。

Keystone 一个开源的汇编引擎（Capstone 的姊妹项目），可以将汇编代码转换为机器码。Frida 的 Writer 组件承担了类似 Keystone 的功能。

Linjector Frida 在 Linux/Android 平台上的代码注入器。Lin 来自 Linux。对应 `frida-core/src/linux/linjector.vala`。

NativeFunction / NativeCallback JavaScript 端用于调用原生函数和创建原生回调的 API。`NativeFunction` 将一个地址包装为可调用的 JS 函数，`NativeCallback` 将一个 JS 函数包装为可被原生代码调用的函数指针。

Ninja 一个小型高速构建系统，由 Meson 生成构建文件后由 Ninja 执行实际编译。在 Frida 开发中，增量编译时直接运行 `ninja` 比运行完整的 `make` 更快。

Spawn（启动） 由 Frida 启动目标进程的操作（与 attach 附加已有进程相对）。spawn 模式下，Frida 可以在程序 main 函数执行之前就完成 hook 设置。

Transformer（变换器） Stalker 的代码变换接口。用户可以通过 Transformer 自定义代码翻译逻辑，例如在特定指令前后插入回调。

TypeScript JavaScript 的超集语言，添加了静态类型。Frida 支持直接运行 TypeScript 脚本（通过内置的编译器），并提供了完整的类型定义文件 `@types/frida-gum`。

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

附录 B：源码关键文件索引

> 本附录列出了 Frida 源码中最重要的文件，按子项目和功能模块分类。路径相对于 Frida 主仓库根目录（即 `frida/subprojects/` 下的各子项目）。“相关章节”列指出了本书中详细分析该文件的章节。

本索引共收录 70+ 个关键文件，覆盖了 Frida 的所有核心子项目。建议读者在阅读每章时，对照本索引找到对应的源码文件进行参考。

> 提示：使用编辑器的全局搜索功能（如 VS Code 的 Ctrl+Shift+F），在 Frida 源码目录中搜索下表中的文件名，可以快速定位到对应文件。

---

frida-gum：核心引擎

文件路径	功能描述	相关章节
`frida-gum/gum/gum.c`	Gum 引擎初始化与全局状态管理	第3章
`frida-gum/gum/gum.h`	Gum 公共头文件，定义核心类型	第3章
`frida-gum/gum/guminterceptor.c`	Interceptor 核心实现——函数 hook 引擎	第7-8章
`frida-gum/gum/guminterceptor.h`	Interceptor 公共 API 定义	第7章
`frida-gum/gum/guminterceptor-priv.h`	Interceptor 内部数据结构	第8章
`frida-gum/gum/gumstalker.c`	Stalker 公共接口与调度逻辑	第11章
`frida-gum/gum/gumstalker.h`	Stalker 公共 API 定义	第11章
`frida-gum/gum/gumstalker-priv.h`	Stalker 内部数据结构定义	第12章
`frida-gum/gum/gummemory.c`	内存操作：分配、读写、搜索、保护属性修改	第15章
`frida-gum/gum/gumprocess.c`	进程信息查询：模块列表、线程列表、内存范围	第16章
`frida-gum/gum/gummodule.c`	模块（已加载的动态库）信息查询	第16章
`frida-gum/gum/gumapiresolver.c`	API 符号解析器	第16章
`frida-gum/gum/gumbacktracer.c`	调用栈回溯接口	第17章
`frida-gum/gum/gumcloak.c`	Cloak 机制——隐藏 Frida 自身的内存和线程	第22章

文件路径	功能描述	相关章节
`frida-gum/gum/guminvocationlistener.c`	函数调用监听器接口	第7章
`frida-gum/gum/gumcodeallocator.c`	代码内存分配器——管理可执行内存	第9章
`frida-gum/gum/gumcodesegment.c`	代码段管理（与代码签名相关）	第22章

frida-gum：架构后端

文件路径	功能描述	相关章节
`frida-gum/gum/backend-arm64/gumstalker-arm64.c`	Stalker ARM64 实现——代码翻译核心	第12-13章
`frida-gum/gum/backend-x86/gumstalker-x86.c`	Stalker x86/x64 实现	第12章
`frida-gum/gum/backend-arm64/guminterceptor-arm64.c`	Interceptor ARM64 后端——trampoline 生成	第9章
`frida-gum/gum/backend-x86/guminterceptor-x86.c`	Interceptor x86/x64 后端	第9章
`frida-gum/gum/arch-arm64/gumarm64writer.c`	ARM64 指令生成器（代码写入器）	第10章
`frida-gum/gum/arch-arm64/gumarm64relocator.c`	ARM64 指令重定位器	第10章
`frida-gum/gum/arch-x86/gumx86writer.c`	x86/x64 指令生成器	第10章
`frida-gum/gum/arch-x86/gumx86relocator.c`	x86/x64 指令重定位器	第10章
`frida-gum/gum/arch-arm/gumarmwriter.c`	ARM32 指令生成器	第10章
`frida-gum/gum/arch-arm/gumarmrelocator.c`	ARM32 指令重定位器	第10章
`frida-gum/gum/arch-arm/gumthumbwriter.c`	Thumb 指令生成器	第10章
`frida-gum/gum/arch-arm/gumthumbrelocator.c`	Thumb 指令重定位器	第10章
`frida-gum/gum/backend-darwin/gumprocess-darwin.c`	Darwin 平台进程操作实现	第16章

文件路径	功能描述	相关章节
`frida-gum/gum/backend-linux/gumprocess-linux.c`	Linux 平台进程操作实现	第16章

frida-gum：GumJS 绑定层

文件路径	功能描述	相关章节
`frida-gum/bindings/gumjs/gumscript.c`	Script 生命周期管理	第18章
`frida-gum/bindings/gumjs/gumscriptbackend.c`	脚本后端接口（V8 / QuickJS 选择）	第18章
`frida-gum/bindings/gumjs/gumquickinterceptor.c`	Interceptor 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickstalker.c`	Stalker 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickcore.c`	核心类型（NativePointer 等）的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickmodule.c`	Module API 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickmemory.c`	Memory API 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumcmodule.c`	CModule——在 JS 中编写并编译 C 代码	第20章
`frida-gum/bindings/gumjs/gumffi.c`	FFI（外部函数接口）绑定	第19章
`frida-gum/bindings/gumjs/gumscriptscheduler.c`	脚本线程调度器	第18章

frida-core：服务与会话管理

文件路径	功能描述	相关章节
`frida-core/src/frida.vala`	Frida 入口——DeviceManager、Device 等核心类	第4章
`frida-core/src/host-session-service.vala`	HostSession 服务——管理进程附加与注入	第5章
`frida-core/src/control-service.vala`		第5章

文件路径	功能描述	相关章节
	控制服务——frida-server 的 D-Bus 服务端	
`frida-core/src/portal-service.vala`	Portal 服务——集群模式支持	第5章
`frida-core/src/agent-container.vala`	Agent 容器——管理注入到目标进程的 Agent	第6章
`frida-core/src/darwin/darwin-host-session.vala`	macOS/iOS 平台的 HostSession 实现	第21章
`frida-core/src/darwin/fruitjector.vala`	Darwin 平台代码注入器	第21章
`frida-core/src/linux/linux-host-session.vala`	Linux/Android 平台的 HostSession 实现	第21章
`frida-core/src/linux/linjector.vala`	Linux 平台代码注入器	第21章
`frida-core/src/linux/linjector-glue.c`	Linux 注入器的 C 层胶水代码（ptrace 操作）	第21章
`frida-core/src/system.vala`	系统信息查询	第4章
`frida-core/src/async-task.vala`	异步任务基础设施	第5章
`frida-core/src/file-monitor.vala`	文件系统监控	第16章
`frida-core/src/compiler/compiler.vala`	TypeScript 编译器集成	第20章

frida-python 与 frida-tools

文件路径	功能描述	相关章节
`frida-python/frida/__init__.py`	Python 绑定入口——定义顶层 API	第2章
`frida-python/frida/core.py`	Python 端核心类：Device、Session、Script 等	第2章
`frida-tools/frida_tools/application.py`	CLI 工具基础框架	第2章
`frida-tools/frida_tools/itracer.py`	frida-trace 工具实现	第2章
	frida-discover 工具实现	第2章

文件路径	功能描述	相关章节
frida-tools/frida_tools/discoverer.py		
frida-tools/frida_tools/reactor.py	事件循环与脚本重载机制	第2章
frida-tools/frida_tools/creator.py	frida-create 项目模板生成器	第2章

frida-core：平台注入与辅助

文件路径	功能描述	相关章节
frida-core/src/darwin/frida-helper-backend-glue.m	macOS helper 的 ObjC 胶水代码	第21章
frida-core/src/darwin/frida-helper-service.vala	macOS helper 服务进程	第21章
frida-core/src/darwin/policy-softener.vala	iOS 代码签名策略软化	第22章
frida-core/src/darwin/system-darwin.m	Darwin 平台系统信息实现	第21章
frida-core/src/linux/helpers/	Linux 辅助进程集合	第21章
frida-core/src/linux/spawn-gater.vala	Linux spawn 控制门控	第21章
frida-core/src/linux/proc-maps.vala	/proc/pid/maps 解析	第16章
frida-core/src/linux/symbol-resolver.vala	Linux 符号解析	第16章
frida-core/src/barebone/	Barebone（无操作系统）模式支持	第27章
frida-core/src/fruity/	iOS USB 通信协议实现	第21章
frida-core/src/socket/	网络套接字传输层	第5章
frida-core/src/package-manager.vala	应用包管理（枚举已安装应用）	第4章

frida-gum：其他重要文件

文件路径	功能描述	相关章节
`frida-gum/gum/gumexceptor.c`	异常处理器——拦截硬件异常	第17章
`frida-gum/gum/gummemorymap.c`	内存映射查询	第15章
`frida-gum/gum/gummetalarray.c`	无 GLib 依赖的底层数组实现	第24章
`frida-gum/gum/gummetalhash.c`	无 GLib 依赖的底层哈希表实现	第24章
`frida-gum/gum/gumtls.c`	线程本地存储封装	第24章
`frida-gum/gum/gumspinlock.c`	自旋锁实现	第24章
`frida-gum/gum/gumsymbolutil.c`	符号解析工具函数	第16章
`frida-gum/gum/gumswiftapiresolver.c`	Swift 符号解析器	第16章
`frida-gum/gum/gumdarwinmodule.c`	Mach-O 模块解析	第16章
`frida-gum/gum/gumelfmodule.c`	ELF 模块解析	第16章
`frida-gum/tests/core/interceptor.c`	Interceptor 单元测试	第7章
`frida-gum/tests/core/stalker.c`	Stalker 单元测试	第11章

构建系统与配置

文件路径	功能描述	相关章节
`meson.build`	主仓库构建入口	第3章
`frida-gum/meson.build`	Gum 构建配置	第3章
`frida-core/meson.build`	Core 构建配置	第3章
`releng/`	发布工程脚本——依赖管理、交叉编译配置	第3章
`frida-gum/meson.options`	Gum 编译选项（启用/禁用功能）	第3章
`frida-core/src/embed-agent.py`	Agent 嵌入脚本——将 Agent 打包进二进制	第6章
`frida-gum/bindings/gumjs/generate-bindings.py`	绑定代码生成脚本	第19章
	运行时代码生成脚本	第18章

文件路径	功能描述	相关章节
`frida-gum/bindings/gumjs/generate-runtime.py`		

frida-gum：GumJS 扩展绑定

文件路径	功能描述	相关章节
`frida-gum/bindings/gumjs/gumquickprocess.c`	Process API 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickfile.c`	File API 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquicksocket.c`	Socket API 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickstream.c`	Stream API 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickapiresolver.c`	ApiResolver 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickchecksum.c`	Checksum API 的 QuickJS 绑定	第19章
`frida-gum/bindings/gumjs/gumquickcloak.c`	Cloak API 的 QuickJS 绑定	第22章
`frida-gum/bindings/gumjs/guminspectorserver.c`	Chrome DevTools 调试协议支持	第26章
`frida-gum/bindings/gumjs/gummemoryvfs.c`	内存虚拟文件系统——SQLite 集成	第20章

其他子项目

文件路径	功能描述	相关章节
`frida-clr/`	.NET/CLR 绑定	第28章
`frida-go/`	Go 语言绑定	第28章
`frida-qml/`	Qt/QML 绑定	第28章
`frida-swift/`	Swift 语言绑定	第28章
`frida-node/`	Node.js 绑定（使用 N-API）	第2章

按功能快速查找

如果你想了解某个特定功能的实现，下表可以帮助你快速定位：

我想了解…	核心文件	辅助文件
函数 hook 是怎么工作的	`guminterceptor.c`	`guminterceptor-arm64.c`
代码追踪是怎么实现的	`gumstalker.c`	`gumstalker-arm64.c`
代码注入到目标进程的过程	`linjector.vala`	`linjector-glue.c`
JavaScript 如何调用 C 函数	`gumquickcore.c`	`gumffi.c`
ARM64 指令是怎么生成的	`gumarm64writer.c`	`gumarm64relocator.c`
Agent 在目标进程中如何启动	`agent-container.vala`	`embed-agent.py`
RPC 调用是怎么实现的	`core.py`	`frida.vala`
内存搜索是怎么工作的	`gummemory.c`	`gumquickmemory.c`

> 提示：以上文件路径基于 Frida 主仓库的 `subprojects/` 目录结构。在实际源码中，完整路径为 `frida/subprojects/frida-gum/gum/...` 等。部分文件路径可能因版本更新而略有变化，请以实际源码为准。

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

附录 C：全书知识地图

> 本附录提供了 Frida 架构的全景视图、组件依赖关系、常用 API 速查表，以及全书章节之间的关系图。适合在阅读过程中随时翻阅，帮助你定位自己所处的位置。

C.1 Frida 全景架构图

下图展示了 Frida 从用户脚本到操作系统底层的完整架构：

```
┌──────────────
```

```
│
│        └──────────────┴──────────────┴────────────────┘
│
│                            │
│
│                      ┌─────┴──────────┐
│
│                      │   frida-core       │
│
│                      │  (Vala / C)        │
│
│                      │                    │
│
│                      │  DeviceManager     │
│
│                      │  Device            │
│
│                      │  Session           │
│
│                      │  Script            │
│
│                      │  HostSession       │
│
│                      │  AgentSession      │
│
│                      └────────────────────┘
│
│                            │
│
│                            │
│
│   ┌──────────────────────────────────────────────────────┐
│
│   │                      frida-gum                       │
│
│   │                                                      │
│
│   │   ┌──────────────────── GumJS 绑定层 ────────────────┐    │  │
│   │   │  gumquickcore.c    gumquickinterceptor.c         │    │
│
│   │   │  gumquickstalker.c gumquickmemory.c    ...       │    │
│
│   │   │                    (或 gumv8*.cpp)               │    │
│
│   │   └──────────────────────────────────────────────────┘    │
│
│   │                          │                               │
│
│   │   ┌──────────────────── Gum 核心引擎 ────────────────┐    │  │
│   │   │  guminterceptor.c  gumstalker.c   gummemory.c    │    │
│
│   │   │  gumprocess.c      gummodule.c    gumcloak.c     │    │
│
│   │   │  gumcodeallocator.c               gumbacktracer.c │    │
```

```
│
│   │   └──────────────────────────┬──────────────────────────────┘     │
│
│   │                              │                                    │
│
│   │   ┌──────────────────── 架构后端层 ────────────────────────┐    │   │
│   │   │  backend-arm64/    backend-x86/    backend-arm/      │    │
│
│   │   │  arch-arm64/       arch-x86/       arch-arm/         │    │
│
│   │   │  (Writer / Relocator / Stalker / Interceptor)        │    │
│
│   │   └──────────────────────────┬──────────────────────────────┘     │
│
│   │                              │                                    │
│
│   │   ┌──────────────────── 平台后端层 ────────────────────────┐    │   │
│   │   │  backend-darwin/   backend-linux/  backend-windows/│    │
│
│   │   │  (内存/进程/模块 的平台特定实现)                       │    │   │
│   │   └──────────────────────────────────────────────────────┘     │
│
│   └──────────────────────────────────────────────────────────────────┘
│
│                                  │
│
├──────────────────────────────────┼──────────────────────────────────────
│                                 操作系统层
│
│                                  │
│
│   ┌──────────┐   ┌──────────┬──┐   ┌──────────────┐                  │
│   │  Darwin  │   │   Linux  │  │   │   Windows    │
│
│   │ mach_vm  │   │   ptrace │  │   │   NtAPI      │
│
│   │ task_port│   │   /proc  │  │   │   Debug API  │
│
│   │ XPC      │   │   memfd  │  │   │              │
│
│   └──────────┘   └──────────┴──┘   └──────────────┘                  │
└──────────────────────────────────────────────────────────────────────────
```

C.2 组件依赖关系图

下图展示了 Frida 各组件之间的依赖关系（箭头表示”依赖于”）：

```
frida-python ──────> frida-core ──────> frida-gum
frida-node   ──────>     │                   │
frida-swift  ──────>     │                   │
frida-tools  ──────>     │                   │
```

```
                          |                  |
                          v                  v
                       GLib/GIO          GLib/GObject
                          |                  |
                          v                  v
                     D-Bus 协议      capstone (反汇编)
                          |
                          v
            平台注入机制
                   (ptrace / mach / ...)
```

frida-gum 内部依赖:

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│   GumJS      │ ──> │  Gum Core    │ ──> │ Arch Backend │
│ (bindings)   │    │ (引擎层)     │    │ (指令处理)   │
└──────────────┘    └──────────────┘    └──────────────┘
      │                                        │
      v                                        v
 V8 / QuickJS                        capstone / keystone
 (JS 引擎)                           (反汇编 / 汇编)
```

C.3 常用 API 速查表

JavaScript API（在目标进程内使用）

```
┌──────────────────────────────────────────────────────────────┐
│                      Interceptor（函数 Hook）                 │
├──────────────────────────────────────────────────────────────┤
│ Interceptor.attach(target, {                                 │
│   onEnter(args) { },          // 函数调用前                  │
│   onLeave(retval) { }         // 函数返回后                  │
│ })                                                           │
│ Interceptor.replace(target, replacement)  // 替换整个函数     │
│ Interceptor.detachAll()                   // 移除所有 hook    │
├──────────────────────────────────────────────────────────────┤
│                      Stalker（代码追踪）                      │
├──────────────────────────────────────────────────────────────┤
│ Stalker.follow(threadId, {                                   │
│   events: { call: true, ret: true },                         │
│   onReceive(events) { },      // 批量接收事件                 │
│   onCallSummary(summary) { }  // 调用统计                    │
│ })                                                           │
│ Stalker.unfollow(threadId)                                   │
│ Stalker.addCallProbe(addr, callback)  // 在指定地址插入探针    │
├──────────────────────────────────────────────────────────────┤
│                      Memory（内存操作）                       │
├──────────────────────────────────────────────────────────────┤
│ Memory.alloc(size)                    // 分配 RWX 内存        │
│ Memory.copy(dst, src, n)              // 复制内存             │
│ Memory.scan(addr, size, pattern, cb)  // 模式扫描             │
```

```
│ Memory.protect(addr, size, prot)      // 修改保护属性          │
├──────────────────────────────────────────────────────────────┤
│                    Module（模块查询）                         │
├──────────────────────────────────────────────────────────────┤
│ Module.findBaseAddress(name)          // 模块基地址            │
│ Module.findExportByName(mod, name)    // 查找导出函数          │
│ Module.enumerateExports()             // 枚举导出              │
│ Module.enumerateImports()             // 枚举导入              │
├──────────────────────────────────────────────────────────────┤
│                    Process（进程信息）                        │
├──────────────────────────────────────────────────────────────┤
│ Process.id                            // 当前进程 PID          │
│ Process.arch                          // CPU 架构              │
│ Process.platform                      // 操作系统              │
│ Process.enumerateModules()            // 枚举已加载模块        │
│ Process.enumerateThreads()            // 枚举线程              │
├──────────────────────────────────────────────────────────────┤
│                    NativePointer（指针操作）                  │
├──────────────────────────────────────────────────────────────┤
│ ptr("0x1234")                         // 创建指针              │
│ p.readU8() / p.readS32() / ...        // 读取值                │
│ p.writeU8(val) / p.writeS32(val)      // 写入值                │
│ p.readUtf8String() / p.readUtf16String()  // 读字符串         │
│ p.add(n) / p.sub(n)                  // 指针算术               │
├──────────────────────────────────────────────────────────────┤
│                    RPC（远程调用）                            │
├──────────────────────────────────────────────────────────────┤
│ rpc.exports = {                                               │
│   myFunc() { return "hello"; }        // 暴露给外部调用        │
│ }                                                             │
│ // Python 端: script.exports.my_func()                        │
└──────────────────────────────────────────────────────────────┘
```

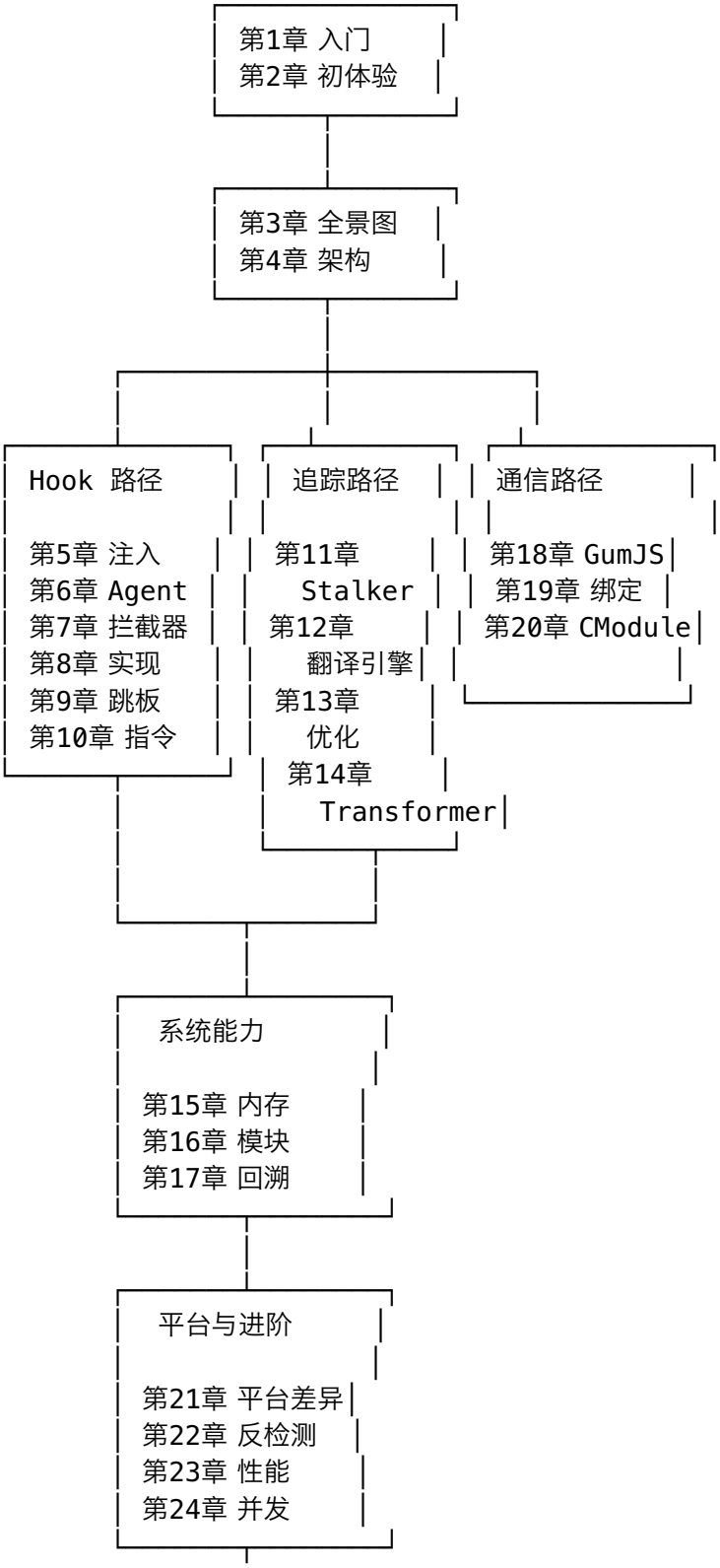
Python API（在控制端使用）

```
┌──────────────────────────────────────────────────────────────┐
│                    设备与会话管理                             │
├──────────────────────────────────────────────────────────────┤
│ frida.enumerate_devices()             // 列出可用设备          │
│ frida.get_usb_device()                // 获取 USB 设备         │
│ frida.get_local_device()              // 获取本地设备          │
│ device.enumerate_processes()          // 列出进程              │
│ device.attach(pid)                    // 附加到进程            │
│ device.spawn(program)                 // 启动进程              │
│ device.resume(pid)                    // 恢复进程执行          │
├──────────────────────────────────────────────────────────────┤
│                    脚本管理                                   │
├──────────────────────────────────────────────────────────────┤
│ session.create_script(source)         // 创建脚本              │
│ script.load()                         // 加载脚本              │
│ script.unload()                       // 卸载脚本              │
```

```
│ script.on('message', callback)        // 监听消息                │
│ script.exports_sync.func()            // 调用 RPC 导出          │
└──────────────────────────────────────────────────────────────────┘
```

C.4 章节关系图

本书30章可以分为几个主要阅读路径：



```
          │
          ┌──────────────────┐
          │  应用与总结       │
          │                  │
          │ 第25章 案例       │
          │ 第26章 调试       │
          │ 第27章 扩展       │
          │ 第28章 生态       │
          │ 第29章 模式       │
          │ 第30章 展望       │
          └──────────────────┘
```

推荐阅读路径

路径 A：我想快速上手 Hook > 第1章 -> 第2章 -> 第7章 -> 第9章 -> 第25章

路径 B：我想深入理解 Stalker > 第1章 -> 第3章 -> 第11章 -> 第12章 -> 第13章 -> 第14章

路径 C：我想理解 Frida 的整体架构 > 第1章 -> 第3章 -> 第4章 -> 第5章 -> 第6章 -> 第29章

路径 D：我想为 Frida 贡献代码 > 第3章 -> 第4章 -> 第29章 -> 第30章 -> 选择感兴趣的模块深入

路径 E：我想学习跨平台系统编程 > 第3章 -> 第10章 -> 第16章 -> 第21章 -> 第24章 -> 第29章

C.5 核心数据流

一次完整的 `Interceptor.attach()` 调用，数据在各层之间的流动：

```
用户 JS 脚本
│
│  Interceptor.attach(addr, callbacks)
│
v
GumJS 绑定层 (gumquickinterceptor.c)
│
│  解析 JS 参数 -> 创建 GumInvocationListener
│
v
Gum 核心层 (guminterceptor.c)
│
│  查找/创建 FunctionContext -> 准备 trampoline
│
v
架构后端层 (guminterceptor-arm64.c)
│
│  生成 trampoline 代码 (使用 GumArm64Writer)
│  重定位原始指令 (使用 GumArm64Relocator)
```

```
|
v
代码分配器 (gumcodeallocator.c)
|
|   分配可执行内存 -> 写入 trampoline
|
v
目标函数入口
|
|   原始指令被替换为跳转到 trampoline 的指令
|
v
Hook 生效!
```

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载

关于本书

这本书是怎么写成的

这本书的写作方式可能和你读过的大多数技术书不太一样。

传统的技术书写作流程是：作者花几个月甚至几年时间阅读源码、做笔记、写初稿、反复修改、找人审阅、最终出版。这个过程漫长而艰辛，也是很多优秀的技术内容最终没能变成书的原因——门槛太高了。

这本书采用了一种新的方式：**人机协作**。

作者 everettjf 使用 Claude Code（Anthropic 公司的 AI 编程助手）来辅助分析 Frida 源码并编写本书。具体的工作流程是：

1. **作者确定主题和大纲**：每一章要讲什么、从什么角度切入、用什么比喻，这些是人来决定的
2. **AI 辅助源码分析**：Claude Code 直接阅读 Frida 源码，提取关键代码片段，理解函数调用关系
3. **协作编写内容**：作者提供写作方向和质量要求，AI 生成初始内容
4. **作者审阅和调整**：确保技术准确性和阅读体验

这种方式有几个好处：

- **效率**：一本30章的源码分析书，传统方式可能需要一年以上，协作方式大大缩短了周期
- **覆盖面**：AI 可以快速扫描大量源码文件，不容易遗漏重要细节
- **一致性**：全书的写作风格、术语使用、代码示例格式保持一致

当然，这种方式也有局限：

- AI 可能对某些代码的理解不够深入，需要作者把关
- 某些需要实际运行和调试才能获得的经验，AI 无法提供

- 源码随版本更新，书中的分析可能与最新版本有出入

我们坦诚地告诉读者这一点，是因为我们相信：**工具不重要，重要的是内容的质量和对读者的价值。**

开源精神

这本书秉持开源精神：

- **免费阅读**：本书完全免费，不设任何付费门槛
- **自由转载**：只要保留出处信息，你可以自由转载本书的任何内容
- **鼓励改进**：如果你发现错误或有更好的表述，欢迎贡献

Frida 本身是一个开源项目，我们认为关于它的知识也应该是开放的。

如何贡献和勘误

本书托管在 GitHub 上。如果你发现了问题，有以下几种贡献方式：

发现错误

如果你发现了技术错误、代码示例问题或表述不清的地方：

1. 在 GitHub 仓库中提交一个 Issue，描述你发现的问题
2. 如果你知道正确的答案，可以直接提交 Pull Request
3. 也欢迎通过其他渠道（博客评论、社交媒体等）告知作者

改进建议

如果你觉得某个概念可以解释得更好，或者某个比喻不太恰当：

1. 提交 Issue 说明你的建议
2. 如果可能，附上你认为更好的表述

补充内容

如果你觉得某个话题值得深入讨论，或者有实践经验想要分享：

1. 可以提交 Pull Request 添加补充内容
2. 也可以在自己的博客上写文章，我们很乐意在书中添加链接

为什么写这本书

市面上关于 Frida 使用教程的文章不少，但深入到源码层面的系统性分析却很少见。这不奇怪——Frida 的代码库涉及 C、Vala、JavaScript、Python 等多种语言，横跨多个操作系统和 CPU 架构，完整阅读是一项巨大的工程。

但正是这种复杂性，让 Frida 成为了一个极好的学习素材。通过阅读 Frida 源码，你能同时接触到：

- 操作系统底层机制（进程管理、内存操作、信号处理）
- 编译器技术（指令编码、代码生成、重定位）

- 跨平台工程实践（抽象层设计、构建系统、条件编译）
- 网络通信（RPC、序列化、异步 I/O）
- 语言互操作（C/JavaScript 桥接、FFI、GObject 类型系统）

这些知识在任何一本单独的教科书中都很难同时覆盖。而 Frida 把它们有机地组合在了一个真实的项目中。

我们希望这本书能降低源码阅读的门槛，让更多人能够从 Frida 的代码中学到东西。

本书的局限性

我们诚实地承认本书的局限：

- **版本时效性**：源码分析基于特定时间点的代码，Frida 持续演进中
- **深度取舍**：30 章的篇幅无法覆盖每一行代码，某些细节必然被省略
- **平台偏重**：由于作者的经验背景，某些平台的分析可能比其他平台更深入
- **AI 辅助的固有局限**：某些需要实际调试和运行才能获得的洞察，可能不够深入

如果你在阅读过程中发现了错误或有更好的理解，非常欢迎你提出来。

致谢

感谢以下个人和项目：

- **Ole Andre Vadla Ravnas** 和 Frida 社区的所有贡献者——没有 Frida 就没有这本书
- **NowSecure** 团队——持续维护和发展 Frida 项目
- **Anthropic** 和 Claude Code——本书的写作辅助工具
- **GLib/GNOME 社区**——Frida 依赖的基础设施
- **V8 和 QuickJS 的开发者**——JavaScript 引擎是 Frida 的灵魂
- **所有读者**——你们的阅读让这些文字有了意义
- 所有在 GitHub、博客、论坛上分享 Frida 使用经验的安全研究者

许可协议

本书采用自由转载许可：

- 你可以自由地阅读、分享、转载本书的全部或部分内容
- 唯一的要求是：**保留原始出处信息**
- 不限制商业或非商业用途
- 不需要事先获得许可

简单来说：**保留出处即可自由转载。**

我们选择这种宽松的许可方式，是因为我们相信知识应该自由流动。如果这本书的某一章帮助了某个人理解了一个概念，那它就实现了自己的价值——无论那个人是在个人博客上引用了它，还是在公司内部培训中使用了它。

版本信息

- 本书基于 Frida 源码编写，源码仓库：https://github.com/nicedayzhu/frida
- 写作工具：Claude Code (Anthropic)
- 作者：everettjf

联系方式

如果你有任何问题、建议或勘误，欢迎通过以下方式联系：

- GitHub Issues：在本书的 GitHub 仓库中提交 Issue
- Pull Request：直接提交修改建议

期待在开源社区中与你相遇。

本书由 [everettjf] 使用 Claude Code 分析源码编写 | 保留出处即可自由转载